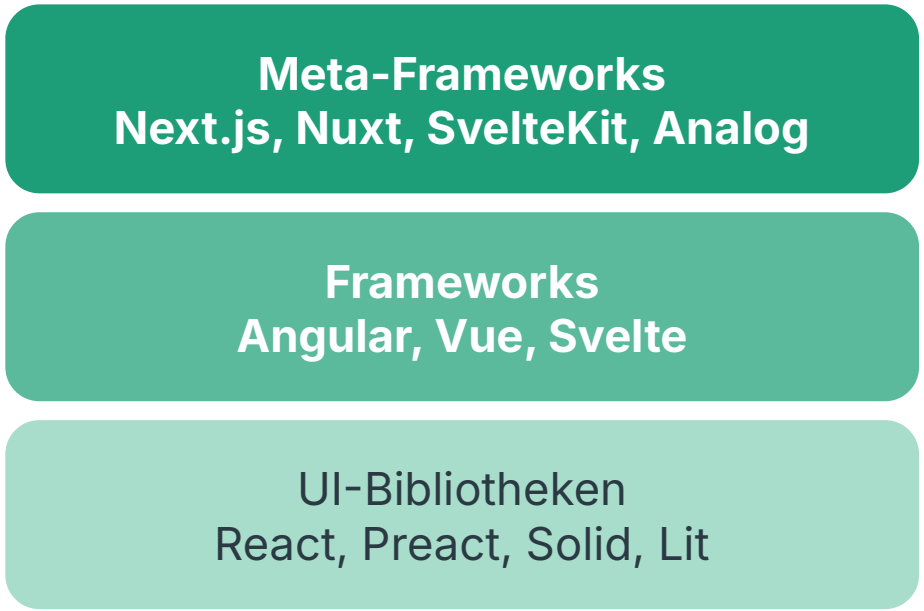
Einordnung moderner Angular-Entwicklung

Wo steht Angular im Frontend-Umfeld?

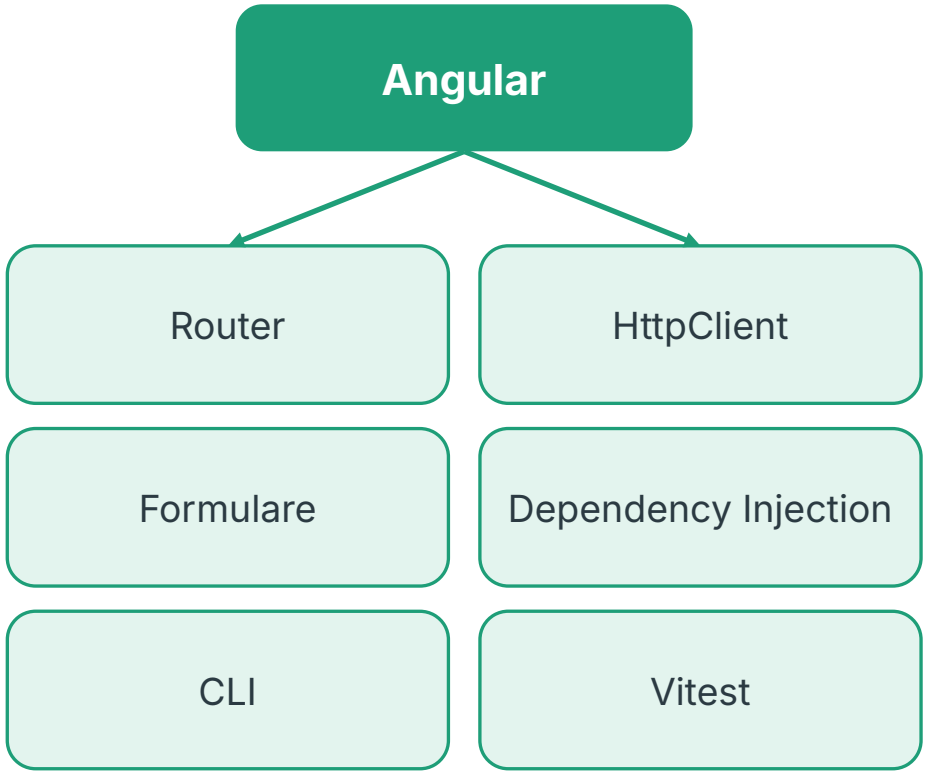
- Werkzeuge für die Oberfläche lassen sich grob in drei Ebenen einteilen:
  - UI-Bibliotheken: React, Preact, Solid, Lit – sie zeigen nur die Oberfläche an
  - Frameworks: Angular, Vue, Svelte – Oberfläche und weitere Bausteine
  - Meta-Frameworks: Next.js, Nuxt, SvelteKit, Analog – Routing, Server-Rendering, Build vorgegeben
- Angular ist ein vollständiges Framework: Router, Formulare, HTTP, Dependency Injection, CLI
- Folge: weniger Auswahl- und Integrationsaufwand, dafür mehr Konvention



Je höher, desto mehr ist vorgegeben

Batteries included: Was Angular mitbringt

- Routing: @angular/router (sonst z. B. React Router oder TanStack Router)
- HTTP: HttpClient mit Interceptors (sonst z. B. fetch oder axios)
- Formulare: Template-Driven Forms und Reactive Forms (sonst z. B. Formik)
- Zustand: Signals, Services und NgRx (sonst z. B. Redux, Zustand oder MobX)
- Abhängigkeiten: Dependency Injection (sonst z. B. Context oder Module-Singletons)
- Werkzeuge: Angular CLI mit dem Befehl ng (sonst z. B. Vite oder eigene Konfiguration)
- Tests: Vitest-Setup ab Version 21 und TestBed (sonst z. B. Jest mit Testing Library)



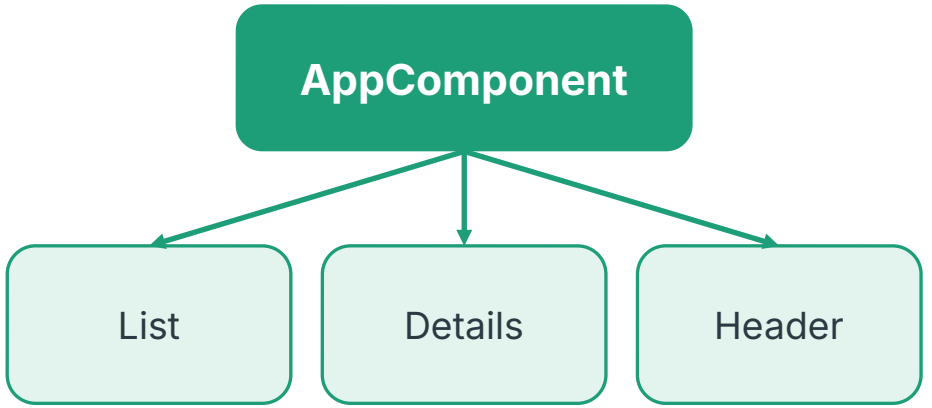
Einsatzgebiete: Wann passt Angular, wann nicht?

- Passt gut: große, langlebige Unternehmensanwendungen mit vielen Teams
- Passt gut: Dashboards und formularlastige Fachanwendungen mit hohen Qualitätsanforderungen
- Passt weniger: reine Inhaltsseiten und Blogs (z. B. Astro oder statische Generatoren)
- Passt weniger: sehr kleine Widgets
- Passt weniger: Teams ohne TypeScript-Erfahrung und ohne Zeit zum Einarbeiten
- Entscheidungskriterien: Teamgröße, Lebensdauer, vorhandene Skills, Integrationsbedarf
- Außerdem: Rendering-Anforderungen wie Suchmaschinen-Optimierung (SEO) und Server-Rendering



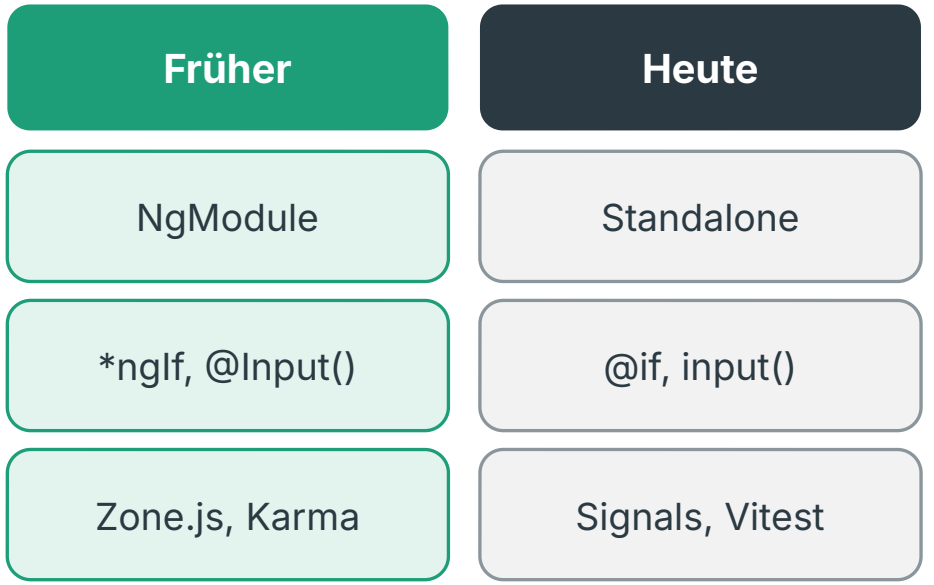
Architekturprinzipien von Angular

- Komponentenbasiert: Die Oberfläche ist ein Baum aus Komponenten (Template, Klasse, Stil)
- Dependency Injection: Abhängigkeiten werden bereitgestellt, nicht selbst erzeugt
  - Dadurch sind sie austauschbar und gut testbar
- Einseitiger Datenfluss: Daten fließen nach unten (input), Ereignisse nach oben (output)
- Starke Typisierung: TypeScript im strict-Modus, typisierte Templates und Formulare
- Konvention vor Konfiguration: gleiche Struktur in jedem Projekt, schnelle Einarbeitung



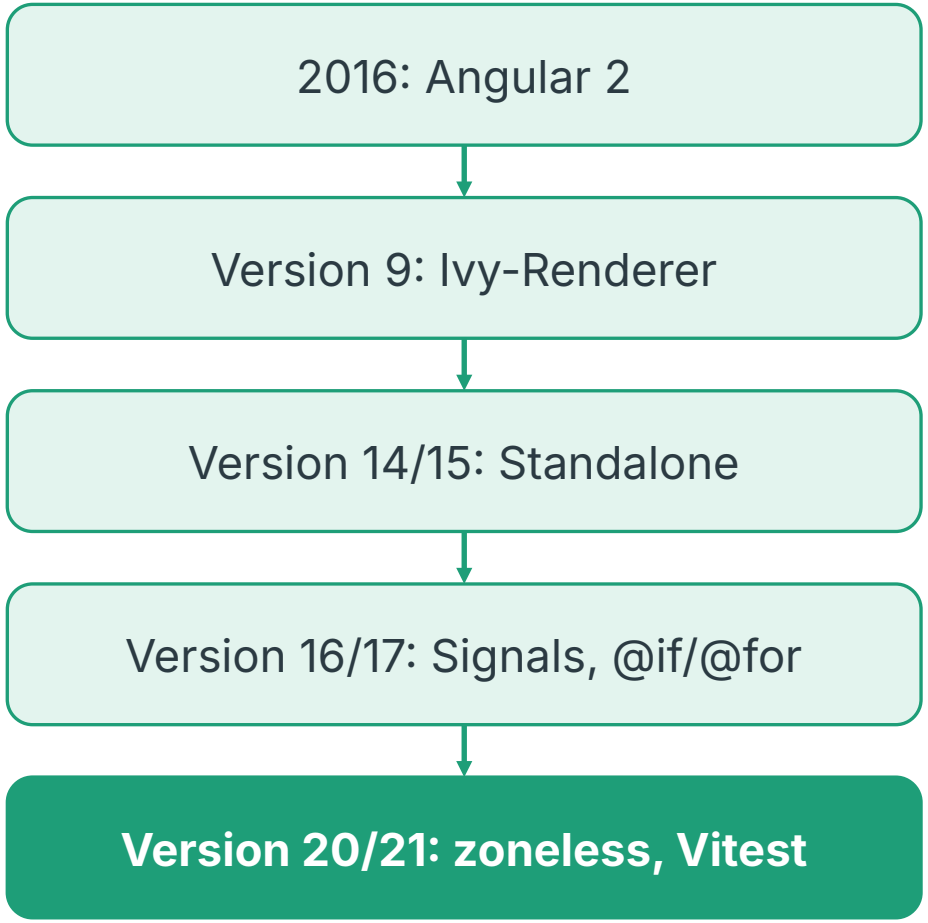
Die Angular-Denkweise heute

- Leitgedanke: weniger Boilerplate, mehr Reaktivität
- NgModule wird zu Standalone-Komponenten (imports: [])
- Zone.js und implizite Change Detection werden zu Signals und zoneless
- *ngIf und *ngFor werden zu @if, @for und @switch
- @Input(), @Output() und EventEmitter werden zu input(), output() und model()
- Konstruktor-Injection wird zu inject()
- Klassen-Guards und -Interceptors werden zu funktionalen Guards und Interceptors
- Karma und Jasmine werden zu Vitest



Entwicklung in Etappen

- 2016: Angular 2 als Neuentwurf gegenüber AngularJS
- Ivy-Renderer (Version 9), Standalone-Komponenten (ab Version 14/15)
- Signals und neuer Control Flow (Version 16/17)
- Zoneless (Version 20/21), Vitest als Standard-Testrunner (Version 21)
- Release-Rhythmus: zwei Major-Versionen pro Jahr, jede etwa 18 Monate unterstützt
- Updates mit ng update und Migrations-Schematics
- Wissen veraltet nicht schlagartig, aber die Muster wandeln sich
- Der Kurs vermittelt deshalb Prinzipien und nicht nur Programmierschnittstellen



Toolchain im Überblick

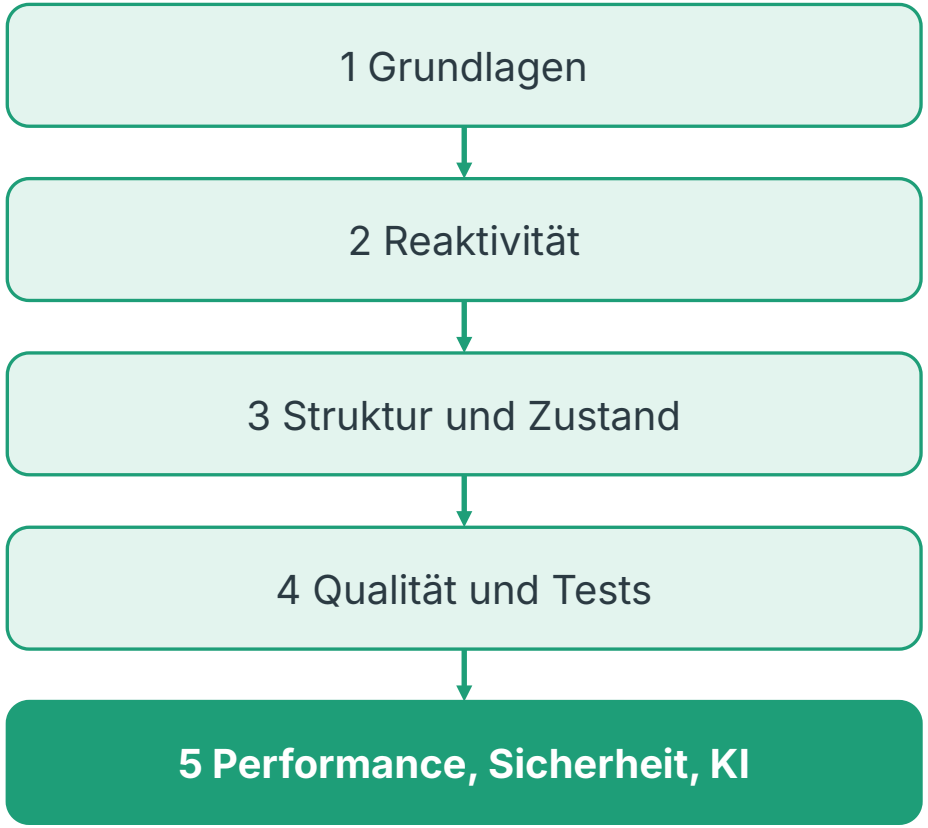
- Angular CLI: ng new, ng serve, ng generate, ng build, ng test, ng update
- Build: Application Builder auf Basis von esbuild und Vite
  - Schneller Dev-Server mit HMR (Hot Module Replacement)
  - HMR: geänderter Code erscheint, ohne die Seite vollständig neu zu laden
- TypeScript im strengen Modus, dazu ESLint und Prettier als Teamstandard
- Angular DevTools im Browser: Komponentenbaum und Profiler

Terminal

```
ng new invoice-app
ng serve        # Dev-Server, HMR
ng generate component customer-list
ng test         # Vitest
ng build        # fertige App
ng update       # neue Version
```

Der rote Faden: die Invoice-App

- Alle Übungen entwickeln eine Anwendung weiter: Kundenverwaltung mit Liste, Details, Login
- Jede Übung startet mit dem Ergebnis der vorherigen (Ordner 04_... bis 38_...)
- Wer eine Übung nicht schafft, startet die nächste im Lösungsordner der vorigen Übung
- Aufbau des Kurses:
  - 1. Grundlagen: Komponenten, Templates, Services, Routing
  - 2. Reaktivität und Datenfluss
  - 3. Anwendungsstruktur und Zustandsverwaltung
  - 4. Qualität, Tests, Wartbarkeit
  - 5. Performance, Sicherheit, KI-gestützte Entwicklung



Eine App, die in jeder Übung weiterwächst

Erwartungen und Vorerfahrung

- Wer hat bisher Weboberflächen entwickelt, und womit?
- Welche Anwendung soll nach dem Kurs mit Angular entstehen oder gepflegt werden?
- Welche zwei Themen sind besonders wichtig?
- Das Ergebnis kommt auf ein Flipchart
- Am Kursende wird es mit dem Erlernten abgeglichen
  - Grundkurs: Ende Tag 3, Kompaktkurs: Ende Tag 5

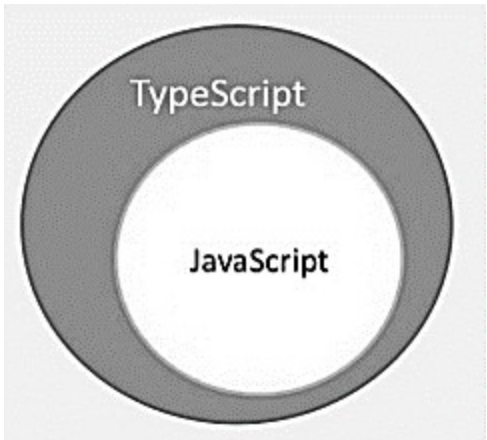


Typescript Primer

Einführung in Typescript

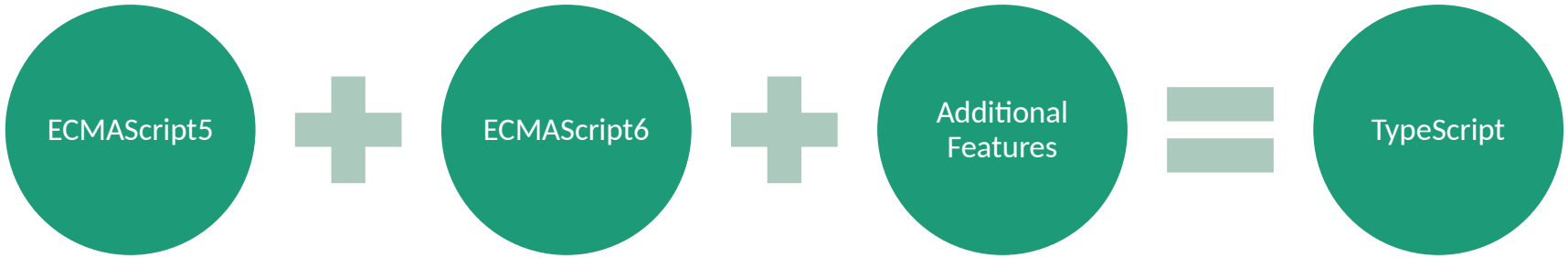
TypeScript is just JavaScript

TypeScript is portable.



TypeScript supports other JS libraries

JavaScript is TypeScript.


ECMAScript5
ECMAScript6
Additional Features
TypeScript



OO Concepts



Supports Type Definitions



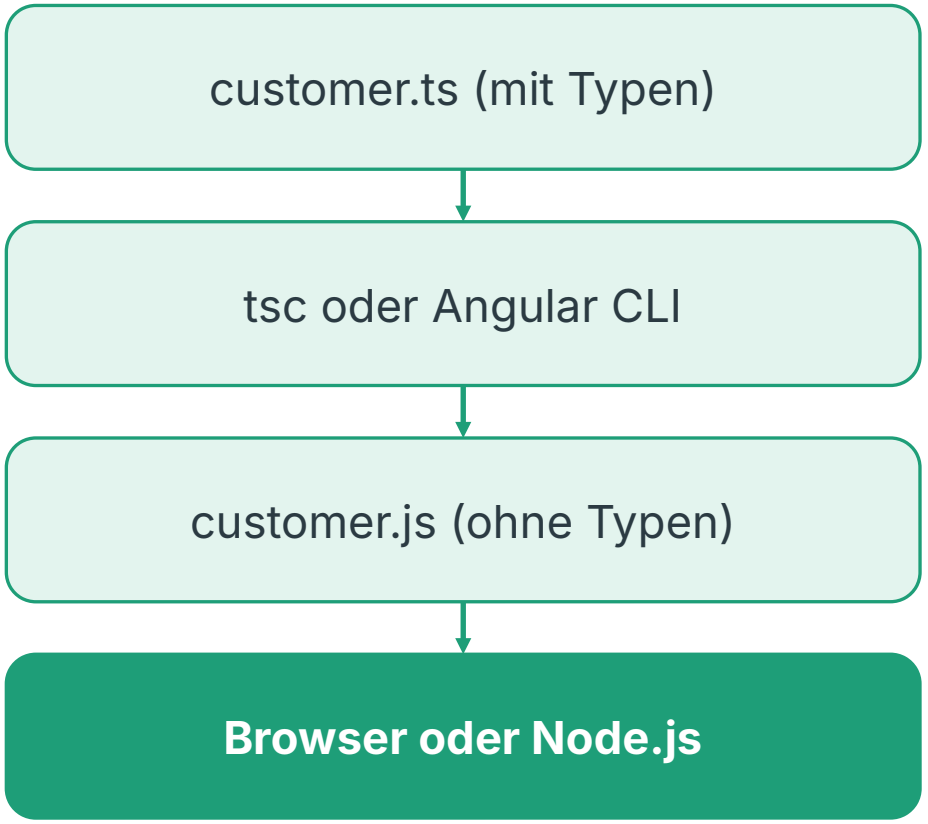
Strong static Type



Compilation

Vom Quelltext zum Browser

- Wir schreiben TypeScript-Dateien mit der Endung .ts
- Der Compiler „tsc“ übersetzt sie in JavaScript-Dateien (.js)
- Browser und Node.js verstehen nur JavaScript, kein TypeScript
- Beim Übersetzen verschwinden alle Typen: Im fertigen Programm gibt es sie nicht mehr
- Die Angular CLI übernimmt das Übersetzen für uns
- Typen prüfen ist nicht dasselbe wie das Programm ausführen



Typen gibt es nur beim Entwickeln

Der Kursordner und die Befehle

- Ordner: 01_typescript_primer, darin einmalig: npm install
- npm run check: prüft alle Typen in src/ und meldet Fehler
- npm run watch: wie check, prüft bei jeder Änderung neu
- npm start -- src/<datei>.ts: führt eine Datei aus
- Achtung: npm start prüft die Typen NICHT
- Fehler zeigen der Editor (rote Wellenlinien) und npm run check
- Aufgaben liegen in src/, Musterlösungen in loesung/

tsconfig.json der Invoice-App

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "target": "ES2022",
    "experimentalDecorators": true
  },
  "angularCompilerOptions": {
    "strictTemplates": true
  }
}
```

Ein erster Typfehler

```
function greet(firstname: string, lastname: string): string {
  return `Hallo ${firstname} ${lastname}`;
}

greet("Lisa", "Simpson");   // richtig

greet("Lisa", 42);
// Fehler TS2345: Argument of type 'number' is not
// assignable to parameter of type 'string'.
```

Typen

Typen und Typableitung

```
const firstname: string = "Lisa";
const age: number = 42;
const active: boolean = true;
const since: Date = new Date(2024, 0, 15);
const names: string[] = ["Lisa", "Hans", "Ned"];

// Typableitung: Der Compiler erkennt den Typ selbst
const city = "Springfield";   // string
const count = names.length;   // number
const ids = [1, 2, 3];        // number[]
```

const und let

```
const company = "Simpson GmbH";   // Standard: const
let counter = 0;                  // let nur, wenn sich der Wert ändert
counter = counter + 1;            // erlaubt

// company = "Andere GmbH";       // Fehler: const nicht neu zuweisbar

const names: string[] = ["Lisa"];
names.push("Hans");               // erlaubt: der Inhalt ändert sich
// names = [];                    // Fehler: die Variable bleibt fest
```

Warum nicht mehr var

```
for (var i = 0; i < 10; i++) {
  setTimeout(() ⇒ console.log("var:", i), 0);
}
// Ausgabe: zehnmal "var: 10"

for (let j = 0; j < 10; j++) {
  setTimeout(() ⇒ console.log("let:", j), 0);
}
// Ausgabe: "let: 0" bis "let: 9"
```

any, unknown und never

```
let anything: any = "Lisa";
anything.toUpperCase();     // kein Fehler, auch wenn es knallen kann

function show(value: unknown): string {
  if (typeof value === "string") {
    return value.toUpperCase();   // hier ist value ein string
  }
  return String(value);
}

function fail(message: string): never {
  throw new Error(message);       // kehrt nie zurück
}
```

null und undefined im strikten Modus

```
function cityLabel(city: string | undefined): string {
  return city.toUpperCase();
  // Fehler TS18048: 'city' is possibly 'undefined'.
}

function safeCityLabel(city: string | undefined): string {
  if (city === undefined) {
    return "unbekannt";
  }
  return city.toUpperCase();   // hier ist city sicher ein string
}
```

Sicher mit fehlenden Werten: ?., ?? und !

```
const customer = customers.find(c ⇒ c.id === "3");  // Customer | undefined

const city = customer?.city;                  // ?. : stoppt bei fehlendem Wert
const label = customer?.city ?? "unbekannt";  // ?? : Ersatzwert

const lisa = customers.find(c ⇒ c.id === "1")!;  // ! : "ich weiß es sicher"
console.log(lisa.firstname);

// Im Kurs:  customer()?.firstname
//           customer()!.imageUrl!
```

TS-Übung 1: Typfehler beheben

- **Öffne 01_typescript_primer, Datei src/uebung-1-typfehler.ts**
- **1. Führe npm run check aus und lies die 6 Fehlermeldungen**
- **2. Behebe jeden Fehler, ohne any zu benutzen**
- **3. Manche Fehler behebst du im Code, bei anderen musst du den Typ anpassen**
- **4. Starte das Programm: npm start -- src/uebung-1-typfehler.ts**
- Tipp: string | undefined, customer.city?.toUpperCase() ?? "UNBEKANNT"
- **Musterlösung: loesung/uebung-1-typfehler.ts**

Funktionen

Funktionen typisieren

```
function cityOf(customer: Customer, fallback = "unbekannt"): string {
  return customer.city ?? fallback;
}

function greet(name: string, greeting?: string): string {
  return `${greeting ?? "Hallo"} ${name}`;
}

function logMessage(message: string): void {
  console.log(message);
}

cityOf(customers[2]);                  // "unbekannt"
cityOf(customers[2], "Ogdenville");    // "Ogdenville"
```

Rest-Parameter und Arrow-Funktionen

```
function sum(...nums: number[]): number {
  return nums.reduce((a, b) ⇒ a + b, 0);
}
sum(1, 2, 3);                // 6

const double = (n: number): number ⇒ n * 2;
const add = (a: number, b: number) ⇒ a + b;

function applyTwice(value: number, fn: (n: number) ⇒ number): number {
  return fn(fn(value));
}
applyTwice(3, double);       // 12
applyTwice(3, n ⇒ n + 10);  // 23
```

Listen verarbeiten: filter, map, find

```
const springfield = customers.filter(c ⇒ c.city === "Springfield");
// zwei Kunden

const names = customers.map(c ⇒ `${c.firstname} ${c.lastname}`);
// ["Lisa Simpson", "Hans Maulwurf", "Ned Flanders"]

const ned = customers.find(c ⇒ c.id === "3");   // Customer | undefined
console.log(ned?.firstname ?? "nicht gefunden");

const active = customers
  .filter(c ⇒ !c.discontinued)
  .map(c ⇒ c.lastname);                    // ["Simpson", "Flanders"]
```

TS-Übung 2: Funktionen typisieren

- **Öffne 01_typescript_primer, Datei src/uebung-2-funktionen.ts**
- **1. Schreibe fullName und filterCustomers (Tipp: filter, includes, toLowerCase)**
- **2. Schreibe cityOf mit einem Standardwert für den zweiten Parameter**
- **3. Schreibe joinNames mit einem Rest-Parameter (...customers), map und join**
- **4. Bis Aufgabe 3 und 4 gelöst sind, meldet npm run check zwei Fehler: das ist gewollt**
- Start: npm start -- src/uebung-2-funktionen.ts
- **Musterlösung: loesung/uebung-2-funktionen.ts**

Objekte und Listen

Interfaces: die Form eines Objekts

```
interface Customer {
  id: string;
  firstname: string;
  lastname: string;
  city?: string;
  discontinued?: boolean;
  lastOrderDate?: Date;
}

const lisa: Customer = { id: "1", firstname: "Lisa", lastname: "Simpson" };

// Fehler: Die Pflicht-Eigenschaft "lastname" fehlt
// const hans: Customer = { id: "2", firstname: "Hans" };
```

Optional und readonly

```
interface Invoice {
  readonly id: string;     // nur lesbar, nach dem Anlegen nicht änderbar
  customerId: string;
  note?: string;           // optional, darf fehlen
}

const invoice: Invoice = { id: "R-1", customerId: "1" };
invoice.note = "bezahlt";  // erlaubt
// invoice.id = "R-2";     // Fehler TS2540: id ist readonly

const ids: readonly string[] = ["1", "2"];
// ids.push("3");          // Fehler: Eine readonly-Liste hat kein push
const more = [...ids, "3"];  // neue Liste ist erlaubt
```

Destructuring

```
const [first, , third] = customers;      // Liste zerlegen, zweiten überspringen

const { firstname, lastname } = first;   // Objekt zerlegen
const { city = "unbekannt" } = third;    // Standardwert, falls city fehlt
const { id, ...rest } = first;           // rest: alle übrigen Eigenschaften

// Zerlegen direkt in der Parameterliste
function label({ firstname, lastname }: Customer): string {
  return `${lastname}, ${firstname}`;
}

console.log(firstname, lastname, city, id);   // Lisa Simpson unbekannt 1
console.log(rest.city, label(third));         // Springfield Flanders, Ned
```

Spread: kopieren statt verändern

```
const lisa = customers[0];

const renamed = { ...lisa, firstname: "Liesel" };  // neues Objekt
const more = [...customers, renamed];              // neue Liste

console.log(lisa.firstname, renamed.firstname);    // Lisa Liesel
console.log(customers.length, more.length);        // 3 4

// Achtung: Spread kopiert nur flach, Verschachteltes wird geteilt
const copy = { ...lisa };
console.log(copy.lastOrderDate === lisa.lastOrderDate);  // true
```

Template-Strings

```
const c = customers[0];

// Backticks: Ausdrücke in ${...} werden in den Text eingesetzt
const line = `${c.lastname}, ${c.firstname} (${c.city ?? "unbekannt"})`;

// Auch mehrere Zeilen sind möglich
const mail = `Guten Tag ${c.firstname},
Ihre Rechnung ist fertig.`;

const total = 3 * 19.99;
console.log(`Summe: ${total.toFixed(2)} Euro`);   // Summe: 59.97 Euro
console.log(line);                                 // Simpson, Lisa (Springfield)
console.log(mail);
```

TS-Übung 3: Objekte und Listen

- **Öffne 01_typescript_primer, Datei src/uebung-3-objekte.ts**
- **1. renameCustomer: neue Liste, ein Kunde hat den neuen Vornamen (map und Spread)**
- **2. addCustomer: neue Liste mit einem Kunden mehr am Ende ([...customers, customer])**
- **3. describe: Parameter in der Signatur zerlegen, city-Standardwert, Template-String**
- **4. Prüfe in der Ausgabe: Die ursprüngliche Liste ist unverändert geblieben**
- Ausführen: npm start -- src/uebung-3-objekte.ts
- **Musterlösung: loesung/uebung-3-objekte.ts**

Typen kombinieren

Union Types und Narrowing

```
function show(value: string | number): string {
  return typeof value === "string" ? value.toUpperCase() : value.toFixed(2);
}

type State =
  | { status: "loading" }
  | { status: "loaded"; data: Customer[] }
  | { status: "error"; message: string };

function render(state: State): string {
  switch (state.status) {
    case "loading": return "Lade …";
    case "loaded": return `${state.data.length} Kunden`;
    case "error": return state.message;
  }
}
```

Literal-Typen und type

```
type Status = "active" | "discontinued";   // nur diese zwei Texte sind erlaubt
type CustomerId = string;                  // ein Alias, also ein zweiter Name

function setStatus(c: Customer, status: Status): Customer {
  return { ...c, discontinued: status === "discontinued" };
}

setStatus(customers[0], "discontinued");
// setStatus(customers[0], "gesperrt");    // Fehler TS2345: nicht erlaubter Wert

enum Role { Admin, User }                  // enum gibt es auch
const role: Role = Role.Admin;
```

interface oder type?

- Beide beschreiben die Form eines Objekts, mit optionalen und readonly-Eigenschaften
- Beide lassen sich erweitern: interface mit extends, type mit & (Schnittmenge)
- Nur type kann Unions, Literal-Typen und Alias für einfache Typen: type Status = "a" | "b"
- Nur interface kann später um Eigenschaften ergänzt werden (declaration merging)
- Klassen können ein interface oder einen Objekt-type mit implements erfüllen
- Faustregel im Kurs: Objekt-Formen als interface (Customer)
- Faustregel im Kurs: Unions und Aliase als type (Status, State)
- Wichtiger als die Wahl: im Projekt einheitlich bleiben

Faustregel im Kurs

```
interface Customer {
  id: string;
  lastname: string;
}

type Status = "active" | "gesperrt";

type State =
  | { status: "loading" }
  | { status: "error" };
```

Generics

```
const names: Array<string> = ["Lisa", "Hans"];       // dasselbe wie string[]
const pending: Promise<Customer[]> = Promise.resolve(customers);

function first<T>(items: T[]): T | undefined {
  return items[0];
}

const firstCustomer = first(customers);   // Typ: Customer | undefined
const firstName = first(names);           // Typ: string | undefined

// In Angular begegnen uns dieselben spitzen Klammern:
// signal<Customer[]>([]), input<string>(), Observable<Customer>
```

Hilfstypen: Partial, Pick, Omit

```
type CustomerChanges = Partial<Customer>;       // alles optional
type CustomerName = Pick<Customer, "firstname" | "lastname">;  // nur diese
type NewCustomer = Omit<Customer, "id">;        // alles außer id

function update(c: Customer, changes: Partial<Customer>): Customer {
  return { ...c, ...changes };
}

const changed = update(customers[0], { city: "Shelbyville" });
const draft: NewCustomer = { firstname: "Ned", lastname: "Flanders" };
const label: CustomerName = changed;     // Customer hat diese Felder auch
```

Klassen

Klassen: Attribute, Konstruktor, Methoden

```
class Customer {
  id: string;
  firstname: string;
  lastname: string;
  city?: string;
  constructor(firstname: string, lastname: string, city?: string) {
    this.id = crypto.randomUUID();
    this.firstname = firstname;
    this.lastname = lastname;
    this.city = city;
  }
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
const lisa = new Customer("Lisa", "Simpson", "Springfield");
```

Parameter Properties

```
class Customer {
  id: string;
  constructor(
    public firstname: string,
    public lastname: string,
    public street?: string,
    public zip?: string,
    public city?: string,
  ) {
    this.id = crypto.randomUUID();
  }
}
// Später in Angular:
// constructor(private service: CustomerService) {}
```

Sichtbarkeit und readonly

```
class Account {
  readonly id = crypto.randomUUID();
  protected owner: string;
  private balance = 0;
  constructor(owner: string) {
    this.owner = owner;
  }
  deposit(amount: number): void {
    this.balance += amount;
  }
}
const account = new Account("Lisa");
account.deposit(50);
console.log(account.balance);
```

Getter und Setter

```
class Customer {
  private _discount = 0;
  constructor(public firstname: string, public lastname: string) {}
  get fullName(): string {
    return `${this.firstname} ${this.lastname}`;
  }
  get discount(): number {
    return this._discount;
  }
  set discount(value: number) {
    if (value < 0 || value > 100) throw new Error("0 bis 100");
    this._discount = value;
  }
}
```

implements: ein Interface erfüllen

```
interface Labelled {
  getLabel(): string;
}
class Customer implements Labelled {
  constructor(public firstname: string, public lastname: string) {}
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
function printLabel(item: Labelled): void {
  console.log(item.getLabel());
}
printLabel(new Customer("Lisa", "Simpson"));
```

extends und super

```
class VipCustomer extends Customer {
  constructor(
    firstname: string,
    lastname: string,
    public discount: number,
  ) {
    super(firstname, lastname);
  }
  override getLabel(): string {
    return super.getLabel() + ` - VIP -${this.discount}%`;
  }
}
const ned = new VipCustomer("Ned", "Flanders", 10);
console.log(ned.getLabel());
```

Dekoratoren: ein Blick auf Angular

```
function Component(config: { selector: string }) {
  return (target: Function) => {
    console.log(`${target.name} → <${config.selector}>`);
  };
}

@Component({ selector: "customer-list" })
class CustomerListComponent {
  title = "Kunden";
}
```

TS-Übung 4: Die Klasse Customer

- Öffne 01_typescript_primer, Datei src/uebung-4-klassen.ts
- 1. Schreibe die Klasse Customer mit Parameter properties (firstname, lastname, city?)
- 2. Setze id im Konstruktor mit crypto.randomUUID()
- 3. Ergänze den Getter fullName und die Methode getLabel()
- 4. Customer implementiert das Interface Labelled
- 5. VipCustomer extends Customer mit discount: super(...) zuerst, getLabel() ergänzen
- 6. Entferne die Kommentarzeichen am Dateiende und starte: npm start -- src/uebung-4-klassen.ts
- Musterlösung: loesung/uebung-4-klassen.ts

Module

Module: export und import

```
// customer.ts
export interface Labelled {
  getLabel(): string;
}
export class Customer implements Labelled {
  constructor(public firstname: string, public lastname: string) {}
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
// main.ts
import { Customer, type Labelled } from "./customer";
```

Regeln für import und export

- Imports sind statisch: sie stehen oben in der Datei, nicht in einer if-Abfrage
- Pfad zu eigenen Dateien beginnt mit ./ oder ../ und hat keine Endung
- Benannte Exporte: export class Customer, Import mit geschweiften Klammern
- Standard-Export: export default, Import ohne Klammern (im Kurs selten)
- import type { Labelled } holt nur einen Typ, der nach dem Übersetzen verschwindet
- Pakete wie @angular/core importiert man ohne Pfad: import { Component } from "@angular/core"
- Eine Datei mit import oder export ist ein Modul; ihre Namen stören andere Dateien nicht

TS-Übung 5: Module

- **Öffne 01_typescript_primer, Datei src/uebung-5-module.ts**
- **1. Lege die Datei src/customer.ts an**
- **2. Verschiebe die Klasse Customer dorthin und exportiere sie mit export**
- **3. Importiere sie in src/uebung-5-module.ts: import { Customer } from "./customer";**
- **4. Starte: npm start -- src/uebung-5-module.ts**
- Zusatz: Exportiere und importiere auch das Interface Labelled
- **Musterlösung: loesung/uebung-5-module/**

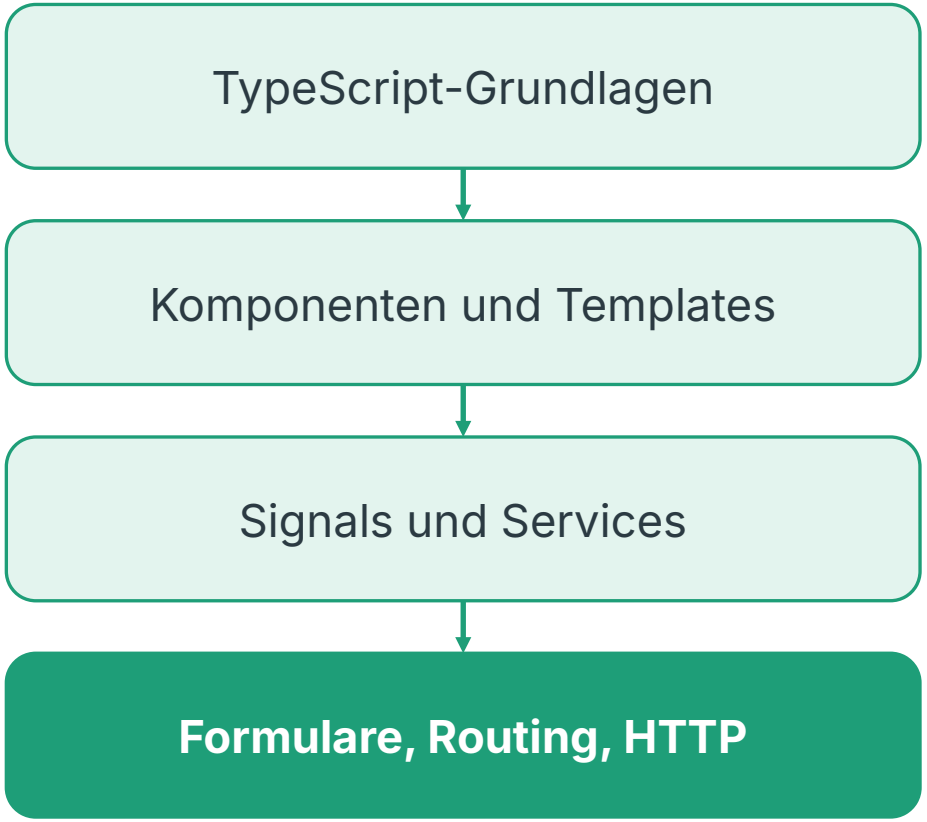
Von TypeScript zu Angular

```
import { Component } from "@angular/core";
import { Customer } from "./customer";

@Component({
  selector: "app-customer-list",
  template: `<h1>Kunden</h1>`,
})
export class CustomerListComponent {
  customers: Customer[] = [];
  constructor(private service: CustomerService) {}
}
```

Zusammenfassung: Was wir für Angular mitnehmen

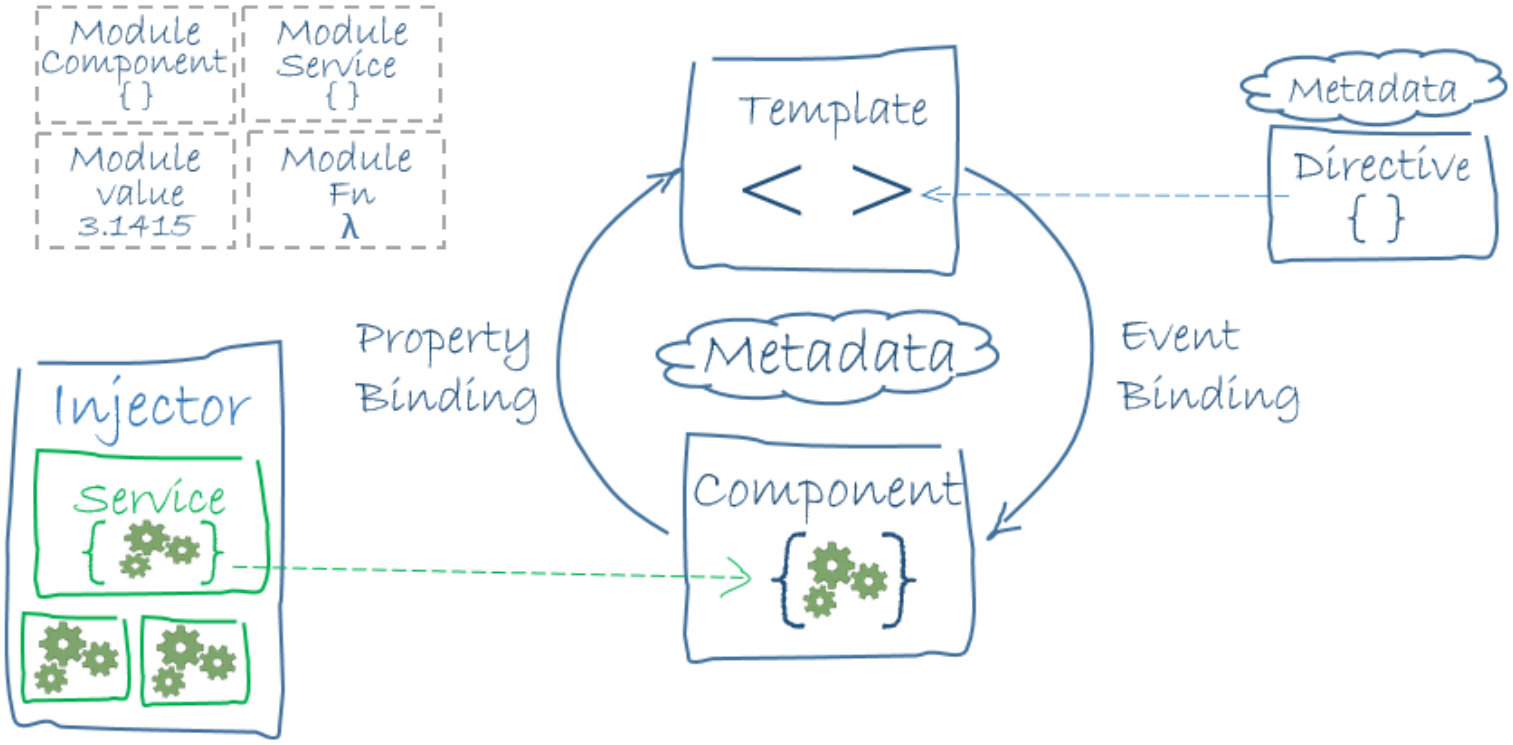
- Typen und Typableitung, strict, null und undefined: Grundlage jeder Komponente
- ?. und ??: im Template, zum Beispiel customer()?.firstname
- Funktionen, Arrow-Funktionen, filter und map: Listen im Kurs verarbeiten
- Interfaces und Destructuring: Datenmodelle und Eingaben
- Spread: neue Objekte statt Veränderung, wichtig für Signals (Reaktivität)
- Union Types und Literal-Typen: Zustände wie loading und loaded
- Generics: signal<T>(), input<T>(), Observable<T>
- Klassen und Parameter properties: customer.model.ts, Konstruktor-Injection
- Dekoratoren und Module: @Component, import und export in jeder Datei



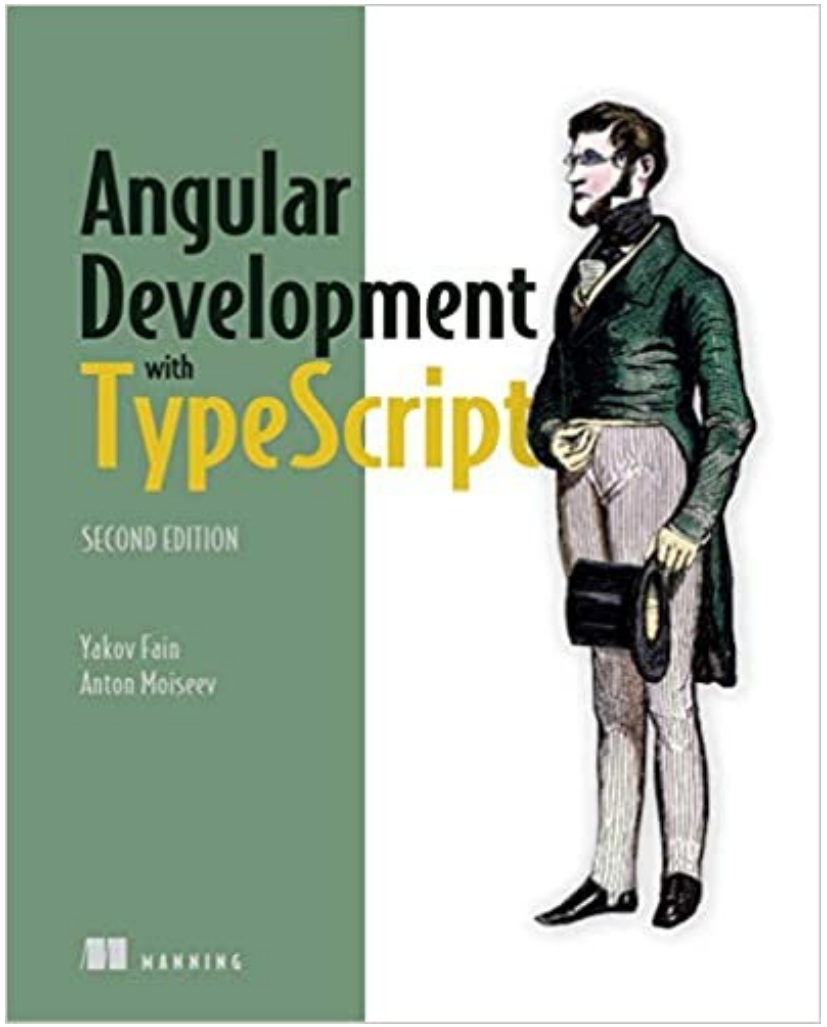
Weiter mit Übung 4

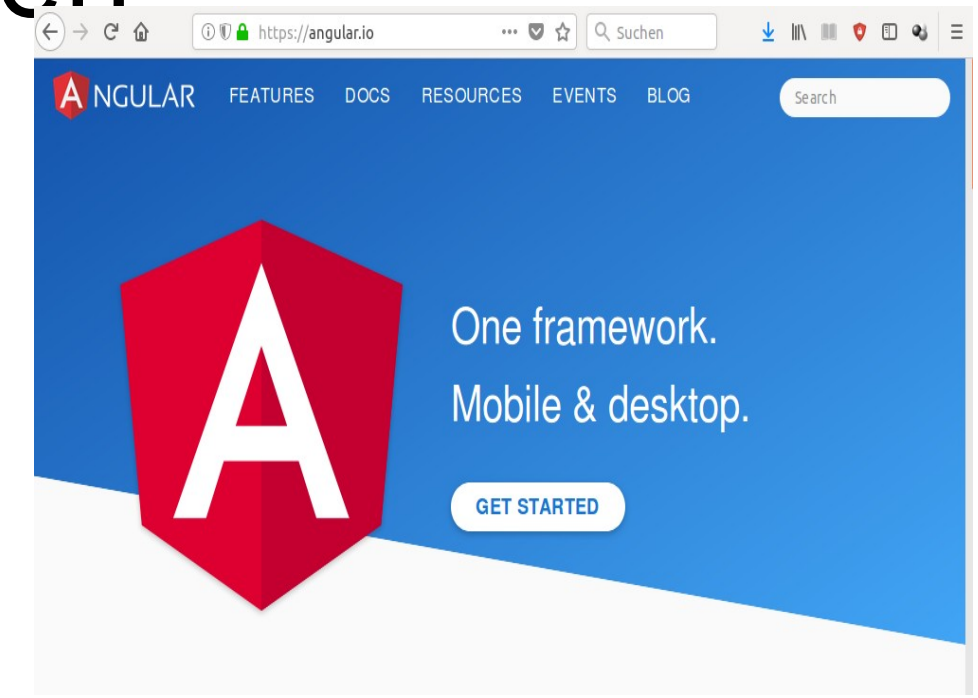

Angular

Architektur Überblick



Literatur und Quellen







Installation und Setup

JavaScript-Grundinstallation

node und npm sind auf einem Entwicklerrechner zu installieren

- Näheres hierzu im Anhang

Damit steht ein ausgefeilter Buildprozess zur Verfügung

- Verzeichnisstruktur und Projekt-Organisation
- Automatische Transpilation
- Browser-Update bei Änderungen an den Quellen

Ein spezieller Editor ist nicht notwendig

- Empfohlen wird Atom oder ähnliches

Installation des Angular Command Line Interfaces

- `npm install -g @angular/cli`
- Damit ist der Angular-Projektassistent `ng` installiert
  - Anlegen eines neuen Projekts
    - `ng new <projektname>`
  - Erzeugen neuer Angular-Programmteile
    - Services
    - Direktiven
    - …
- Freunde des Handmade Manifesto:
  - Das Anlegen eines Angular-Projektes kann natürlich auch ohne dieses Werkzeug erfolgen

Überblick über ng command line

- Development server
  - `ng serve startet einen Dev-Server (Vite/esbuild) auf http://localhost:4200/`
- Code Scaffolding
  - `ng generate component component-name (standalone ist Default)`
  - `directive|pipe|service|class|guard|interceptor|interface|enum|resolver`
- Build
  - `ng build (Production ist Default, Application Builder mit esbuild)`
- Unit Tests
  - `ng test – neue Projekte ab v21 mit Vitest, Bestandsprojekte mit Karma/Jasmine`
- End-to-End Tests
  - `ng e2e – Runner per ng add hinzufügen, z. B. Cypress oder Playwright`
- Updates
  - `ng update @angular/core @angular/cli – inkl. automatischer Migrationen`

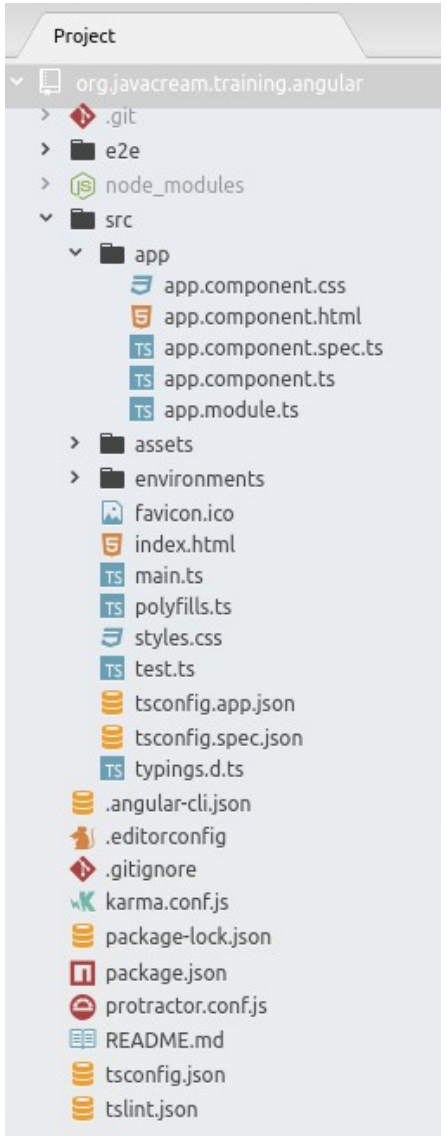


Ein erstes Projekt

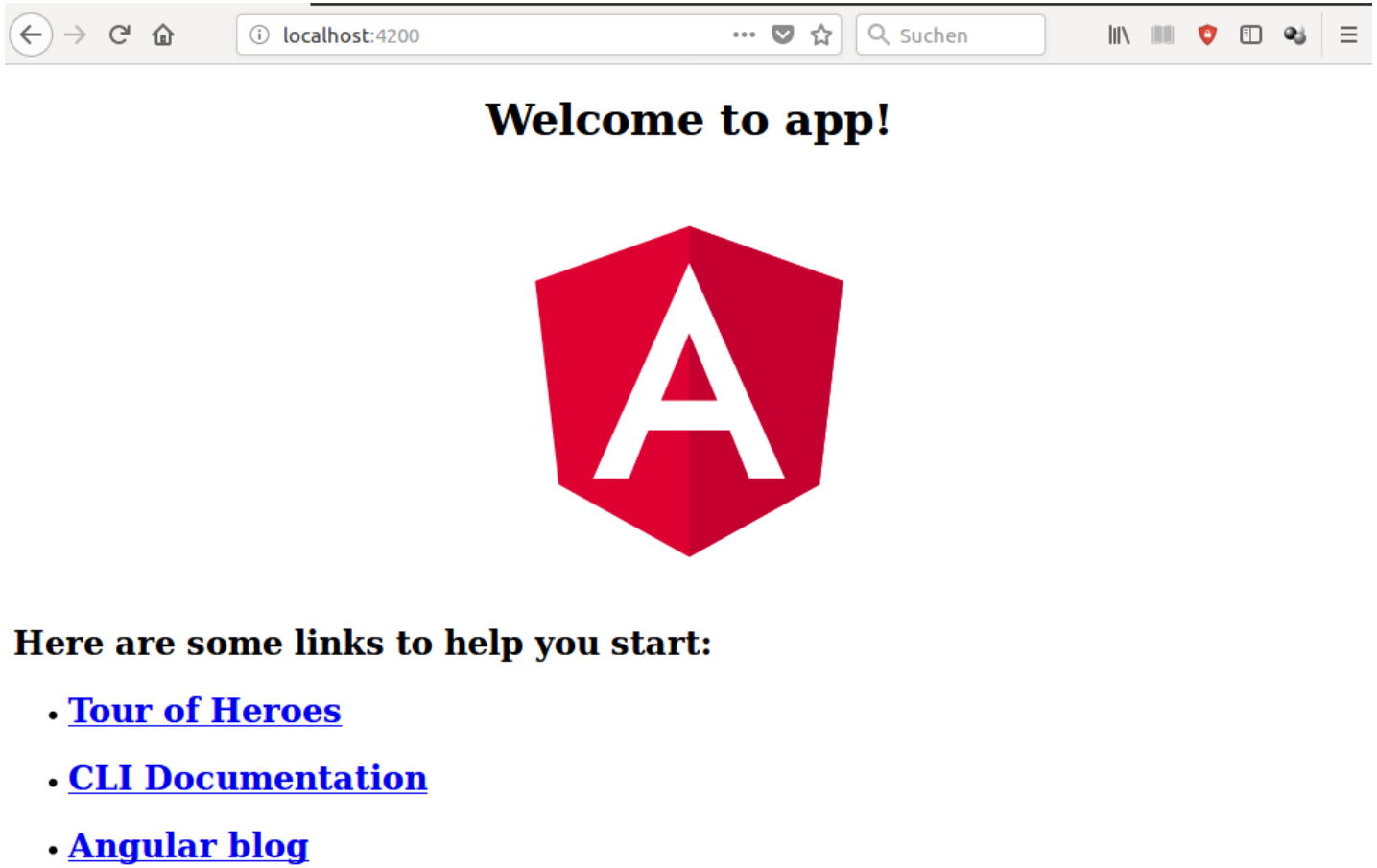
Anlegen des Projekts

- `ng new org.somepackage.angular`
- Damit wird ein node-Projekt erzeugt
- Ebenso werden automatisch alle Abhängigkeiten installiert
  - Insbesondere TypeScript

Projektstruktur



Die Anwendung unter localhost:4200



Übung 4: Erstelle Initiales Projekt

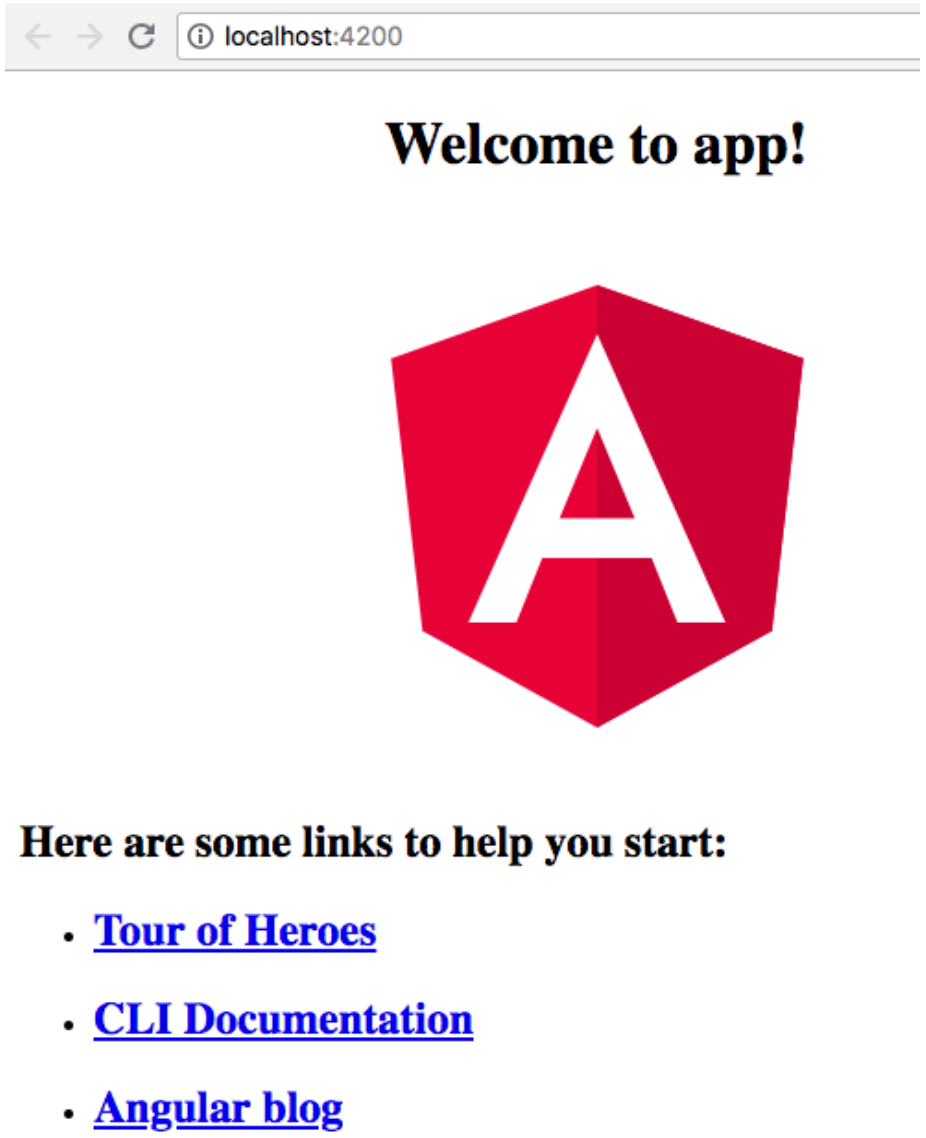
- Öffne ein Terminal
- Installiere, falls noch nicht geschehen, das Angular CLI global (siehe *Benötigte Software*)
- Navigiere zu einem Ordner in dem das neues Projekt anzulegen werden soll
- Lege eine neues Projekt an: ng new invoice-app
- Wechsel in das erzeugte Verzeichnis invoice-app
- Führe ng serve aus
- Öffne Anwendung im Browser unter *http://localhost:4200*



Reload der App im Hintergrund bei Change

- ng serve –o (observe)
- Beispiel:
  - Ändern des Seitentitels sowie der Willkommens-Nachricht
    - *src/index.html*
    - *src/app/app.component.html*
    - *src/app/app.component.ts*

Die aktualisierte Seite



Projektüberblick

Standalone Components

- Komponenten, Direktiven und Pipes sind standardmäßig standalone
  - standalone: true muss seit Angular 19 nicht mehr angegeben werden
- Kein NgModule zur Deklaration nötig
  - Jede Komponente deklariert ihre Abhängigkeiten selbst
- imports: [] direkt in der @Component-Annotation
  - Andere Komponenten, Direktiven, Pipes (z. B. RouterLink, FormsModule)
  - Für @if / @for ist kein CommonModule nötig
- Vorteile
  - Weniger Boilerplate, besseres Tree-Shaking, einfaches Lazy Loading
- NgModules: siehe Legacy-Kapitel am Ende (Übung 30)

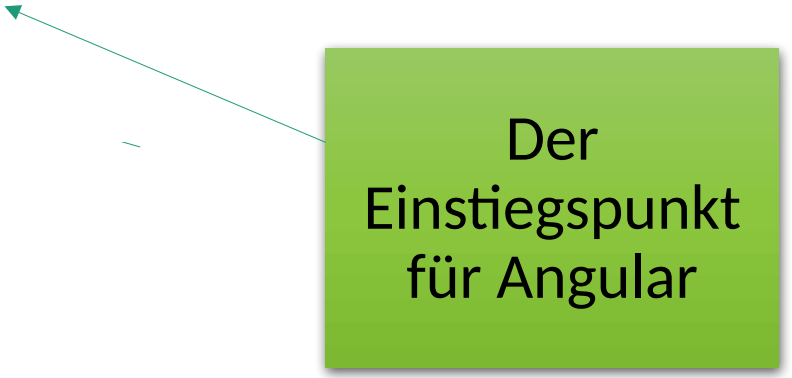
Übersicht der Dateien eines Projekts

- `src/index.html`
  - Die Hauptseite
- `src/main.ts`
  - *Start der Anwendung per bootstrapApplication – ersetzt app.module.ts. Module siehe Projekt 25 (Legacy)*
- `src/app/app.component.ts`
  - Definition des dynamischen Anteils einer Komponente mit TypeScript
- `src/app/app.component.html`
  - Definition des HTML-Templates einer Komponente
- `src/app/app.component.css`
  - Stylesheet der Komponente
- `src/app/app.component.spec.ts`
  - Definition von Unit-Tests (Vitest bzw. Jasmine/Karma)

Referenz auf Bootstrap Component in Index-Seite

index.html

```
1   <!doctype html>
2   <html lang="en">
3   <head>
4     <meta charset="utf-8">
5     <title>org.Javacream.Training.Angular</title>
6     <base href="/">
7
8     <meta name="viewport" content="width=device-width, initial-scale=1">
9     <link rel="icon" type="image/x-icon" href="favicon.ico">
10  </head>
11  <body>
12    <app-root></app-root>
13  </body>
14  </html>
15
```



Component-Annotation

- Eine Component ist eine TypeScript-Klasse mit der `@Component`-Annotation
  - Annotationen enthalten Metadaten, die der Klasse zugeordnet werden
  - Diese müssen von einem Framework interpretiert werden
- Annotationen sind damit so etwas wie statische Eigenschaften einer Klasse
  - Allerdings sind diese syntaktisch nicht Bestandteil der Klassendeklaration
  - Damit ergibt sich eine sauber Trennung
- Eine typische Component definiert
  - Den Selector als Bezug zum Template
  - Die Lokation des Templates
- Vollständige Liste unter
  - *`https://angular.io/api/core/Component`*

Aufgabe der Component

- Die Eigenschaften der Component-Klasse dienen als Schnittstelle zum HTML-Code
  - Werte werden über Interpolation oder Direktiven gebunden
  - Funktionen dienen als Event-Handler
- Bei Bedarf implementiert die Component-Klasse Lifecycle-Funktionen
  - `ngOnChanges`
  - `ngOnInit`
  - *`https://angular.io/guide/lifecycle-hooks`*

Beispiel: Component Klasse

```
import {ApplicationRef, Component, OnDestroy, OnInit} from "@angular/core";

@Component({
   selector: “app-root",
   templateUrl: "./app.component.html",
   styleUrls: ["./app.component.scss"],
   animations: [fadeAnimation]
})
export class AppComponent implements OnInit, OnDestroy {

   public title = "theFrontend";
```

TypeScript-Import der Angular-Core-Bibliothek

Referenz auf den Einstiegspunkt

Referenz auf Template und CSS

Definition einer Variablen und Wert-Zuweisung

Das HTML-Template einer Component

app.component.html

```
<!--The content below is only a placeholder and can be replaced.-->
<div style="text-align:center">
  <h1>
    Welcome to the great {{ title }}!
  </h1>
  <img width="300" alt="Angular Logo"
  src="
  3IDQwLjl6IiAvPgogIDwvc3ZnPg==">
</div>
<h2>Here are some links to help you start: </h2>
<ul>
  <li>
    <h2><a target="_blank" rel="noopener" href="https://angular.io/t
  </li>
  <li>
    <h2><a target="_blank" rel="noopener" href="https://github.com/a
  </li>
  <li>
    <h2><a target="_blank" rel="noopener" href="https://blog.angular
  </li>
</ul>
```

Referenzieren der Component-Variable mit Interpolation

Kompletter Ablauf

- Ablauf insgesamt
  - Aufruf bootstrapApplication mit Component (AppComponent)
  - Komponenten-Klasse hat Decorator
  - Komponenten-Decorator hat Feld selector
  - Selector wird in DOM aus index.html gesucht
  - Komponente wird instanziiert

Bootstrapping einer Standalone-App

```
// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideZonelessChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient()
  ]
}).catch(err => console.error(err));
// Kein AppModule mehr nötig – Legacy: siehe Übung 30
```

Einfache Erweiterungen

Mehrere Komponenten

- Auf einer Seite können auch mehrere Anwendungen dargestellt werden
- Diese sind komplett voneinander isoliert
  - Der Datenaustausch zwischen verschiedenen Komponenten wird später beschrieben

Export von Benutzerdefinierten Typen

- Eine Komponente kann auch eigene Datentypen definieren und Exportieren
  - Ein Export ist nicht notwendig, so lange der Typ nicht anderen Komponenten bekannt gemacht werden muss
- Ebenso können die Eigenschaften der Komponente komplexe Objekte sein
- Der Zugriff auf die Eigenschaften des komplexen Objekts erfolgen über die normale JavaScript-Syntax
  - Also über den `.`-Operator
  - Aber Vorsicht!
    - Die Deklarativen unterstützen nicht die komplette Syntax!
      - `if`
      - `for`
      - Deklarationen

Auflistungen

- Innerhalb der `{{}}`-Interpolation wird das `for`-Konstrukt nicht unterstützt
- Allerdings gibt es eine spezielle `forEach`-Anweisung

```
<ul>
      @for(person of people; track person.id){
       <li >
              <span>{{person.firstname}}</span> {{person.lastname}}
       </li>
      }
</ul>
```

UI-Programmierung

Data Binding

- Template nicht statisch sondern dynamisch
- Komponenten-Klasse stellt Daten bereit
  - Felder / Instanz Variablen
- Template zeigt Daten an
  - Interpolation mit {{ expression }}
  - Gibt Ergebnis der Expression als String aus
  - Zugriff nur auf Instanz-Variablen der eigenen Klasse
  - Template-Variablen

Die ngModel-Direktive

- Bisher wurden die Werte der Komponente mit der `{{}}`-Interpolation dargestellt
- Dieses Binding ist dynamisch (!)
  - Änderungen der Werte werden in der Oberfläche dargestellt
- Eigenschaften der Komponente können auch bidirektional gebunden werden
- Dazu dient die Direktive `ngModel`
  - Notwendig hierzu ist der Import von FormsModule aus @angular/forms im imports-Array der Komponente

Data Binding: Beispiel

```
import { Track } from "./../tracks/track.model";
@Component({
  selector: "music-app",
  template: `
    <h1>Music App</h1>
    Music App has {{tracks.length}} tracks to play  `
})
export class MusicAppComponent {
  public tracks: Track[];
  constructor() {
    this.tracks = [
    new Track("Sound of Silence"), new Track("I Wish"),
    new Track("Over My Shoulder"), new Track("Take Five")
   ];
 }}
```

Übung 5: Databinding

- Erstelle ein Datenmodell
  - Lege eine neue Datei src/app/customer/customer.model.ts an.
  - Erstelle und exportiere darin ein Klasse Customer mit id, firstname, lastname, street, zip, city.
  - Vergebe für jeden erzeugten Customer eine eindeutige ID.
- Erweitere AppComponent
  - Ersetze im Template den dynamischen Titel gegen statisches HTML (<h1>Invoice Manager App</h1>).
  - Importiere in der AppComponent Klasse die Klasse Customer.
  - Stelle für das Template eine Liste mit Cutomer bereit (Feld + Initializierung im Konstruktor).
  - Zeige im Template die Länge der Liste an.

Direktiven

- Allgemeines Konzept
  - Komponenten
  - Strukturelle Direktiven
  - Attribut-Direktiven
- *Direktiven* meint meist strukturelle und Attribut-Direktiven
  - Wie Komponenten Teil des UI
  - Unterschied zu Komponenten: kein eigenes Template

Direktiven

- Direktiven sind Erweiterungen von HTML, die vom Angular Framework gesucht und interpretiert werden
  - Während des Bootstrap-Vorgangs wird die Startseite der Anwendung nach Direktiven analysiert
  - Erweiterung durch
    - HTML-Elemente
    - Attribute
- Was genau beim Auftreten einer Direktive passiert hängt von der Direktiven-Implementierung ab
  - `ngModel`
    - Data Binding
  - `@for`
    - Iteration

Beispiel

```
@for (track of tracks; track track.id) {
  <li>{{track.name}}</li>
} @empty {
  <p>Keine Tracks gefunden.</p>
}
```

Direktiven des Angular-Frameworks

- Angular unterscheidet die folgenden Direktiven
  - Components sind Direktiven!
    - sie werden an ein HTML-Element gebunden
  - Structural Directives
    - Modifizieren das DOM der Anwendung
    - `for` oder `if` sind Beispiele hierfür
  - Attribute Directives
    - Ändern die Darstellung oder das Verhalten eines HTML-Elements
    - `ngStyle` als Beispiel
- Vollständige Liste unter *`https://angular.io/api?status=stable&type=directive`*

Built-in Control Flow: @if, @for, @switch

```
@if (customers.length > 0) {
  <p>{{ customers.length }} Kunden</p>
} @else {
  <p>Keine Kunden</p>
}
@for (customer of customers; track customer.id) {
  <li>{{ customer.firstname }}</li>
} @empty {
  <li>Keine Kunden gefunden.</li>
}
@switch (customer.status) {
  @case ('active') { <span>aktiv</span> }
  @default { <span>inaktiv</span> }
}
// Kein Import von CommonModule nötig, bessere Typprüfung und Performance
```

Noch ein paar Beispiele

```
@if (track.playable) {
  <button>play</button>
} @else {
  <span>nicht verfügbar</span>
}
```

Früher (Legacy, CommonModule nötig)

```
<div *ngIf="track.playable">...</div>
```

Attribut Direktiven

- Keine Veränderung der DOM Struktur

```
<form #form="ngForm">
        <input type="email" name="mail" ngModel/>
</form>
```

Übung 6

- Erweitere das Template von AppComponent
  - Zeige eine Liste aller Customer an



Child Components

Divide and conquer



Goals

- Listen Darstellung in eigene Komponente auslagern
  - Komplexere Darstellung ermöglichen
  - Wachsendes UI aufteilen
  - Wiederverwendbarkeit erreichen

Verschachtelung

- Gesamtes UI ist aus Baum von Komponenten aufgebaut
  - "Anwendung" ist Top-Level-Komponente
  - Komponenten können geschachtelt werden
- Vater- und Kind-Komponente sind isoliert
  - Kein Zugriff auf Daten des anderen
  - Daten müssen an Kind übergeben werden

Komponente als Direktive einbinden

- Komponenten sind auch Direktiven (eine Art)
- Direktiven (alle Arten) müssen bekannt gemacht werden
  - Per imports: [] in der @Component-Annotation
- Nur bekannt gemacht Direktiven werden ausgeführt

Mittels ng-generate

- Angular CLI legt neue Komponenten an
  - Klasse + alle benötigte Annotation
  - Html-Template
  - Leeres Styling
  - Test
- ng generate component *packetname/component-name*

Übung 7: Komponenten schachteln

- Erstelle eine Komponente CustomerListComponent
  - Generieren per ng generate component customer/customer-list
- Binde die neue Komponente ein
  - Importiere CustomerListComponent im imports-Array von AppComponent
  - Verwende CustomerListComponent im Template von AppComponent

Input einer Component

- Components können über das Template verschachtelt werden
  - Innerhalb des Templates wird ein Root-Element einer weiteren Angular-Component definiert
- Der Kind-Controller kann Input-Parameter definieren
  - Dazu wird eine Eigenschaft mit der `input`-Signal versehen
- Diese werden im Template gesetzt

Beispiel: Input einer Child-Component

```
<ul>
  @for(person of people; track person.id) {
    <li><people-detail
[person]="person"></people-detail></li>
  }
</ul>
```

```
@Component({
  selector: 'people-detail',
  imports: [],
  templateUrl: './people-detail.html',
  styleUrl: './people-detail.css',
})
export class PeopleDetail {
  person = input.required<Person>();

}
```

```
<h3>{{person().firstname}}
{{person().lastname}}</h3>
```

AppComponent:
Neues Root-Element für die Kind-Component

PeopleDetail Component: Angabe des Root-Elements

Deklaration einer Input-Eigenschaft

Inputs

- Daten können an Komponente übergeben werden
  - *Property Binding* über [propertyName]="expression"
  - [propertyName] Property in der "inneren" Komponente *release-card*
  - expression Property in der "äußeren / aufrufenden" Komponente
- Data-Binding in Angular immer unidirektional
  - Objekt veränderbar
  - Referenz bleibt außen immer gleich

Übung 8: Inputs

- Erweitere die Komponente CustomerListComponent
  - Nimm als Input ein Array von Customer Objekten entgegen
  - Verschiebe die bisherige Darstellung aus AppComponent in das Template von CustomerListComponent
    (Text und Liste, nicht die h1 Überschrift)
- Passe die Verwendung von CustomerListComponent in AppComponent an
  - Lösche den entsprechenden Teil des Templates
  - Übergebe app-customer-list die Daten per Property-Binding

Events

Output aus Components

Event Binding

- Data-Binding unidirektional
  - Daten von außen nach innen
  - Wie kommen Informationen von innen nach außen?
- Komponenten können Events nach außen geben
  - *Event Binding* mit (eventName)="statement"
  - Zuweisung oder Funktionsaufruf
  - Zugriff auf das Event immer über $event möglich

Event Handler

- Im Gegensatz zum Standard-Eventmodell im Browser können beliebige Funktionen mit Parametern als Handler aufgerufen werden
- Die Event-behandelnde Funktion ist Bestandteil der `Component`

Event Handler: Beispiel

```
export class UiAppComponent {
  //…
  selectionList: string[] = ["Item1", "Item2", "Item3"];
  setItem(item: string) {
    this.data.boundData = item;
  }
}
<ul>
  @for (item of selectionList; track item) {
    <li (click)="setItem(item)">{{item}}</li>
  }
</ul>
```

Übung 9: Events

- Reagiere in CustomerListComponent auf Klicks auf die Listen-Elemente
  - Rufe eine Methode (listItemClicked) in der Component-Klasse auf
  - Übergebe das Dom-Event ($event vom Typ MouseEvent) und das Training des geklickten Listen-Eintrags
  - Logge den Namen des Customer und das Event auf die Konsole
    z.B. mit console.log(event, customer.firstname)

Output -- Events aus Komponenten

- Komponenten können Events nach außen werfen
- Eigenschaft wird mit der Funktion output<T>() angelegt
  - Typ-Parameter gibt Typ des Events an
  - Rückgabetyp ist OutputEmitterRef<T>
- Events werden über emit(wert) gefeuert
- Legacy: @Output() mit new EventEmitter<T>()

Output Beispiel

```
import { Component, input, output } from "@angular/core";
@Component({
  selector: "release-card", template: `<h3>{{release().name}}</h3>`
})
export class ReleaseCardComponent {
  release = input.required<Release>();
  // we use domain object as event, real world: ReleaseLikedEvent
  releaseLiked = output<Release>();
  likeRelease() {
    this.releaseLiked.emit(this.release());
  }
}
```

Nutzende Komponente

```
<div class="releases">
        @for (release of artist.releases; track release.id) {
    <release-card
        [release]="release"
        (releaseLiked)="onReleaseLiked($event, release)">
    </release-card>
    }
    </div>
```

Methode im aufrufendem Controller
NICHT in der Release-Card Komponente

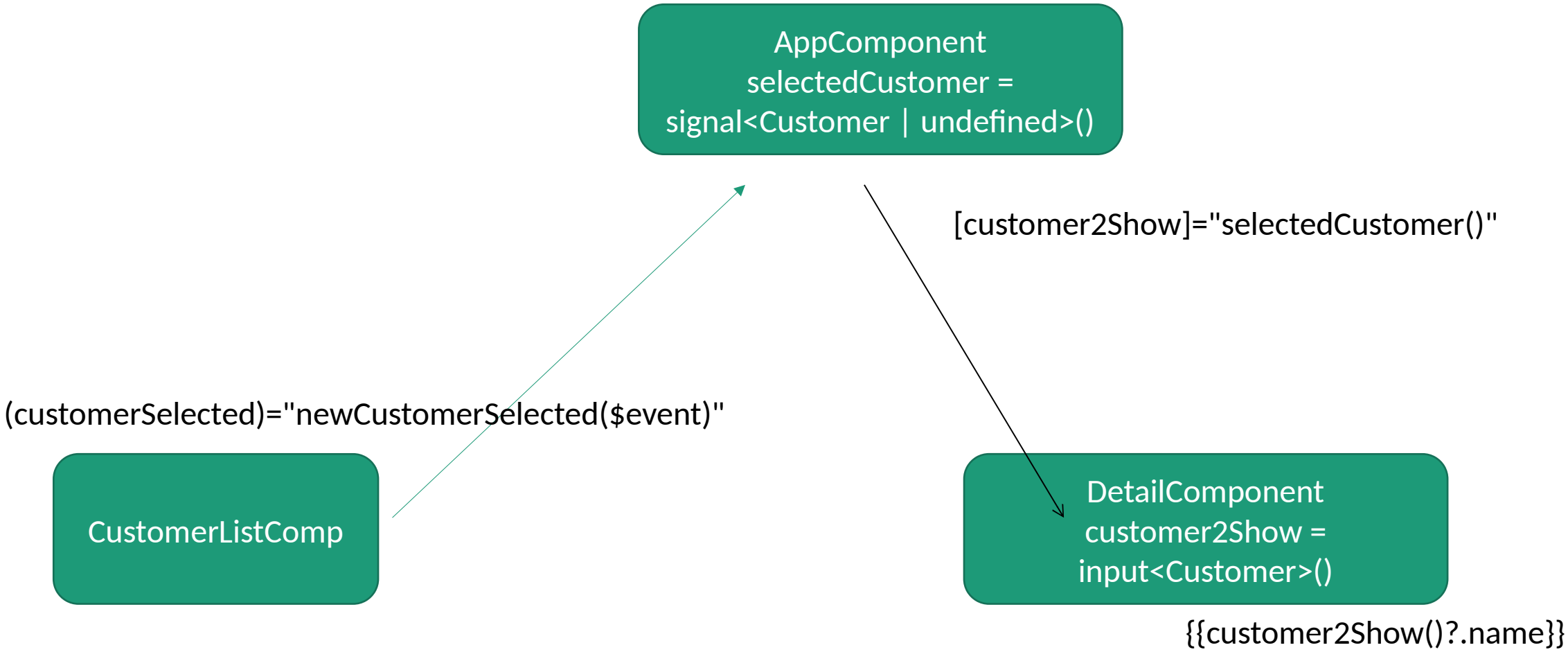
Lokaler State mit Signals

```
import { Component, computed, signal } from '@angular/core';
export class AppComponent {
  selectedCustomer = signal<Customer | undefined>(undefined);
  hasSelection = computed(() => this.selectedCustomer() !== undefined);

  newCustomerSelected(customer: Customer) {
    this.selectedCustomer.set(customer);   // oder .update(old => ...)
  }
}
<!-- Template: Signals werden wie Funktionen gelesen -->
<customer-details [customer]="selectedCustomer()" />
// Signals + zoneless: Angular weiß genau, was sich geändert hat
```

Übung 10: Output

- Erweitere CustomerListComponent
  - Erstelle in CustomerListComponent ein Output customerSelected = output<Customer>()
  - Emitte in CustomerListComponent#listItemClicked customerSelected ein Event
  - Erstelle in AppComponent ein Event-Binding für customerSelected an CustomerListComponent
  - Speichere in AppComponent den zuletzt selektierten Customer in einem signal()
- Erstelle eine Komponente CustomerDetailsComponent
  - Generieren per ng generate component customer/customer-details
  - Erstelle einen Signal-Input customer = input<Customer>() und zeige im Template dessen Properties an
  - Importiere CustomerDetailsComponent im imports-Array von AppComponent
- Verbinde die beiden Komponenten in AppComponent
  - Binde CustomerDetailsComponent in AppComponent ein (parallel zu CustomerListComponent)
  - Reiche das selektierte Customer an CustomerDetailsComponent weiter ([customer]="selectedCustomer()")
  - Zeige in AppComponent die Detail-Ansicht nur an, wenn ein Customer selektiert wurde (NPE vermeiden)


AppComponent
selectedCustomer =
signal<Customer | undefined>()
[customer2Show]="selectedCustomer()"
(customerSelected)="newCustomerSelected($event)"
CustomerListComp
DetailComponent
customer2Show =
input<Customer>()
{{customer2Show()?.name}}

Theorie: Warum Komponenten aufteilen?

- Eine Komponente sollte genau einen Grund zur Änderung haben
- Zu groß: schwer zu verstehen, schwer zu testen, jede Änderung ist riskant
- Zu fein: viele Dateien, viel Durchreichen von Daten, der Überblick geht verloren
- Ziel: so teilen, dass Änderungen lokal bleiben
- Dieser Teil ist reine Theorie
- Praxis dazu: Übungen 7 bis 10 (Komponenten schachteln, Inputs, Events, Outputs)

Theorie: Kriterien für die Aufteilung

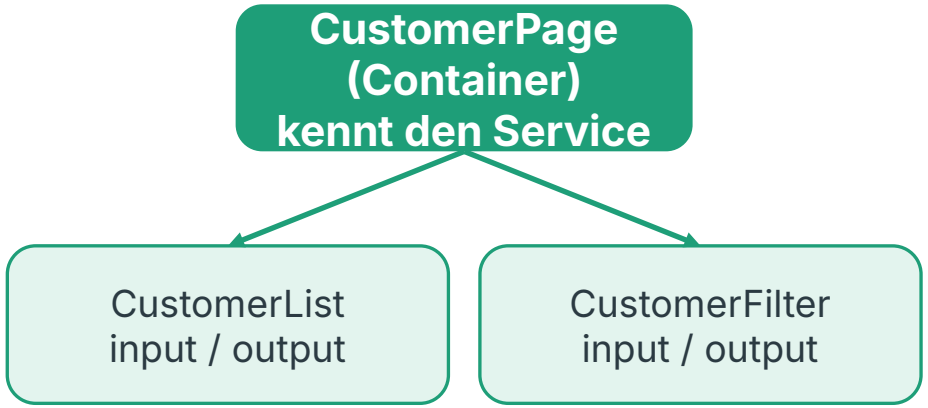
- Verantwortung: mehr als eine Sache? Beispiel: Liste lädt Daten, filtert und zeigt an
- Wiederverwendung: Taucht das Muster mehrfach auf? Beispiel: CardComponent
- Änderungsgrund: Ändern sich Teile aus verschiedenen Gründen? Beispiel: Filter und Anzeige
- Lesbarkeit: Passt das Template auf einen Bildschirm? Warnzeichen: über 100 Zeilen
- Testbarkeit: Braucht ein Test viel Aufbau für wenig Prüfung? Beispiel: 5 Services
- Rendering: Soll ein Teil anders aktualisiert werden? Beispiel: Kopfbereich und 1000 Zeilen

Theorie: Typische Fehlformen

- God Component: alles in einer Komponente (Daten, Logik, Layout)
- Prop Drilling: Daten werden durch viele Ebenen durchgereicht
  - Beispiel: 4 Ebenen, obwohl nur die letzte die Daten braucht
  - Besser: Service oder Content Projection
- Zu feine Teilung: eine Komponente pro span-Element
- Verdeckte Kopplung: Kind greift auf Eltern zu, per inject(ParentComponent) oder über das DOM

Theorie: Container und Darstellungskomponenten

- Container („smart“): kennt Services, lädt Daten, ist selten wiederverwendbar
  - Eingänge: Route und Services; Ausgänge: Aufrufe an Services
  - Test: Service durch einen Mock ersetzen
- Darstellung („dumb“): kennt keine Services, nur input() und output()
  - Oft wiederverwendbar
  - Test: Inputs setzen und Outputs prüfen



Theorie: Darstellungskomponente im Beispiel

```
// Darstellungskomponente: kennt weder Service noch Router
@Component({
  selector: 'customer-filter',
  template: `
    <input [value]="value()"
           (input)="changed.emit($any($event.target).value)">
  `
})
export class CustomerFilterComponent {
  value = input('');           // Eingang: aktueller Filtertext
  changed = output<string>();   // Ausgang: neuer Text nach einer Eingabe
}
```

Theorie: Schnittstellen sauber gestalten

- input.required<T>() für Pflichtdaten, statt im Template auf undefined zu prüfen
- Typen exportieren (z. B. Customer), kein any
- Outputs als Ereignis benennen: customerSelected statt onClick
- Eingaben im Kind nicht verändern; Änderungen als Output melden (oder model() bewusst nutzen)
- Kleine Schnittstelle: lieber 3 klare Inputs als ein riesiges Konfigurationsobjekt
- Kein Zugriff auf Interna des Kindes; wenn nötig, öffentliche Methode oder viewChild()

Theorie: Checkliste „Soll ich aufteilen?“

- 1. Kann ich in einem Satz sagen, was die Komponente tut – ohne das Wort „und“?
- 2. Gibt es Teile, die ich an anderer Stelle wiederverwenden würde?
- 3. Ändern sich Teile aus unterschiedlichen Gründen?
- 4. Ist die Schnittstelle zur Eltern-Komponente klein und typisiert?
- 5. Kann ich die Komponente isoliert testen?



Expression Language

Was wir fast alles schon kennen

Interpolation	{{ expression }}
Property Bindings	[propertyName]="expression"
Event-Bindings	(eventName)="statement"
Two-Way-Data-Binding	[(ngModel)]="assignable expression"
In Control Flow	@if(expression)

Expression

Template Operatoren

- Pipe Operator |
- Elivs Operator als Safe Navigation ?.

Evaluation Kontext

- Globaler Namensraum nicht zugreifbar
- this --> Instanzvariablen der @Component Klasse
- Eigene Template Variablen

Was geht nicht

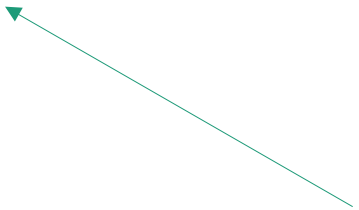
- new Operator
- Bit-Operatoren | und &
- Inkrement bzw. Dekrement

Best Practice

Halte Expressions **einfach und lesbar**
Komplexe Logik → in den Component (am besten als **computed Signal**)

Safe Navigator

- {{bean?.name}}



Falls bean null oder undefined dann keine Exception sondern Leerstring

- Auch mehrfach
  - {{bean?.complexProperty?.name}}



Performanc e Impact

Schlecht, wird bei jeder Change Detection gecalled

```
@if (isUserActive(user)) { ... }
```

Besser

```
// Im Component
activeUser = computed(() =>
this.isUserActive(this.currentUser()));
// Im Template
@if (activeUser()) { ... }
```

One Way Property Bindings bei Images

```
<img [src]="release.cover">
<button [disabled]="!track.playable"></button>
```

Event Binding

- One-Way nach oben im Komponenten-Baum
- $event als Handle auf das Event-Objekt
- Ziele
  - Standard-DOM-Events
    - <input type="text" (keyup)="logInput($event.target.value)">
  - Custom-Events an Komponenten
    - <release (releaseLiked)="highlight($event)"></release>

Style Bindings

- Class-Bindings
  - <div class="selected"
  - <div [class.selected]="isSelected(track)">
- Style-Bindings
  - <img [style.visibility]="release.cover ? 'visible' : 'hidden'">
- Alternativ: [class]="{ selected: isSelected(track) }" bzw. [style]-Bindings (ngClass/ngStyle nicht mehr nötig)

Data Binding in Formularen

- Ziel:
  - Formularfelder an Properties binden
- Manuelles Synchronisieren würde bedeuten
  - <input [value]="bean.name"
  - (input)="bean.name=$event.target.value">
- Zwei-Wege-Data-Binding mit ngModel
  - <input [(ngModel)]="bean.name">

- *Merke: Banana in a Box [()]*

Deferrable Views: @defer

```
@defer (on idle) {
  <customer-details [customer]="selectedCustomer()" />
} @placeholder {
  <div>Lade Kundendetails...</div>
} @loading (minimum 500ms) {
  <p>Lädt...</p>
} @error {
  <p>Fehler beim Laden</p>
}
// Trigger: on idle | on viewport | on interaction | on hover | on timer(2s) | when bedingung
// Die Komponente landet in einem eigenen Bundle und wird erst bei Bedarf geladen
```

Übung 11: Bindings auf Images und mehr

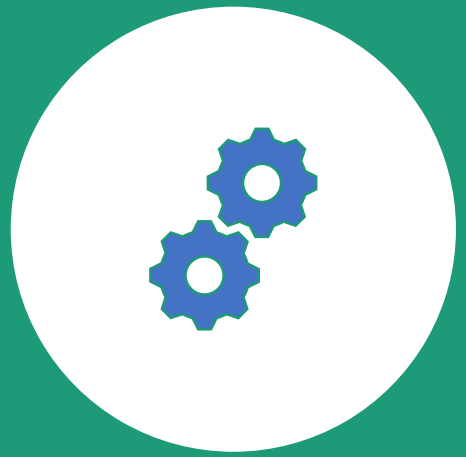
- Erweitere das Datenmodell
  - Füge der Klasse Customer eine optionale Eigenschaft imageUrl: string hinzu
  - Nehme imageUrl im Konstruktor als Parameter entgegen und setzte das Feld
- Passe AppComponent an
  - Kopiere aus der Musterlösung (11_image) die Bilder aus src/assets/images/customer
  - Setze beim Erzeugen der Testdaten bei manchen Customer ein Bild.
    Pfad z.B. assets/images/customer/carl.svg
- Erweitere die Darstellung in CustomerDetailsComponents
  - Zeige das Bild mit einer festen Größe an, falls vorhanden
  - Blende das Bild aus, falls kein Bild vorhanden
  - Optional: Lade CustomerDetailsComponent in AppComponent per @defer (on idle) mit @placeholder

Übung 12: Class Binding

- Erweitere das Datenmodell
  - Lege in Customer ein Feld discontinued: boolean an, das per Konstruktor gesetzt wird
  - Erzeuge in AppComponent mindestens ein Customer, an den nichts mehr geliefert wird
  - Setze für diesen Customer discontinued auf true
- Passe die Darstellung in CustomerListComponent an
  - Setze an nicht länger gewünschte Customer eine Klasse
  - Stelle nicht länger gewünschte Customer anders dar
    (über Klasse an Element und CSS in customer-list.component.css, z.B. ausgegraut oder durchgestrichen)
- Hebe in der Liste das selektierte Customer hervor
  - Speichere in CustomerListComponent den zuletzt selektierten Customer in einem signal()
  - Prüfe am <li> ob der aktuelle Customer das selektierte ist und setze ensprechend eine Klasse

Übung 13: Formulare

- Biete in der Detail-Ansicht die Möglichkeit an, den Namen eines Customers zu verändern
  - Importiere FormsModule aus @angular/forms im imports-Array von CustomerDetailsComponent
  - Erstelle ein input mit ngModel



Services

Eigenschaften eines Services



Services sind TypeScript-Klassen



Diese werden ausschließlich vom Angular-Framework instanziert

Diese Aufgabe übernimmt der Angular-Context



Benötigt eine Component den Zugriff auf einen Service, so wird dieser automatisch gesetzt

Dieses Verfahren heißt Dependency Injection



Insgesamt stellt Angular mit dem Service-Mechanismus ein Dependency-Injection-Framework zur Verfügung

Services? Was

Sehr allgemeines Konzept

- Irgendwelche Funktionalität kapseln
- Nicht UI spezifisch
- Implementierungsdetails verstecken

Einsatzmöglichkeiten

- Höhere Abstraktion schaffen (REST)
- Integration anderer Bibliotheken (WebSocket)
- Datenhaltung, Datenzugriff, Caching, Logik, ...

Können Abhängigkeit zu anderen Services haben

Aufgaben eines Services

- Standard-Services des Angular-Frameworks stellen nützliche Routinen zur Verfügung
  - Beispielsweise wird die Client-Server-Kommunikation über den `HttpClient`-Service realisiert
- Eigene Services werden in der Anwendung häufig benutzt, um in einer sauberen Architektur ein Datenmodell zu definieren
  - Dieses wird dann von allen Components konsistent benutzt
  - Services ermöglichen damit eine Inter-Component-Kommunikation, die unabhängig von Input-Parametern sind
    - Wesentlich leichter wartbar

Ein simpler Service

- Die Erzeugung eines Services kann das Angular-CLI übernehmen
  - ng generate service <ServiceName>
- Ein Service selbst ist eine @Injectable-Klasse
  - @Injectable({ providedIn: 'root' }) – anwendungsweiter Singleton, tree-shakeable
- Nutzung in jeder anderen Angular-Klasse per inject() (oder Konstruktor)
  - Components
  - Andere Services, Guards, Interceptors
- Alternativen zur Registrierung
  - providers in bootstrapApplication (main.ts)
  - providers der @Component-Annotation
  - https://angular.dev/guide/di

Methode in Services

```
import {Artist} from "./artist.model";


export class ArtistService {
    getAllArtists(): Artist[] {
            return [];
    }
    }
```

API Stabilität

- Service versteckt Details über Datenhaltung vor Komponenten
- API stabil, Datenquelle austauschbar
- Best Practice: Mock-Daten parallel zu Service in eigene Datei ablegen

```
import {Artist} from "./artist.model";
import {ARTISTS} from './artist.mock';

export class ArtistService {
    getAllArtists(): Artist[] {
        return ARTISTS;
    }
}
```

Objektnetze in OO- Systemen

- Aufbau der Netze komplex.
- Testbarkeit der Komponenten wird eingeschränkt.
- Änderbarkeit und Wartbarkeit nur bedingt gegeben.
- Favor Delegation over inheritance.
- Verschiedene Möglichkeiten Netze aufzubauen.

Assignment by Third Party

```
public static void main(String[] args)
{
    MyService service = new MyService();
    service.setDAO(new MyDAO());
}
```

Creating itself - DIRECT

```
public MyService()
{
      this.myDao = new MyDao();
}
```

Factory- Pattern

```
public MyService()
{
  this.myDao = Factory.getMyDao();
}
```

Dependency Injection

- Objekte werden zur Laufzeit „injiziert“.
- Objekte werden passiv, statt sie zum Beispiel mit einer Factory aktiv zu erzeugen.
- Abhängigkeiten sind explizit an der Schnittstelle durch entsprechende set-Methoden zu erkennen.
- Es können beliebige andere Objekte zugewiesen werden(z.B. Mocks / an Bedürfnisse angepasste Objekte).
- Sehr flexibel wenn Objekte nur noch von Interfaces abhängen.
- Externe Konfiguration der Beziehungen zwischen Objekten.
- Objekte bekommen ihre abhängigen Objekte statt sie selbst zu suchen.

Beispiel: Service

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable()
4  export class PeopleModelService {
5
6    constructor() { }
7    people: Person[] = [new Person("Gärtner", "Hans")
8
9  }
```

@Injectable macht diese Klasse dem Angular-Context bekannt

```
providers: [
  PeopleModelService
],
```

Im Modul oder der `@Component`-Annotation

```
export class ServiceAppComponent {
  constructor(readonly peopleModel: PeopleModelService ){

  }
  people:Person[] =  this.peopleModel.people
}
```

Dependency Injection in einer Component

DI Konfig

- Woher weiß Angular was zu injizieren ist?
- Muss konfiguriert werden: Providers
  - Injector → Provider → Instance
- Konfiguration direkt am Service

```
@Injectable({ providedIn: 'root' })
export class TrackService { }
```

- oder global in main.ts

```
bootstrapApplication(AppComponent, { providers:
[ TrackService ] });
```

- Wie sieht ein Provider aus?

Wie geht DI in Angular

- Direkte Abhängigkeit
  - einfach nur die Klasse angeben: [ConsoleLogger]
  - Kurz für [{ provide: ConsoleLogger, useClass: ConsoleLogger}]
- Allgemein: Provider-Objekt
  - provide: Unter welchem Namen steht es zur Verfügung → Klasse oder InjectionToken
  - Ein Rezept, wie es erzeugt werden soll → useClass, useValue, useFactory oder useExisting

```
bootstrapApplication(AppComponent, {
  providers: [
    { provide: ConsoleLogger, useClass: ConsoleLogger }, // same as shortcut
    { provide: Logger, useExisting: ConsoleLogger } // different token, same instance
  ]
});
```

Moderne Dependency Injection mit inject()

```
import { Component, inject } from '@angular/core';

@Component({ selector: 'app-root', templateUrl: './app.component.html' })
export class AppComponent {
  private readonly customerService = inject(CustomerService);
  customers = this.customerService.getAll();
}
// Früher: constructor(private customerService: CustomerService) {}

// inject() funktioniert auch in Funktionen (Injection Context):
export const authGuard: CanActivateFn = () ⇒ inject(AuthService).isLoggedIn();
// Vorteile: weniger Boilerplate, einfache Vererbung, funktionale Guards & Interceptors
```

Kontextbasierte Injektion

- Injektoren bilden einen Baum
  - Jede Komponente ein Child-Injector vom Injector der Vater-Komponente
  - Instanzen sind Singletons in einem Injektor
- Provider für Child-Injektoren können an Komponenten definiert werden
  - Child-Injector kann Provider des Vaters überschreiben / ergänzen
  - providers Property in @Component Decorator
- Einsatzmöglichkeiten
  - Auflösen von Namenskonflikten
  - Unterschiedliche Konfiguration (z.B. Log-Level)

Übung 14

- Erstelle einen Service CustomerService
  - Generieren per ng generate service customer/customer
- Erstelle Mock-Daten
  - Erstelle eine Datei src/app/customer/customer.mock.ts
  - Exportiere darin ein Array von Customer (kopieren aus AppComponent)
- Verwende im Service die Mock-Daten
  - Importiere die Mock-Daten aus customer.mock
  - Erstelle in CustomerService eine Methode getAll(): Customer[]
  - Gebe darin die Mock-Daten zurück
- Verwende den Service noch nicht!

Übung 15

- Lasse CustomerService in AppComponent injizieren
  - Stelle den CustomerService über @Injectable({ providedIn: 'root' }) zur Verfügung (Default von ng generate)
  - Injiziere CustomerService in AppComponent per inject(CustomerService)
- Verwende CustomerService in AppComponent um die Daten zu beziehen
  - Lösche den Code zum Erzeugen der Test-Daten in AppComponent
  - Rufe CustomerService#getAll auf um Daten zu bekommen

Komponenten Lebenszyklus

- Komponenten haben einen Lebenszyklus
  - **Aufruf des Konstruktors**
  - **ngOnChanges**
  - **ngOnInit**
  - ngDoCheck
  - ngAfterContentInit
  - ngAfterContentChecked
  - ngAfterViewInit
  - ngAfterViewChecked
  - [ngDoCheck und ggf. ngOnChanges]*
  - **ngOnDestroy**

Was soll das?

- Bestimmte Informationen stehen erst später zur Verfügung
  - ID des Trainings beim Routing
  - Kind-Komponenten
- Instanziierung und Initialisierung trennen
  - Gut für Testbarkeit

- Achtung: keine Lifecycle-Hooks für Services

Best Practices: OnInit

```
import { Component, OnInit, inject } from "@angular/core";
@Component({
  selector: "music-app", template: `<h1>Music App</h1>`,
  providers: [ ArtistService ]
})
export class MusicAppComponent implements OnInit {
  private readonly service = inject(ArtistService);
  public artists: Artist[] = [];
  ngOnInit() {
    this.artists = this.service.getAll();
  }
}
```

Übung 16: OnInit

Initialisiere die Daten in AppComponent im ngOnInit Hook statt im Konstruktor

- Implementiere das Interface OnInit aus @angular/core
- Verschiebe das Laden der Daten aus dem Konstruktor in die Hook-Methode

Pipes

Daten für UI aufbereiten

Allgemein

- Daten-Transformation für das UI
  - Kleine, wiederverwendbare Operationen für das UI
  - An Funktionale Programmierung angelehnt
    - Daten und Konfiguration per Parameter
    - Pipe verändert ursprünglichen Wert nicht
- Pipes werden an Expression mit | angehängt
  - {{ *property* | json }}
- Pipes können per : konfiguriert werden
  - {{ release.releaseDate | date:'dd.MM.yyyy' }}
- Pipes können verkettet werden
  - {{ release.releaseDate | date | uppercase }}

Was sind Pipes?

- Pipes formatieren das Ergebnis beispielsweise einer Interpolation
  - `{{dateOfBirth | date}}`
    - Hier wird das Geburtsdatum als Datum formatiert dargestellt
  - Pipes sowie der Operator `|` sind aus der Linux-Welt übernommen
- Vordefinierte Pipes umfassen
  - `date[:format[:timezone[:locale]]]`
  - `currency[:currencyCode[:display[:digitInfo[:locale]]]]`
  - `lowercase`
  - Vollständige Liste und Dokumentation unter *`https://angular.io/api?status=stable&type=pipe`*

Decimal Pipe

- Formatiertung von Zahlenwert
- Parameter: Anzahl Nachkommastellen

  {{ track.size | decimal:2 }}

Date Pipe

- Für Datum-String, Date-Object, Timestamp-Number
- Nimmt als Parameter Shortcut (z.B. short) oder Format-String entgegen
- Kurzschreibweise: Zusammenspiel mit locale des Browsers

```
{{ release.releaseDate | date:'short' }}
{{ release.releaseDate | date:'dd.MM.yyyy' }}
```

Übung 17: Pipe

- Erweitere das Datenmodell
  - Lege in Customer ein Feld / Konstruktor-Parameter lastOrderDate vom Typ Date an
- Passe die Mock-Daten an
  - Setze für jeden Customer ein unterschiedliches Datum (new Date(year, month - 1, day))
  - Für mind. 1 Customer weniger als 1 Woche zurück
  - Setze für nicht länger gewünschte Customer natürlich ein Datum in der weiteren Vergangenheit
- Erweitere das Template von CustomerDetailsComponent
  - Gib das Datum lastOrderDate formatiert aus
  - Probiere verschiedene Formattierungen aus (Pattern, Shortcuts)

Pipes sind Klassen

- Pipes sind Klassen
  - Implementieren Interface PipeTransform aus @angular/core
  - Werden mit @Pipe Decorator aus @angular/core markiert

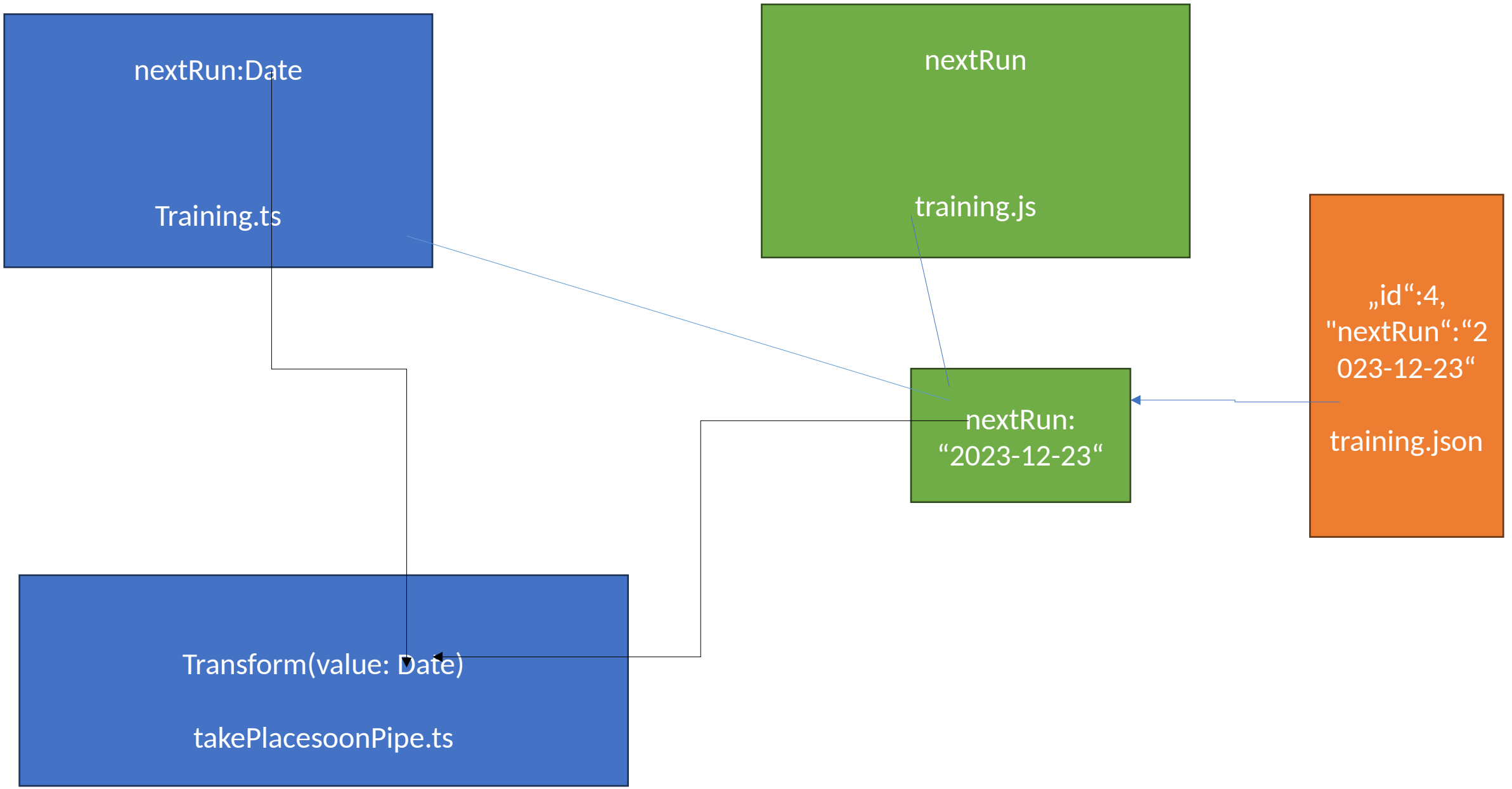
```
import {Pipe, PipeTransform} from "@angular/core";
@Pipe({ name: "hasLikes" })
export class HasLikesPipe implements PipeTransform {
        transform(withLikes: {likes :number}): boolean {
        return withLikes.likes > 0;
        }
}
```

Verwendung von Pipes in Komponenten

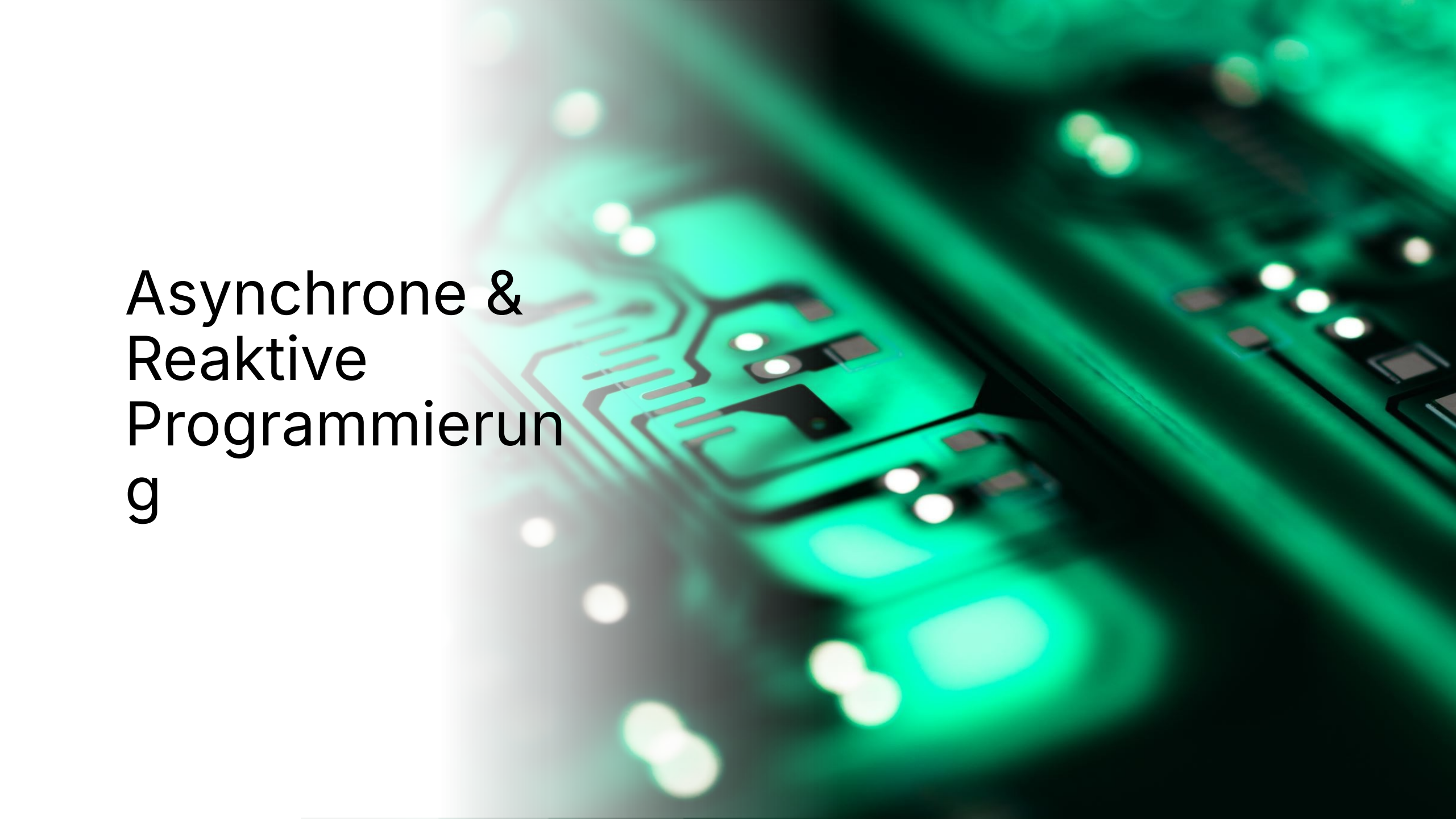
```
import { DatePipe } from '@angular/common';
import { HasLikesPipe } from './has-likes.pipe';
@Component({
  selector: 'release-card',
  imports: [ HasLikesPipe, DatePipe ],
  template: `@if (release() | hasLikes) { <span>❤</span> }`
})
export class ReleaseCardComponent { ... }
// Pipes sind standalone – Import direkt in der verwendenden Komponente
// Legacy: declarations/exports im NgModule (siehe Anhang)
```

Übung 18: Eigene Pipe

- Schreibe eine eigene Pipe currentActiveCustomers
  - Generieren per ng generate pipe customer/current-active-customers-pipe
  - Implementiere PipeTransform mit der Methode transform(date: Date, daysInPast = 7): boolean
  - Gib true zurück, falls der Cutomer innerhalb der letzten x Tage bestellt hat (Parameter soon)const timeDiff = Date.now() - date.getTime().
  - const maxTimeDiffInMS = daysInPast * 1000 * 60 * 60 * 24;
  - const isActive = timeDiff < maxTimeDiffInMS;
- Wende die currentActiveCustomers Pipe auf das lastOrderDate Datum in CustomerDetailsComponent an (Pipe im imports-Array importieren)
  - Verpacke die Ausgabe des lastOrderDate Datums in einen div-Container
  - Setze an diesem div eine Klasse, wenn der Customer aktiv ist ([class.active]="...")
  - Übergebe z.B. 5 (Tage) als Parameter daysInPast an currentActiveCustomers
  - Stelle über CSS mithilfe der gesetzten Klasse Customer, die active sind, anders dar (z.B. rote Schrift)


nextRun:Date
Training.ts
nextRun
training.js
„id“:4,
"nextRun“:“2
023-12-23“
training.json
nextRun:
“2023-12-23“
Transform(value: Date)
takePlacesoonPipe.ts

Asynchrone & Reaktive Programmierung



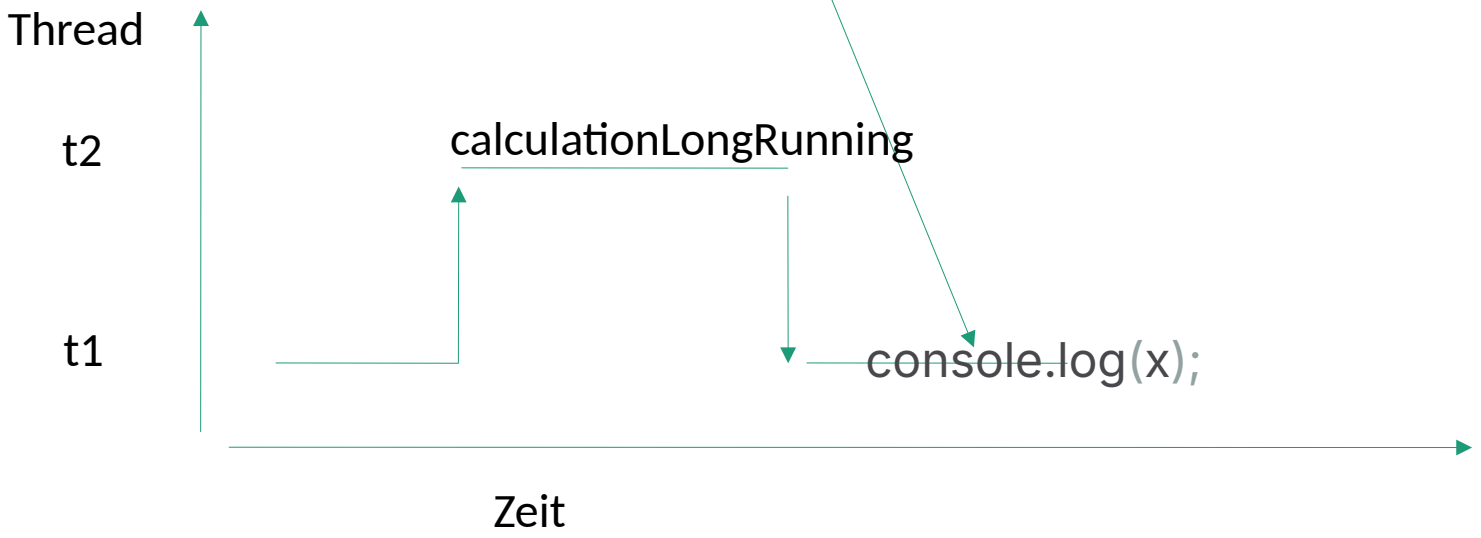
Welche Eigenschaft hat die Variable X

```
var x = 1;
console.log(x);
```

```
var x = [1, 2, 3];
x.forEach(alert);
```

Und hier?

```
asyncFunction(function callback(x) {
        console.log(x);
});
```



Threading im Browser

- Javascript per Definition Single-Threaded
- Event-Loop Thread mit Queue, die vom Interpreter abgearbeitet wird
- Beispiel: Ajax Aufruf
  - JavaScript startet HTTP Request
  - Interpreter wartet nicht bis Server geantwortet hat, UI würde einfrieren
  - Interpreter arbeitet andere Sachen ab (Event-Loop)
  - Nach Response, wird weiterer Code in Event-Loop-Queue eingestellt
  - Code läuft erst wenn in Event-Loop an der Reihe

Callback API

- Asynchroner Funktion wird Callback als zusätzlicher Parameter mitgegeben
- Callback wird aufgerufen sobald Ergebnis bereit steht
- Probleme
  - Keine standardisierte API (Wie werden Fehler behandelt?)
  - Mehrmalige oder spätere Nutzung kompliziert ("Callback-Hell")
  - Abhängigkeit von mehreren asynchronen Ergebnissen sehr schwierig zu handhaben

Lösung sind Promises

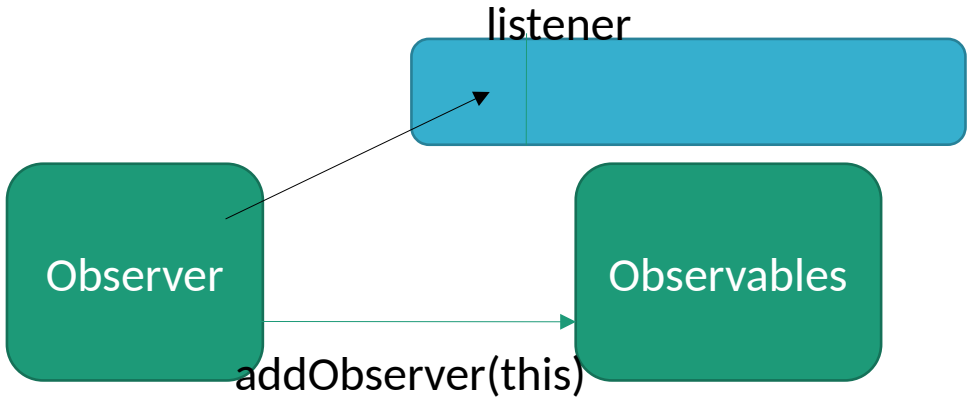
- Stellvertreter für Ergebnis, das zunächst unbekannt ist (z.B. http-Aufruf)
- Asynchronen Funktionen geben Promise zurück, kein Callback als Parameter
  - Können erfüllt oder abgelehnt werden
  - Bieten API für success- und error-Funktionen
- Promise kann genau ein Mal aufgelöst werden
  - 🚫 *Nicht geeignet für mehrere asynchrone Werte*

Promise Beispiel

```
// call to async function returns a promise
const promise = doSomethingAsynchronous();
promise.then(
        (result) ⇒ { return filter(result) }, // success handler
        (error) ⇒ { } // error handler (optional)
).then((result) ⇒);
```

Observable als Promise++

- Observables sind Streams von Events und Daten
  - Aber auch Fehlern
- --a---b-c---d---X|
  - -- Zeitline
  - | Endsignal
  - a,b .. Ereignisse
- Schönere und interaktive Visualisierung
- https://rxmarbles.com




```
for(l in listener){
  l.notify(oldWeight,
this.weight)
}
```

Methoden für Zwischenverarbeitung

- Häufig genutzte Methoden:
  - map für Transformation
  - debounce für Verzögerung
  - filter zum Aussortieren
- Jede Transformation gibt wieder Observable zurück

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

--------3---4---|

Observables sind lazy

- Observables brauchen Abnehmer
  - forEach(dataHandler): Promise
  - subscribe(dataHandler, errorHandler?): Subscription
  - Handler: Funktionen mit Seiteneffekten
- Intermediate Transformationen werden nur ausgeführt wenn Abnehmer da ist

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

--------3---4---|

subscribe console.log

Intermediate vs Terminal

intermediate	terminal
filter	count
map	foreach
sorted	sum
distinct	min/max
limit	reduce
peek	findFirst

Unsubscribe Nicht vergessen

- subscribe() gibt Subscription zurück
- Subscription hat unsubscribe() Methode
- Wenn completed, werden alle intern alle Handler gelöscht

Nutze ngOnDestroy für unsubscribe

Modern: takeUntilDestroyed() aus @angular/core/rxjs-interop oder toSignal() – unsubscribe automatisch

Automatisches Unsubscribe

```
{{ aPromise | async }}
{{ anObservable | async }}
<trainingList [trainings]="releaseObservable$ | async"></
trainingLis
```

Training[]

Observable<Training[]>

Alternative ohne async-Pipe: toSignal() (siehe RxJS Interop)

Reactive Extensions – RxJS

- Entwickelt von MS für verschiedene Sprachen
- Implementierung von Observables
  - Wird in Angular verwendet
  - http://reactivex.io/

RxJS Interop: toSignal()

```
import { toSignal } from '@angular/core/rxjs-interop';

export class AppComponent {
  private readonly customerService = inject(CustomerService);
  // Observable → Signal, Subscription wird automatisch beendet
  customers = toSignal(this.customerService.getAll(), { initialValue: [] as
Customer[] });
}
<!-- Template: keine async-Pipe nötig -->
<customer-list [customers]="customers()" />
// Gegenrichtung: toObservable(signal)
// RxJS für asynchrone Streams (HTTP, Events) – Signals für den Template-State
```

Übung 19: Async CustomerService

- Verpasse CustomerService eine Observable basierte API
  - Lege im Service ein ReplaySubject als Feld an
  - Rufe im Konstuktor am Subject next mit den Mock-Daten auf
  - Lasse getAll das Observable<Customer[]> des Subject zurückgeben
- Passe alle Aufrufe an (toSignal(), alternativ subscribe oder async-Pipe)
  - Eigentlich nur:
    - AppComponent: customerMockList = toSignal(customerService.getAll(), { initialValue: [] })
    - app.component.html: <customer-list [customers]="customerMockList()">

Routing und Navigation

Arbeitsweise

- Routen definieren Pfade der Anwendung
- Jede Route verbindet einen Pfad mit einer Component
  - Auch Redirects können definiert werden
  - Die Pfade unterstützen Platzhalter
- Die Routen werden in app.routes.ts definiert und per provideRouter(routes) in main.ts für die gesamte Applikation bereitgestellt
  - dazu werden Klassen importiert
  - `import { Routes, provideRouter } from '@angular/router'`
- Routen-Pfade müssen nicht disjunkt sein
  - Der erste Treffer eines Pfads wird benutzt
  - Damit sind in der Routen-Definition speziellere Pfade vor allgemeinen zu platzieren

Features von Routen

- Deep Linking über URL
- Browser Historie (Vor / Zurück)
- Links über symbolische Namen, nicht über hart codierte URLs
- Pfad- oder Hash-Navigation
- Hierarchischer Aufbau, nicht zwingend absolute Routen

Routendefinition

- Route
  - Relativer path (ohne führenden Slash)
  - component für die Anzeige

```
import { Routes } from '@angular/router';


export const routes: Routes = [
{ path: 'release', component: AllReleasesComponent },
{ path: 'artist', component: AllArtistsComponent }
];
```

Ablauf

- URL des Browsers wird überprüft / auf Änderungen überwacht
- Passende Route wird gesucht
- Komponente wird angezeigt

Verlinken von Routen

- Dazu gibt es die `routerLink`-Direktive
- Die anzuzeigende Seite wird in einem `router-outlet`-Element angezeigt

```
@Component({
        selector: "music-app",
        template: `
        <h1>Music App</h1>
        <router-outlet></router-outlet>`
})
export class MusicAppComponent {}
```

Default / Fallback Route

- Keine gültige Route gefunden?
  - Einstieg über URL ohne Route
  - ungültige Route
- Fallback konfigurierbar
  - path mit **
  - redirectTo: auf anderen Pfad
  - Achtung: Reihenfolge der Routen spielt eine Rolle,
    - **Fallback als Letzte**

Routenkonfiguration

- provideRouter(routes, ...features) erzeugt die Provider für den Router-Service
- Aufruf einmalig in bootstrapApplication (main.ts)
- Features als optionale Funktionen
  - withComponentInputBinding() – Routen-Parameter als Inputs
  - withHashLocation(), withDebugTracing(), withPreloading(...)
- Lazy Loading per loadComponent / loadChildren
- Legacy: RouterModule.forRoot / forChild im AppModule (siehe Anhang)

Beispiel: Routen-Definition

```
// app.routes.ts
export const routes: Routes = [
  { path: 'path1', component: Component1 },
  { path: 'lazy', loadComponent: () => import('./lazy.component').then(m =>
m.LazyComponent) },
  { path: '', redirectTo: '/index', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];
// main.ts
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes, withDebugTracing()) // <-- debugging purposes only
  ]
});
```

Location-Strategy

- LocationStrategy festlegen
  - PathLocationStrategy (default)
    - Bessere SEO Indizierung
  - HashLocationStrategy: default AngularJS 1
    - Simpler
- Verwendung von PathLocationStrategy
  - <base href="/"> als erstes Element im head der index.html
- import { provideRouter, withHashLocation } from '@angular/router';
- import { routes } from './app/app.routes';
- bootstrapApplication(AppComponent, {
- providers: [provideRouter(routes, withHashLocation())]
- });

Übung 20: Routen für Listenansicht

- Bereite die Anwendung auf Routing vor
  - Erstelle eine Datei src/app/app.routes.ts
  - Definiere und exportiere eine Konstante des Typs Routes (export const routes: Routes = [])
  - Registriere die Routen in main.ts per provideRouter(routes)
  - Importiere RouterOutlet im imports-Array von AppComponent
- Erstelle eine Route für die Listen-Ansicht
  - Füge in app.routes.ts eine Routen-Konfiguration für path: "customer" und component: CustomerListComponent hinzu
  - Erstelle auch eine Redirect-Route für path: "**“ mit redirect auf customer
  - Lösche CustomerListComponent und CustomerDetailsComponent aus dem Template von AppComponent
  - Füge in das Template von AppComponent ein router-outlet ein
- Passe CustomerListComponent an die Veränderungen an
  - Daten können nicht mehr per Input übergeben werden, da keine direkte Verwendung
  - Lade in CustomerListComponent die Daten vom Service (inject() + toSignal())
  - Kein ngOnDestroy nötig – toSignal() beendet die Subscription automatisch
  - Lösche das Laden der customers aus AppComponent

Pfadparameter für dynamische Information

Problem:

- Routen Konfiguration zur Entwicklungszeit
- Somit: Keine statische Route pro Datensatz möglich

Lösung:

- Dynamische Routen mit Parametern
- Routen Konfiguration kann in Pfadvariablen enthalten

Parameter vs URL-Parameter

- Parameter in Route für Pflichtparameter
- URL-Parameter für optionale Parameter

Beispiel Pfadparameter

```
{
    path: 'artist/:id',
    component: ArtistDetailsComponent
    }
```

Details zu den Routen

- Modern: Routen-Parameter, Query-Parameter und Route-Data direkt als Input der Komponente
  - `provideRouter(routes, withComponentInputBinding())`
  - `id = input<string>() in der Ziel-Komponente`
- Alternativ: ActivatedRoute per inject() – Zugriff auf die Routen-Definition, die zum Aufruf geführt hat
  - `url, paramMap, queryParamMap, data, …`
- Router-Events werden an interessierte Listener delegiert
  - `NavigationStart, RoutesRecognized, NavigationEnd, …`
- Details unter https://angular.dev/guide/routing

Beispiel: Component Input Binding

```
// main.ts
provideRouter(routes, withComponentInputBinding())
// app.routes.ts
{ path: 'artist/:id', component: ArtistDetailsComponent }
// artist-details.component.ts
export class ArtistDetailsComponent {
  private readonly service = inject(ArtistService);
  id = input<string>();          // wird aus :id befüllt
  artist = computed(() => ...);   // reagiert auf Änderungen von id()
}
```

Weiterverarbeitung von Routenparametern

```
export class ArtistDetailsComponent {
  private readonly service = inject(ArtistService);
  id = input.required<string>();
  artist = toSignal(
    toObservable(this.id).pipe(
      map(id ⇒ parseInt(id)),
      switchMap(id ⇒ this.service.getById(id))
    )
  );
}
// Legacy: this.route.params.pipe(map(...), switchMap(...)).subscribe(...)
```

Navigation im html

```
1.<a [routerLink]="['/artist']">Artists</a>
2.<a [routerLink]="['/artist', artist.id]">Some Artist</a>
3.<a [routerLink]="[artist.id]">Artist with ID</a>
```

- Absolute Links mit führendem Slash (1,2)
- Zur aktuellen Route relative Links ohne führenden Slash (3)

Navigation im Code – Component Klasse

```
private readonly router = inject(Router);
save() {
  // Navigate back to list
  this.router.navigate(['artist']);
}
```

Übung 21: Routing mit Parametern

- Erstelle eine Route für die Detail Ansicht
  - Konfiguriere eine Route für /customer/:id die auf die CustomerDetailComponent verweist
  - Erweitere CustomerService um eine Methode getById : Observable<Customer>
  - Lese in CustomerDetailsComponent den Routen-Parameter per Signal-Input id = input<string>() aus (withComponentInputBinding() in main.ts)
  - Hole das Customer für die Anzeige per ID vom Service
- Verlinke die Listen-Ansicht mit der Detail-Ansicht
  - Importiere RouterLink im imports-Array von CustomerListComponent
  - Füge den Listen-Elementen einen routerLink hinzu
  - Navigiere im routerLink zur Detail-Ansicht für das Element
- Verlinke die Detail-Ansicht mit der Listen-Ansicht
  - Füge im Template von CustomerDetailsComponent einen Link zurück zur Liste ein
  - Verwende einen Button mit einem Click-Handler
  - Navigiere im Handler per router.navigate (Router per inject())



Formulare und Eingabefelder

Bearbeiten von Daten

Rückblick auf Standard html

- Formular (form) einfacher Aufbau
  - Mit Feldern (input, select, ..)
  - FieldSet für visuelle Gruppierung
- Validierung mittels HTML 5 (teilweise)
- Posten der Daten mittels Pagereload oder AJAX

Formulare inAngular

Arten

- Template-Driven
- Model-Driven

Interne Forms API

- Binding (bzw. ChangeDetection)
- Validierung
- Aggregation

Formulare, Gruppen, Felder

- Ein Formular = Gruppe von Feldern + Submit-Aktion
- Der Zustand der Gruppe hängt von den enthaltenen Feldern ab
  - z. B. Gruppe ungültig, falls mindestens ein Feld ungültig ist
- Ein Feld ist ein Wert + weitere Zustände: dirty, touched, valid
- Zwei Arten in Angular: Template-Driven Forms (Hauptpfad), Reactive Forms (Wahlthema)

Template-Driven Forms: FormsModule und ngModel

- Template-Driven Form: das Formular wird im Template beschrieben, die Klasse bleibt schlank
- FormsModule aus @angular/forms im imports-Array der Komponente importieren
- Jedes Feld bekommt die Direktive ngModel und ein name-Attribut
  - Innerhalb eines form-Tags ist das name-Attribut Pflicht, sonst gibt es einen Fehler
- Template-Referenz #f="ngModel" gibt Zugriff auf den Zustand eines Feldes
  - z. B. f.invalid, f.dirty, f.touched, f.errors
- Angular legt das Formular (NgForm) und die Felder automatisch im Hintergrund an
- Im Hauptpfad des Kurses bleiben wir bei dieser einfachen Variante

customer-form.html

```
<form>
  <input name="firstname"
    [(ngModel)]="draft.firstname"
    #f="ngModel" required>
  @if (f.invalid && f.dirty) {
    <small>Pflichtfeld</small>
  }
</form>
```

Template-Driven Form: Aufbau im Template

```
@if (draft(); as c) {
  <form #customerForm="ngForm">
    <input name="firstname" [(ngModel)]="c.firstname">
    <input name="city" [(ngModel)]="c.city">

    <p>Formular: {{ customerForm.valid ? 'gültig' : 'ungültig' }}</p>
  </form>
}
<!-- c: bearbeitbare Kopie des Kunden, das Original bleibt unverändert -->
<!-- #customerForm="ngForm": Zugriff auf das ganze Formular, z. B. .valid -->
```

Bearbeitungskopie mit linkedSignal

```
// Der geladene Kunde (nur lesen!)
customer = computed(() ⇒ {
  const id = this.id();
  return id ? this.customerService.getById(id) : undefined;
});

// Bearbeitbare Kopie: folgt dem Original, kann aber überschrieben werden
draft = linkedSignal(() ⇒ {
  const customer = this.customer();
  return customer ? { ...customer } : undefined;
});
// Wechselt customer (z. B. andere ID), entsteht draft neu aus dem Original
```

Template-Driven oder Reactive Forms?

- Template-Driven: Beschreibung im Template, wenig Code, gut für einfache Formulare
- Reactive Forms: Beschreibung in der Klasse (FormGroup, FormControl), mehr Kontrolle
- Reactive Forms lohnen sich bei dynamischen Feldern, vielen Abhängigkeiten, komplexen Regeln
- Beide Varianten kennen dieselben Zustände: valid, dirty, touched, errors
- Beide Varianten verwenden dieselben Validatoren
- Im Hauptpfad: Template-Driven Form in Übung 22
- Reactive Forms: Wahlthema am Ende des Kurses, mit Wahlübung Z1



Validierung

Prüfen der Formulareingaben

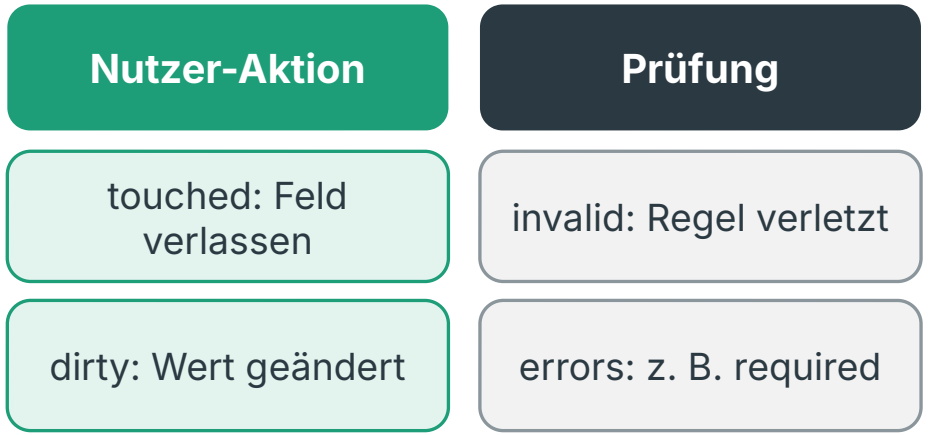
- 2-fache Validierung ist nicht redundant
- Client-seitig Validierung für User Experience
- Server-seitig Validierung für Sicherheit
- Validatoren
  - Built-in (required, pattern, minlength, maxlength, email)
  - Custom
- Verwendung
  - Template-Driven Forms: mittels HTML-Attributen bzw. Direktiven (Hauptpfad)
  - Reactive Forms: mit Hilfe der Forms API (Wahlthema)

Validierung mit HTML-Attributen

- Template-Driven Forms prüfen Eingaben über HTML-Attribute am Feld
- required: Pflichtfeld
- pattern="[0-9]*": Eingabe muss zum regulären Ausdruck passen (z. B. nur Ziffern für eine PLZ)
- minlength und maxlength: Mindest- und Höchstlänge
- email: Eingabe muss wie eine E-Mail-Adresse aussehen
- Angular wertet diese Attribute mit eingebauten Validator-Direktiven aus
- Eigene Prüfregeln (Custom Validators) sind möglich und werden als Direktive angelegt
- Die Prüfung im Client verbessert die Bedienung, die Prüfung im Server bleibt Pflicht

Feldzustände: dirty, touched und invalid

- Jedes Feld hat unabhängige Zustände:
  - untouched / touched: das Feld wurde noch nicht / schon einmal verlassen
  - pristine / dirty: der Wert ist noch unverändert / wurde verändert
  - valid / invalid: der aktuelle Wert ist gültig / ungültig
- errors enthält die konkrete Fehlerursache, z. B. required oder pattern
- Angular setzt passende CSS-Klassen: ng-valid, ng-invalid, ng-dirty, ng-touched
- Fehlermeldungen erst zeigen, wenn der Benutzer das Feld verändert hat (dirty)
- Das ganze Formular ist ungültig, sobald ein Feld ungültig ist



Fehlermeldung und Formularzustand

```
<input name="firstname" [(ngModel)]="c.firstname" required #firstname="ngModel">
@if (firstname.invalid && firstname.dirty) {
  <p class="error">Bitte einen Vornamen eingeben.</p>
}

<input name="zip" [(ngModel)]="c.zip" pattern="[0-9]*" #zip="ngModel">
@if (zip.invalid && zip.dirty) {
  <p class="error">Die PLZ darf nur aus Ziffern bestehen.</p>
}

<p>Formular: {{ customerForm.valid ? 'gültig' : 'ungültig' }}</p>
<!-- Übung 23: <button type="submit" [disabled]="customerForm.invalid"> ⟶
```

Übung 22: Kunde bearbeiten (Template-Driven Form)

- Öffne das Projekt 22_forms und arbeite in CustomerDetailsComponent
- 1. FormsModule im imports-Array importieren (für das Datumsfeld auch DatePipe)
- 2. Bearbeitbare Kopie als linkedSignal anlegen: draft = linkedSignal(() ⇒ ({ ...customer }))
- 3. Felder mit name-Attribut und [(ngModel)] an die Kopie binden
- 4. Validierung per HTML-Attribut: Vor- und Nachname required, PLZ pattern="[0-9]*"
- 5. Fehlermeldung je Feld mit @if (feld.invalid && feld.dirty)
- 6. Formularzustand ausgeben: #customerForm="ngForm" und customerForm.valid
- Musterlösung: 22_forms (baut auf 21_dynamic_routes auf)



Modulare wiederverwendbare Komponenten

Kind Komponenten

Best Practice

- Smarte Komponenten und einfache (dumme) Komponenten
  - Smarte Komponenten wissen etwas über Umgebung
    - z.B. Abhängigkeiten von Services
  - Dumme Komponenten haben nur Inputs und Outputs
    - z.B. keine Abhängigkeiten von Services
- Bessere Wiederverwendbarkeit bei dummen Komponenten

- In unserer Beispielanwendung Formular aus Detailkomponente auslagern. **Wie läuft die Interaktion topDown und bottomUp**

Rückblick: Input Output

- Vater zu Kind über Property-Binding
  - Daten von oben nach unten übergeben
  - Kind: Signal-Input input() – Reaktion auf Änderungen per computed() / effect() statt ngOnChanges
- Kind zu Vater über Event-Binding mit output()
  - Events von unten nach oben reichen

viewChild() – Signal Query

- Injection von Kindkomponente zur einfacheren Interaktion

```
export class ParentComponent {
  listComponent = viewChild(ListComponent);   // Signal<ListComponent | undefined>
  refresh() { this.listComponent()?.reload(); }
}
```

- Legacy: @ViewChild(ListComponent) private listComponent: ListComponent;

Kommunikation über ThirdParty

- Kommunikationskanal
  - Service stellt Observables bereit
- Auch andere Kommunikationsstrukturen möglich z.B. 1:n
- Vaterkomponente kann als Provider vom Service dienen (Scoped Service Instanziierung)

Beispiel Kommunikations-Service

```
@Component({ selector: 'list', ... })
class ListComponent {
  private readonly com = inject(Communication);
  constructor() {

this.com.detailToList.pipe(takeUntilDestroyed())
      .subscribe((update: DetailUpdate) =>
console.log("update received"));
  }
  doSomething() {
    this.com.listToDetail.next(new ListUpdate());
  }
}
```

```
@Injectable()
class Communication {
  listToDetail = new
Subject<ListUpdate>();
  detailToList = new
Subject<DetailUpdate>();
}
@Component({
  imports: [ListComponent,
DetailComponent],
  template: `<list /><detail />`,
  providers: [Communication]
})
class MasterDetailComponent {}
```

Übung 23

- Stelle in CustomerService eine Methode update bereit
  - Rufe die Methode beim Speichern des Formulares in CustomerDetailsComponent auf
  - Ersetze in der Methode das ursprüngliche Customer-Objekt durch das neue
  - Feuere einen neuen Datensatz am internen Subject
- Zeige in AppComponent immer den letzt aktiven Customer an
  - Implementiere in CustomerService die Logik (Methode getNext(): Observable<Customer>)
  - Nutze dafür das Observable von getAll
  - Nutze eine flache Kopie des Arrays zum Sortieren (array.slice().sort)
  - Hole in AppComponent das Observable von getNext und wandle es per toSignal() in ein Signal um
  - Zeige im UI den Namen des nächsten Customer an

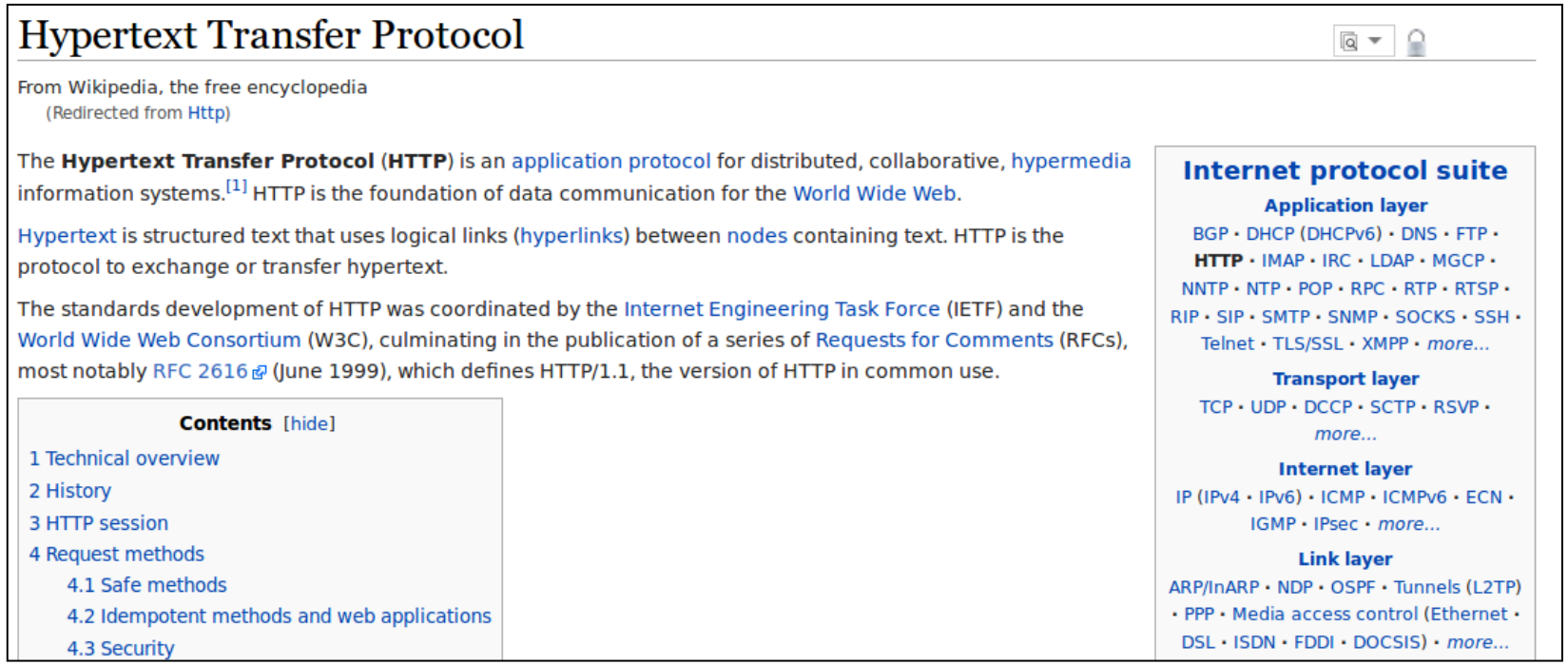


Remote-Kommunikation


Exkurs REST

Das http-Protokoll

- Eine umfassende Spezifikation des w3w-Konsortiums
  - Siehe http://en.wikipedia.org/wiki/Http



Hypertext Transfer Protocol

From Wikipedia, the free encyclopedia
(Redirected from Http)

The **Hypertext Transfer Protocol** (**HTTP**) is an application protocol for distributed, collaborative, hypermedia information systems.[1] HTTP is the foundation of data communication for the World Wide Web.

Hypertext is structured text that uses logical links (hyperlinks) between nodes containing text. HTTP is the protocol to exchange or transfer hypertext.

The standards development of HTTP was coordinated by the Internet Engineering Task Force (IETF) and the World Wide Web Consortium (W3C), culminating in the publication of a series of Requests for Comments (RFCs), most notably RFC 2616 (June 1999), which defines HTTP/1.1, the version of HTTP in common use.

Contents [hide]

1 Technical overview
2 History
3 HTTP session
4 Request methods
  4.1 Safe methods
  4.2 Idempotent methods and web applications
  4.3 Security

Internet protocol suite

Application layer
BGP • DHCP (DHCPv6) • DNS • FTP • **HTTP** • IMAP • IRC • LDAP • MGCP • NNTP • NTP • POP • RPC • RTP • RTSP • RIP • SIP • SMTP • SNMP • SOCKS • SSH • Telnet • TLS/SSL • XMPP • *more...*

Transport layer
TCP • UDP • DCCP • SCTP • RSVP • *more...*

Internet layer
IP (IPv4 • IPv6) • ICMP • ICMPv6 • ECN • IGMP • IPsec • *more...*

Link layer
ARP/InARP • NDP • OSPF • Tunnels (L2TP) • PPP • Media access control (Ethernet • DSL • ISDN • FDDI • DOCSIS) • *more...*

Elemente der http-Spezifikation

- Definition von URIs
  - Pfad
  - Parameter
- http-Request und http-Response
  - Daten-Container mit Header und Body
  - Encodierung
- Umfassender Satz von Header-Properties
  - Content-Length
  - Accepts
  - Content-Type

Elemente der http-Spezifikation II

- http-Methoden
  - PUT
  - GET
  - POST
  - DELETE
  - OPTIONS
  - HEAD
- Statuscodes für Aufrufe
  - 404: „Not found“
  - 204: „Created“
  - …

MimeTypes

- Definition der Datentypen des Internet
  - Nicht zu verwechseln mit einem XML-Schema
  - Ein MimeType ist „nur“ eine strukturierte Zeichenkette
  - Eigene Erweiterungen sind möglich

REST und http

- REST hat mit http prinzipiell nichts zu tun
  - REST ist eine abstrakte Architektur
  - http ist ein konkretes Kommunikationsprotokoll
- Aber
  - http passt als Kommunikations-Protokoll der „Referenz-Implementierung“ Internet natürlich perfekt zum REST-Stil

GET

- Liefert die Ressource

DELETE

- Löscht die Ressource

PUT

- Erzeugt aktualisiert die Ressource
- Client spezifiziert resourceIdentifier
- PUT /resources/<newResourceId> HTTP/1.1

POST

- Erzeugt ein neues Objekt unterhalb (Collection) der spezifizierten Ressource
- POST /resources HTTP/1.1

Der http Client

Der Service HttpClient

- HttpClient ist ein Service
  - und steht damit via Dependency Injection zur Verfügung: inject(HttpClient)
- provideHttpClient() in main.ts registrieren
  - optional withFetch(), withInterceptors([...])
- Das API
  - umfasst die Standard-http-Methoden
  - ist asynchron konzipiert (Observables)

```
this.http.get<Data>('/endpoint').subscribe(data => {
  this.results = data.jsonAttribute;
});
```

Json Daten extrahieren

```
private readonly http = inject(HttpClient);
getAll(): Observable<Release[]> {
  return this.http.get<{ data: Release[] }>('api/release').pipe(
    map(response ⇒ response.data),
    catchError(this.handleError)
  );
}
// JSON-Parsing übernimmt HttpClient automatisch
```



Exception Handler

```
private handleError(error: HttpErrorResponse) {
    let message;
    if (error.message) {
            message = error.message;
    }
    else if (error.status) {
            message = '${error.status} - ${error.statusText} ';
    }
    else {
            message = 'Server error‘;
    }

    // Rethrow in intermediate Methods
    return throwError(() ⇒ new Error(message));
    }
```

Header

```
const headers = new HttpHeaders({ 'Content-Type': 'application/json' });
return this.http.post<{ data: MyObject }>('api/objects', object, { headers }).pipe(
  map(response ⇒ response.data),
  catchError(this.handleError)
);
// JSON.stringify übernimmt HttpClient automatisch
```



Übung 24: Starten des Servers

- Zip-Datei *server-js.zip* entpacken
- Konsole öffnen und in Server Verzeichnis wechseln
- Installieren per npm install
- Starten über node server.js
- Backend läuft unter localhost:3000
- dev.proxy.json: { "/api": { "target": "http://localhost:3000" } }
- Dev-Server mit Proxy starten (CORS) ng serve --proxy dev.proxy.json

Übung 24: Endpunkte

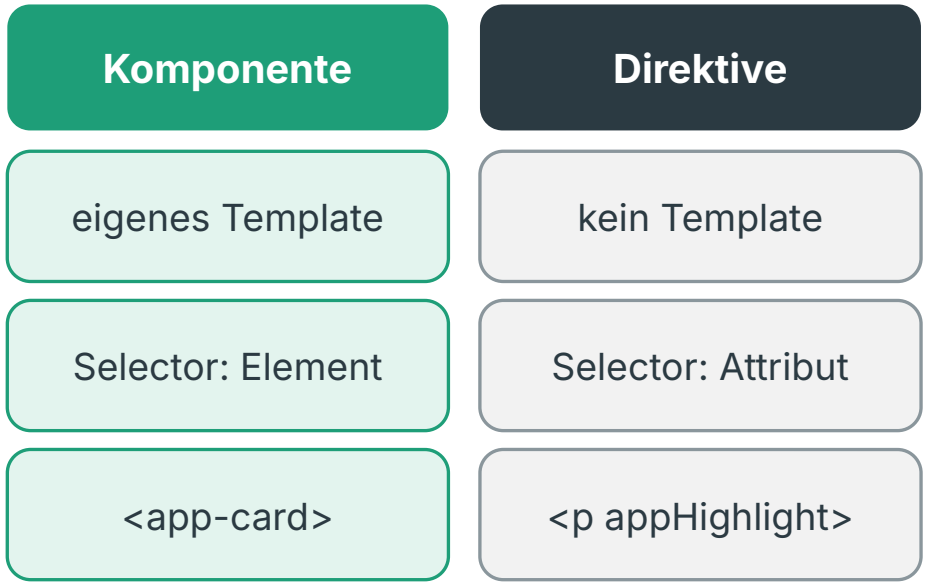
- GET
  - api/customer (GET_ALL)
  - api/customer/:id (GET_BY_ID)
- POST api/customer (INSERT)
- PUT/PATCH/DELETE api/customer/:id (UPDATE, PARTIAL_UPDATE, REMOVE)

Übung 24: http

- Verwende in CustomerService das Backend (inject(HttpClient)) statt der Mock-Daten
- Registriere provideHttpClient() in main.ts
  - Dev-Server mit Proxy starten
  - Lade die Liste initial vom Backend
  - Achtung: Date Felder vom Server werden serialisiert als String gesendet
    - Müssen noch in Date umgewandelt werden
  - Schicke die geänderten Daten in update per put zum Backend
  - Lade (der Einfachheit halber) nach einem update die Liste neu vom Backend
  - Musterlösung: 23_communication (enthält bereits die HTTP-Anbindung)

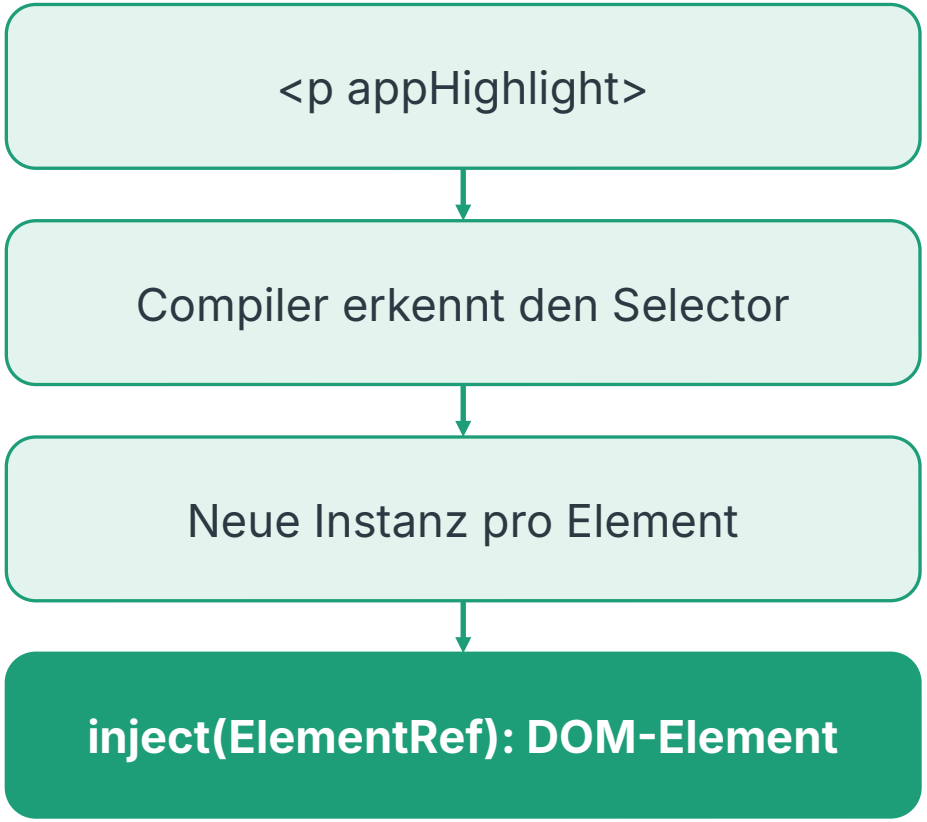
Eigene Direktiven: Wozu?

- Direktive: Klasse, die einem vorhandenen Element Verhalten gibt
- Komponente = Direktive mit eigenem Template
- Attribut-Direktive: ändert Aussehen oder Verhalten
  - Beispiele: Hervorheben, Tooltip, Autofokus
- Strukturelle Direktive: fügt Elemente hinzu oder entfernt sie
  - Heute meist durch @if / @for ersetzt
- Wiederverwendbar: einmal schreiben, überall als Attribut nutzen



Direktive erzeugen und einbinden

- Erzeugen: ng generate directive highlight
- @Directive({ selector: '[appHighlight]' })
  - Eckige Klammern: CSS-Selector für ein Attribut
- Einbinden: Klasse im imports-Array der Komponente
- Angular erzeugt pro passendem Element eine eigene Instanz
- inject(ElementRef) liefert das Element, auf dem sie sitzt



Direktive: einfaches Beispiel

```
import { Directive, ElementRef, inject } from '@angular/core';

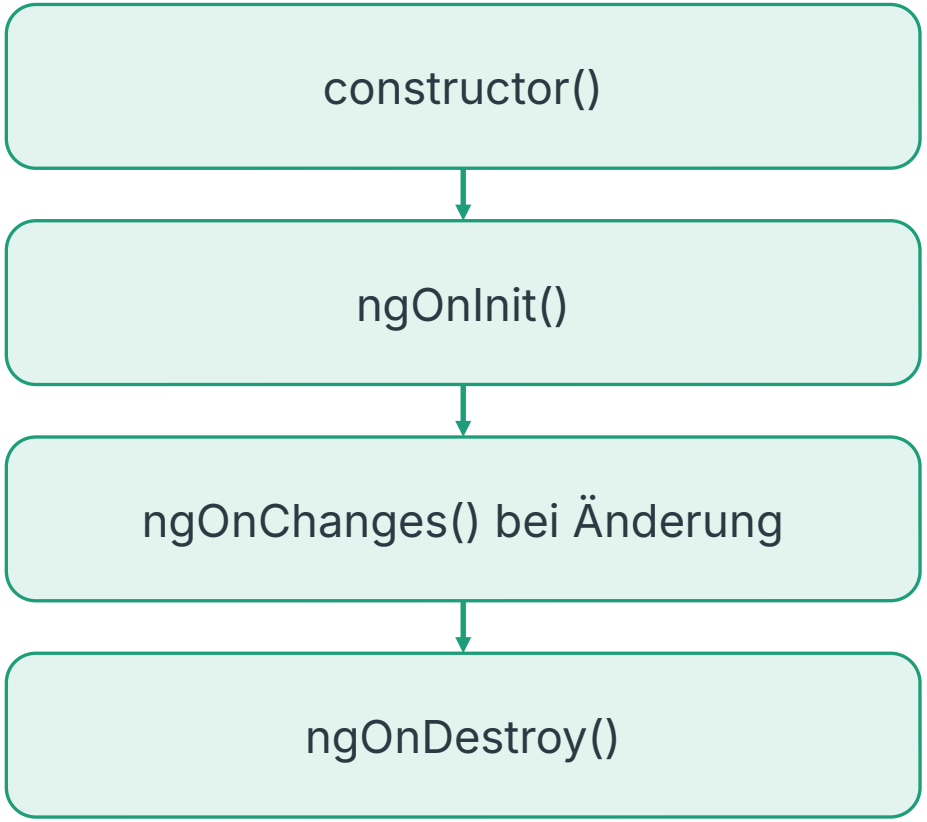
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  // Referenz auf das Element, auf dem die Direktive sitzt
  private readonly el = inject(ElementRef<HTMLElement>);

  constructor() {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }
}

// Verwendung (HighlightDirective im imports-Array):
// <p appHighlight>Dieser Text wird gelb hinterlegt</p>
```

Lebenszyklus einer Direktive

- Direktiven haben denselben Lebenszyklus wie Komponenten
- constructor: inject() möglich, Inputs noch nicht gesetzt
- ngOnInit: Inputs haben ihren ersten Wert
- ngOnChanges: ein Input hat sich geändert
- ngOnDestroy / DestroyRef: Element verschwindet, aufräumen
- Mit input() statt Hooks: computed() oder effect() nutzen



Signal-Inputs: Hooks meist unnötig

Direktive mit input() und Host-Bindings

```
import { Directive, input, signal } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  host: {
    '[style.backgroundColor]': 'active() ? highlightColor() : null',
    '(mouseenter)': 'active.set(true)',
    '(mouseleave)': 'active.set(false)'
  }
})
export class HighlightDirective {
  highlightColor = input('yellow');     // Farbe von außen, Standard gelb
  protected active = signal(false);     // Maus gerade über dem Element?
}

// Verwendung:
// <p appHighlight>gelb beim Darüberfahren</p>
// <p appHighlight highlightColor="lightblue">hellblau</p>
```

Host-Bindings erklärt

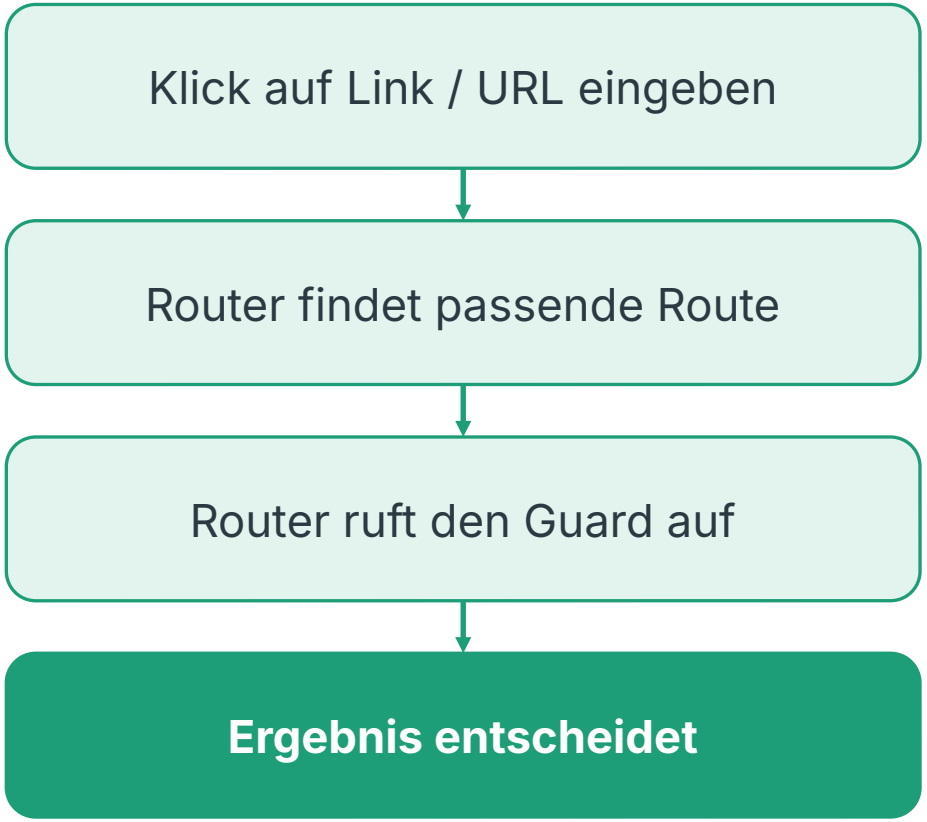
- host: { ... } beschreibt Bindungen am Element der Direktive
- '[eigenschaft]': 'ausdruck' – wie [ ] im Template
  - z. B. '[class.active]', '[attr.aria-label]', '[style.color]'
- '(ereignis)': 'anweisung' – wie ( ) im Template
  - z. B. '(click)', '(keydown.enter)', '(window:resize)'
- Vorteil: kein direkter DOM-Zugriff, funktioniert auch bei SSR
- @HostBinding / @HostListener: ältere Schreibweise

host-Objekt

```
host: {
  '[class.active]': 'active()',
  '[attr.role]': "'button'",
  '(click)': 'toggle()',
  '(keydown.enter)': 'toggle()'
}
```

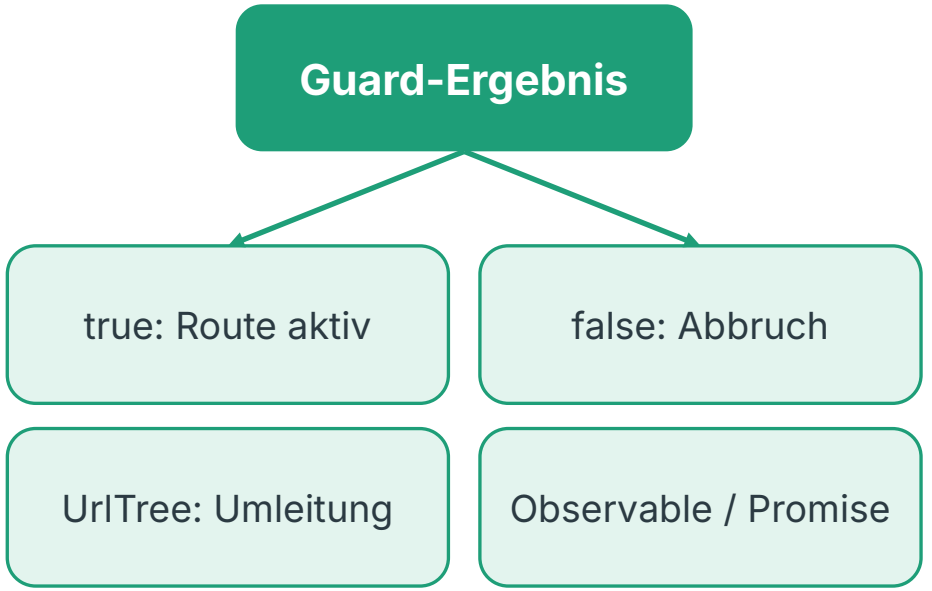
Route Guards: Wozu?

- Guard: Funktion, die der Router vor einer Navigation fragt
- Typische Fragen: Ist der Nutzer angemeldet? Hat er die Rolle?
- canActivate: darf die Route betreten werden?
- canDeactivate: darf sie verlassen werden (ungespeichert)?
- canMatch: darf diese Route überhaupt passen?
- Nur Komfort: Der Server muss Zugriffe selbst prüfen



Guard-Ablauf und Rückgabewerte

- Der Router wartet auf das Ergebnis des Guards
- true: Navigation geht weiter, Komponente wird angezeigt
- false: Navigation wird abgebrochen, Seite bleibt
- UrlTree: Abbruch und Umleitung auf eine andere Adresse
- Auch asynchron: Observable oder Promise mit diesen Werten
- Mehrere Guards im Array: alle müssen zustimmen



Funktionaler Guard: authGuard

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from './auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }
  // Umleitung zur Login-Seite, Ziel-URL für später merken
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};

// app.routes.ts – Guard ohne Klammern eintragen:
{ path: 'customer', component: CustomerListComponent, canActivate: [authGuard] }
```

Parameter route und state erklärt

- Der Router ruft den Guard bei jeder Navigation zur Route auf
- route: Schnappschuss der Zielroute (ActivatedRouteSnapshot)
  - route.params, route.queryParams, route.data
- state: Zustand des gesamten Ziels (RouterStateSnapshot)
  - state.url: komplette Ziel-Adresse als Text
- Nicht benötigte Parameter darf man weglassen: () ⇒ ...
- inject() geht, weil der Router im Injection Context aufruft

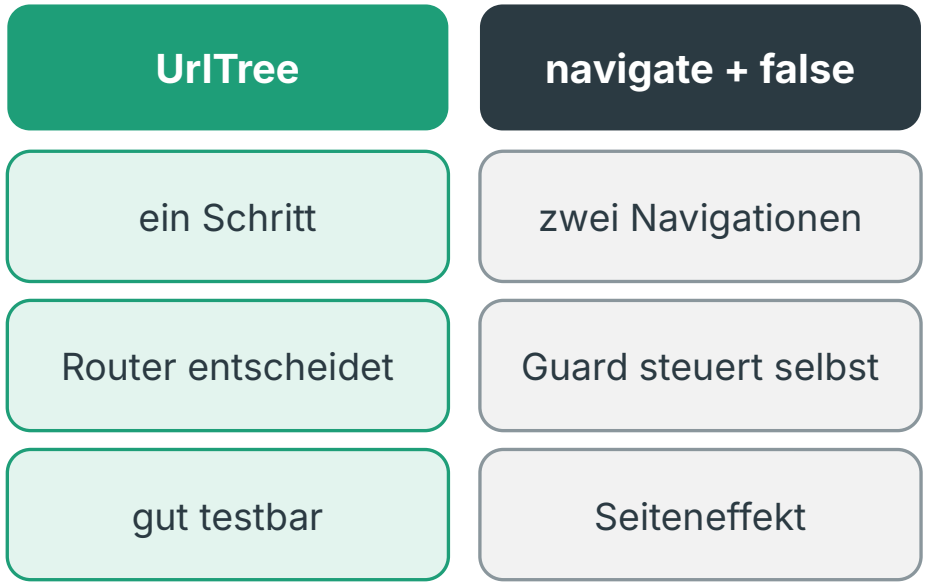
Signatur von CanActivateFn

```
type CanActivateFn = (
  route: ActivatedRouteSnapshot,
  state: RouterStateSnapshot
) => MaybeAsync<GuardResult>;

// GuardResult:
//   boolean | UrlTree
//   | RedirectCommand
```

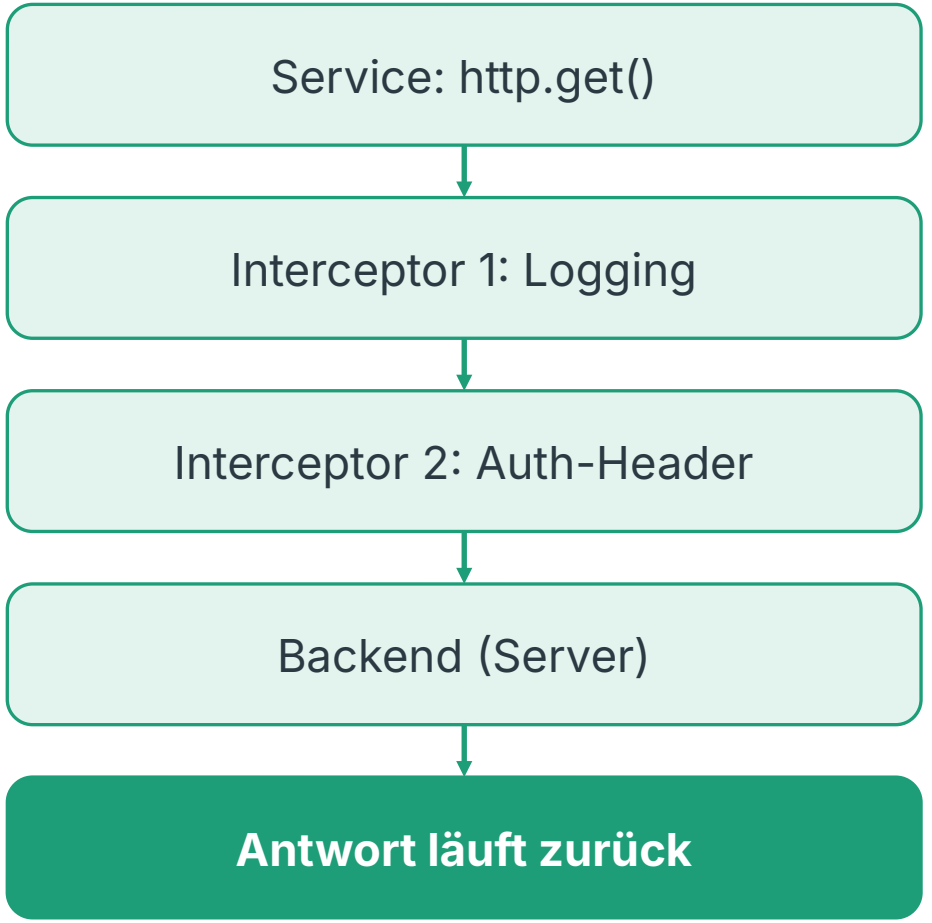
UrlTree statt router.navigate

- Möglich: router.navigate(['/login']) und dann return false
  - Startet eine zweite Navigation, während die erste noch läuft
- Besser: return router.createUrlTree(['/login'])
  - Router bricht ab und leitet in einem Schritt um
  - Bei mehreren Guards entscheidet der Router sauber
  - Leicht testbar: Rückgabewert prüfen statt Aufruf
- Musterlösung 25 nutzt navigate – beides funktioniert



HTTP-Interceptors: Wozu?

- Interceptor: Funktion, durch die jede HttpClient-Anfrage läuft
- Querschnittsaufgaben an einer zentralen Stelle:
  - Token in den Authorization-Header schreiben
  - Anfragen und Antwortzeiten protokollieren
  - Fehler zentral behandeln, z. B. 401 → Login
  - Ladeanzeige ein- und ausschalten
- Komponenten und Services bleiben frei davon



Die Interceptor-Kette

- Reihenfolge = Reihenfolge im Array von withInterceptors
- Anfrage läuft vorwärts: 1 → 2 → Backend
- Antwort läuft rückwärts: Backend → 2 → 1 → Aufrufer
- Jeder Interceptor entscheidet selbst:
  - unverändert weiterreichen: next(req)
  - verändert weiterreichen: next(req.clone(...))
  - Antwort beobachten oder umbauen: next(req).pipe(...)

withInterceptors([logging, auth])

```
Anfrage (vorwärts):
  Aufrufer
  → loggingInterceptor
  → authInterceptor
  → Backend

Antwort (rückwärts):
  Backend
  → authInterceptor
  → loggingInterceptor
  → Aufrufer
```

Funktionaler Interceptor: Logging

```
import { HttpInterceptorFn, HttpEventType } from '@angular/common/http';
import { tap } from 'rxjs';

export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const started = Date.now();
  console.log(`[HTTP] ${req.method} ${req.url} gestartet`);

  return next(req).pipe(
    tap({
      next: event => {
        if (event.type === HttpEventType.Response) {
          console.log(`[HTTP] ${req.url} ok (${Date.now() - started} ms)`);
        }
      },
      error: err => console.log(`[HTTP] ${req.url} fehlgeschlagen`, err)
    })
  );
};
```

Parameter req und next erklärt

- Der HttpClient ruft die Kette bei jeder Anfrage auf
- req: die Anfrage (HttpRequest) – unveränderlich
  - method, url, headers, body, params
- next: Funktion für das nächste Glied (HttpHandlerFn)
  - next(req) liefert ein Observable mit den Antwort-Ereignissen
- Rückgabe: genau dieses Observable (ggf. mit pipe)
- Registrierung: provideHttpClient(withInterceptors([...]))

Signatur von HttpInterceptorFn

```
type HttpInterceptorFn = (
  req: HttpRequest<unknown>,
  next: HttpHandlerFn
) => Observable<HttpEvent<unknown>>;

// main.ts
provideHttpClient(
  withInterceptors([
    loggingInterceptor,
    authInterceptor
  ])
)
```

Header ergänzen mit req.clone

```
import { inject } from '@angular/core';
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token();   // angenommen: Token als Signal

  // Nur eigene API-Aufrufe erhalten das Token
  if (!token || !req.url.startsWith('api/')) {
    return next(req);
  }

  // req ist unveränderlich: veränderte Kopie erzeugen
  const authReq = req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  });
  return next(authReq);
};
```

Interceptors: Stolperfallen

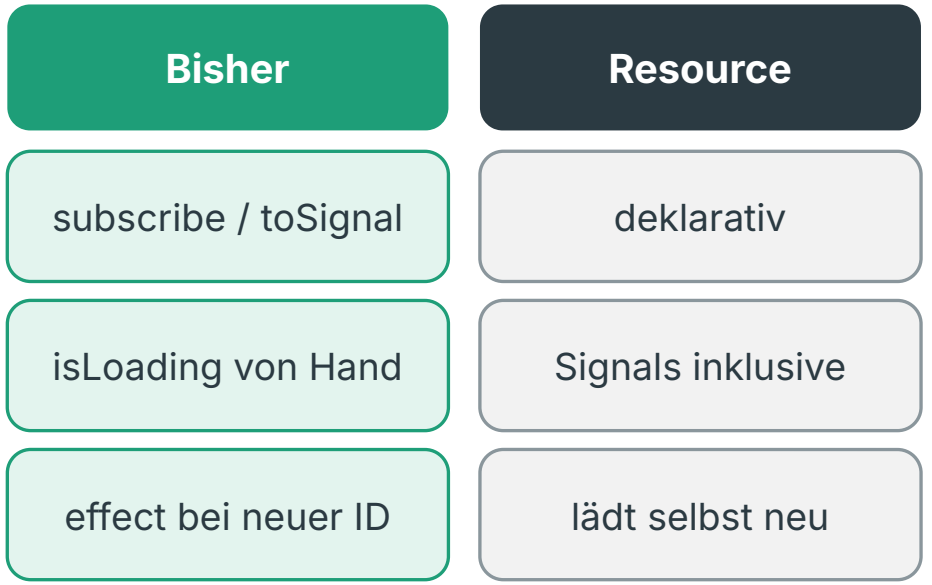
- Immer next(...) aufrufen und das Observable zurückgeben
- Anfrage nie direkt ändern, sondern req.clone({ ... })
- Reihenfolge im Array bewusst wählen (Auth vor Logging?)
- Gilt nur für HttpClient – auch für httpResource, nicht für fetch()
- Fehler zentral mit catchError behandeln, dann weiterwerfen
- Alte Klassen-Interceptors: nur mit withInterceptorsFromDi()

Übung 25: Funktionale Guards & Interceptors

- **1. AuthService: Login-Status als Signal mit isLoggedIn(), login(), logout()**
- **2. authGuard (CanActivateFn): inject(AuthService) und inject(Router)**
  - Nicht angemeldet: Umleitung zur Login-Seite (navigate + false oder UrlTree)
- **3. In app.routes.ts: canActivate: [authGuard] für customer und customer/:id**
- **4. LoginComponent: Button ruft login() auf und navigiert zur Kundenliste**
- **5. loggingInterceptor (HttpInterceptorFn): Start, Erfolg, Fehler mit Dauer loggen**
- **6. Registrieren in main.ts: provideHttpClient(withInterceptors([loggingInterceptor]))**
- Test: Kundenliste ohne Login aufrufen, Konsole beobachten
- **Musterlösung: 25_httpInterceptorGuard (baut auf 23_communication auf)**

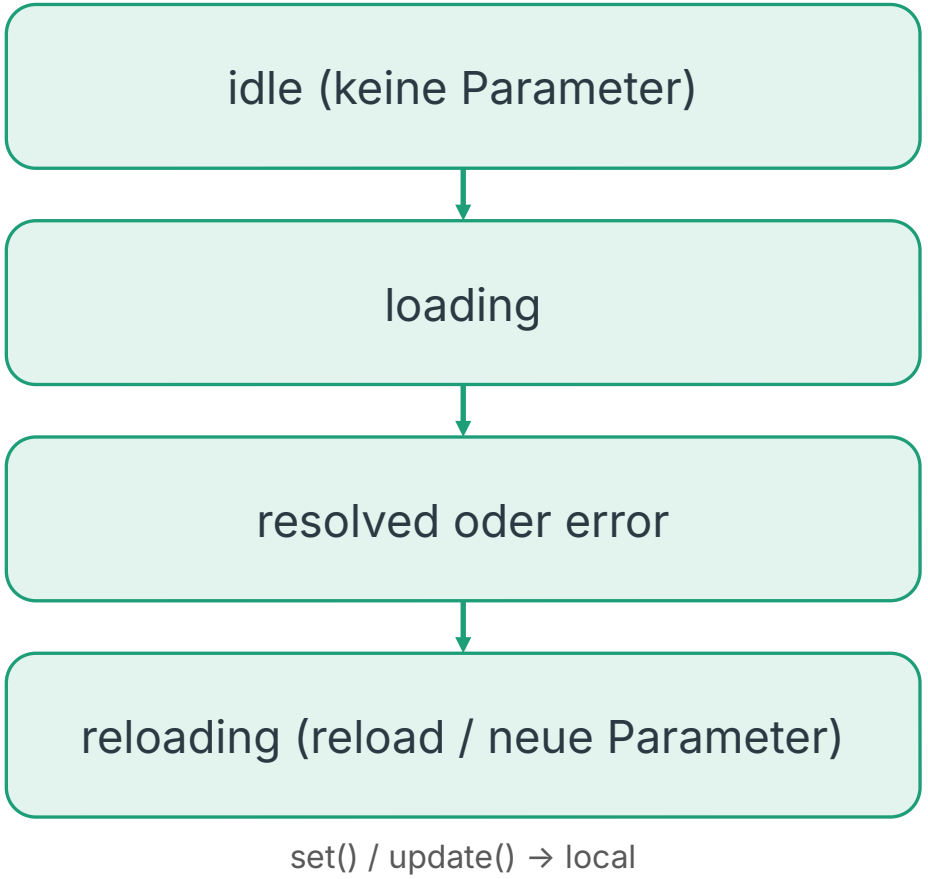
Resource API: Daten laden mit Signals

- Bisher: Service + Observable + subscribe() oder toSignal()
  - Ladezustand und Fehler in eigenen Variablen pflegen
  - Bei neuer ID: effect() + erneutes subscribe()
- Resource: deklarativ beschreiben, was abhängig wovon geladen wird
  - Angular lädt, lädt neu und bricht alte Anfragen ab
  - Ergebnis, Ladezustand und Fehler kommen als Signals



Zustände einer Resource

- status() liefert den aktuellen Zustand als Text:
  - idle: keine Parameter, es wird nichts geladen
  - loading: erster Ladevorgang läuft
  - resolved: Daten sind da
  - error: Laden ist fehlgeschlagen
  - reloading: reload() oder neue Parameter, alter Wert bleibt
  - local: Wert wurde lokal mit set() / update() geändert



Was eine Resource bereitstellt

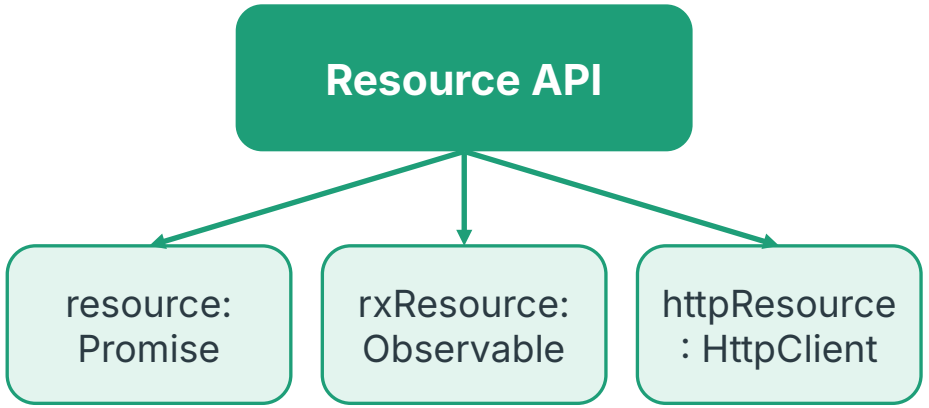
- value(): die geladenen Daten (vorher undefined)
- hasValue(): true, wenn ein Wert vorliegt
  - Vor value() prüfen – im Zustand error wirft value()
- isLoading(): true bei loading und reloading
- error(): der Fehler oder undefined
- reload(): Laden mit denselben Parametern wiederholen
- set() / update(): Wert lokal überschreiben

Template

```
@if (customer.isLoading()) {
  <p>Lade…</p>
} @else if (customer.error()) {
  <p>Fehler beim Laden</p>
} @else if (customer.hasValue()) {
  <p>{{ customer.value().firstname }}</p>
}
```

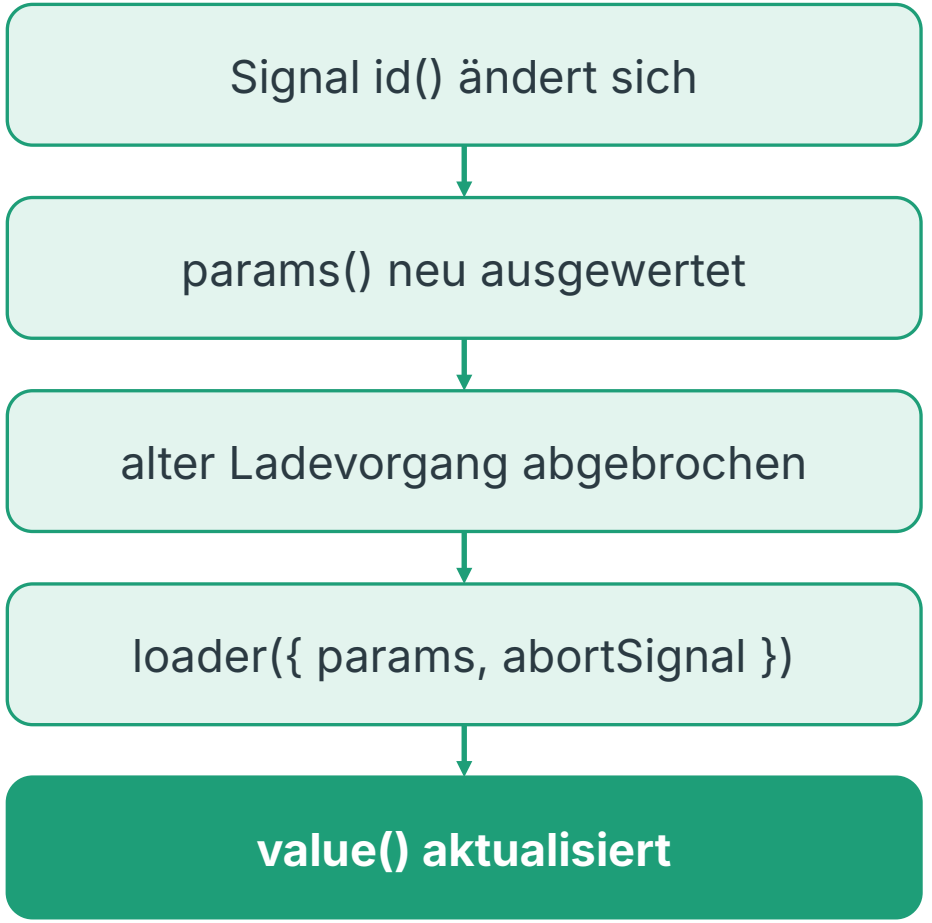
Drei Varianten der Resource API

- resource({ params, loader })
  - loader liefert ein Promise, z. B. mit fetch() – @angular/core
- rxResource({ params, stream })
  - stream liefert ein Observable – @angular/core/rxjs-interop
  - Ideal für bestehende Services mit Observables
- httpResource(() ⇒ url)
  - Nutzt HttpClient, Interceptors greifen – @angular/common/http



Ablauf: params, loader und stream

- params: Funktion, die Signals liest und Parameter liefert
  - Läuft reaktiv wie computed()
  - Liefert sie undefined: Zustand idle, kein Laden
- loader / stream: wird von Angular aufgerufen
  - bei jeder Änderung von params, Argument: { params, abortSignal }
- Laufender Ladevorgang wird dabei abgebrochen



resource() mit loader und abortSignal

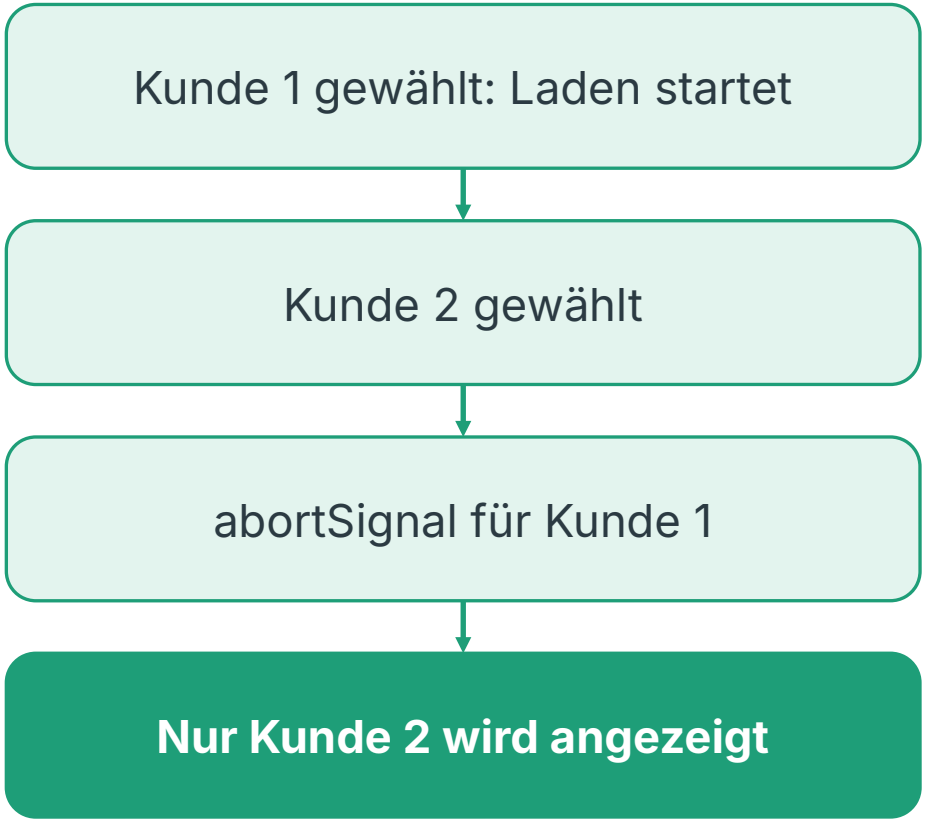
```
export class CustomerDetailsComponent {
  id = input<string>();

  customer = resource({
    // 1. Parameter: reaktiv, undefined = nichts laden
    params: () ⇒ this.id() ? { id: this.id()! } : undefined,

    // 2. Loader: Angular ruft ihn bei jeder Parameter-Änderung auf
    loader: async ({ params, abortSignal }) ⇒ {
      const res = await fetch(`api/customer/${params.id}`, { signal: abortSignal });
      if (!res.ok) {
        throw new Error(`HTTP ${res.status}`);   // → Zustand error
      }
      return (await res.json()) as Customer;     // → Zustand resolved
    }
  });
}
```

abortSignal erklärt

- AbortSignal: Standard-Objekt des Browsers zum Abbrechen
- Angular erzeugt pro Ladevorgang ein neues Signal
- Ausgelöst, wenn sich params ändert oder die Komponente endet
- fetch(url, { signal: abortSignal }) bricht die Anfrage ab
- rxResource / httpResource: Abbruch automatisch per Unsubscribe
- Verhindert veraltete Antworten (Race Condition)



Resources im Kursprojekt

```
// customer-list.component.ts – Liste per httpResource
private readonly customerResource = httpResource<ResponseArray>(() ⇒ 'api/customer');
customers = () ⇒ this.customerResource.value()?.data ?? [];
isLoading = this.customerResource.isLoading;
error = this.customerResource.error;

// customer-details.component.ts – Details per rxResource
id = input<string>();
private readonly customerResource = rxResource({
  params: () ⇒ this.id(),
  stream: ({ params }) ⇒ {
    if (!params) { throw new Error('Keine Customer-Id angegeben'); }
    return this.customerService.getById(params);   // Observable<Customer>
  }
});
customer = this.customerResource.value;
isLoading = this.customerResource.isLoading;
```

httpResource im Detail

- httpResource(() ⇒ url): Funktion, keine feste Adresse
  - Liest sie Signals, wird bei Änderung neu geladen
  - Liefert sie undefined, wird nicht geladen
- Statt Adresse auch ein Anfrage-Objekt möglich
  - { url, method, params, headers, body }
- Optionen: defaultValue, parse (z. B. Prüfung mit Zod)
- Varianten: httpResource.text(), .blob(), .arrayBuffer()

Abhängig von einem Signal

```
id = input<string>();

customer = httpResource<Customer>(
  () => this.id()
    ? `api/customer/${this.id()}`
    : undefined
);
```

Resource: Grenzen und Best Practices

- Für das Lesen gedacht, nicht für Speichern (PUT/POST)
  - Speichern weiter über den Service, danach reload()
- Bearbeitbare Kopie mit linkedSignal() aus value() ableiten
- value() nicht lesen, ohne hasValue() zu prüfen
- params schlank halten: nur die Werte, die das Laden steuern
- Teile der API sind noch als experimentell markiert

Kopie für das Formular

```
customer = this.res.value;

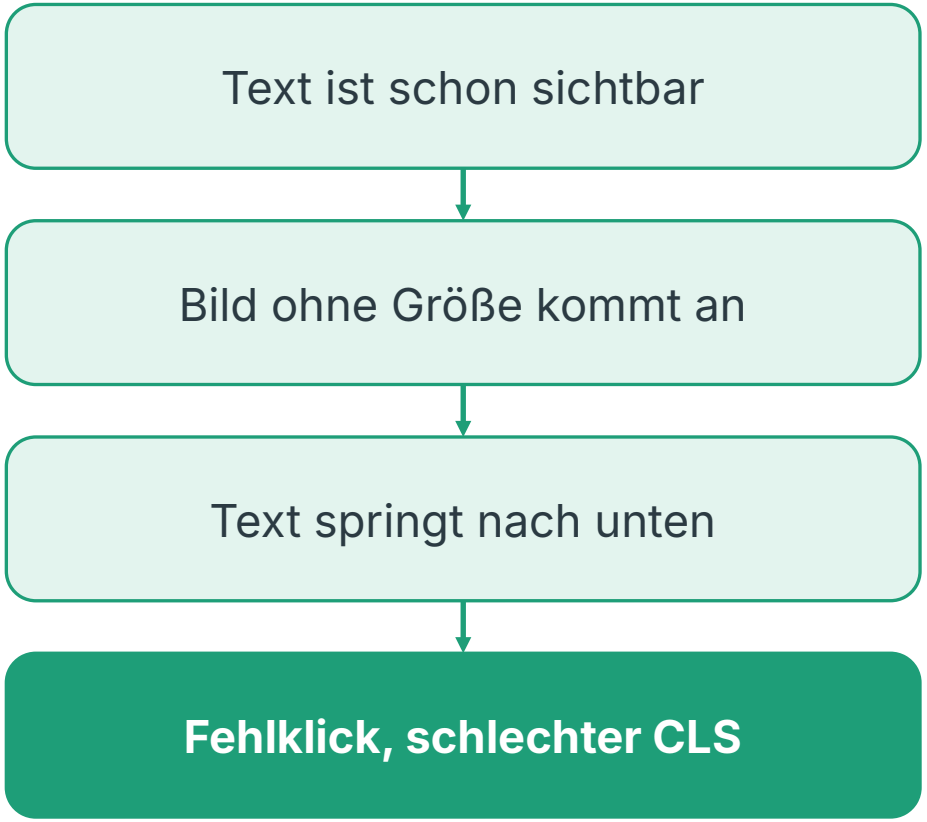
draft = linkedSignal(() => {
  const c = this.customer();
  return c ? { ...c } : undefined;
});
```

Übung 26: Resource API

- **1. CustomerListComponent: toSignal(getAll()) durch httpResource() ersetzen**
- **2. Mit isLoading() und error() Ladehinweis und Fehlermeldung anzeigen**
- **3. CustomerDetailsComponent: effect() + subscribe() durch rxResource() ersetzen**
  - params: () ⇒ this.id(), stream: ({ params }) ⇒ customerService.getById(params)
- **4. Formular bleibt an draft (linkedSignal aus customer()), nur Ladehinweis neu**
- **5. Speichern (update()) bleibt eine einmalige Mutation über CustomerService**
- Testen: zwischen Kunden wechseln und das automatische Nachladen beobachten
- **Musterlösung: 26_resourceApi (baut auf 25_httpInterceptorGuard auf)**

Bilder und Web-Performance: LCP und CLS

- Core Web Vitals: Messwerte von Google für Nutzererlebnis
- LCP (Largest Contentful Paint)
  - Zeit, bis das größte sichtbare Element gezeichnet ist
  - Oft ein Bild – Ziel: unter 2,5 Sekunden
- CLS (Cumulative Layout Shift)
  - Summe aller unerwarteten Layout-Sprünge – Ziel: unter 0,1
  - Ursache oft: Bild ohne Größenangabe



NgOptimizedImage: Was die Direktive tut

- Direktive aus @angular/common, aktiv durch ngSrc statt src
- Erzwingt width/height oder fill → Platz wird reserviert
- Lädt standardmäßig lazy: erst wenn das Bild nahe am Sichtbereich ist
- priority: sofort und mit hoher Priorität laden
- Mit Bild-Loader (CDN): automatisches srcset
- Warnungen im Entwicklungsmodus bei falschen Größen



NgOptimizedImage im Kursprojekt

```
// customer-details.component.ts
import { NgOptimizedImage } from '@angular/common';

@Component({
  selector: 'customer-details',
  imports: [DatePipe, FormsModule, NgOptimizedImage],
  templateUrl: './customer-details.component.html'
})
export class CustomerDetailsComponent { ... }

<!-- customer-details.component.html -->
@if (c.imageUrl) {
  <img class="logo" [ngSrc]="c.imageUrl" width="200" height="200" priority>
}
```

ngSrc, width/height, fill und priority

- width / height: Größe in Pixeln, ergibt das Seitenverhältnis
  - Anzeigegröße kann per CSS abweichen
- fill: Bild füllt das Elternelement aus, ohne width/height
  - Eltern brauchen position: relative, absolute oder fixed
- priority: nur für das LCP-Bild, meist eins pro Seite
- Ohne Größe oder fill: Fehler zur Laufzeit – gewollt

fill-Modus

```
<div class="hero">
  <img ngSrc="banner.jpg"
       fill priority>
</div>

/* CSS */
.hero {
  position: relative;
  height: 300px;
}
```

Übung 27: NgOptimizedImage

- **1. NgOptimizedImage aus @angular/common in CustomerDetailsComponent importieren**
- **2. [src]="c.imageUrl" durch [ngSrc]="c.imageUrl" ersetzen**
  - Bild in @if (c.imageUrl) { ... } setzen, da ngSrc einen String erwartet
- **3. width und height ergänzen (oder fill mit positioniertem Elternelement)**
- **4. Im Browser prüfen, dass die Bilder weiter korrekt angezeigt werden**
- **5. Optional: priority für das sofort sichtbare Bild setzen**
- **Musterlösung: 27_ngOptimizedImage (baut auf 26_resourceApi auf)**

Signal Queries: viewChild & contentChild

- Manchmal braucht die Klasse direkten Zugriff auf ein Element
  - Fokus setzen, Größe messen, Bibliothek anbinden
  - Methode einer Kind-Komponente aufrufen
- viewChild() / viewChildren(): im eigenen Template
- contentChild() / contentChildren(): im projizierten Inhalt
- Ergebnis ist ein Signal – ersetzt @ViewChild und Co.

Neu (Signal)	Alt (Dekorator)
viewChild()	@ViewChild
viewChildren()	@ViewChildren
contentChild()	@ContentChild

View oder Content? Der Unterschied

- View: das eigene Template der Komponente
  - Steht in templateUrl bzw. template
  - Abfrage mit viewChild()
- Content: was die Elternkomponente zwischen die Tags schreibt
  - Wird per <ng-content> angezeigt
  - Abfrage mit contentChild()

Aus Sicht von CardComponent

```
<!-- card.component.html: View -->
<div class="card" #frame>
  <ng-content />
</div>

<!-- Elterntemplate: Content -->
<app-card>
  <h3 #title>Kunde</h3>
</app-card>

frame = viewChild('frame');
title = contentChild('title');
```

Locator, required und read

- Erster Parameter (Locator): wonach wird gesucht?
  - Name einer Template-Variable: 'searchInput' für #searchInput
  - Oder eine Klasse: viewChild(CustomerFilterComponent)
- Rückgabe: Signal<T | undefined> – kann leer sein
- viewChild.required(...): Signal<T>, Fehler wenn nicht da
- Option read: welches Objekt man erhält
  - z. B. { read: ElementRef } statt der Komponenten-Instanz

Varianten

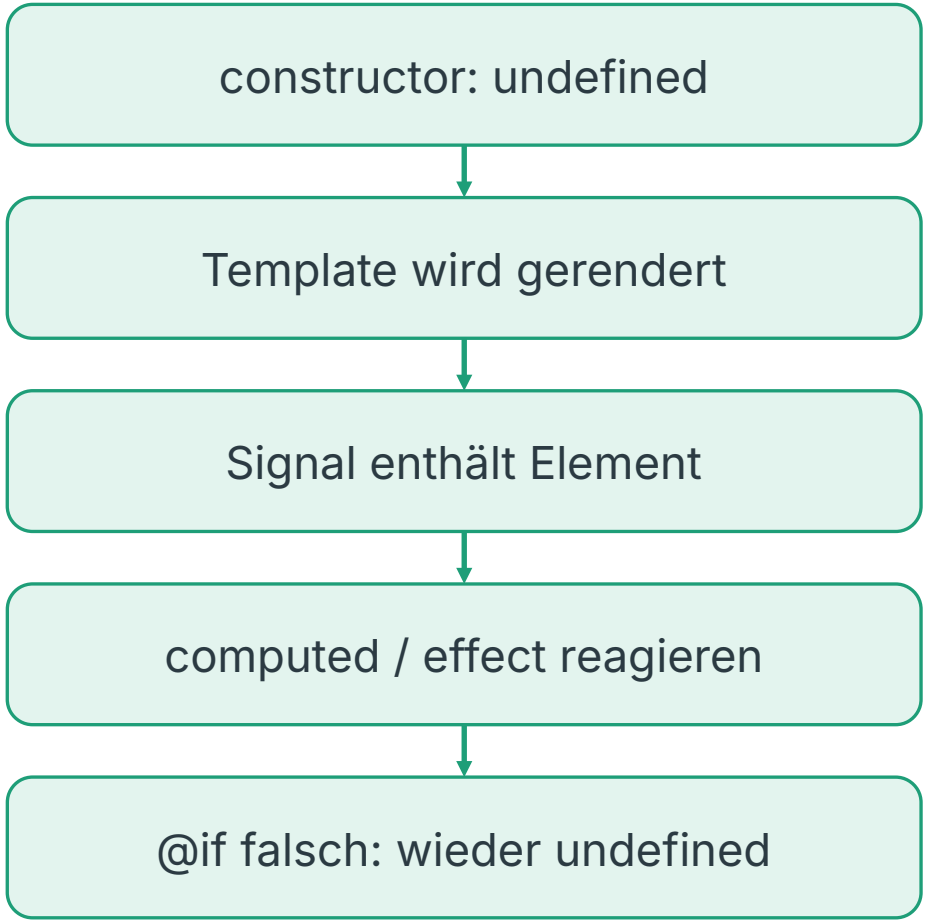
```
input = viewChild<
  ElementRef<HTMLInputElement>>(
  'searchInput');

filter = viewChild.required(
  CustomerFilterComponent);

filterEl = viewChild(
  CustomerFilterComponent,
  { read: ElementRef });
```

Wann ist der Wert verfügbar?

- Im constructor noch nicht: das Template ist nicht gerendert
- Nach dem ersten Rendern setzt Angular den Wert
- Das Signal meldet die Änderung: computed()/effect() reagieren
- Element in @if: verschwindet es, wird das Signal undefined
- Einmalige DOM-Arbeit nach dem Rendern: afterNextRender()
- Event-Handler (Klick) laufen später: Wert ist dann da



Beispiel: Suchfeld mit viewChild()

```
<!-- customer-list.component.html -->
<input #searchInput type="text" placeholder="Nach Vorname filtern..."
       [value]="filterText()" (input)="onFilterInput($event)">
<button (click)="clearFilter()">Filter leeren &amp;amp; fokussieren</button>

// customer-list.component.ts
searchInput = viewChild<ElementRef<HTMLInputElement>>('searchInput');
filterText = signal('');

onFilterInput(event: Event) {
  this.filterText.set((event.target as HTMLInputElement).value);
}

clearFilter() {
  this.filterText.set('');
  this.searchInput()?.nativeElement.focus();   // Fokus zurück ins Feld
}
```

Signal Queries: Best Practices

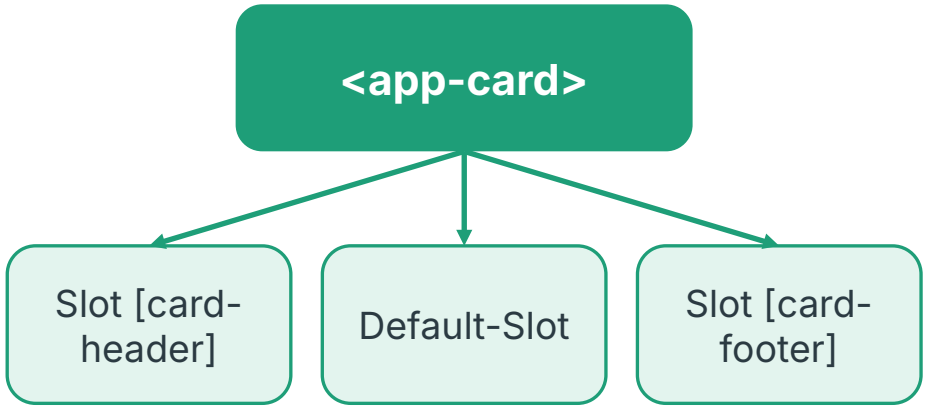
- Erst fragen: Geht es mit Bindungen im Template?
  - Werte über Signals setzen, nicht über nativeElement.value
- Direkten DOM-Zugriff auf wenige Fälle beschränken
  - Fokus, Scrollen, Messen, Drittbibliotheken
- Name im Locator muss exakt der Template-Variable entsprechen
- Signal immer mit () lesen: this.searchInput()

Übung 28: viewChild()

- **1. Im Template von CustomerListComponent ein <input #searchInput> ergänzen**
- **2. searchInput = viewChild<ElementRef<HTMLInputElement>>('searchInput')**
- **3. filteredCustomers(): Kundenliste nach dem eingegebenen Text filtern**
- **4. clearFilter(): Filtertext leeren, dann Fokus setzen**
  - this.searchInput()?.nativeElement.focus()
- **5. Im Template @for über filteredCustomers() statt customers()**
- Testen: filtern, Filter leeren, Cursor steht wieder im Feld
- **Musterlösung: 28_viewChild (baut auf 27_ngOptimizedImage auf)**

Content Projection: Das Slot-Prinzip

- Komponente liefert den Rahmen, Elternteil den Inhalt
- <ng-content> markiert die Stelle, an der Inhalt erscheint
- Mehrere Stellen (Slots) mit select unterscheiden
- Typisch: Card, Dialog, Tabs, Akkordeon, Layout
- Trennt Aufbau und Gestaltung vom fachlichen Inhalt



Card-Komponente mit drei Slots

```
// shared/card/card.component.ts
@Component({
  selector: 'app-card',
  templateUrl: './card.component.html',
  styleUrls: ['./card.component.css']
})
export class CardComponent {}

<!-- shared/card/card.component.html -->
<div class="card">
  <div class="card-header"><ng-content select="[card-header]" /></div>
  <div class="card-body"><ng-content /></div>
  <div class="card-footer"><ng-content select="[card-footer]" /></div>
</div>
```

Card verwenden: Inhalt in die Slots

```
<!-- customer-details.component.html -->
<app-card>
  <span card-header>
    Kundendetails: {{ customer()?.firstname }}
  </span>

  <form id="customer-form" #customerForm="ngForm"
        (ngSubmit)="onSubmit(customerForm)">
    <!-- Eingabefelder ... landen im Default-Slot -->
  </form>
  <div card-footer>
    <button type="submit" form="customer-form"
          [disabled]="customerForm.invalid">Speichern</button>
    <button type="button" (click)="goBack()">Zurück zur Liste</button>
  </div>
</app-card>
```

select, Default-Slot und ngProjectAs

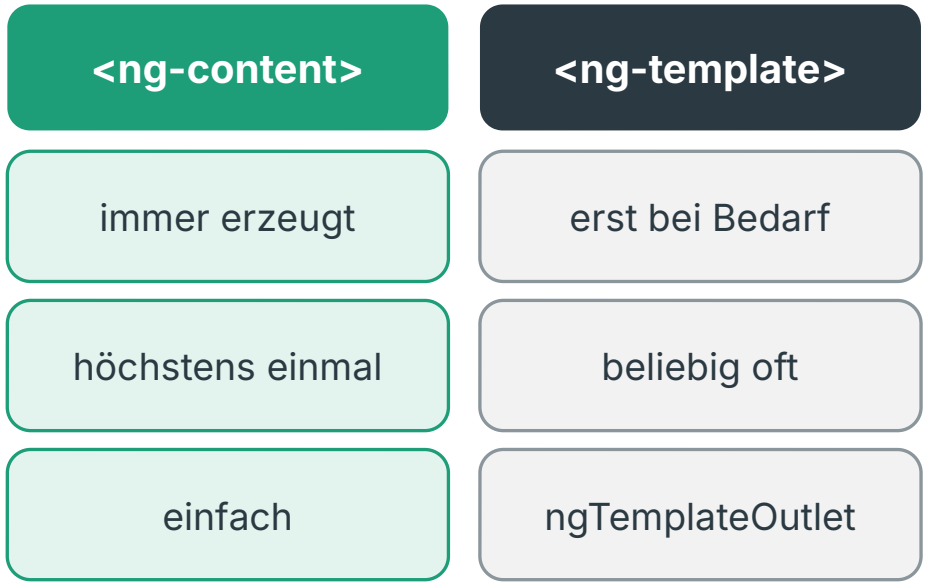
- select nimmt einen CSS-Selector: [attr], tag, .klasse
- Inhalt ohne passenden Slot landet im Default-Slot
  - Ohne Default-Slot wird er nicht angezeigt
- ngProjectAs: Wrapper tut so, als passe er zu einem Selector
  - Mehrere Elemente gemeinsam in einen Slot geben
- Standardinhalt im Slot: <ng-content>Text</ng-content>

ngProjectAs

```
<app-card>
  <ng-container
      ngProjectAs="[card-header]">
    <h3>{{ title }}</h3>
    <small>{{ sub }}</small>
  </ng-container>
  …
</app-card>
```

Wichtig: Projizierter Inhalt wird immer erzeugt

- Inhalt gehört zur Elternkomponente, nicht zur Card
  - Bindungen und Variablen aus dem Eltern-Template
- Er wird immer erzeugt – auch wenn die Card ihn nicht zeigt
  - <ng-content> nicht in @if setzen: Inhalt entsteht trotzdem
- Jeder Inhalt erscheint höchstens einmal
- Bedingt oder mehrfach anzeigen: ng-template statt ng-content



contentChild in der Card

```
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="card-header" [hidden]="!hasHeader()">
        <ng-content select="[card-header]" />
      </div>
      <div class="card-body"><ng-content /></div>
    </div>`
})
export class CardComponent {
  header = contentChild<ElementRef>('header');  // #header im Content
  hasHeader = computed(() => !!this.header());
}
<!-- Verwendung -->
<app-card><h3 #header card-header>Kunde</h3> ... </app-card>
```

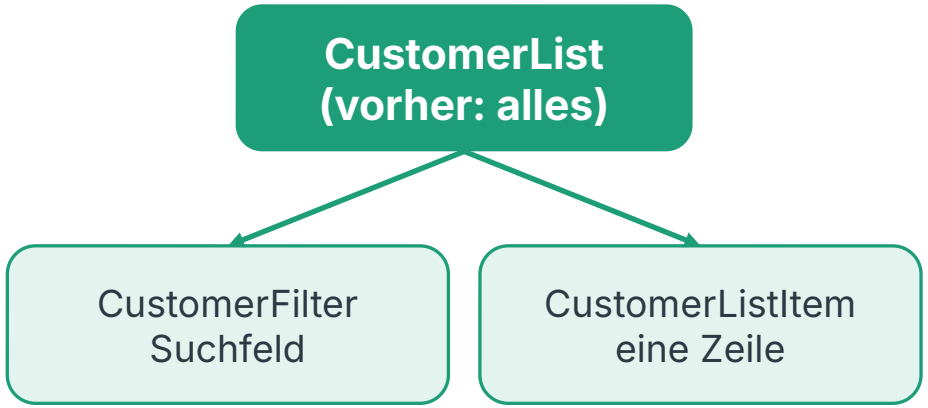
Übung 29: Content Projection

- **1. Standalone-Komponente CardComponent in src/app/shared/card/ erstellen**
- **2. Drei Slots: <ng-content select="[card-header]">, Default, select="[card-footer]"**
- **3. Styling: Rahmen, Schatten, getrennte Bereiche für Header, Body, Footer**
- **4. In CustomerDetailsComponent einbinden (imports-Array!):**
  - Kundenname im Slot card-header, Formular aus Übung 22 im Default-Slot
  - Buttons „Speichern“ und „Zurück zur Liste“ im Slot card-footer
  - Tipp: Formular mit id, Button mit type="submit" form="customer-form"
- **Musterlösung: 29_contentProjection (baut auf 28_viewChild auf)**

Komponenten sinnvoll aufteilen

Warum Komponenten aufteilen?

- Komponenten wachsen: Template, Zustand, Laden, Filtern, Auswahl in einer Datei
- Folgen einer zu großen Komponente:
  - schwer zu lesen und zu ändern
  - Tests brauchen viel Vorbereitung (HTTP, Router, DOM)
  - Teile lassen sich nicht wiederverwenden
- Ziel: kleine Bausteine mit klarer Aufgabe und klarer Schnittstelle

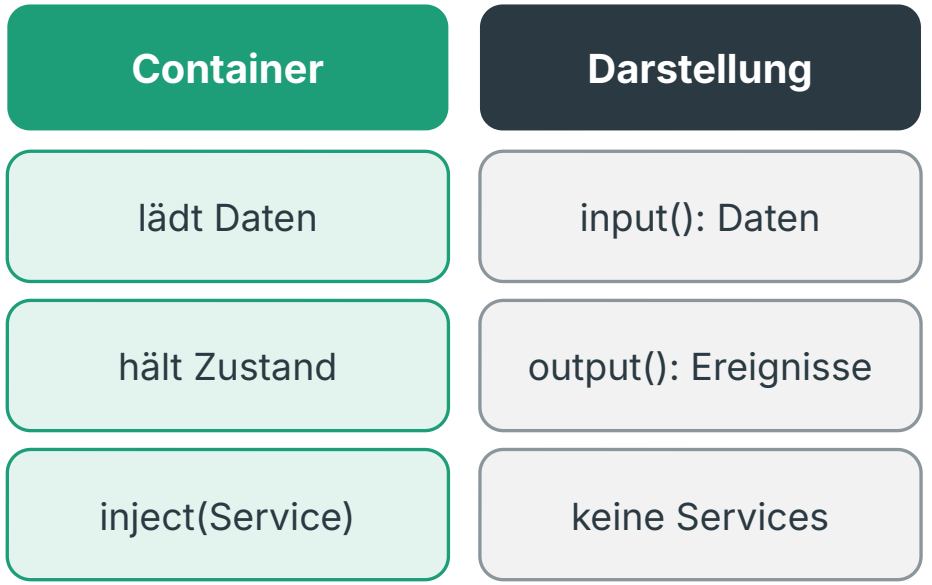


Kriterien für das Aufteilen

- Ein Satz ohne „und“: Lässt sich die Aufgabe so beschreiben?
- Wiederverwendung: Wird der Teil an mehreren Stellen gebraucht?
- Unterschiedliche Änderungsgründe: Ändern sich Teile aus verschiedenen Gründen?
- Größe als Warnsignal: langes Template, viele Felder, viele inject()
- Testbarkeit: Lässt sich der Teil allein prüfen?
- Nicht übertreiben: Jede neue Komponente kostet eine Schnittstelle

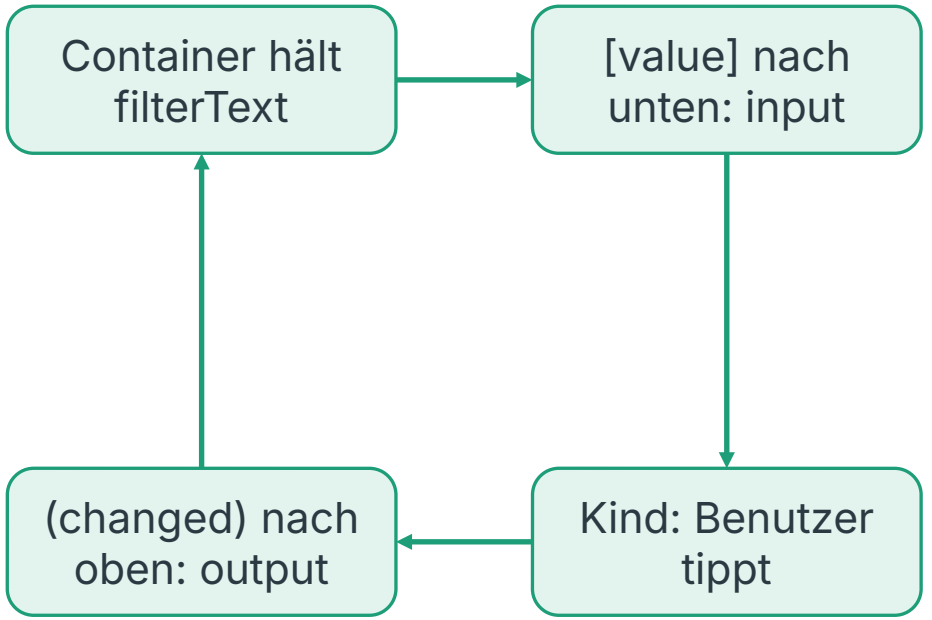
Container- und Darstellungskomponenten

- Container (smart): lädt Daten, hält Zustand, ruft Services
  - Beispiel: CustomerListComponent
- Darstellung (presentational): zeigt an, meldet Ereignisse
  - kennt keine Services, nur input() und output()
  - Beispiele: CustomerFilter, CustomerListItem
- Darstellungskomponenten sind leicht wiederzuverwenden und zu testen



Datenfluss: Daten runter, Ereignisse hoch

- Container besitzt den Zustand: filterText = signal('')
- Nach unten per Property Binding: [value]="filterText()"
- Kind liest ihn über value = input('')
- Benutzer tippt: Kind ruft changed.emit(text)
- Nach oben per Event Binding: (changed)="onFilterChanged($event)"
- Nur der Container ändert den Zustand: eine Richtung, ein Besitzer



Darstellungskomponente: CustomerFilter

```
@Component({
  selector: 'customer-filter', templateUrl: './customer-filter.component.html'
})
export class CustomerFilterComponent {
  value = input('');                // Eingabe: aktueller Filtertext
  changed = output<string>();       // Ausgabe: neuer Text
  cleared = output<void>();         // Ausgabe: ohne Nutzdaten

  searchInput = viewChild<ElementRef<HTMLInputElement>>('searchInput');

  onInput(event: Event) {
    this.changed.emit((event.target as HTMLInputElement).value);
  }
  clear() {
    this.cleared.emit();
    this.searchInput()?.nativeElement.focus();
  }
}
```

input() und output() im Detail

- input(''): Angular liefert ein schreibgeschütztes Signal (InputSignal<string>)
  - Startwert '' gilt, solange die Eltern nichts binden
- input.required<T>(): Eltern müssen binden, sonst Fehler beim Kompilieren
- output<T>(): liefert ein OutputEmitterRef<T> mit emit(wert)
  - output<void>(): emit() ohne Argument
- Im Eltern-Template: $event ist genau der Wert aus emit(...)

customer-list.component.html

```
<customer-filter
  [value]="filterText()"
  (changed)=
    "onFilterChanged($event)"
  (cleared)=
    "onFilterCleared()" />

// in der Klasse:
onFilterChanged(t: string) {
  this.filterText.set(t);
}
```

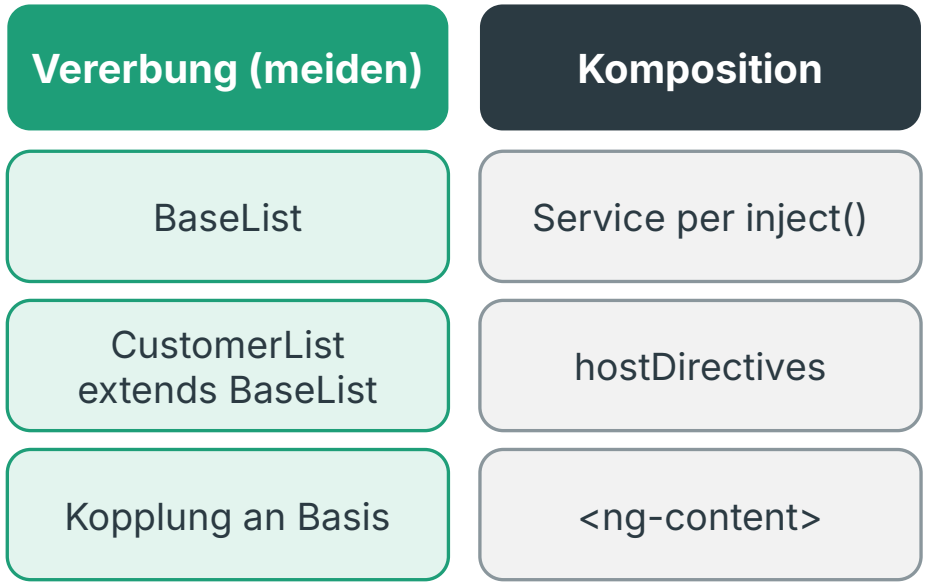
Zeilenkomponente: Attribut-Selektor und host

```
@Component({
  selector: 'li[customer-list-item]',     // Attribut-Selektor auf <li>
  imports: [RouterLink],
  templateUrl: './customer-list-item.component.html',
  host: {
    '[class.discontinued]': 'customer().discontinued',  // Klasse am Host-Element
    '[class.selected]': 'selected()',
    '(click)': 'chosen.emit(customer())'                // Klick am Host-Element
  }
})
export class CustomerListItemComponent {
  customer = input.required<Customer>();
  selected = input(false);
  chosen = output<Customer>();
}

// Container: <li customer-list-item [customer]="customer"
//   [selected]="isSelectedCustomer(customer)" (chosen)="listItemClicked($event)">
```

Komposition statt Vererbung

- Komposition: größere Bausteine aus kleinen zusammensetzen
  - <ng-content> (Übung 29), Kind-Komponenten, Direktiven, Services
- Vererbung: Komponente erweitert eine Basisklasse (extends)
  - versteckte Kopplung: Änderung an der Basis trifft alle
  - nur eine Basisklasse möglich, Template wird nicht geerbt
- Faustregel: Verhalten hineinholen statt erben



Vererbung: so sieht es aus

```
// Basisklasse mit Angular-Funktionen braucht einen eigenen Dekorator
@Directive()
export abstract class BaseListComponent<T> {
  selected = signal<T | undefined>(undefined);
  select(item: T) { this.selected.set(item); }
  isSelected(item: T) { return item === this.selected(); }
}

@Component({ selector: 'customer-list', templateUrl: './customer-list.component.html' })
export class CustomerListComponent extends BaseListComponent<Customer> {
  // erbt selected, select, isSelected
}

// Probleme:
// - Template und Styles werden nicht geerbt, nur Klassenlogik
// - Konstruktor-Parameter der Basis müssen per super(...) durchgereicht werden
// - Jede Änderung an BaseListComponent trifft alle Listen der Anwendung
// - Eine zweite Basisklasse (z. B. für Sortierung) ist nicht möglich
```

Komposition: Verhalten als Service

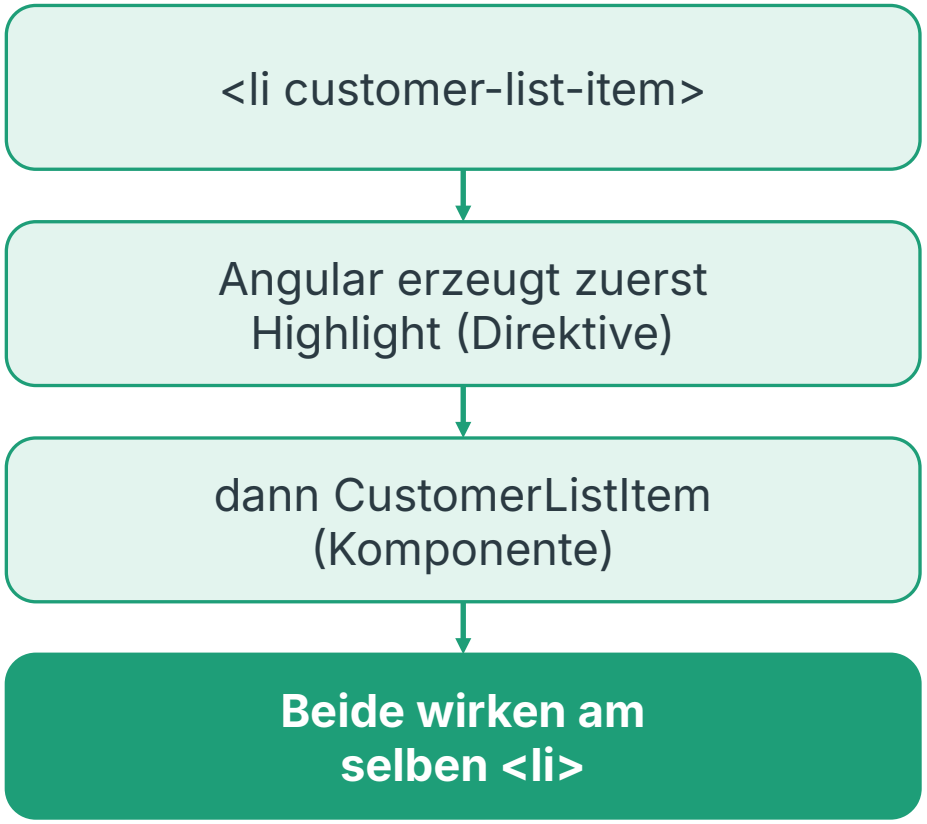
```
// Gemeinsames Verhalten in einer eigenen Klasse, ohne providedIn
@Injectable()
export class Selection<T> {
  readonly selected = signal<T | undefined>(undefined);
  select(item: T) { this.selected.set(item); }
  isSelected(item: T) { return item === this.selected(); }
}

@Component({
  selector: 'customer-list', templateUrl: './customer-list.component.html',
  providers: [Selection]            // jede Liste bekommt ihre eigene Instanz
})
export class CustomerListComponent {
  readonly selection = inject<Selection<Customer>>(Selection);
}

// Template: [selected]="selection.isSelected(customer)"
// Weitere Bausteine (Sortierung, Paging) einfach zusätzlich injizieren
```

hostDirectives: Direktiven mitbringen

- Direktive: fügt einem Element Verhalten hinzu (z. B. Hervorheben)
- Normal: Nutzer schreibt sie ins Template: <li appHighlight>
- hostDirectives: Komponente bringt die Direktive selbst mit
  - Angular erzeugt sie automatisch am Host-Element
- Inputs und Outputs der Direktive nur, wenn ausdrücklich freigegeben
- Komponente kann die Direktive per inject() nutzen



Host-Direktiven laufen vor der Komponente

hostDirectives im Code

```
@Directive({
  selector: '[appHighlight]',
  host: {
    '[style.background]': 'active() ? color() : null',
    '(mouseenter)': 'active.set(true)',
    '(mouseleave)': 'active.set(false)'
  }
})
export class Highlight { color = input('lightyellow'); active = signal(false); }

@Component({
  selector: 'li[customer-list-item]',
  hostDirectives: [{ directive: Highlight, inputs: ['color: highlightColor'] }]
})
export class CustomerListItemComponent { /* wie bisher */ }
// Nutzung: <li customer-list-item highlightColor="lightblue" [customer]="c">
```

hostDirectives: Regeln

- Nur Standalone-Direktiven (in Angular 21 der Standard)
- Der Selektor der Direktive spielt keine Rolle
- Inputs und Outputs: inputs: ['name'] oder ['name: alias']
- Reihenfolge: erst Host-Direktiven, dann die Komponente
- Zugriff aus der Komponente: inject(Highlight)
- Nutzen: gemeinsames Verhalten ohne Basisklasse, mehrere kombinierbar

Trennung von Oberfläche und Fachlogik

- Ebene 1: Komponente – Darstellung und Eingaben
- Ebene 2: Service oder Store – Fachlogik und Zustand
- Ebene 3: API-Client (HttpClient) – Zugriff nach außen
- Abhängigkeiten zeigen nur nach unten
- Der Service kennt keine Komponente
- Reine Berechnungen als pure Funktion oder Pipe: ohne Angular testbar
- Keine Fachregeln im Template: lieber computed oder Pipe



Wartbarkeit: Merkmale stabiler Codebasen

- Namen sagen, was etwas ist (CustomerListComponent), nicht wie (Helper2)
- Dateistruktur nach Features (customer/, auth/, shared/), nicht nach Typ
- TypeScript strict, kein any ohne Begründung
- Kleine Funktionen und Komponenten, wenig Verschachtelung
- Automatische Prüfung: ESLint, Prettier und Tests in der Pipeline
- Entscheidungen festhalten: kurze Notiz (ADR), z. B. „Warum Store statt Service?“

Struktur und Testbarkeit hängen zusammen

- Kleine Komponente mit klaren Inputs und Outputs: Test in wenigen Zeilen
- Viele versteckte Abhängigkeiten: Test braucht viel Mocking
- Pflicht-Input vor dem ersten Rendern setzen: setInput(...)
- Output prüfen: abonnieren und Wert vergleichen
- Faustregel: Schwer zu testen heißt oft schlecht geschnitten
- Vertiefung im Kapitel zur Testbarkeit

customer-list-item.spec.ts

```
const f = TestBed.createComponent(
  CustomerListItemComponent);
f.componentRef.setInput(
  'customer', anna);
f.detectChanges();

let got: Customer | undefined;
f.componentInstance.chosen
  .subscribe(c => got = c);
f.nativeElement.click();
expect(got).toBe(anna);
```

Stolperfallen beim Aufteilen

- Kind ändert Eingabedaten selbst: input() ist schreibgeschützt
  - Stattdessen Ereignis per output() melden
- Werte durch viele Ebenen durchreichen: lieber Service oder Store
- Ungültiges HTML durch Komponenten-Tags in <ul> oder <table>
  - Lösung: Attribut-Selektor wie li[customer-list-item]
- Zu fein zerlegt: Komponenten ohne eigene Aufgabe wieder zusammenlegen

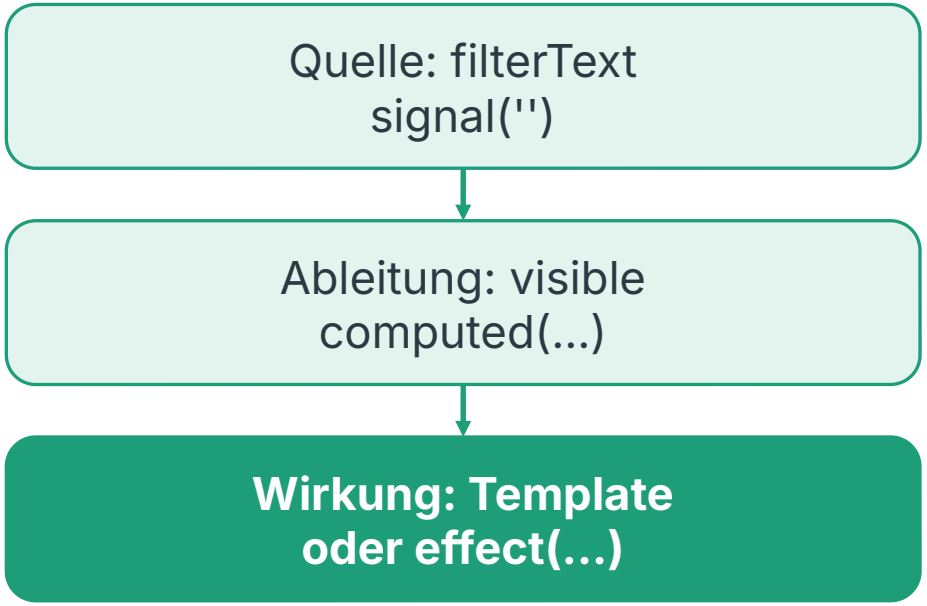
Übung 31: Kundenliste aufteilen

- **1. CustomerFilterComponent: Eingabefeld und Button „Filter leeren & fokussieren“**
  - value = input(''), changed = output<string>(), cleared = output<void>(); viewChild wandert mit
- **2. CustomerListItemComponent für eine Zeile (Stile discontinued und selected wandern mit)**
  - customer = input.required<Customer>(), selected = input(false), chosen = output<Customer>()
- **3. CustomerListComponent behält nur: Daten laden, Filterzustand, Auswahlzustand**
- **4. Smoke-Test: Laden, Filtern, Fokus, Markieren, Details; ng build und ng test grün**
- **5. Reflexion: Welche Komponente ist jetzt leicht testbar? Welche Abhängigkeiten sind weg?**
- **Musterlösung: 31_komponenten_aufteilen (baut auf 29_contentProjection auf)**

Reaktivität und Datenfluss

Grundmodell: Quelle, Ableitung, Wirkung

- Quelle (signal): hält Zustand und wird gesetzt
- Ableitung (computed): berechnet aus Quellen neue Werte
- Wirkung (Template oder effect): greift nach außen
  - Oberfläche, Browser-Speicher, Logging
- Angular verfolgt selbst, wer welches Signal liest
- Nichts muss von Hand abonniert oder abgemeldet werden



Änderungen fließen in Pfeilrichtung

Grundmodell im Code

```
export class CustomerListComponent {
  // Quellen: halten Zustand
  customers = signal<Customer[]>([]);
  filter = signal('');

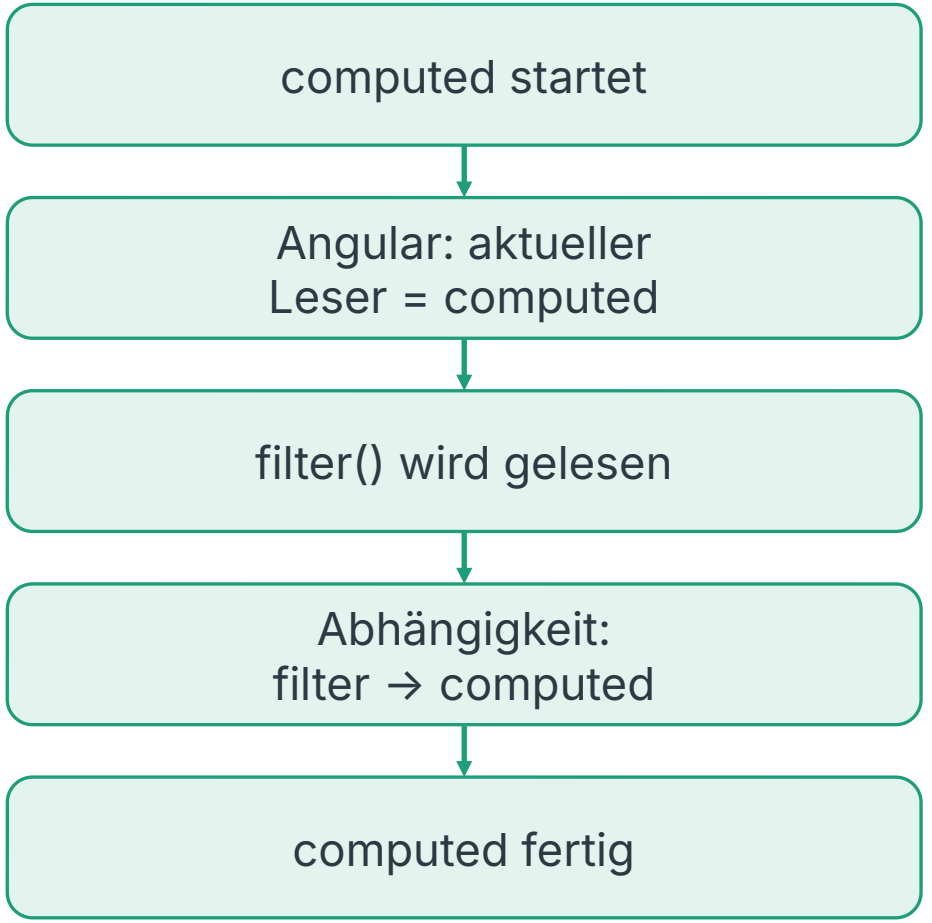
  // Ableitung: liest zwei Quellen
  visible = computed(() ⇒
    this.customers().filter(c ⇒ c.lastname.includes(this.filter()))
  );

  constructor() {
    // Wirkung: liest die Ableitung, schreibt nach außen
    effect(() ⇒ console.log('Treffer:', this.visible().length));
  }
}

// Template (ebenfalls eine Wirkung):  @for (c of visible(); track c.id) { ... }
```

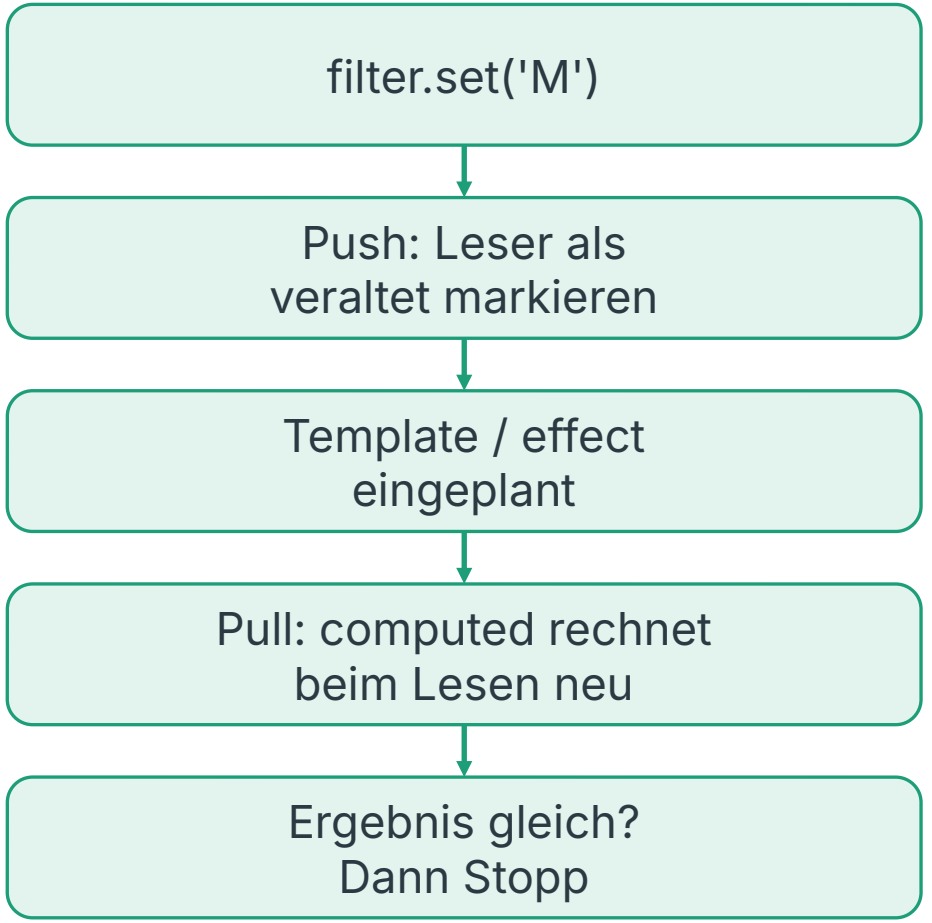
Wie Angular Abhängigkeiten erkennt

- Ein Signal lesen heißt: es aufrufen, z. B. filter()
- Reaktiver Kontext: computed, effect, Template, linkedSignal
- Angular merkt sich den gerade laufenden Leser
- Jedes gelesene Signal meldet sich bei diesem Leser an
- Kein reaktiver Kontext: Konstruktor, Event-Handler, setTimeout
  - Dort gelesen: nur der aktuelle Wert, keine Abhängigkeit



Push und Pull: wie Änderungen ankommen

- set() ändert den Wert der Quelle
- Push: Angular benachrichtigt alle Leser „du bist veraltet“
  - Es wird dabei noch nichts berechnet
- Template und effect werden zur Ausführung eingeplant
- Pull: Ein computed rechnet erst, wenn jemand es liest
- Gleiches Ergebnis wie vorher: Kette stoppt hier



Glitch-Freiheit: keine Zwischenzustände

- Glitch: kurzer, falscher Mischwert aus alt und neu
- Beispiel rechts: c hängt direkt und über b von a ab
- Signals: Werte werden erst beim Lesen geholt (Pull)
  - c sieht immer a und b im selben Stand
- effect gibt nur 3 und danach 6 aus, nie 4
- Mehrere set() hintereinander: ein einziger Lauf

Diamant-Abhängigkeit

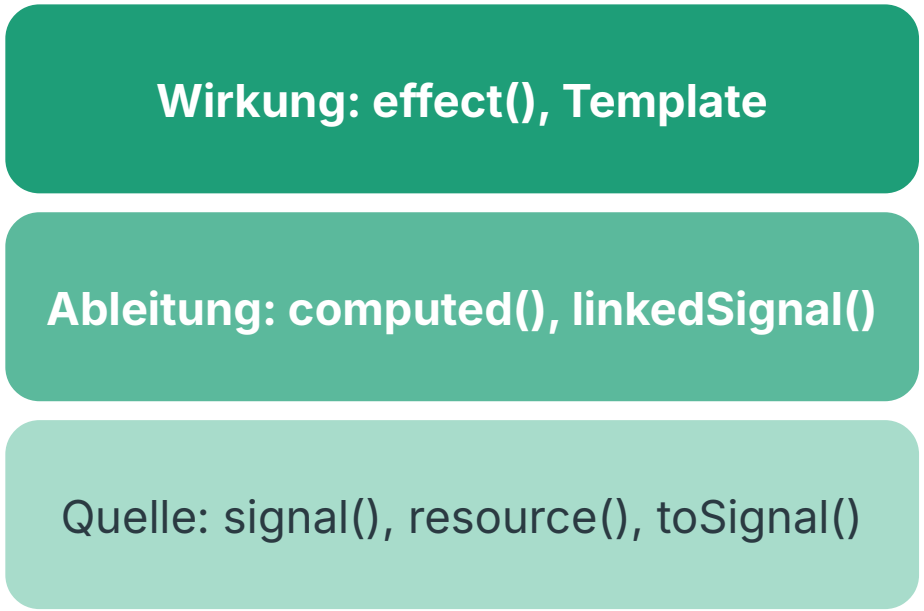
```
const a = signal(1);
const b = computed(() => a() * 2);
const c = computed(() =>
  a() + b());

effect(() => console.log(c()));
// Ausgabe: 3

a.set(2);
// Ausgabe: 6  (nie 4)
```

Landkarte der Reaktivitäts-Bausteine

- signal(): Zustand halten, schreibbar – Filtertext, Auswahl
- computed(): Wert ableiten, nur lesbar – gefilterte Liste, Summen
- linkedSignal(): abgeleitet, aber überschreibbar – Auswahl mit Rücksprung
- effect(): Nebenwirkung – localStorage, Logging, Drittbibliothek
- resource() / httpResource(): asynchron laden – Daten vom Server
- toSignal() / toObservable(): Brücke zu RxJS – z. B. Router-Ereignisse



Daten fließen von unten nach oben

signal(): die vollständige API

```
import { signal, isSignal, Signal, WritableSignal } from '@angular/core';

const count: WritableSignal<number> = signal(0);    // Startwert 0, Typ wird abgeleitet
const selected = signal<Customer | undefined>(undefined);   // Typ ausdrücklich angeben

count();                    // lesen: 0
count.set(5);               // neuen Wert setzen: 5
count.update(n => n + 1);     // aus dem alten Wert berechnen: 6

const ro: Signal<number> = count.asReadonly();
ro();                       // lesen geht: 6
// ro.set(1);               // Fehler beim Kompilieren: Signal hat kein set

isSignal(count);            // true

// Optionen als zweiter Parameter: signal(0, { equal: ..., debugName: 'count' })
```

set, update und asReadonly erklärt

- set(wert): ersetzt den Wert, Leser werden benachrichtigt
- update(fn): Angular ruft fn mit dem aktuellen Wert auf
  - Rückgabewert von fn wird der neue Wert
- Gleicher Wert wie vorher: keine Benachrichtigung
- asReadonly(): gleiche Daten, aber ohne set und update
  - Muster: privat schreiben, öffentlich nur lesen

cart.store.ts

```
@Injectable({providedIn:'root'})
export class CartStore {
  private _items =
    signal<Item[]>([]);
  items =
    this._items.asReadonly();

  add(item: Item) {
    this._items.update(list =>
      [...list, item]);
  }
}
```

Die Option equal: wann gilt ein Wert als neu?

- Standard: Vergleich mit Object.is (fast wie ===)
  - Objekte und Arrays: nur neue Referenz zählt als Änderung
- equal: eigene Funktion (a, b) ⇒ boolean
  - Angular ruft sie mit altem und neuem Wert auf
  - true: gilt als gleich, keine Benachrichtigung
- Gibt es bei signal, computed und linkedSignal
- Vorsicht: falsches equal verschluckt echte Änderungen

equal-Option

```
const current = signal<Customer>(
  anna,
  { equal: (a, b) =>
      a.id === b.id });

current.set({ ...anna });
// gleiche id: keine
// Benachrichtigung
```

Lokaler und abgeleiteter Zustand

- Lokaler Zustand (Quelle): Benutzer oder Ereignis ändert ihn direkt
  - Beispiele: Filtertext, ausgewählter Kunde
- Abgeleiteter Zustand: folgt zwingend aus anderem Zustand
  - Beispiele: gefilterte Liste, Anzahl der Treffer
  - Nicht speichern, sondern berechnen
- Regel: Single Source of Truth
  - Derselbe Fakt wird nur an einer Stelle gespeichert

Doppelter oder abgeleiteter Zustand

```
// Falsch: doppelter Zustand, muss von Hand synchron gehalten werden
customers = signal<Customer[]>([]);
filtered  = signal<Customer[]>([]);   // wird irgendwo "mitgepflegt"

onFilterChanged(text: string) {
  this.filterText.set(text);
  this.filtered.set(this.customers().filter(...));   // vergessen = Fehler
}

// Richtig: abgeleitet, kann nicht auseinanderlaufen
filtered = computed(() => {
  const text = this.filterText().toLowerCase();
  return this.customers().filter(c => c.firstname.toLowerCase().includes(text));
});
```

Befund im Startprojekt der Übung 32

```
// Startstand nach Übung 31 (CustomerListComponent)

// Pfeilfunktion statt computed: kein Zwischenspeicher
customers = () ⇒ this.customerResource.value()?.data ?? [];

// Normale Methode: läuft bei jeder Change Detection neu
filteredCustomers() {
  const filter = this.filterText().toLowerCase();
  return this.customers().filter(c ⇒ c.firstname?.toLowerCase().includes(filter));
}

// Das funktioniert, aber:
//  - das Ergebnis wird nicht zwischengespeichert
//  - jeder Aufruf im Template rechnet erneut
//  - Abhängigkeiten sind nicht als Graph sichtbar
// Umbau zu computed ist Teil A von Übung 32
```

computed im Detail

- Lazy: rechnet erst, wenn der Wert zum ersten Mal gelesen wird
- Memoized: liefert den gespeicherten Wert, solange nichts sich ändert
- Rein: keine Nebenwirkungen, kein Schreiben auf Signals
  - Schreiben im computed: Fehler NG0600
- Ergebnis ist Signal<T>: nur lesbar, kein set
- Option equal wie bei signal

computed

```
const count = signal(2);

const double = computed(() => {
  console.log('rechne');
  return count() * 2;
});

double(); // rechnet: 4
double(); // aus dem Speicher
count.set(3);
double(); // rechnet neu: 6
```

computed: dynamische Abhängigkeiten

- Abhängigkeiten werden bei jedem Lauf neu ermittelt
- Es zählt nur, was im letzten Lauf wirklich gelesen wurde
  - Signal in nicht ausgeführtem Zweig: keine Abhängigkeit
- Fehler in der Funktion: wird gespeichert und beim Lesen geworfen
- Kein async: Signals nach await werden nicht erfasst
  - Asynchrones Laden: resource oder httpResource

dynamische Abhängigkeit

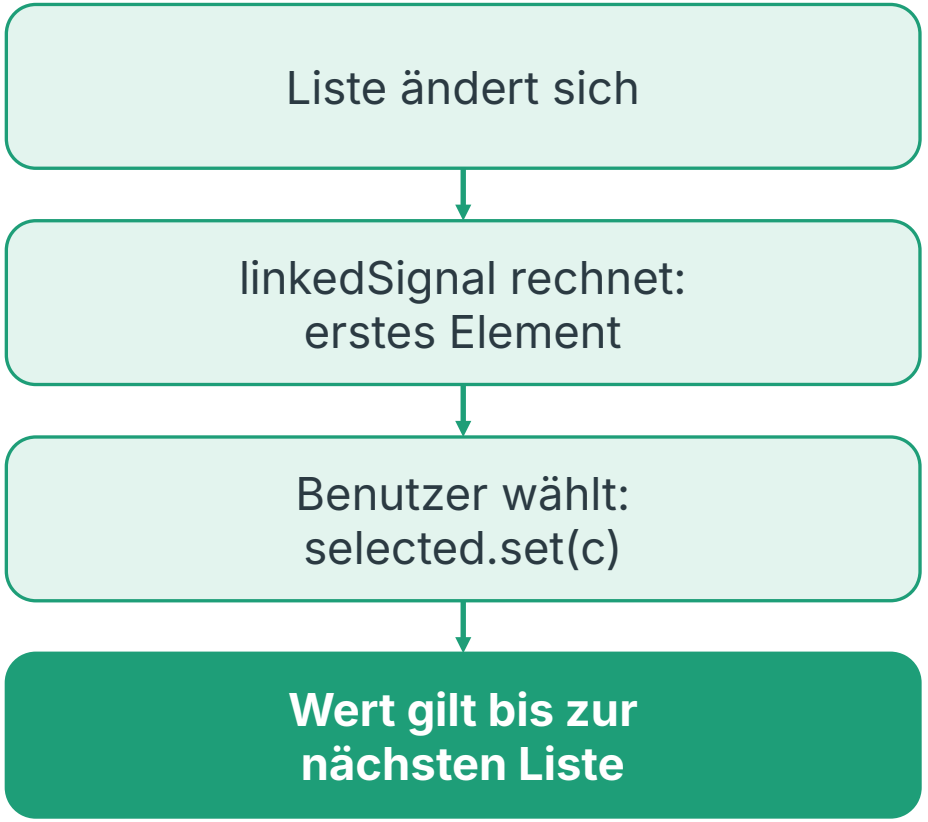
```
const showDetails = signal(false);
const name = signal('Ada');

const label = computed(() =>
  showDetails() ? name() : '-');

label();           // '-'
name.set('Bob'); // kein Lauf
showDetails.set(true);
label();           // 'Bob'
```

linkedSignal: abgeleitet, aber änderbar

- Problem: Auswahl hängt von der Liste ab
  - Neue Liste: Auswahl soll zurückspringen
  - Benutzer soll trotzdem selbst wählen können
- computed: nicht schreibbar – signal: folgt der Liste nicht
- linkedSignal verbindet beides: berechnet und mit set() änderbar
- Ein gesetzter Wert gilt bis zur nächsten Änderung der Quelle



linkedSignal: Kurzform

```
customers = input.required<Customer[]>();

// Kurzform: nur eine Berechnungsfunktion
// Auswahl springt auf das erste Element, sobald sich die Liste ändert
selected = linkedSignal(() ⇒ this.customers()[0]);

onPick(c: Customer) {
  this.selected.set(c);         // Benutzer überschreibt den Wert
}

// Typ: WritableSignal<Customer> – lesen mit selected(), schreiben mit set/update

// Ersetzt das Muster:
//   selected = signal<Customer | undefined>(undefined);
//   effect(() ⇒ this.selected.set(this.customers()[0]));   // effect setzt Signal
```

linkedSignal: Langform

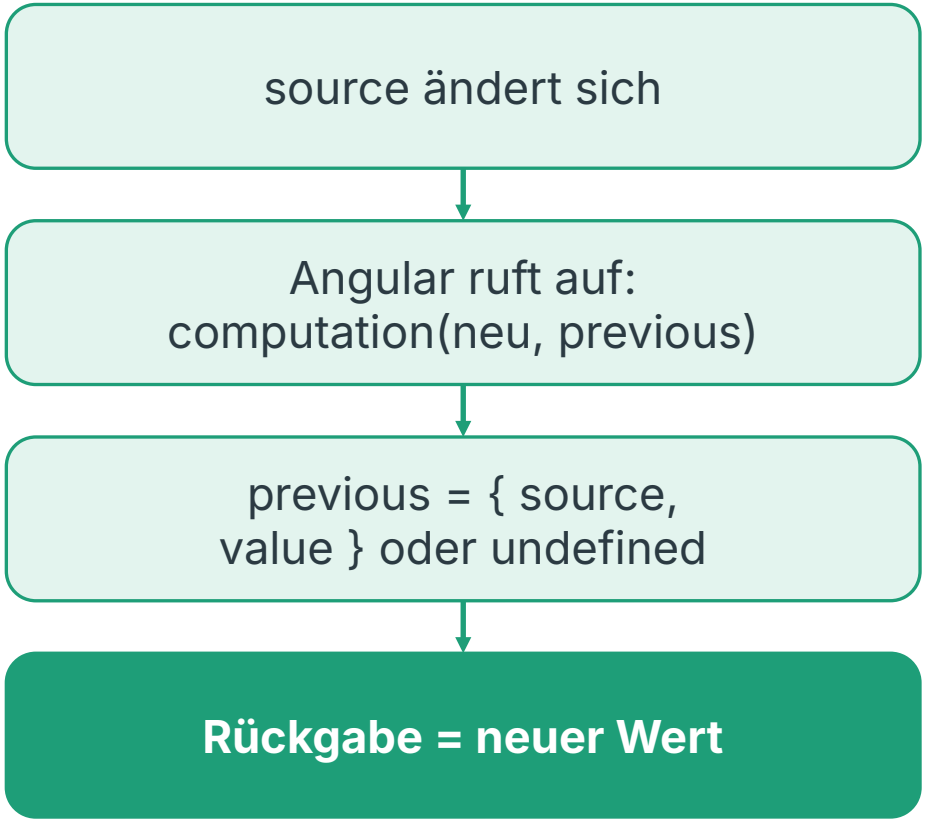
```
selected = linkedSignal<Customer[], Customer | undefined>({
  // source: was beobachtet wird
  source: this.customers,

  // computation: Angular ruft sie auf, wenn sich source ändert
  computation: (newList, previous) => {
    // previous ist beim ersten Lauf undefined,
    // danach { source: alte Liste, value: bisherige Auswahl }
    const keep = newList.find(c => c.id === previous?.value?.id);
    return keep ?? newList[0];    // Auswahl behalten, sonst erstes Element
  },

  // optional: equal: (a, b) => a?.id === b?.id
});
```

Die Parameter von computation

- Aufrufer: Angular, beim ersten Lesen und nach Änderung von source
- 1. Parameter: aktueller Wert von source (hier die neue Liste)
- 2. Parameter previous: vorheriger Stand oder undefined
  - previous.source: alter Wert der Quelle
  - previous.value: letzter Wert, auch einer per set()
- Rückgabe: neuer Wert des linkedSignal
- Namen frei wählbar, Reihenfolge ist fest



Nebenwirkungen und effect()

- Nebenwirkung: verändert etwas außerhalb des Signal-Graphen
- Gute Gründe für einen effect:
  - Zustand in localStorage oder sessionStorage sichern
  - Logging und Analytics
  - Drittbibliothek (Diagramm, Karte, Canvas) mit Daten versorgen
  - Imperativer DOM-Zugriff, der nicht per Binding möglich ist
- Kein guter Grund: ein anderes Signal setzen

effect: Grundbeispiel aus Übung 32

```
const FILTER_STORAGE_KEY = 'customerFilter';

function saveFilter(text: string): void {
  try {
    localStorage.setItem(FILTER_STORAGE_KEY, text);
  } catch { /* localStorage fehlt oder ist gesperrt: ignorieren */ }
}

export class CustomerListComponent {
  filterText = signal(loadFilter());       // Startwert aus dem Speicher

  constructor() {
    // Angular ruft die Funktion einmal auf und danach bei jeder Änderung von filterText
    effect(() ⇒ saveFilter(this.filterText()));
  }
}
```

effect(): Signatur und Rückgabewert

- Parameter 1: die Effekt-Funktion, Angular ruft sie auf
  - Sie bekommt einen Parameter: onCleanup
- Parameter 2: Optionen (injector, manualCleanup, debugName)
- Rückgabe: EffectRef mit der Methode destroy()
- Erster Lauf: kurz nach dem Erzeugen, nicht sofort
- Weitere Läufe: nach Änderung eines gelesenen Signals

Signatur (vereinfacht)

```
function effect(
  effectFn: (onCleanup:
    EffectCleanupRegisterFn)
    => void,
  options?: CreateEffectOptions
): EffectRef;

type EffectCleanupRegisterFn =
  (cleanupFn: () => void)
    => void;
```

Was ist onCleanup?

- Angular ruft die Effekt-Funktion auf und übergibt onCleanup
- onCleanup ist selbst eine Funktion (EffectCleanupRegisterFn)
- Aufruf onCleanup(fn): meldet eine Aufräumfunktion fn an
- Angular ruft fn auf:
  - vor dem nächsten Lauf des effect
  - beim Zerstören (Komponente weg oder destroy())
- Name frei wählbar, Parameter darf auch fehlen

Wer ruft wen?

```
effect((onCleanup) => {
  // Angular hat diese
  // Funktion aufgerufen und
  // onCleanup übergeben

  onCleanup(() => {
    // Angular ruft das
    // später auf
  });
});
```

onCleanup im Einsatz: Polling

```
export class DashboardComponent {
  pollMs = signal(5000);              // Abfrage-Intervall in Millisekunden

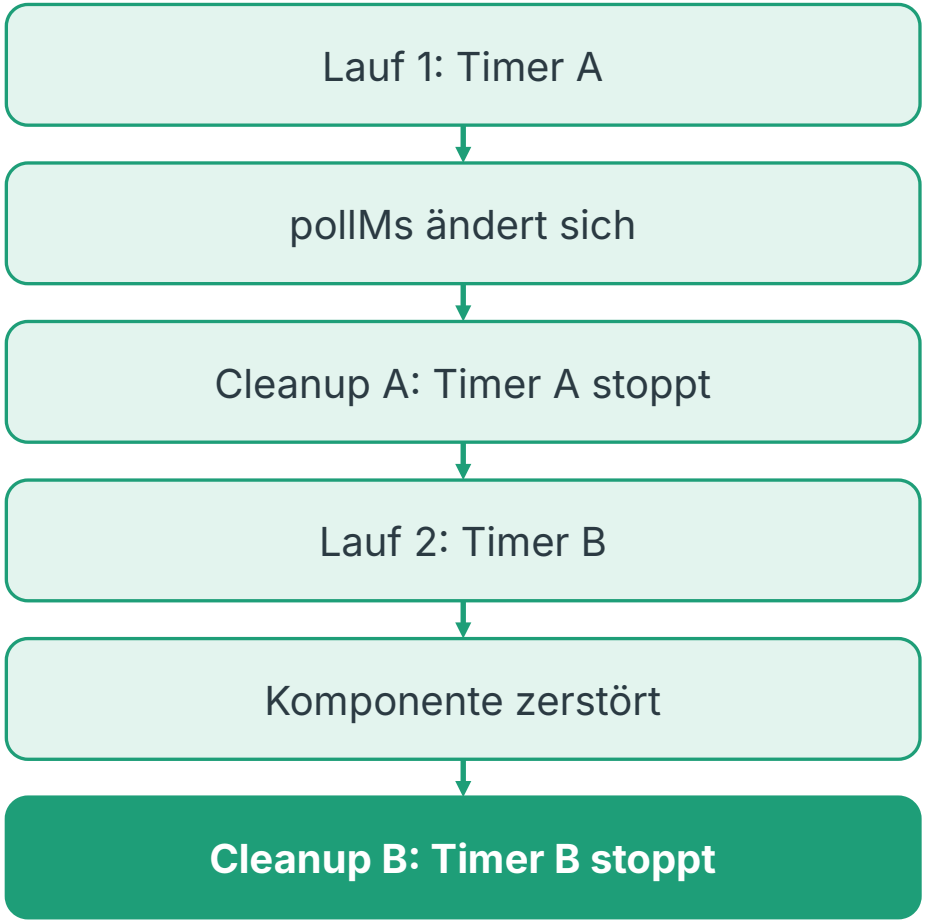
  constructor() {
    effect((onCleanup) => {
      const ms = this.pollMs();                       // Abhängigkeit
      const id = setInterval(() => this.refresh(), ms);    // Nebenwirkung starten
      console.log('Timer gestartet', id, ms);

      onCleanup(() => {                               // Aufräumen anmelden
        clearInterval(id);
        console.log('Timer gestoppt', id);
      });
    });
  }

  refresh() { /* Daten neu laden */ }
}
```

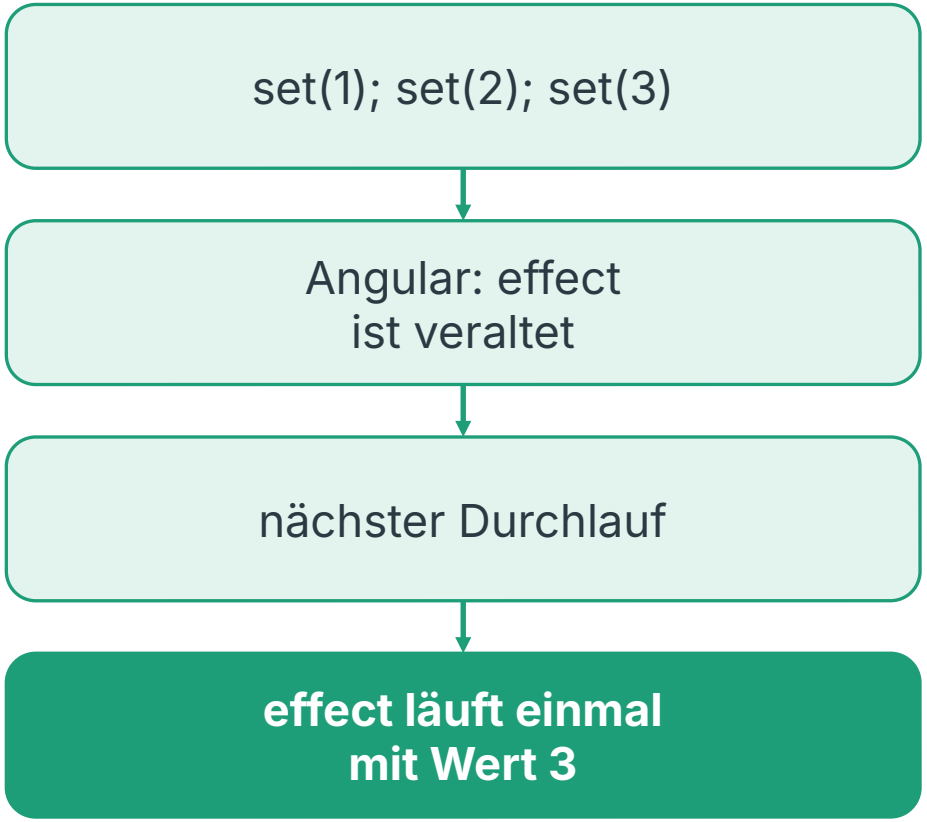
Ablauf: Läufe und Aufräumen

- Lauf 1: Timer A starten, Cleanup A anmelden
- pollMs.set(1000): effect ist veraltet
- Vor Lauf 2: Angular ruft Cleanup A, Timer A stoppt
- Lauf 2: Timer B starten, Cleanup B anmelden
- Komponente wird zerstört: Angular ruft Cleanup B
- Ergebnis: nie zwei Timer gleichzeitig, kein Leck



Wie ein effect ausgeführt wird

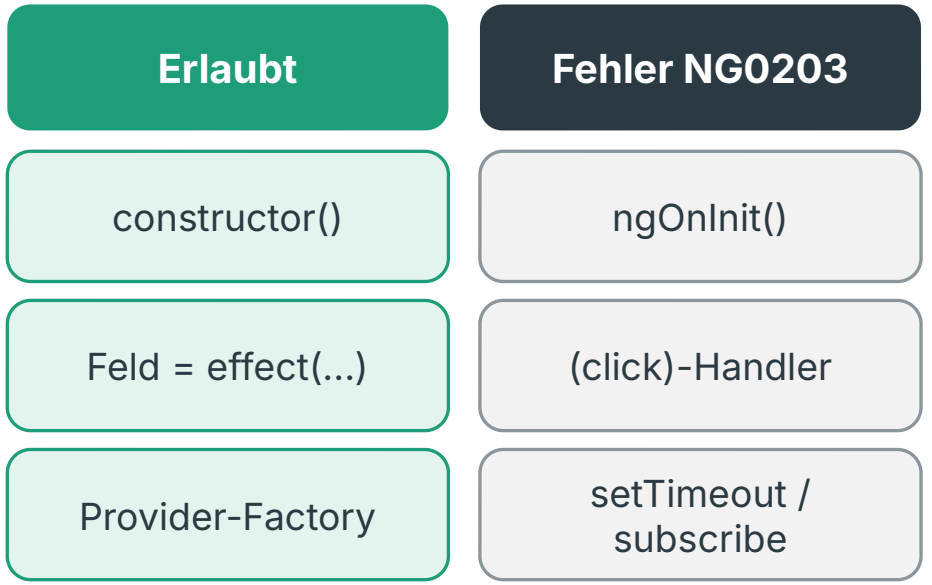
- Läuft mindestens einmal, danach bei Änderung gelesener Signals
- Läuft asynchron, nicht direkt nach set()
  - Angular plant ihn ein und führt ihn im nächsten Durchlauf aus
- Mehrere Änderungen hintereinander: nur ein Lauf
- Effect einer Komponente: lebt so lange wie die Komponente
- Effect in einem Service (root): lebt so lange wie die Anwendung



Mehrere set() ergeben einen Lauf

Injection Context: wo effect() erlaubt ist

- Injection Context: Phase, in der inject() funktioniert
- effect braucht ihn: holt sich den Injector
  - daraus: wann aufräumen (DestroyRef) und Zeitplanung
- Erlaubt: Konstruktor, Feldinitialisierer, Provider-Factory
- Nicht erlaubt: ngOnInit, Klick-Handler, setTimeout, subscribe
  - Fehler NG0203
- Ausweg: Option injector oder runInInjectionContext



effect: Optionen und destroy()

```
@Injectable({ providedIn: 'root' })
export class FilterLogger {
  private store = inject(FilterStore);
  private injector = inject(Injector);      // im Injection Context merken
  private ref?: EffectRef;

  start() {                                 // normale Methode: kein Injection Context
    this.ref = effect(
      () ⇒ console.log('Filter:', this.store.filter()),
      { injector: this.injector, debugName: 'filterLog' }
    );
  }
  stop() {
    this.ref?.destroy();                    // effect sofort beenden, Cleanup läuft
  }
}
// manualCleanup: true – effect endet nur durch destroy(), nicht mit der Komponente
```

untracked(): lesen ohne Abhängigkeit

```
export class AuditComponent {
  currentUser = input.required<User>();
  settings = signal({ verbose: false });
  private logger = inject(Logger);

  constructor() {
    effect(() => {
      const user = this.currentUser();            // Abhängigkeit: Lauf bei neuem User

      untracked(() => {   // Lesen ohne Abhängigkeit: settings löst KEINEN Lauf aus
        const verbose = this.settings().verbose;
        this.logger.log(`Benutzer: ${user.name}`, verbose);  // liest evtl. selbst Signals
      });
    });
  }
}
// Kurzform für einen einzelnen Wert: const v = untracked(this.settings);
```

afterRenderEffect: nach dem Zeichnen

- Für DOM-Arbeit, die das fertig gezeichnete Element braucht
  - Größen messen, Canvas zeichnen, Drittbibliothek
- Läuft nach dem Rendern und nur, wenn gelesene Signals sich ändern
- Läuft nur im Browser, nicht beim Server-Rendering
- Für einmalige Arbeit: afterNextRender
- Normale Nebenwirkungen ohne DOM: effect

chart.component.ts

```
canvas = viewChild.required<
  ElementRef<HTMLCanvasElement>
>('c');
data = input.required<number[]>();

constructor() {
  afterRenderEffect(() => {
    drawChart(
      this.canvas().nativeElement,
      this.data());
  });
}
```

Wann kein effect?

- Wert aus anderen Signals berechnen: computed
- Signal bei Änderung einer Quelle zurücksetzen: linkedSignal
- Auf Klick oder Eingabe reagieren: Event-Handler, z. B. (click)
- Daten laden, wenn sich eine id ändert: resource oder httpResource
- Beim Start einmal etwas tun: Konstruktor oder afterNextRender
- Faustregel: effect schreibt ein Signal? Fast immer gibt es Besseres
  - Technisch erlaubt (seit Version 19), aber ein Warnsignal

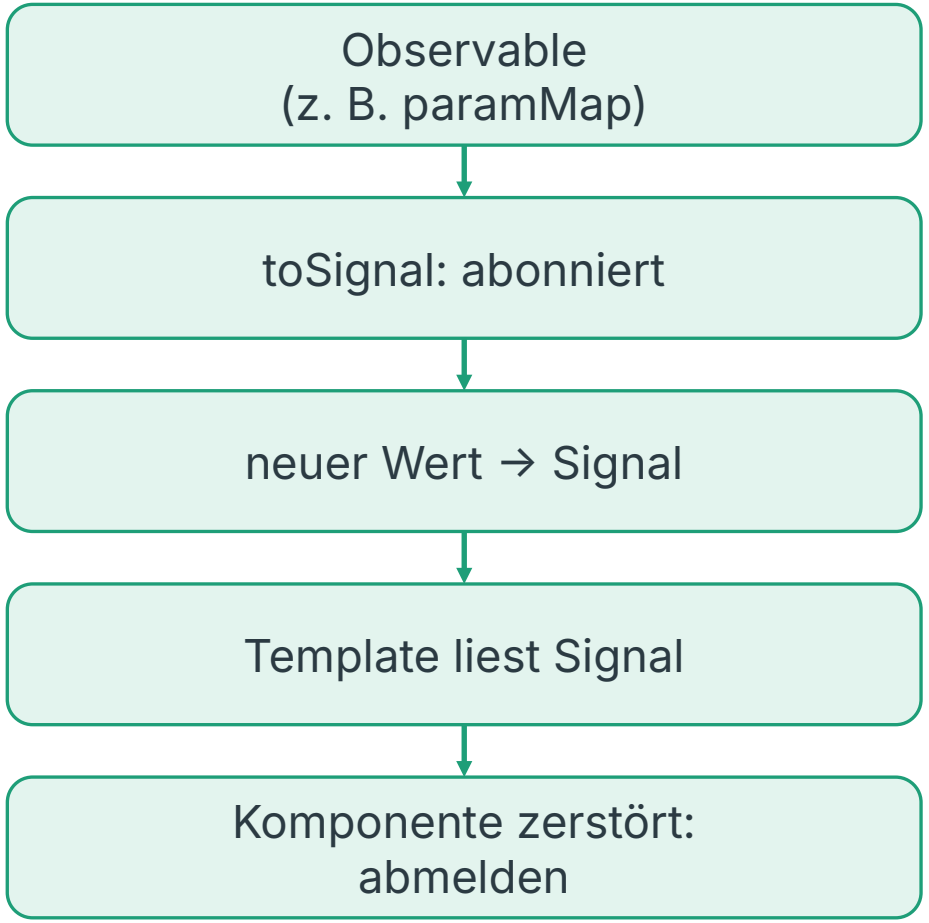
effect ersetzen: vorher und nachher

```
// Vorher: effect setzt ein Signal (abgeleiteter Zustand in Verkleidung)
fullName = signal('');
constructor() {
  effect(() ⇒ this.fullName.set(`${this.first()} ${this.last()}`));
}
// Nachher: computed – kein zusätzlicher Lauf, nichts aufzuräumen
fullName = computed(() ⇒ `${this.first()} ${this.last()}`);

// Vorher: Seite bei neuem Suchbegriff per effect zurücksetzen
page = signal(1);
constructor() {
  effect(() ⇒ { this.query(); this.page.set(1); });
}
// Nachher: linkedSignal – folgt query, bleibt mit set() änderbar
page = linkedSignal({ source: this.query, computation: () ⇒ 1 });
```

toSignal(): Observable wird Signal

- Observable: Datenstrom aus RxJS (Router, Formulare, HttpClient)
- toSignal(obs$) abonniert sofort beim Aufruf
- Jeder neue Wert des Stroms wird der Wert des Signals
- Abmelden automatisch, wenn die Komponente zerstört wird
  - dafür nötig: Injection Context
- Ohne Startwert: Typ T | undefined



toSignal(): die Optionen im Code

```
private route = inject(ActivatedRoute);
private http = inject(HttpClient);

// 1) ohne Option: Typ Signal<ParamMap | undefined>
params = toSignal(this.route.paramMap);

// 2) initialValue: gilt, bis der erste Wert kommt → Signal<Customer[]>
customers = toSignal(this.http.get<Customer[]>('api/customer'), { initialValue: [] });

// 3) requireSync: Observable liefert sofort beim Abonnieren → kein undefined
private state$ = new BehaviorSubject<string>('start');
state = toSignal(this.state$, { requireSync: true });      // Signal<string>

// 4) außerhalb des Injection Context
load() {
  return toSignal(this.http.get<Customer[]>('api/customer'), { injector: this.injector });
}
```

toSignal()-Optionen erklärt

- initialValue: Startwert bis zum ersten Wert des Stroms
- requireSync: true – Strom liefert sofort synchron
  - Liefert er nicht sofort: Fehler NG0601
- injector: für Aufrufe außerhalb des Injection Context
- manualCleanup: true – Abo endet erst mit dem Strom selbst
- equal: eigene Vergleichsfunktion wie bei signal
- Fehler im Strom: werden beim Lesen des Signals geworfen

toObservable(): Signal wird Observable

- Für RxJS-Operatoren: debounceTime, switchMap, retry
- Intern ein effect: braucht den Injection Context
- Erster Wert: der aktuelle Wert des Signals
- Werte kommen asynchron, nicht direkt nach set()
  - Mehrere set() hintereinander: nur der letzte Wert
- Option injector wie bei effect

toObservable

```
const count = signal(0);
const count$ =
  toObservable(count);

count$.subscribe(v =>
  console.log(v));
// später: 0

count.set(1);
count.set(2);
// später nur: 2
```

Brücke zu RxJS: Filter mit Verzögerung

```
filterText = signal('');

// Signal in einen Datenstrom umwandeln, verzögern, wieder als Signal bereitstellen
private filter$ = toObservable(this.filterText).pipe(
  debounceTime(300),            // erst weiterreichen, wenn 300 ms keine Eingabe kam
  distinctUntilChanged()        // gleiche Werte hintereinander ignorieren
);
debounced = toSignal(this.filter$, { initialValue: '' });

// Treffer: rechnet erst nach der Tipp-Pause neu
visible = computed(() ⇒
  this.customers().filter(c ⇒ c.lastname.includes(this.debounced()))
);
```

Stolperfallen der Brücke zu RxJS

- toSignal ohne initialValue: T | undefined im Template behandeln
- toSignal in ngOnInit oder Methode: NG0203 ohne injector
- Fehler im Strom: Signal wirft beim Lesen – vorher catchError
- toSignal abonniert sofort: HTTP-Anfrage startet beim Erzeugen
- Zeit und Ereignisströme: RxJS – Zustand: Signals

Stolperfalle 1: Mutation statt Kopie

```
list = signal<Customer[]>([]);

this.list().push(c);                  // Falsch: gleiches Array, Signal merkt nichts
this.list.update(l ⇒ [...l, c]);      // Richtig: neues Array, neue Referenz

// Eintrag ändern, ebenfalls mit neuer Referenz
this.list.update(l ⇒ l.map(x ⇒ x.id === c.id ? { ...x, ...changes } : x));

// Eintrag entfernen
this.list.update(l ⇒ l.filter(x ⇒ x.id !== id));

// Warum? Signals vergleichen mit Object.is: gleiches Array = "keine Änderung".
// push ändert den Inhalt, aber nicht die Referenz → keine Benachrichtigung,
// computed und Template behalten ihren alten Stand.
```

Stolperfalle 2: Zyklen und Ketten

```
// Falsch: der erste effect schreibt b, der zweite liest b und schreibt a ...
effect(() ⇒ this.b.set(this.a() * 2));
effect(() ⇒ this.a.set(this.b() + 1));    // Endlosschleife

// Warum? Jeder Schreibvorgang macht den anderen effect veraltet,
// beide planen sich gegenseitig immer wieder ein.

// Besser: eine Quelle und ein computed
a = signal(1);
b = computed(() ⇒ this.a() * 2);

// Schreiben auf Signals in einem computed ist verboten (Fehler NG0600)
b2 = computed(() ⇒ { this.a.set(5); return 1; });      // NG0600
```

Stolperfalle 3: Methoden im Template

```
<!-- Normale Methode: wird bei jeder Change Detection aufgerufen -->
<p>Summe: {{ expensiveSum() }}</p>

<!-- computed: rechnet nur neu, wenn sich Abhängigkeiten ändern -->
<p>Summe: {{ total() }}</p>

// Klasse
expensiveSum() { return this.items().reduce((s, i) => s + i.price, 0); }
total = computed(() => this.items().reduce((s, i) => s + i.price, 0));

// Warum? Angular weiß nicht, ob eine Methode ihr Ergebnis geändert hat,
// also ruft es sie bei jeder Prüfung erneut auf.
// Ein computed weiß es: Es kennt seine Abhängigkeiten und hat einen Speicher.
// Ausnahme: reine, sehr billige Aufrufe. Im Zweifel computed oder reine Pipe.
```

Stolperfalle 4: zu grobe Signals

```
// Ein großes Signal für alles
state = signal({
  customers: [] as Customer[], filter: '', selectedId: 0, loading: false
});

state.update(s ⇒ ({ ...s, loading: true }));    // nur loading geändert …
// … aber JEDER, der state() liest, gilt als veraltet: Liste, Filter, Auswahl

// Warum? Die Abhängigkeit hängt am ganzen Signal, nicht an einzelnen Feldern.

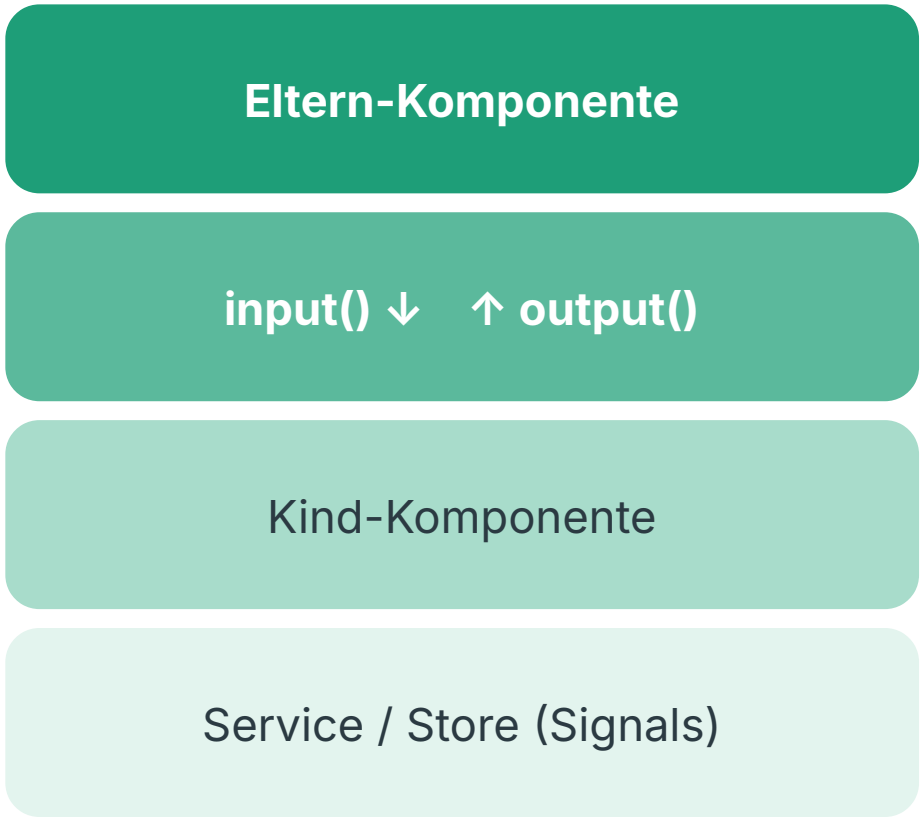
// Besser: mehrere kleine Signals, daraus computed für Ausschnitte
customers = signal<Customer[]>([]);
filter = signal('');
selectedId = signal<number | null>(null);
loading = signal(false);
selected = computed(() ⇒ this.customers().find(c ⇒ c.id === this.selectedId()));
```

Stolperfalle 5: kleine Fehler, große Wirkung

- Klammern vergessen: if (this.loading) ist immer wahr
  - Richtig: if (this.loading())
- Lesen nach await im effect: keine Abhängigkeit mehr
  - Signals vor dem ersten await lesen
- input() setzen wollen: input ist schreibgeschützt
  - lokalen Zustand mit linkedSignal ableiten
- effect in ngOnInit: Fehler NG0203

Nachvollziehbare Datenflüsse

- Eltern zu Kind: input(), Daten nach unten
- Kind zu Eltern: output(), Ereignisse nach oben
- Gemeinsamer Zustand: Service oder Store
- Eine Richtung, ein Besitzer pro Zustand
- Ändern nur über benannte Methoden
- Fehlersuche: Angular DevTools zeigen den Signal-Graphen



eine Richtung, ein Besitzer

Entscheidungsbaum: Welcher Baustein?

- 1. Kommt der Wert vom Benutzer oder von einem Ereignis? Dann signal
- 2. Folgt er aus anderen Werten? Dann computed
  - Soll er trotzdem überschreibbar sein? Dann linkedSignal
- 3. Kommt er asynchron vom Server? Dann resource oder httpResource
- 4. Muss etwas außerhalb von Angular passieren? Dann effect
  - Braucht es das gezeichnete DOM? Dann afterRenderEffect
- 5. Geht es um Zeit und Ereignisströme? Dann RxJS
  - Am Rand der Komponente mit toSignal in ein Signal wandeln

Übung 32: Reaktivitäts-Fallen finden

- **Teil A: customers und filteredCustomers() in CustomerListComponent zu computed umbauen**
  - Prüfen: console.log im computed erscheint nur bei geänderten Abhängigkeiten
- Teil B: Filtertext per effect in localStorage sichern (Schlüssel customerFilter)
  - Beim Start als Startwert des Signals wiederherstellen, Zugriff mit try/catch
- Teil C: Patch von der nächsten Folie einfügen, Button „Demo-Kunde hinzufügen“ testen
  - Je Nummer benennen: Was ist falsch? Welche Regel wird verletzt?
  - Beheben: (1) und (3) mit computed, (2) mit signal plus update und Spread
- Reflexion: Welche Fehler hätte ein Test gefunden, welche nur ein Review?
- **Musterlösung: 32_reaktivitaet (baut auf 31_komponenten_aufteilen auf)**

Übung 32: Patch für Teil C

```
// --- Klassenfelder ---
selectedName = signal('');
total = signal(0);

// --- Konstruktor (der effect aus Teil B steht dort schon) ---
constructor() {
  effect(() ⇒ this.selectedName.set(this.selectedCustomer()?.firstname ?? ''));  // (1)
  effect(() ⇒ this.total.set(this.filteredCustomers().length));                  // (3)
}

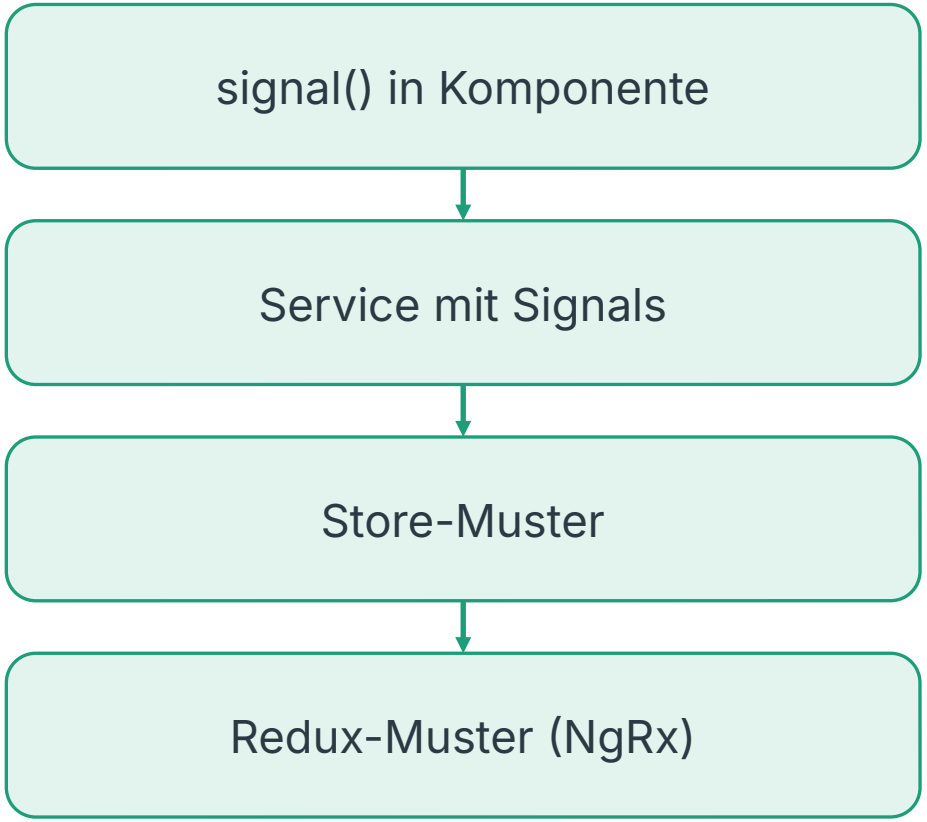
// --- Methode ---
addDemo() { this.customers().push(new Customer('Neu', 'Demo')); }                // (2)

<!-- Template, nach </ul> ⟶
<p>Treffer: {{ total() }} · Ausgewählt: {{ selectedName() || '-' }}</p>
<button (click)="addDemo()">Demo-Kunde hinzufügen</button>
```

Anwendungsstruktur und Zustandsverwaltung

Wie Zustand wächst

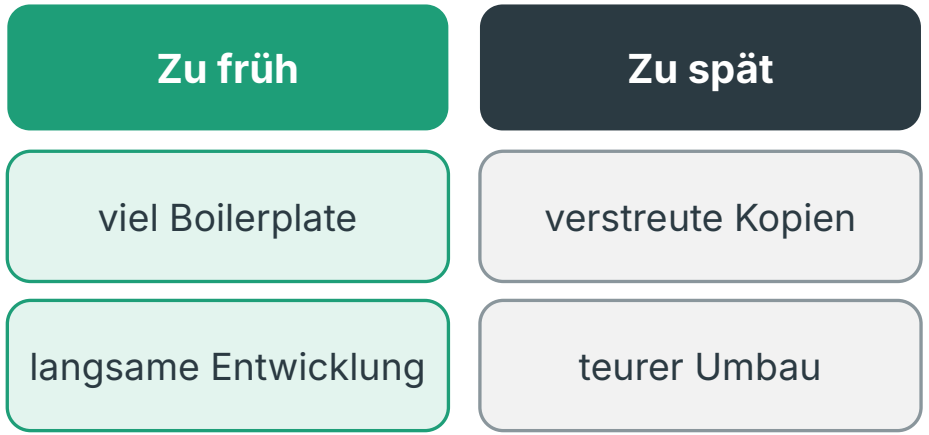
- Phase 1: eine Komponente, wenige Felder
  - signal() direkt in der Komponente genügt
- Phase 2: mehrere Komponenten teilen Daten
  - Service mit Signals
- Phase 3: Daten, Filter, Auswahl, Laden, Fehler
  - Store-Muster: Zustand und Aktionen gebündelt
- Phase 4: viele Teams, Nachvollziehbarkeit
  - Redux-Muster mit NgRx



Nur so weit gehen, wie nötig

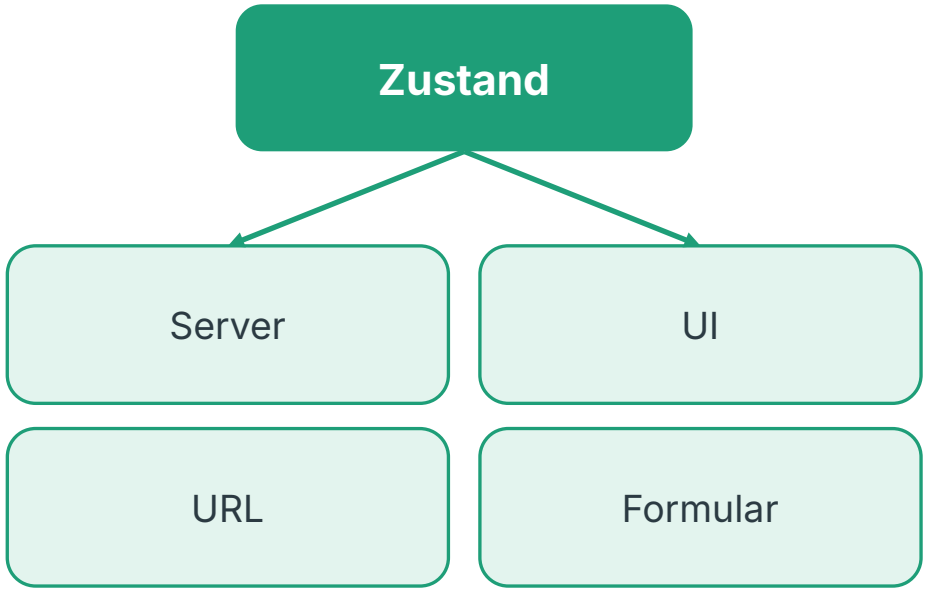
Zu früh oder zu spät strukturieren

- Zu früh: viel Boilerplate für wenig Nutzen
  - Actions, Reducer, Selektoren für drei Felder
- Zu spät: Zustand liegt verstreut in Komponenten
  - Kopien, widersprüchliche Daten, schwerer Umbau
- Warnzeichen für den nächsten Schritt:
  - gleiche Daten in mehreren Komponenten kopiert
  - Fehler wie „Liste zeigt alten Stand“



Welche Arten von Zustand gibt es?

- Server-Zustand: Daten vom Backend
  - haben ein Alter, können veralten oder fehlen
- UI-Zustand: offener Dialog, aktiver Reiter
- URL-Zustand: Route, Parameter, Query
- Formular-Zustand: Eingaben, Gültigkeit
- Jede Art hat einen natürlichen Ort
  - nicht alles gehört in den Store

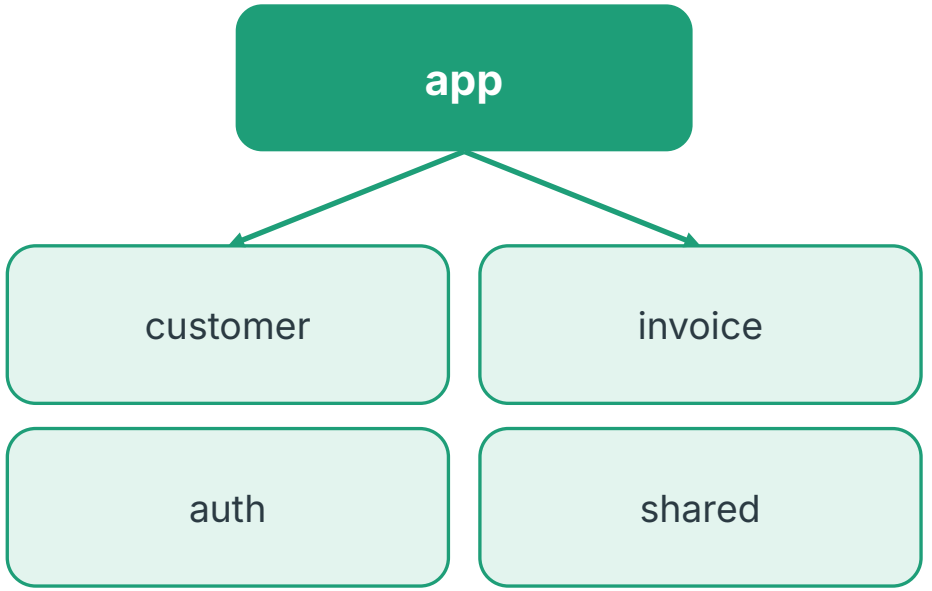


Feature-Ordnerstruktur

```
src/app/
  customer/                ← Feature (Fachbereich)
    customer-list/         ← Container-Komponente
    customer-details/      ← Container-Komponente
    ui/                    ← Darstellungskomponenten (nur input/output)
      customer-card/
    customer.store.ts      ← Zustand und Aktionen
    customer.service.ts    ← HTTP-Zugriff (API-Schicht)
    customer.model.ts      ← Typen
    customer.routes.ts     ← Routen des Features (lazy)
  auth/                    ← weiteres Feature
  shared/                  ← nur wirklich Allgemeines (Card, Pipes)
  app.routes.ts
  app.config.ts
```

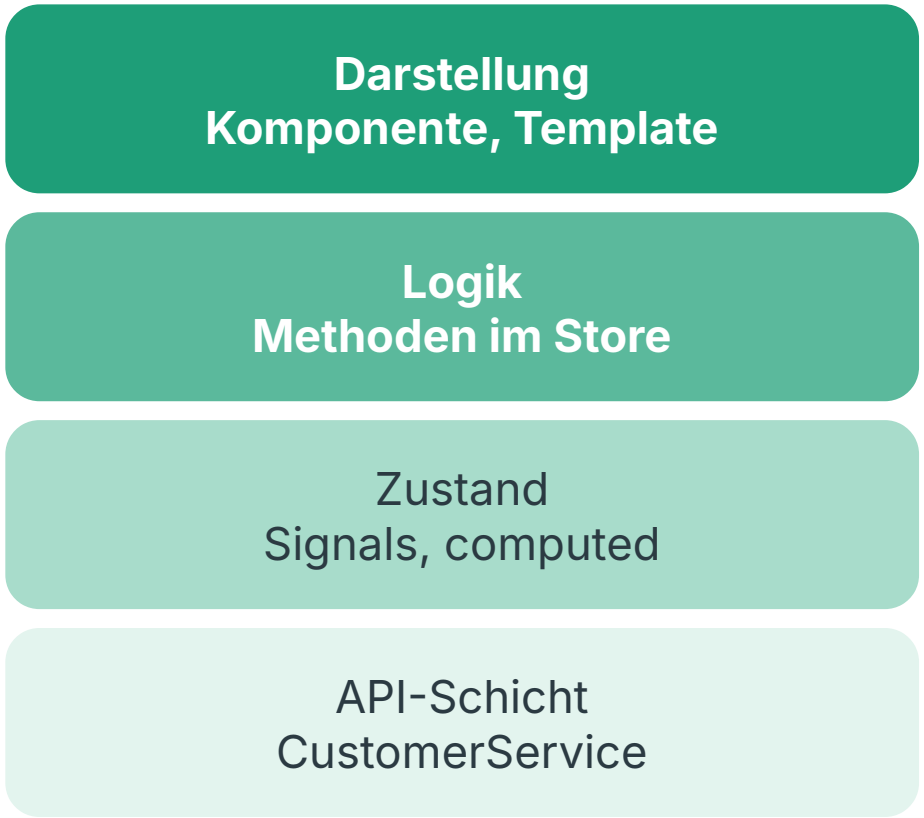
Regeln für die Ordnerstruktur

- Nach Fachlichkeit gliedern, nicht nach Technik
  - nicht: components/, services/, models/
- Features nutzen shared, nie umgekehrt
- Features kennen sich nur über Schnittstellen
  - Route, öffentlicher Service oder Store
- Pro Feature ein Store, kein Gesamt-Store
- Lazy Loading pro Feature über die Routen



Drei Schichten: Darstellung, Logik, Zustand

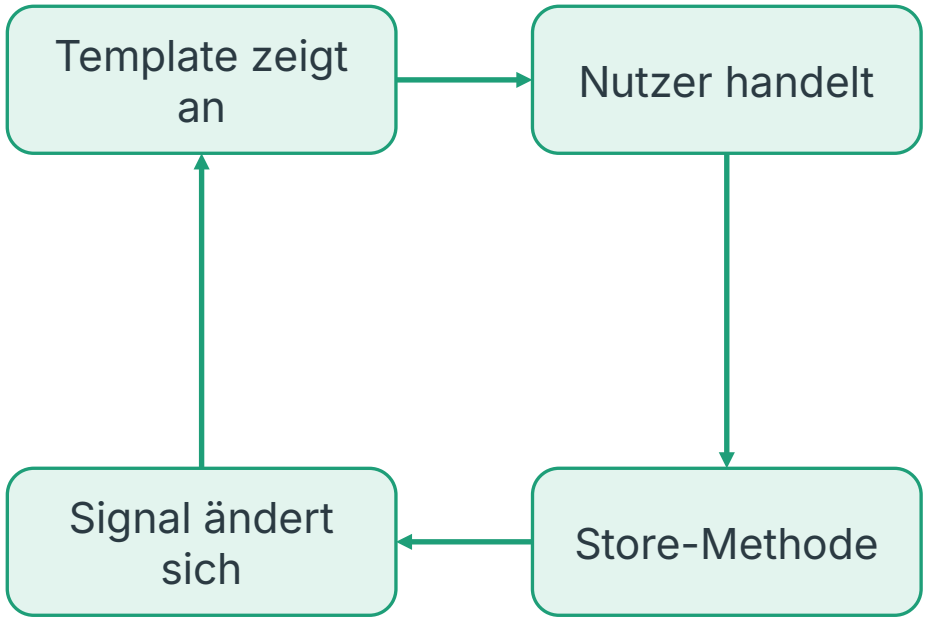
- Darstellung: Komponenten und Templates
  - anzeigen, Eingaben weitergeben
- Logik: Methoden im Store
  - Regeln, Befehle, Laden über die API-Schicht
- Zustand: Signals im Store
  - Daten halten und ableiten (computed)
- Abhängigkeiten zeigen nur nach unten



Abhängigkeiten nur nach unten

Einseitiger Datenfluss

- Daten fließen nach unten: Store zu Template
  - Template liest Signals, z. B. store.filtered()
- Ereignisse fließen nach oben: Template zu Store
  - Klick oder Eingabe ruft eine Store-Methode
- Komponente ändert nie direkt den Zustand
- Ergebnis: eine Quelle der Wahrheit



Problem: Zustand in Komponenten verteilt

- Jede Komponente lädt und filtert selbst
- Auswahl liegt in der Liste, Details brauchen sie
- Gleiche Logik mehrfach, leicht widersprüchlich
- Lösung: Store als Service
  - hält den Zustand an einer Stelle
  - bietet Aktionen als Methoden an
  - wird mit inject() geholt



Store als Service (1/2): Zustand

```
@Injectable({ providedIn: 'root' })
export class CustomerStore {
  private readonly customerService = inject(CustomerService);

  // privat und schreibbar: nur der Store darf ändern
  private readonly _customers = signal<Customer[]>([]);
  private readonly _filter = signal('');
  private readonly _selectedId = signal<string | undefined>(undefined);
  private readonly _loading = signal(false);
  private readonly _error = signal<string | undefined>(undefined);

  // öffentlich und nur lesbar
  readonly customers = this._customers.asReadonly();
  readonly filter = this._filter.asReadonly();
  readonly selectedId = this._selectedId.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly error = this._error.asReadonly();
```

Store als Service (2/2): Aktionen

```
  readonly filtered = computed(() => {
    const f = this._filter().toLowerCase();
    return this._customers().filter(c => c.firstname?.toLowerCase().includes(f));
  });
  readonly selected = computed(() =>
    this._customers().find(c => c.id === this._selectedId()));
  readonly total = computed(() => this.filtered().length);

  async load(): Promise<void> {
    this._loading.set(true);
    this._error.set(undefined);
    try { this._customers.set(await firstValueFrom(this.customerService.getAll())); }
    catch { this._error.set('Fehler beim Laden der Kunden.'); }
    finally { this._loading.set(false); }
  }
  setFilter(text: string): void { this._filter.set(text); }
  select(id: string | undefined): void { this._selectedId.set(id); }
}
```

Privat schreibbar, öffentlich lesbar

- signal() liefert ein WritableSignal<T>
  - mit set() und update()
- asReadonly() liefert ein Signal<T>
  - nur lesen: filter(), kein set()
  - kein Kopieren: gleicher Wert, gleiche Quelle
- Fehler beim Übersetzen, nicht erst zur Laufzeit

Kapselung

```
// im Store
private _filter = signal('');
readonly filter =
  this._filter.asReadonly();

// in der Komponente
store.filter();       // ok
store.filter.set('x');
// Fehler: set gibt es
// auf Signal nicht
store.setFilter('x'); // ok
```

Aktionen als Methoden

- Jede Änderung hat einen fachlichen Namen
  - setFilter, select, load statt set() überall
- Parameter: nur was die Aktion braucht
- Rückgabe: meist void, load() ein Promise
  - Aufrufer kann await store.load() schreiben
- Regeln an einer Stelle, z. B. Filter kürzen
- Leicht zu testen und zu protokollieren

Aktion mit Regel

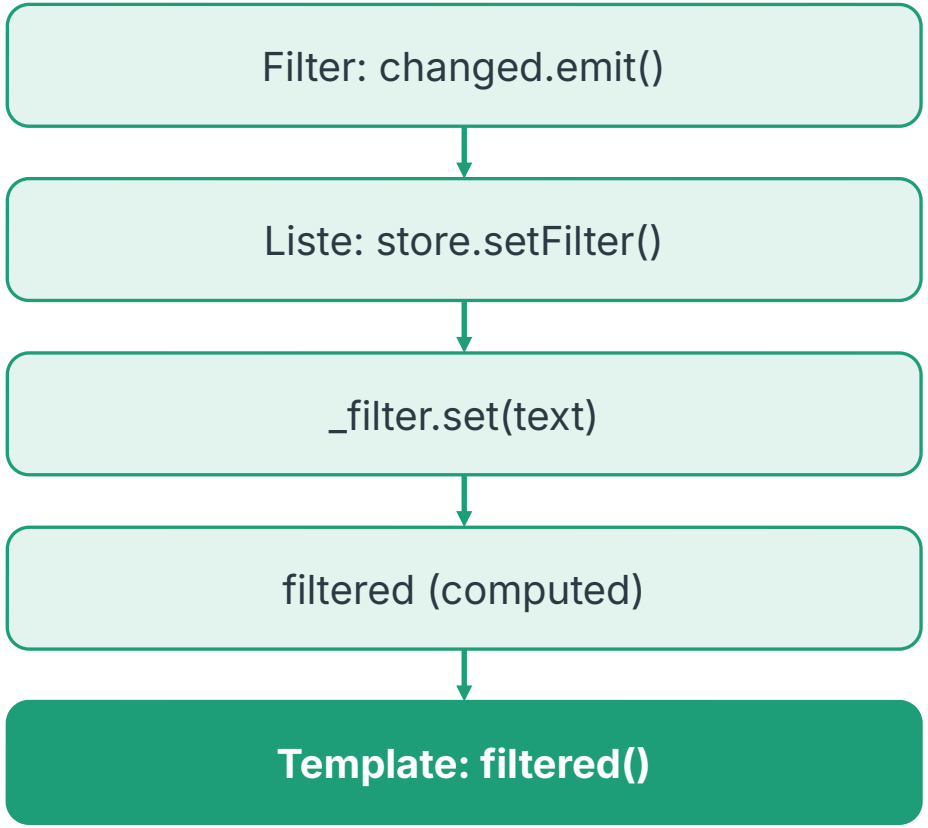
```
setFilter(text: string): void {
  this._filter.set(text.trim());
}

select(id?: string): void {
  this._selectedId.set(id);
}

// Aufrufer
await store.load();
```

Ablauf: vom Tippen bis zur Anzeige

- 1. Filter-Komponente meldet Text per output
- 2. Liste ruft store.setFilter(text)
- 3. _filter.set() ändert das Signal
- 4. filtered und total sind nun veraltet
- 5. Template liest filtered() neu
  - computed rechnet erst jetzt (lazy)
- Kein subscribe, kein manuelles Neuzeichnen



Angular verfolgt die Abhängigkeiten

Komponente nutzt nur den Store

```
export class CustomerListComponent implements OnInit {
  protected readonly store = inject(CustomerStore);
  customerSelected = output<Customer>();

  ngOnInit(): void { this.store.load(); }   // beim Öffnen neu laden
  onFilterChanged(text: string) { this.store.setFilter(text); }
  listItemClicked(c: Customer) {
    this.store.select(c.id);
    this.customerSelected.emit(c);
  }
}

<!-- Template -->
@if (store.loading()) { <p>Lade Kunden…</p> }
@for (customer of store.filtered(); track customer.id) { … }
<p>Treffer: {{ store.total() }}</p>
```

Nebenwirkung im Store: effect

- Filter soll im localStorage gespeichert werden
- effect() im Konstruktor des Stores
  - läuft einmal sofort, dann bei jeder Änderung
- Konstruktor = Injection Context
  - dort darf effect() erzeugt werden
- Warum im Store? Der Zustand gehört dorthin
  - jede Komponente bekommt das Speichern mit

customer.store.ts

```
private readonly _filter =
  signal(loadFilter());

constructor() {
  effect(() =>
    saveFilter(this._filter()));
}

// saveFilter: localStorage
// .setItem in try/catch
```

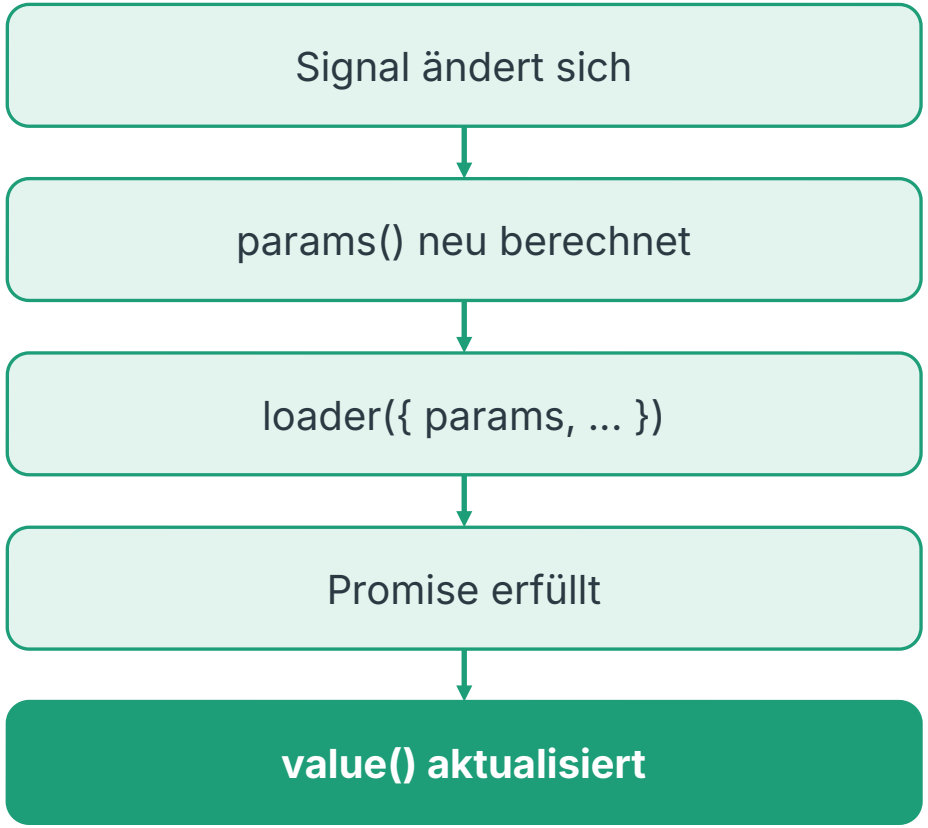
Laden mit resource im Store

```
@Injectable({ providedIn: 'root' })
export class CustomerStore {
  private readonly api = inject(CustomerService);
  private readonly _selectedId = signal<string | undefined>(undefined);
  // Liste: Observable direkt verwenden
  private readonly listRes = rxResource({ stream: () => this.api.getAll() });
  readonly customers = computed(() =>
    this.listRes.hasValue() ? this.listRes.value() : []);
  readonly loading = this.listRes.isLoading;
  readonly error = this.listRes.error;

  // Details: lädt neu, sobald sich selectedId ändert
  readonly detailsRes = resource({
    params: () => this._selectedId(),
    loader: ({ params: id }) => firstValueFrom(this.api.getById(id)),
  });
  reload() { this.listRes.reload(); }
}
```

resource: Parameter und Rückgabe erklärt

- params: Funktion, liest Signals
  - Änderung startet den loader neu
  - undefined: kein Laden, Status idle
- loader: von Angular aufgerufen mit Objekt
  - params, abortSignal, previous
  - muss ein Promise liefern
- Ergebnis: value(), isLoading(), error(),
  - status(), hasValue(), reload()



Alte Anfrage wird abgebrochen

Asynchronen Zustand als Union-Typ

```
type LoadState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'error'; message: string }
  | { status: 'loaded'; data: T };

private readonly _state = signal<LoadState<Customer[]>>({ status: 'idle' });
readonly state = this._state.asReadonly();

async load() {
  this._state.set({ status: 'loading' });
  try {
    const data = await firstValueFrom(this.api.getAll());
    this._state.set({ status: 'loaded', data });
  } catch {
    this._state.set({ status: 'error', message: 'Laden fehlgeschlagen' });
  }
}
```

Wie der Union-Typ funktioniert

- status ist das Unterscheidungsmerkmal
- TypeScript grenzt nach einer Prüfung ein
  - nach status === 'loaded' gibt es data
- Unmögliche Kombinationen sind nicht darstellbar
  - nicht gleichzeitig loading und error
- Fehlender Fall: Prüfung mit never

Narrowing

```
const s = store.state();

if (s.status === 'loaded') {
  s.data.length; // ok
}

if (s.status === 'loading') {
  s.data; // Fehler:
  // data gibt es hier nicht
}
```

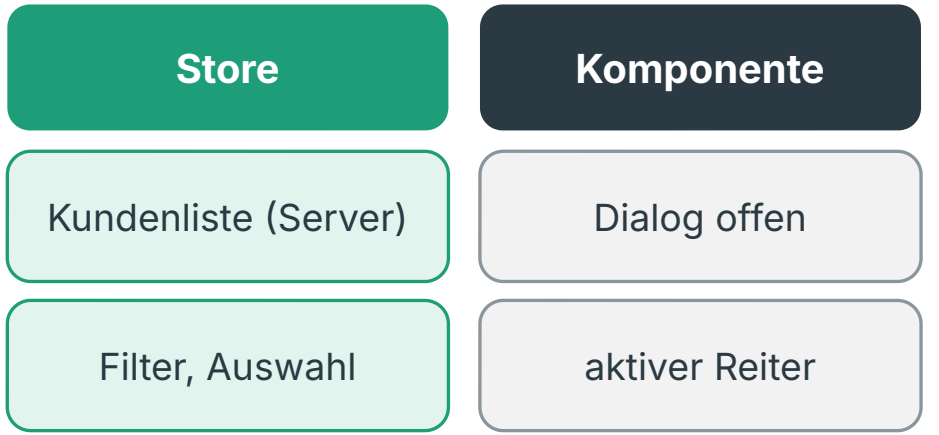
Union-Typ im Template mit @switch

```
@let s = store.state();

@switch (s.status) {
  @case ('idle') {
    <button (click)="store.load()">Kunden laden</button>
  }
  @case ('loading') {
    <p>Lade Kunden …</p>
  }
  @case ('error') {
    <p class="error">{{ s.message }}</p>
    <button (click)="store.load()">Erneut versuchen</button>
  }
  @case ('loaded') {
    @for (c of s.data; track c.id) { <customer-card [customer]="c" /> }
  }
}
```

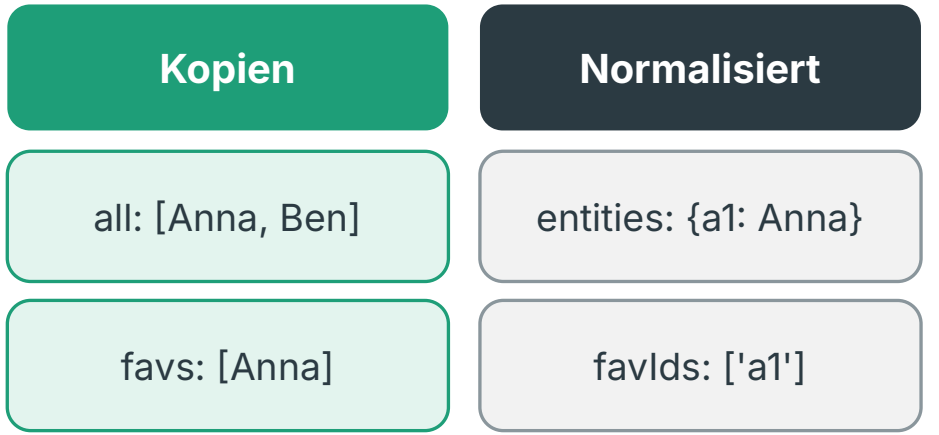
Server-Daten und UI-Zustand trennen

- Server-Daten: geladen, gecacht, veralten
  - resource oder LoadState im Store
- UI-Zustand: Dialog offen, aktiver Reiter
  - signal() in der Komponente
- Gemeinsam genutzte Auswahl und Filter
  - in den Store des Features
- Faustregel: wer braucht den Wert noch?



Normalisierung: das Problem

- Gleicher Kunde in mehreren Listen gespeichert
  - alle Kunden, Favoriten, letzte Aufrufe
- Umbenennen muss jede Kopie ändern
  - eine vergessene Kopie zeigt alten Namen
- Lösung: jede Entität genau einmal speichern
  - Listen enthalten nur noch IDs



Normalisierter Zustand im Store

```
interface CustomerState {
  entities: Record<string, Customer>;   // id → Kunde, jeder genau einmal
  ids: string[];                        // Reihenfolge der Liste
  favoriteIds: string[];
}
private readonly _state =
  signal<CustomerState>({ entities: {}, ids: [], favoriteIds: [] });
readonly all = computed(() ⇒ {
  const s = this._state();
  return s.ids.map(id ⇒ s.entities[id]);
});
readonly favorites = computed(() ⇒
  this._state().favoriteIds.map(id ⇒ this._state().entities[id]));

rename(id: string, firstname: string) {   // nur eine Stelle ändern
  this._state.update(s ⇒ ({ ...s,
    entities: { ...s.entities, [id]: { ...s.entities[id], firstname } } }));
}
```

Weitere Regeln für wachsenden Zustand

- Ableiten statt kopieren
  - selected = computed aus selectedId
- Unveränderlich aktualisieren
  - update(list ⇒ [...list, neu]) statt push
- Zustand klein halten
  - nur speichern, was nicht berechenbar ist
- Ein Store pro Feature

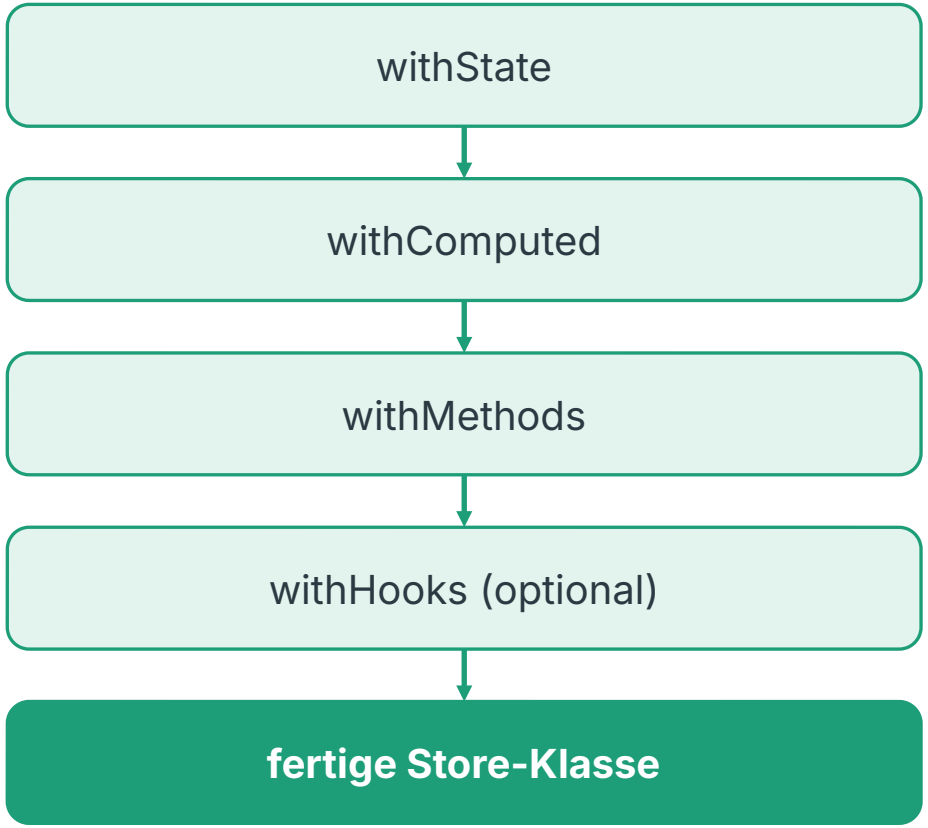
Neue Referenz

```
// falsch: gleiche Liste
this._customers.update(l => {
  l.push(neu); return l;
});

// richtig: neue Liste
this._customers.update(l =>
  [...l, neu]);
```

NgRx Signal Store: Überblick

- Paket @ngrx/signals: npm install @ngrx/signals
- Gleiches Muster wie der eigene Store
  - aber standardisiert und erweiterbar
- signalStore() setzt Bausteine zusammen
  - withState, withComputed, withMethods
- patchState() ändert den Zustand
- Ergebnis: eine injizierbare Klasse



Reihenfolge zählt

Signal Store: CustomerSignalStore

```
export const CustomerSignalStore = signalStore(
  { providedIn: 'root' },
  withState({ customers: [] as Customer[], filter: '',
              selectedId: undefined as string | undefined }),
  withComputed(({ customers, filter, selectedId }) => ({
    filtered: computed(() => customers().filter(
      c => c.firstname.toLowerCase().includes(filter().toLowerCase()))),
    selected: computed(() => customers().find(c => c.id === selectedId())),
  })),
  withMethods((store, service = inject(CustomerService)) => ({
    setFilter(filter: string) { patchState(store, { filter }); },
    select(selectedId: string | undefined) { patchState(store, { selectedId }); },
    async load() {
      patchState(store, { customers: await firstValueFrom(service.getAll()) });
    },
  })),
);
```

Bausteine erklärt (1/2)

- signalStore(config, ...features)
  - liefert eine Klasse, kein Objekt
- withState(anfangszustand)
  - jede Eigenschaft wird ein lesbares Signal
  - store.filter() liest den Filter
- withComputed(store ⇒ ({ ... }))
  - bekommt die bisherigen Signals
  - gibt ein Objekt mit computed-Werten zurück

Was entsteht

```
withState({ filter: '' })
// -> store.filter: Signal

withComputed(({ filter }) => ({
  upper: computed(() =>
    filter().toUpperCase())
}))
// -> store.upper: Signal
```

Bausteine erklärt (2/2)

- withMethods((store, service = inject(X)) ⇒ ...)
  - läuft im Injection Context: inject() erlaubt
  - gibt ein Objekt mit Methoden zurück
- patchState(store, teil)
  - ersetzt nur die genannten Eigenschaften
  - auch mit Funktion: s ⇒ ({ filter: '' })
- Zustand von außen nur lesbar (protectedState)

patchState

```
patchState(store, { filter: 'a' });

patchState(store, s => ({
  customers: [...s.customers,
    neu]
}));

// in Komponente: Fehler
// patchState(store, ...)
```

Signal Store verwenden und erweitern

- Komponente: store = inject(CustomerSignalStore)
  - store.filtered(), store.setFilter('a')
- Ohne providedIn: in providers einer Komponente
  - eigene Instanz pro Komponente
- withHooks: onInit, z. B. load() beim Start
- withEntities: Normalisierung fertig
- rxMethod: Methoden mit RxJS-Operatoren

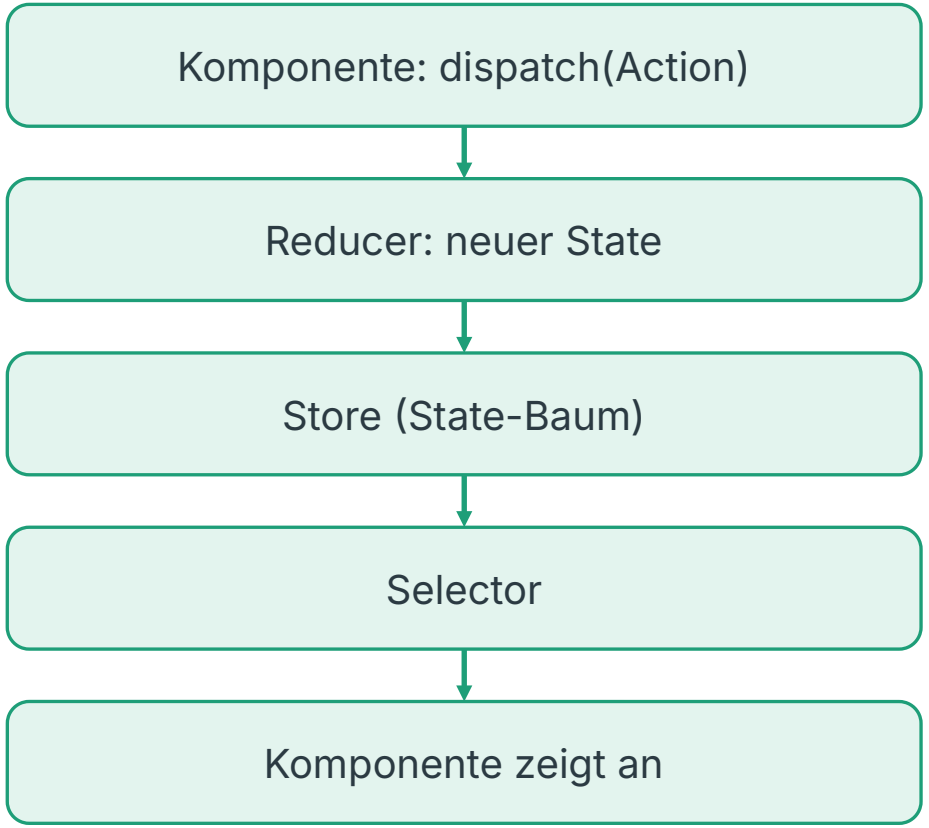
withHooks

```
withHooks({
  onInit(store) {
    store.load();
  },
})

// Komponente
store = inject(
  CustomerSignalStore);
```

Klassisches NgRx: Redux-Muster

- Action: beschreibt, was passiert ist
- Reducer: reine Funktion (state, action) ⇒ neu
- Store: hält einen unveränderlichen Zustandsbaum
- Selector: liest und leitet ab
- Effect: asynchrone Arbeit, liefert neue Action
- Komponente sendet nur Actions (dispatch)



Effects: Action → Server → Action

Klassisches NgRx im Code (Auszug)

```
// Actions: was ist passiert?
export const loadCustomers = createAction('[Customer List] Load');
export const loadSuccess = createAction('[Customer API] Load Success',
  props<{ customers: Customer[] }>());

// Reducer: (state, action) ⇒ neuer State
export const reducer = createReducer(initialState,
  on(loadCustomers, state ⇒ ({ ...state, loading: true })),
  on(loadSuccess, (state, { customers }) ⇒ ({ ...state, customers, loading: false })));

// Selector: lesen und ableiten
export const selectCustomers = createSelector(selectCustomerState, s ⇒ s.customers);

// Komponente
private readonly store = inject(Store);
customers = this.store.selectSignal(selectCustomers);
ngOnInit() { this.store.dispatch(loadCustomers()); }
```

Klassisches NgRx vs. Signal Store

- State: withState
- Selector: withComputed
- Action: Methodenaufruf
- Reducer: patchState in der Methode
- Effect: async-Methode oder rxMethod
- Klassisch: mehr Code, dafür Action-Protokoll
  - Redux DevTools, Time-Travel, strikte Trennung

Klassisch	Signal Store
Action + Reducer	Methode + patchState
Selector	withComputed
Effect	async / rxMethod

Wann strukturierten Zustand einführen? (1/2)

- Konsumenten: 1–3 Komponenten oder viele, über Features verteilt
- Zustandsarten: 1–2 oder mehrere zusammenhängende (Daten, Filter, Status)
- Seiteneffekte: wenige oder viele, die voneinander abhängen
- Nachvollziehbarkeit: unwichtig oder Änderungen müssen protokolliert werden

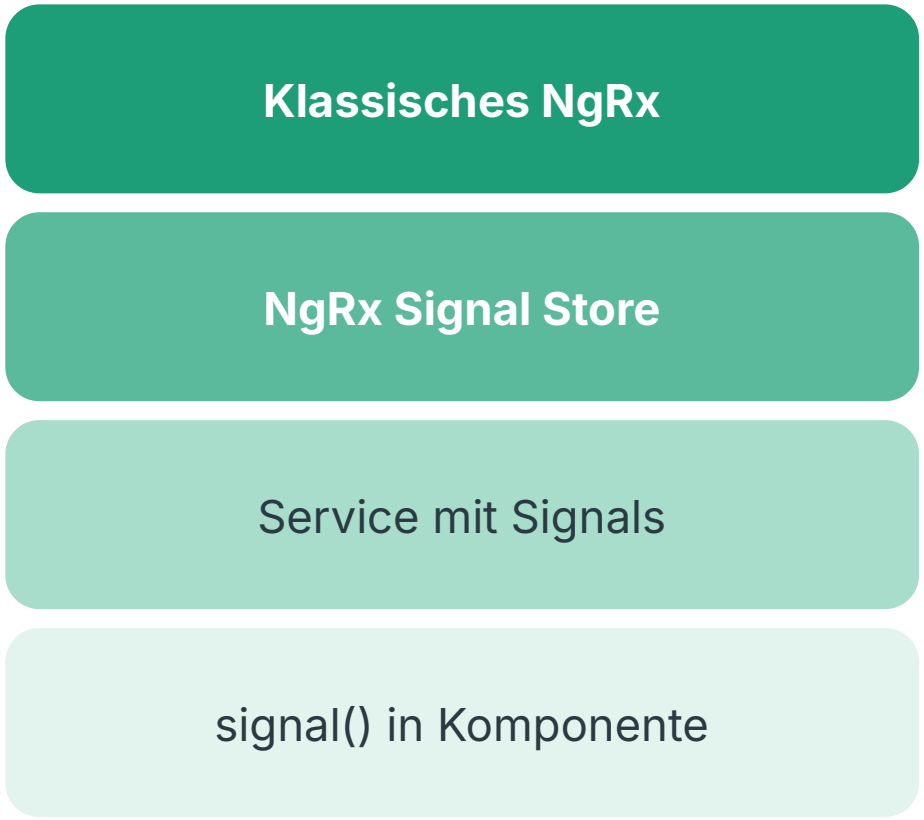
Service reicht	Store lohnt sich
1-3 Komponenten	viele Konsumenten
1-2 Zustandsarten	Daten, Filter, Status
wenige Effekte	Protokoll nötig

Wann strukturierten Zustand einführen? (2/2)

- Teamgröße: 1–3 Personen oder mehrere Teams mit einheitlichen Mustern
- Wiederverwendung: Einzelfall oder gleiche Muster in vielen Features
- Testaufwand: gering oder gezielte Tests der Zustandslogik
- Ergebnis: einfach starten mit Service und Signals
  - strukturieren erst, wenn ein Kriterium zum Problem wird

Welches Werkzeug wann?

- signal() in der Komponente: lokaler UI-Zustand
- Service mit Signals: geteilte Daten, kleines Team
- NgRx Signal Store: viele Stores, gleiche Struktur
  - Erweiterungen wie withEntities, rxMethod
- Klassisches NgRx: große Anwendung, viele Teams
  - Protokoll aller Actions, DevTools
- Mischen ist erlaubt: pro Feature entscheiden



Aufwand und Struktur steigen nach oben

Typische Fehlentscheidungen

- Alles in den Store: UI-Details gehören in die Komponente
- God Store: ein Store für alles statt einer pro Feature
- Doppelte Wahrheit: Daten im Store und als Kopie in der Komponente
- Server-Cache mit Client-Zustand verwechseln
  - Server-Daten veralten und können fehlen: resource nutzen
- Werkzeug vor Problem: erst die Schwierigkeit benennen, dann wählen

Stolperfallen im Code (1/2)

- asReadonly() vergessen
  - Komponenten setzen Werte am Store vorbei
- Liste verändert statt neu erzeugt
  - push() und gleiche Referenz: keine Anzeige
- effect() zum Kopieren zwischen Signals
  - besser computed() oder linkedSignal()
- value() einer resource im Fehlerzustand
  - wirft einen Fehler: vorher hasValue()

Falsch	Richtig
public signal()	asReadonly()
list.push(x)	[...list, x]
effect kopiert	computed()

Stolperfallen im Code (2/2)

- Store in providers einer Komponente
  - jede Komponente bekommt eine neue Instanz
- Überholende Antworten beim Laden
  - alte Antwort überschreibt neue: resource bricht ab
- load() ohne finally
  - loading bleibt bei Fehler für immer true
- Fachlogik doch wieder in der Komponente

Testbarkeit des Stores

- Store ohne Oberfläche testbar
  - kein fixture, kein DOM nötig
- API-Schicht durch Attrappe ersetzen
- Ablauf: Aktion aufrufen, Signal prüfen
- await store.load() wartet auf das Laden
- Vertiefung im Kapitel zur Testbarkeit

customer.store.spec.ts

```
TestBed.configureTestingModule({
  providers: [{
    provide: CustomerService,
    useValue: { getAll: () =>
      of([anna, ben]) } }],
});
const store =
  TestBed.inject(CustomerStore);

await store.load();
store.setFilter('an');
expect(store.total())
  .toBe(1);
```

Übung 33: CustomerStore einführen (1/2)

- **1. CustomerService zur reinen API-Schicht machen**
  - kein ReplaySubject, kein Laden im Konstruktor; getAll() liefert das HTTP-Observable
- **2. customer/customer.store.ts als Service mit Signals anlegen**
  - privat: customers, filter, selectedId, loading, error; nach außen asReadonly()
  - computed: filtered, selected, total
  - Methoden: load(), setFilter(), select(), addDemo()
- **3. CustomerListComponent nutzt nur den Store; load() in ngOnInit()**
- **Musterlösung: 33_state_store (baut auf 32_reaktivitaet auf)**

Übung 33: CustomerStore einführen (2/2)

- **4. localStorage-effect für den Filter aus Übung 32 in den Store verschieben**
  - Diskussion: Warum gehört die Nebenwirkung dorthin?
- **5. Im Browser prüfen: Filter, Auswahl, Details, Speichern und zurück**
- **6. Reflexion: War ein Store hier nötig? Kriterien anwenden, 3 Sätze**
- **7. Optional: Store mit signalStore aus @ngrx/signals skizzieren**
  - Zeilenzahl und Lesbarkeit vergleichen (nur lesen, nicht installieren)
- **Musterlösung: 33_state_store (baut auf 32_reaktivitaet auf)**

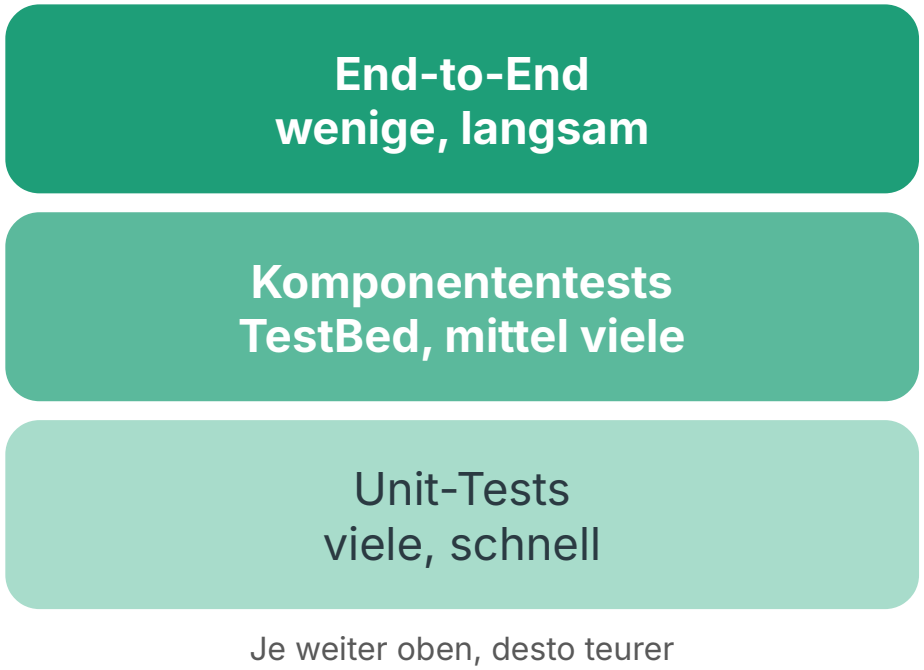
Qualitätssicherung: Testbarkeit und Tests

Wofür Tests?

- Sicherheitsnetz für Änderungen: umbauen, ohne Angst vor versteckten Fehlern
- Dokumentation: Ein Test beschreibt, wie sich der Code verhalten soll
- Frühes Feedback: Fehler fallen beim Entwickeln und in der Pipeline auf
- Mut zum Aufräumen: Refactoring wird erst mit Tests sicher
- Tests sind Code und kosten Pflege
  - Schlechte Tests bremsen mehr, als sie helfen
  - Gute Tests prüfen Verhalten, nicht Details der Umsetzung

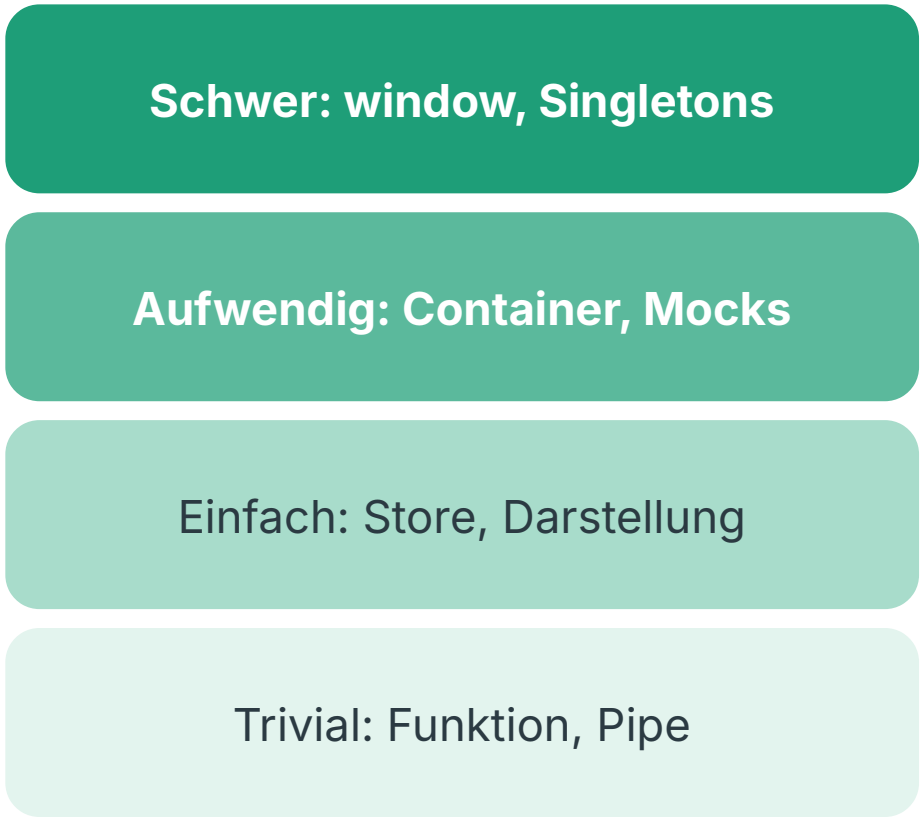
Testpyramide mit Angular-Bezug

- Unten, viele: Unit-Tests für Services, Stores, Pipes, Funktionen
  - laufen in Millisekunden, ohne Browser und Server
- Mitte: Komponententests mit Template (TestBed)
  - prüfen Anzeige, Klicks, Inputs und Outputs
- Oben, wenige: End-to-End-Tests (Playwright, Cypress)
  - echter Browser, echte App, langsam
- Mehr Logik außerhalb von Komponenten: mehr schnelle Tests



Struktur bestimmt Testbarkeit

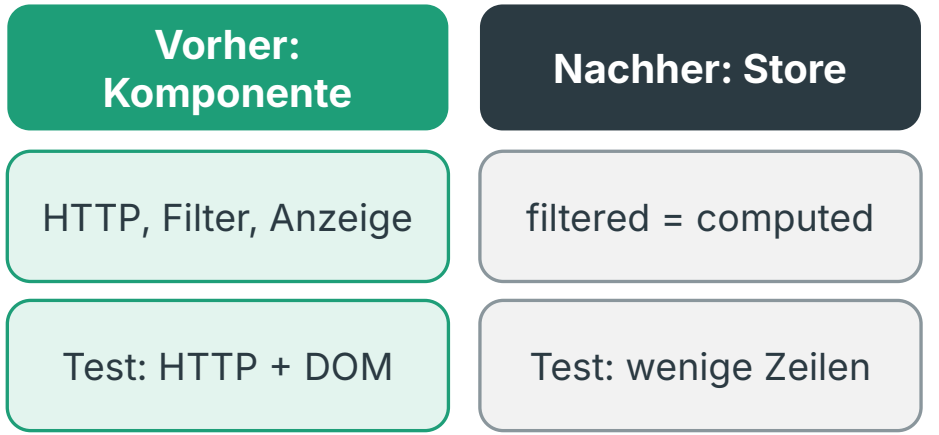
- Reine Funktion oder Pipe: trivial
  - Eingabe hinein, Ausgabe prüfen
- Store oder Service mit Signals: einfach
  - Methode aufrufen, Signal lesen
- Darstellungskomponente: einfach
  - Input setzen, Anzeige und Output prüfen
- Container mit Services: aufwendig (Mocks)
- Mit window, DOM-Tricks, Singletons: schwer



Logik nach unten verlagern

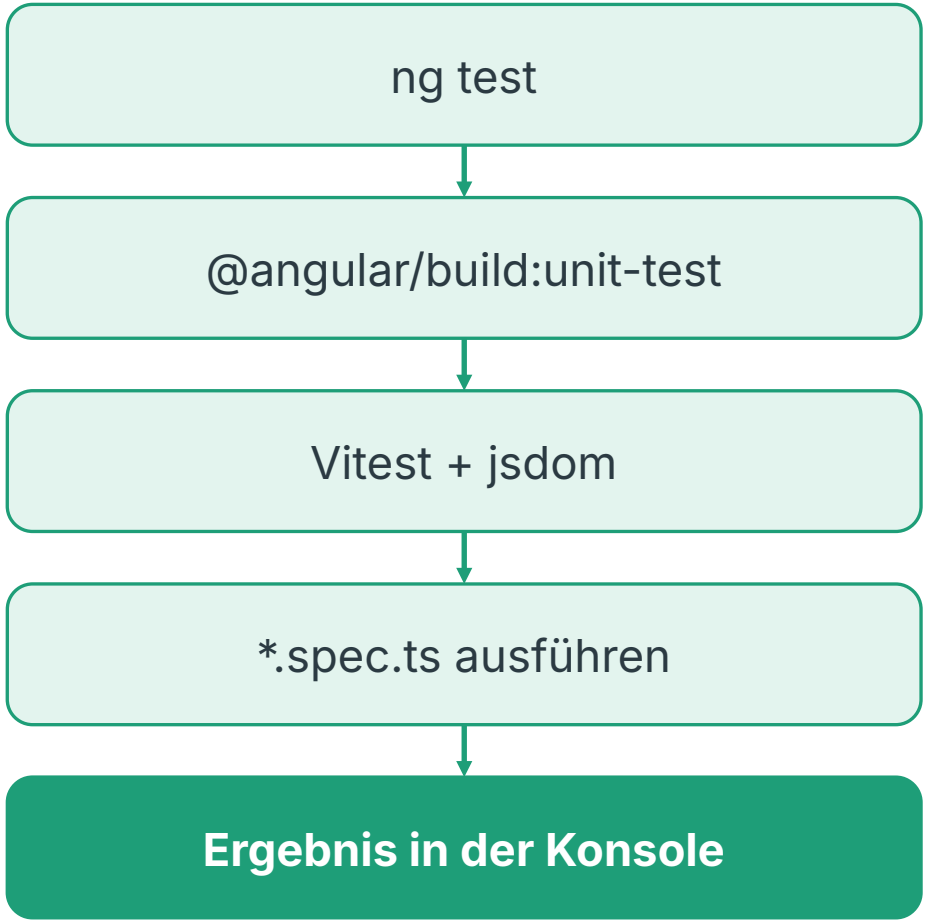
Vorher und nachher: Logik im Store

- Vorher: Komponente ruft HttpClient selbst auf und filtert
  - Test braucht HTTP-Ersatz, DOM und Change Detection
- Nachher: Filterlogik im CustomerStore als computed
  - Test: Store holen, setFilter aufrufen, filtered() lesen
  - kein Template, kein DOM, wenige Zeilen
- Die Komponente zeigt nur noch an, ihr Test bleibt klein



Vitest in Angular 21

- Vitest ist ab Angular 21 der Standard-Test-Runner
- Start mit ng test (einmalig: ng test -- watch=false)
  - Builder @angular/build:unit-test in angular.json
- Tests laufen in jsdom, einem nachgebauten Browser
- Testdateien enden auf .spec.ts, liegen neben dem Code
- describe, it, expect, vi sind global verfügbar
  - über vitest/globals in tsconfig.spec.json



Aufbau eines Vitest-Tests

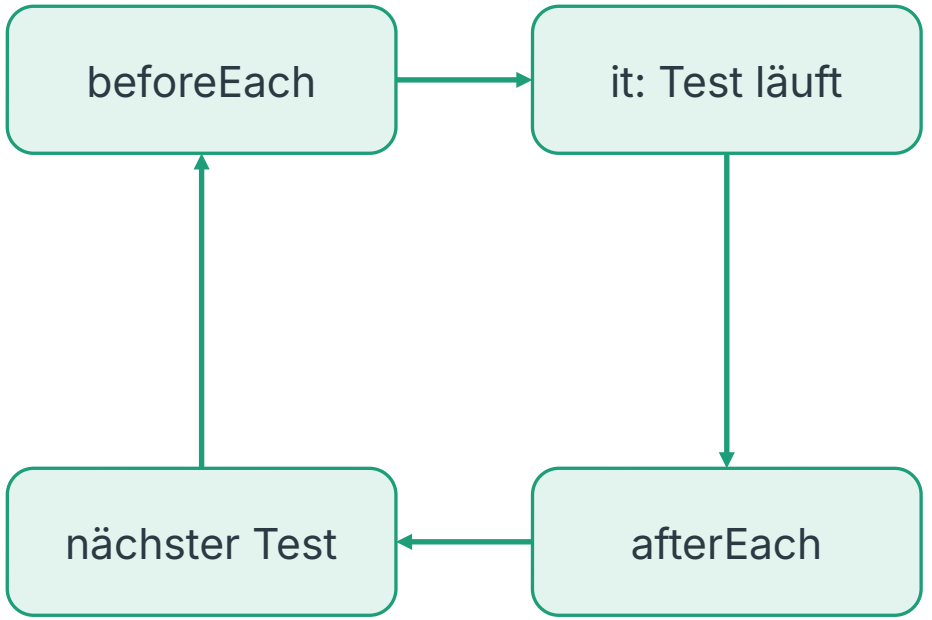
```
describe('CustomerStore', () => {         // Suite: Gruppe von Tests
  const getAll = vi.fn();                 // Ersatz für den Service

  beforeEach(() => {                      // läuft vor JEDEM it
    getAll.mockReset();
    TestBed.configureTestingModule({
      providers: [{ provide: CustomerService, useValue: { getAll } }]
    });
  });

  it('should ignore upper and lower case', async () => {   // ein Test
    getAll.mockReturnValue(of([hans, lisa]));              // Arrange
    const store = TestBed.inject(CustomerStore);
    await store.load();
    store.setFilter('LISA');                               // Act
    expect(store.filtered()).toEqual([lisa]);              // Assert
  });
});
```

describe, it, beforeEach: wer ruft was auf?

- describe(name, fn): Vitest ruft fn sofort auf, sammelt die Tests
- it(name, fn): ein Test; Vitest ruft fn später einzeln auf
  - fn darf async sein: Vitest wartet auf das Promise
- beforeEach(fn) / afterEach(fn): vor bzw. nach jedem it
- beforeAll / afterAll: einmal pro describe
- Test ist rot, wenn fn einen Fehler wirft
  - expect wirft bei falschem Wert einen Fehler



expect und Matcher

- expect(wert) nimmt den tatsächlichen Wert entgegen
- Der Matcher vergleicht mit dem erwarteten Wert
- toBe: gleiche Referenz bzw. gleicher einfacher Wert
- toEqual: gleicher Inhalt (tiefer Vergleich)
- toHaveLength, toContain, toBeUndefined, toThrow
- .not kehrt jeden Matcher um
- toBeTruthy nur sparsam: prüft fast nichts

Matcher

```
expect(store.total()).toBe(1);
expect(store.filtered())
  .toEqual([lisa]);
expect(list).toHaveLength(2);
expect(text).toContain('Hans');
expect(store.selected())
  .toBeUndefined();
expect(store.error())
  .not.toBeUndefined();
```

Arrange, Act, Assert

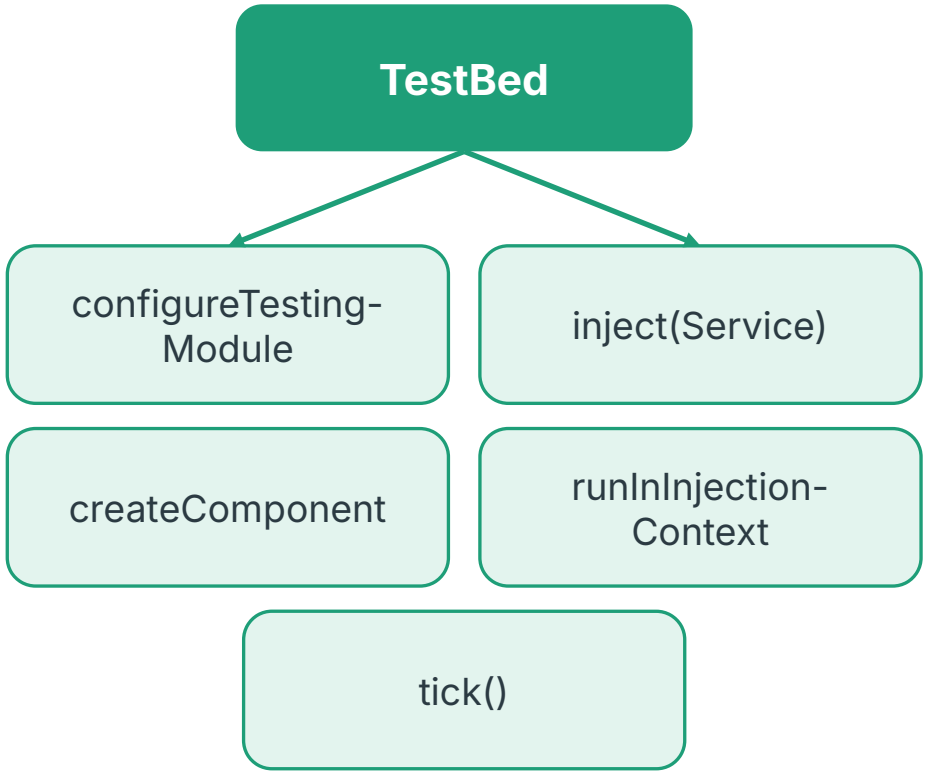
- Arrange: Ausgangslage herstellen (Daten, Ersatzobjekte)
- Act: genau eine Aktion ausführen
- Assert: Ergebnis prüfen
- Ein Test prüft ein Verhalten
- Testnamen beschreiben Verhalten
  - „should ignore upper and lower case“
- Kleine Hilfsfunktionen für wiederholtes Arrange

Arrange, Act, Assert

```
it('should return an empty list
    when nothing matches',
  async () => {
  // Arrange
  const store = await loadedStore();
  // Act
  store.setFilter('xyz');
  // Assert
  expect(store.filtered())
    .toEqual([]);
});
```

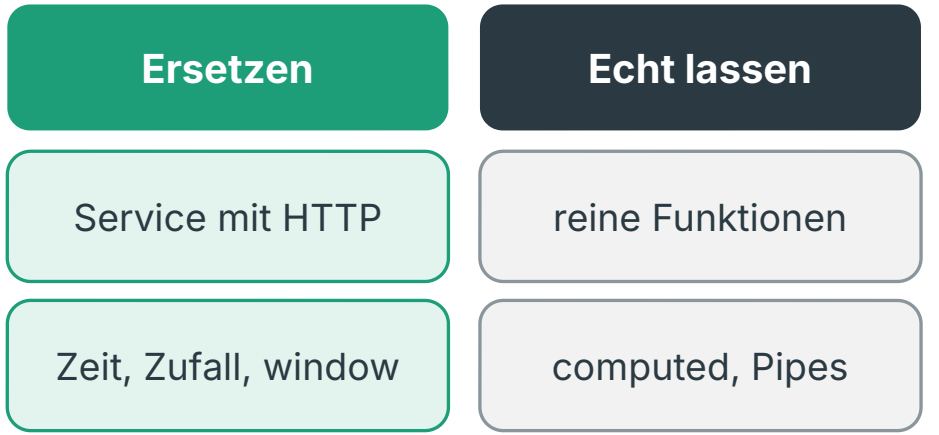
TestBed: Angular im Kleinen

- TestBed baut für jeden Test eine kleine Angular-Umgebung
  - mit eigenem Injector für Dependency Injection
- configureTestingModule({ imports, providers })
  - imports: Standalone-Komponenten, die getestet werden
  - providers: Services oder deren Ersatz
- TestBed.inject(Typ): Instanz aus dem Injector holen
- TestBed.createComponent(Typ): Komponente rendern
- Nach jedem Test setzt Angular das TestBed zurück



Arten von Ersatzobjekten

- Stub oder Fake: Ersatz mit festen Daten
  - z. B. getAll liefert immer zwei Kunden
- Mock (vi.fn): zeichnet Aufrufe auf, Rückgabe steuerbar
  - „Wurde update mit diesem Kunden aufgerufen?“
- Spy (vi.spyOn): beobachtet eine echte Methode
- Echte Abhängigkeit: bei reinen Funktionen kein Ersatz
- Over-Mocking vermeiden: sonst testet man nur Ersatz



vi.fn: eine aufzeichnende Funktion

- vi.fn() liefert eine Funktion ohne eigene Logik
- Sie merkt sich jeden Aufruf mit Parametern
- mockReturnValue(x): Rückgabewert festlegen
- mockImplementation(fn): eigene Logik einsetzen
- mockReset(): Aufrufe und Einstellungen löschen
- Prüfen: toHaveBeenCalledTimes, toHaveBeenCalledWith
- vi.spyOn(obj, 'name'): echte Methode beobachten

vi.fn im Einsatz

```
const getAll = vi.fn();
getAll.mockReturnValue(
  of([hans, lisa]));

getAll();  // liefert das Observable

expect(getAll)
  .toHaveBeenCalledTimes(1);

getAll.mockReturnValue(
  throwError(() => new Error()));
```

Services per providers ersetzen

```
const getAll = vi.fn();

beforeEach(() ⇒ {
  getAll.mockReset();
  TestBed.configureTestingModule({
    providers: [{ provide: CustomerService, useValue: { getAll } }]
  });
});

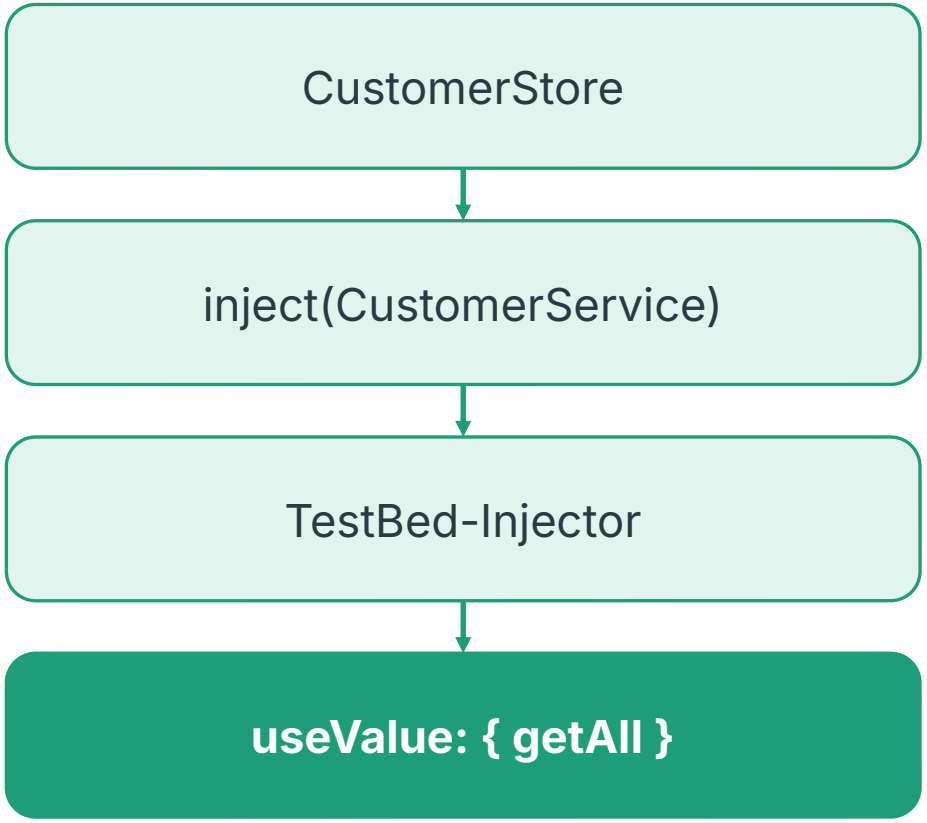
it('should set an error when loading fails', async () ⇒ {
  getAll.mockReturnValue(throwError(() ⇒ new Error('Netzwerk')));
  const store = TestBed.inject(CustomerStore);  // Store bekommt den Ersatz

  await store.load();

  expect(store.error()).toBe('Fehler beim Laden der Kunden.');
  expect(store.loading()).toBe(false);
});
```

Was passiert beim Ersetzen per DI?

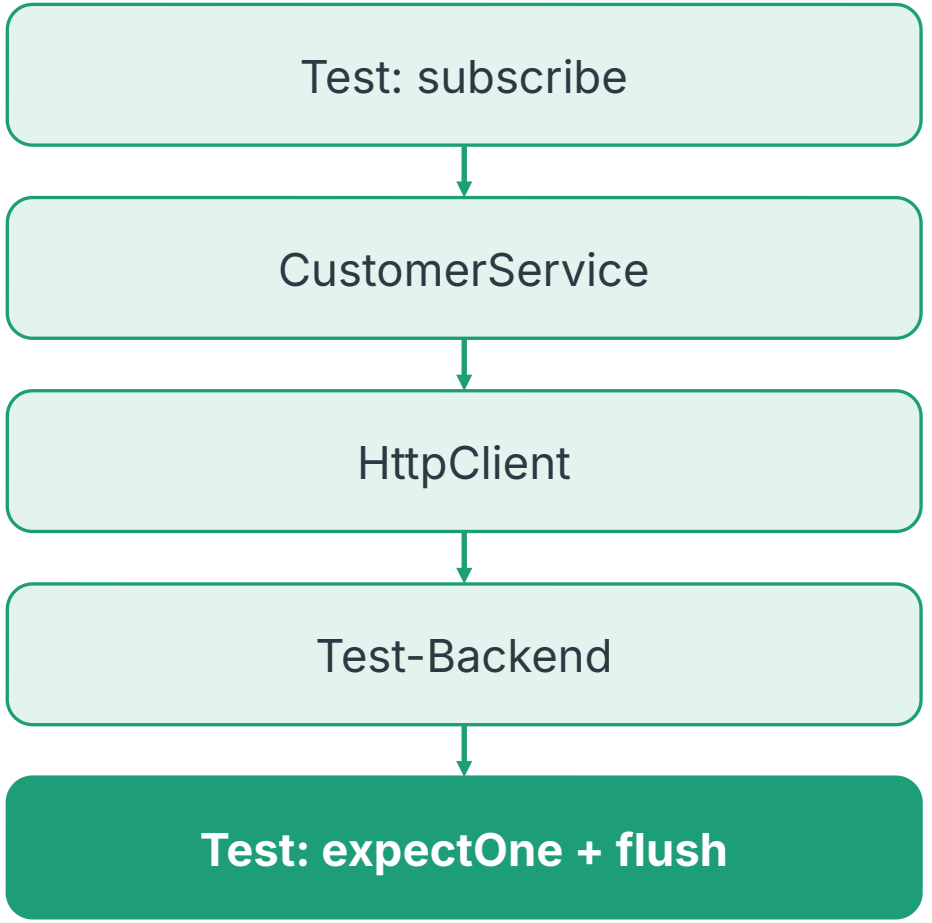
- Store ruft inject(CustomerService) auf
- Der Injector des TestBed sucht einen Provider dafür
- Er findet { provide, useValue } und gibt das Objekt heraus
- Der Store merkt keinen Unterschied, sein Code bleibt gleich
- of(...) liefert Daten, throwError(...) einen Fehler
- mockReset in beforeEach: jeder Test startet sauber



Ersatz statt echtem Service

HTTP testen: der Ablauf

- provideHttpClientTesting ersetzt das Netzwerk
- Der Service sendet wie immer über HttpClient
- Die Anfrage bleibt im Test-Backend hängen
- Der Test holt sie mit expectOne(url)
- flush(daten) schickt die Antwort zurück
- verify(): keine Anfrage darf offen bleiben



HTTP-Aufrufe testen

```
beforeEach(() => {
  TestBed.configureTestingModule({
    providers: [provideHttpClient(), provideHttpClientTesting()]
  });
  service = TestBed.inject(CustomerService);
  httpMock = TestBed.inject(HttpTestingController);
});
afterEach(() => httpMock.verify());   // kein Request darf offen bleiben

it('should send the customer with PUT when updating', () => {
  const customer = { ...new Customer('Lisa', 'Simpson'), id: '1' };
  service.update(customer).subscribe();

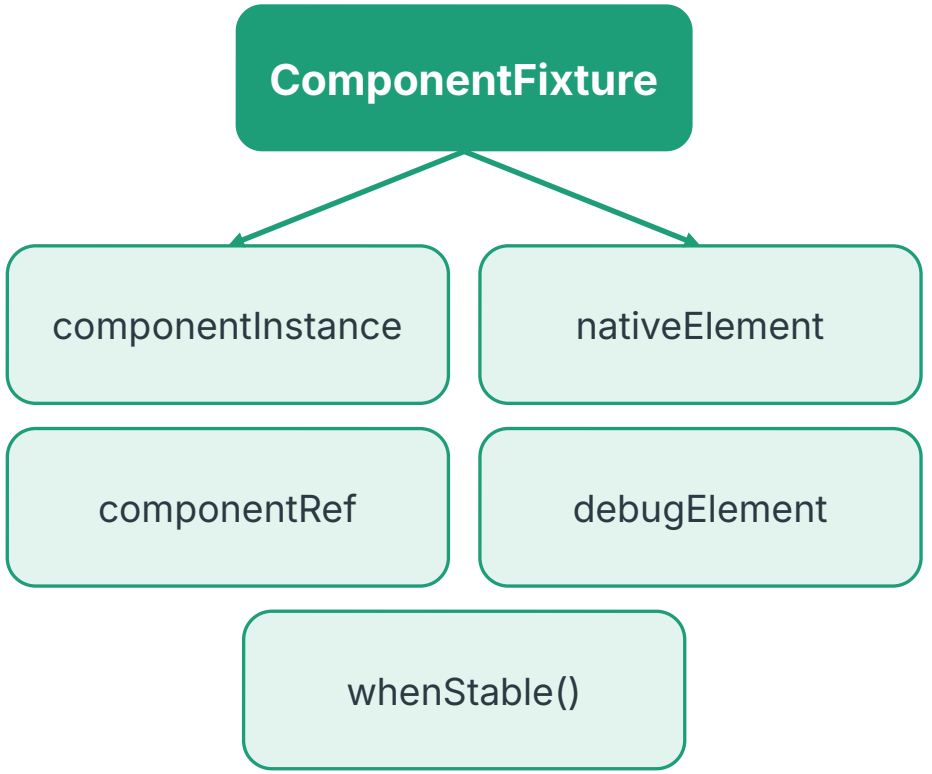
  const request = httpMock.expectOne('api/customer/1');
  expect(request.request.method).toBe('PUT');
  expect(request.request.body).toEqual(customer);
  request.flush({ data: customer });
});
```

HttpTestingController im Detail

- Erst subscribe, dann expectOne
  - Observables vom HttpClient senden erst beim Abonnieren
- expectOne(url) wirft, wenn es 0 oder mehr als 1 Anfrage gibt
- Rückgabe: ein TestRequest
  - request.method, request.body, request.headers prüfen
- flush(body): Antwort mit Status 200 senden
- flush('Fehler', { status: 500, statusText: '...' }): Fehlerfall
- expectNone(url): prüfen, dass nichts gesendet wurde

Komponententest: die Fixture

- TestBed.createComponent(Typ) liefert eine ComponentFixture
- fixture.componentInstance: das Objekt der Klasse
- fixture.nativeElement: das Host-Element im DOM
- fixture.componentRef: zum Setzen von Inputs (setInput)
- fixture.debugElement: Suche mit By.css, triggerEventHandler
- fixture.detectChanges(): Change Detection sofort ausführen
- await fixture.whenStable(): warten, bis alles gerendert ist



Komponententest: Vorbereitung

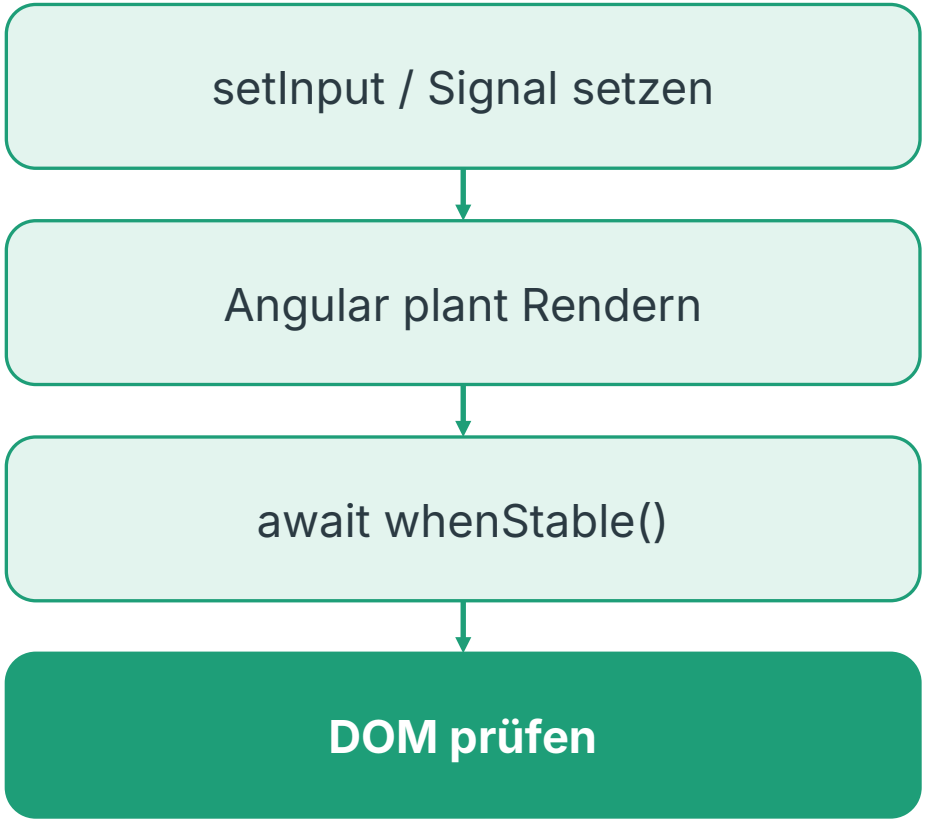
```
describe('CustomerListItemComponent', () ⇒ {
  let fixture: ComponentFixture<CustomerListItemComponent>;
  const el = () ⇒ fixture.nativeElement as HTMLElement;

  beforeEach(async () ⇒ {
    await TestBed.configureTestingModule({
      imports: [CustomerListItemComponent],   // Standalone-Komponente
      providers: [provideRouter([])]          // für routerLink im Template
    }).compileComponents();
    fixture = TestBed.createComponent(CustomerListItemComponent);
  });

  async function render(customer: Customer, selected = false) {
    fixture.componentRef.setInput('customer', customer);
    fixture.componentRef.setInput('selected', selected);
    await fixture.whenStable();
  }
});
```

Change Detection im Test (zoneless)

- Angular 21 läuft ohne Zone.js (zoneless)
- Nach einer Änderung plant Angular das Rendern ein
- await fixture.whenStable() wartet darauf
- fixture.detectChanges() rendert sofort, synchron
- Empfehlung: Zustand ändern, whenStable, DOM prüfen
- Ohne await prüft der Test das alte DOM



Komponenten mit Signal-Inputs testen

```
it('should show the first name and the city', async () ⇒ {
  await render(new Customer('Hans', 'Meyer', 'Hauptstraße 21', '11111', 'Teststadt'));

  expect(el().textContent).toContain('Hans');
  expect(el().textContent).toContain(': Teststadt');
});

it('should emit the customer when it is clicked', async () ⇒ {
  const customer = new Customer('Hans', 'Meyer');
  await render(customer);
  const chosen: Customer[] = [];
  fixture.componentInstance.chosen.subscribe(value ⇒ chosen.push(value));

  el().click();

  expect(chosen).toEqual([customer]);
});
```

Komponententests: wichtige Regeln

- Signal-Inputs nicht direkt zuweisen (component.customer = ...)
  - Inputs sind lesbare Signals ohne set
- Stattdessen fixture.componentRef.setInput('name', wert)
- input.required: vor dem ersten Rendern setzen, sonst Fehler
- Nach Änderungen await fixture.whenStable(), dann prüfen
- Outputs mit subscribe beobachten, Klicks mit click()
- Im DOM nach Inhalt oder Rolle suchen, nicht nach CSS-Details
- Kinder-Komponenten nur einbinden, wenn sie wichtig sind

Signals und computed testen

- signal und computed funktionieren überall, auch ohne TestBed
- computed rechnet beim Lesen neu, wenn sich Quellen geändert haben
- Test: Quelle setzen, computed lesen, Ergebnis prüfen
- Kein Warten nötig: Signals sind synchron
- Stores mit inject() holt man über TestBed.inject
- Readonly-Signals von außen nur über Methoden ändern

ohne TestBed

```
const filter = signal('');
const names = signal(['Hans',
                      'Lisa']);
const visible = computed(() =>
  names().filter(n =>
    n.toLowerCase()
     .includes(filter())));

filter.set('li');
expect(visible())
  .toEqual(['Lisa']);
```

Effects testen mit TestBed.tick()

```
// Im Store: effect(() ⇒ saveFilter(this._filter()));

it('should write the filter to localStorage', () ⇒ {
  const store = TestBed.inject(CustomerStore);   // effect braucht Injection Context

  store.setFilter('lisa');
  TestBed.tick();            // führt anstehende Effects jetzt aus

  expect(localStorage.getItem('customerFilter')).toBe('lisa');
});

it('should restore the filter on creation', () ⇒ {
  localStorage.setItem('customerFilter', 'burns');
  const store = TestBed.inject(CustomerStore);
  expect(store.filter()).toBe('burns');
});
```

Component Harnesses (CDK)

- Harness: Test-API für eine Komponente statt DOM-Selektoren
- Fertige Harnesses für Angular Material (MatButtonHarness ...)
- Paket @angular/cdk/testing
- loader = TestbedHarnessEnvironment.loader(fixture)
- Methoden sind async und warten selbst auf Stabilität
- Vorteil: Tests überleben Änderungen am inneren HTML

Harness (Beispiel Material)

```
const loader =
  TestbedHarnessEnvironment
    .loader(fixture);

const button = await loader
  .getHarness(MatButtonHarness
    .with({ text: 'Speichern' }));

await button.click();
expect(await button.isDisabled())
  .toBe(true);
```

Was macht einen guten Test aus?

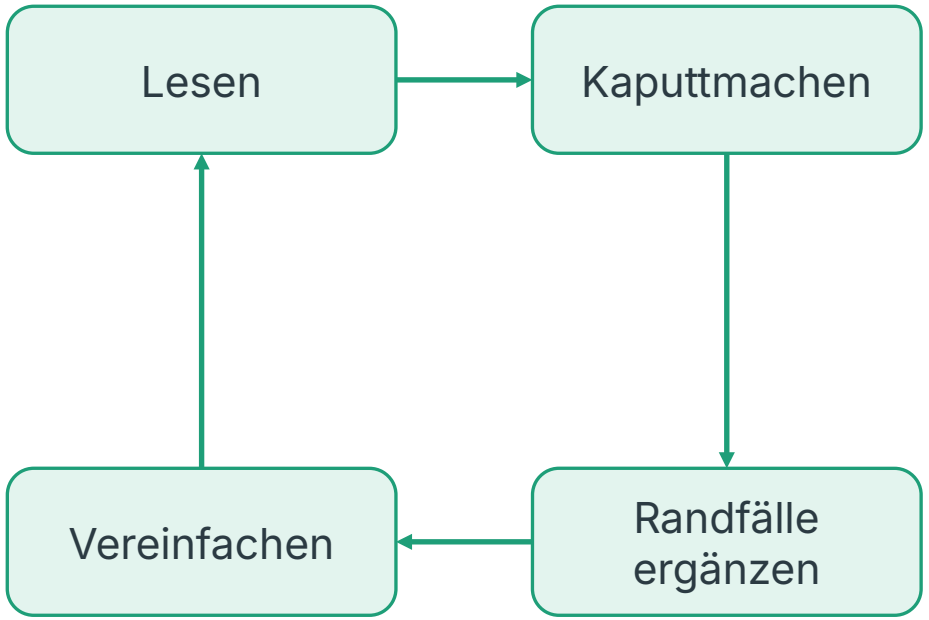
- Er schlägt fehl, wenn das Verhalten kaputt ist, und nur dann
- Er ist unabhängig von Reihenfolge und anderen Tests
- Er ist schnell und deterministisch: keine echten Timer, kein Netz
  - vi.useFakeTimers() ersetzt echte Zeit durch steuerbare Zeit
- Er ist lesbar: Arrange, Act, Assert, sprechende Namen, kleine Testdaten
- Er prüft Verhalten, nicht Implementierungsdetails (private Felder, CSS-Klassen)
- Er deckt Randfälle ab: leere Liste, kein Treffer, Fehler, unbekannte ID
- Coverage ist ein Hinweis, kein Ziel: 100 % ohne Prüfungen beweisen nichts

Automatisch erzeugte Tests richtig einordnen

- Quellen: ng generate (Gerüste), Assistenten in der Entwicklungsumgebung, KI-Werkzeuge
- should create mit toBeTruthy: prüft nur, dass die Instanz entsteht
- Test wiederholt die Implementierung: Tautologie, Fehler fallen nicht auf
- Alles gemockt, nur Aufrufe geprüft: testet nicht das echte Verhalten
- Nur der Normalfall: leere Listen, null, Fehler und Grenzwerte fehlen
- Snapshots großer Templates: brechen oft und werden blind aktualisiert
- Test passt sich dem Fehler an: bestätigt falsches Verhalten

Prüfverfahren für generierte Tests

- 1. Lesen: Was behauptet der Test?
  - Passt das zur Anforderung?
- 2. Kaputtmachen: Produktivcode absichtlich ändern
  - Wird kein Test rot, sind die Tests wertlos
- 3. Randfälle ergänzen: leer, Fehler, Sonderzeichen
- 4. Vereinfachen: Mocks reduzieren, Namen verbessern
- 5. Verantwortung: Wer committet, haftet dafür



Übung 34: Tests schreiben und bewerten (1/2)

- **Öffne das Projekt 34_tests (Test-Runner: Vitest, Start mit ng test)**
- Teil A: Schreibe customer.store.spec.ts für CustomerStore, ohne Oberfläche
  - Laden, Filter (Groß-/Kleinschreibung, Teiltreffer), Filter ohne Treffer, leere Liste
  - Auswahl (auch unbekannte ID), Fehlerfall beim Laden, Filter in localStorage
- Ersetze CustomerService per providers durch ein Test-Double
  - const getAll = vi.fn(); { provide: CustomerService, useValue: { getAll } }
  - Rückgabe steuern: getAll.mockReturnValue(of([...])) bzw. throwError(() ⇒ ...)
- Effect prüfen: nach store.setFilter('x') mit TestBed.tick() die Effects ausführen
- **Musterlösung: 34_tests (baut auf 33_state_store auf)**

Übung 34: Tests schreiben und bewerten (2/2)

- **Teil B: Teste CustomerListItemComponent**
  - Name wird angezeigt, ohne Stadt kein Doppelpunkt, Klick löst chosen aus
  - Klassen discontinued und selected; Inputs per fixture.componentRef.setInput
  - Optional: CustomerFilterComponent und CustomerService (HttpTestingController)
- Teil C: Speichere material/generierter-store-test.spec.ts.txt als Spec und führe sie aus
  - Finde die fünf versteckten Schwächen
  - Ändere in filtered includes zu startsWith: Welche Tests werden nicht rot?
  - Verbessere die Spec, danach die generierte Datei wieder löschen
- **Musterlösung: 34_tests (baut auf 33_state_store auf)**

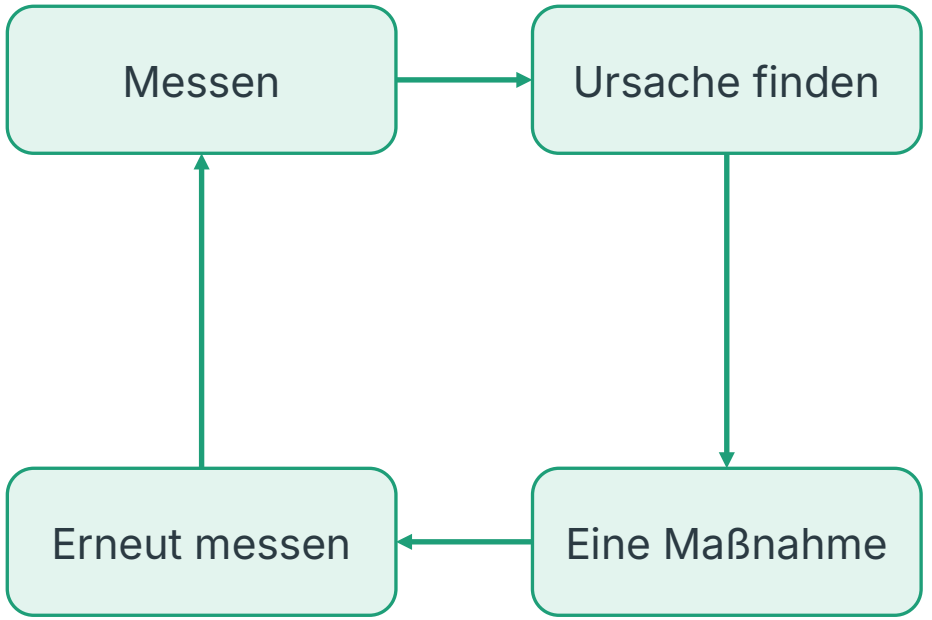
Performance und Laufzeitverhalten

Performance im Anwendungskontext

- Performance ist Nutzererlebnis: Fühlt sich die App schnell an?
- Ladezeit: Wie schnell ist etwas sichtbar und bedienbar?
  - Einflüsse: Bundle-Größe, Netzwerk, Bilder, Rendering
- Laufzeit: Wie flüssig reagiert die App auf Eingaben?
  - Einflüsse: Change Detection, große Listen, Berechnungen
- Regel Nr. 1: Erst messen, dann optimieren
- Vorzeitige Optimierung macht Code komplizierter, ohne zu wirken

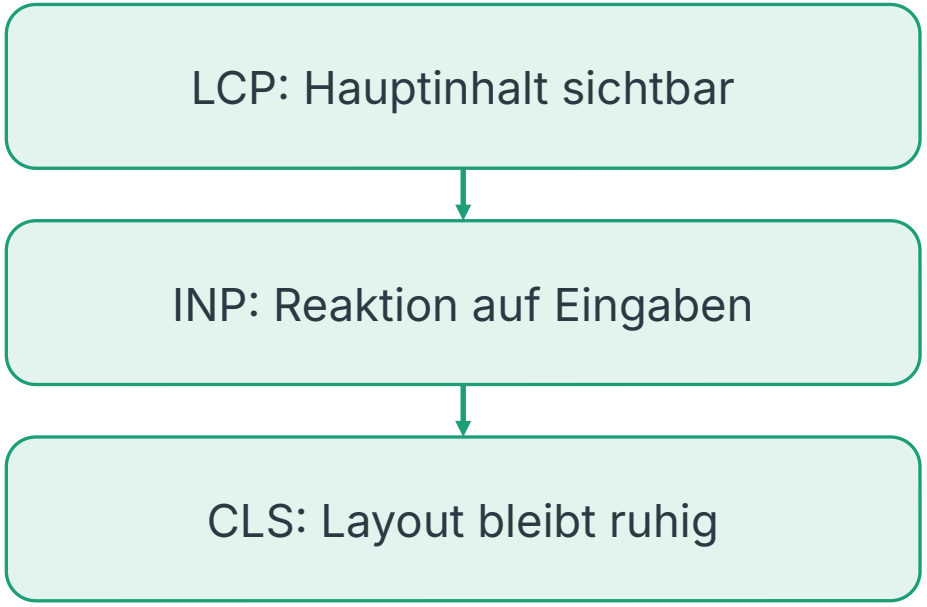
Vorgehen: Messen, ändern, erneut messen

- Ausgangslage messen und notieren
- Ursache suchen: Wo geht die Zeit verloren?
- Genau eine Maßnahme umsetzen
- Unter gleichen Bedingungen erneut messen
  - gleicher Build, gleiche Daten, gleiches Gerät
- Bringt sie nichts: zurücknehmen



Core Web Vitals: drei Kennzahlen

- LCP (Largest Contentful Paint)
  - Wann ist das größte Element sichtbar? Gut: bis 2,5 s
- INP (Interaction to Next Paint)
  - Wie schnell reagiert die Seite auf Eingaben? Gut: bis 200 ms
- CLS (Cumulative Layout Shift)
  - Wie stark springt das Layout? Gut: bis 0,1
- Gemessen bei echten Nutzern und im Labor



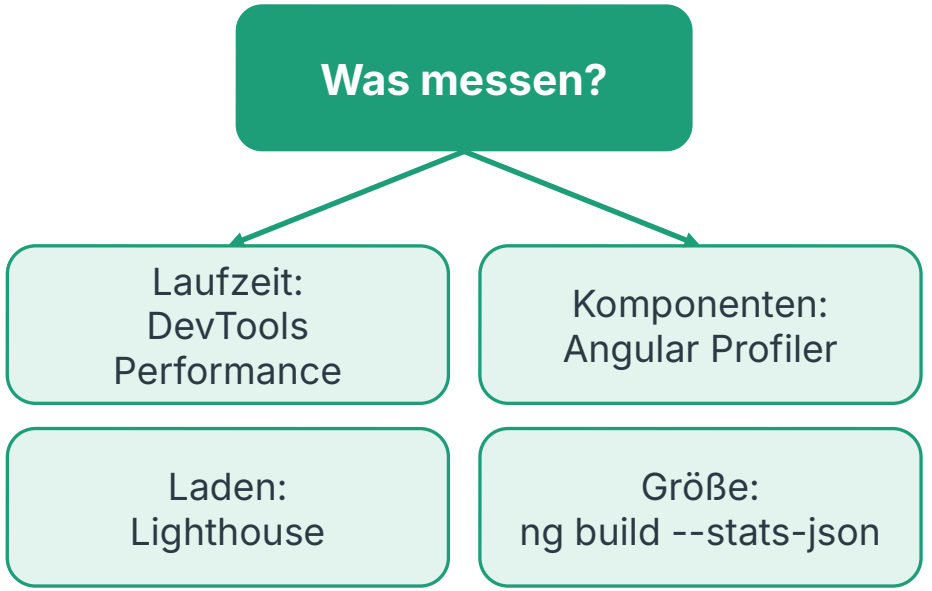
Core Web Vitals

Core Web Vitals: Ursachen und Hebel in Angular

- LCP schlecht: großes Start-Bundle, großes Hauptbild
  - Hebel: Lazy Loading, @defer, NgOptimizedImage mit priority, SSR
- INP schlecht: lange Aufgaben im Haupt-Thread nach einer Eingabe
  - Hebel: weniger Change Detection (Signals, OnPush), Debounce, Virtual Scrolling
- CLS schlecht: Bilder ohne Größe, nachgeladene Inhalte schieben das Layout
  - Hebel: width und height am Bild, @placeholder mit fester Höhe
- Lighthouse misst im Labor; INP am besten mit DevTools oder echten Nutzerdaten

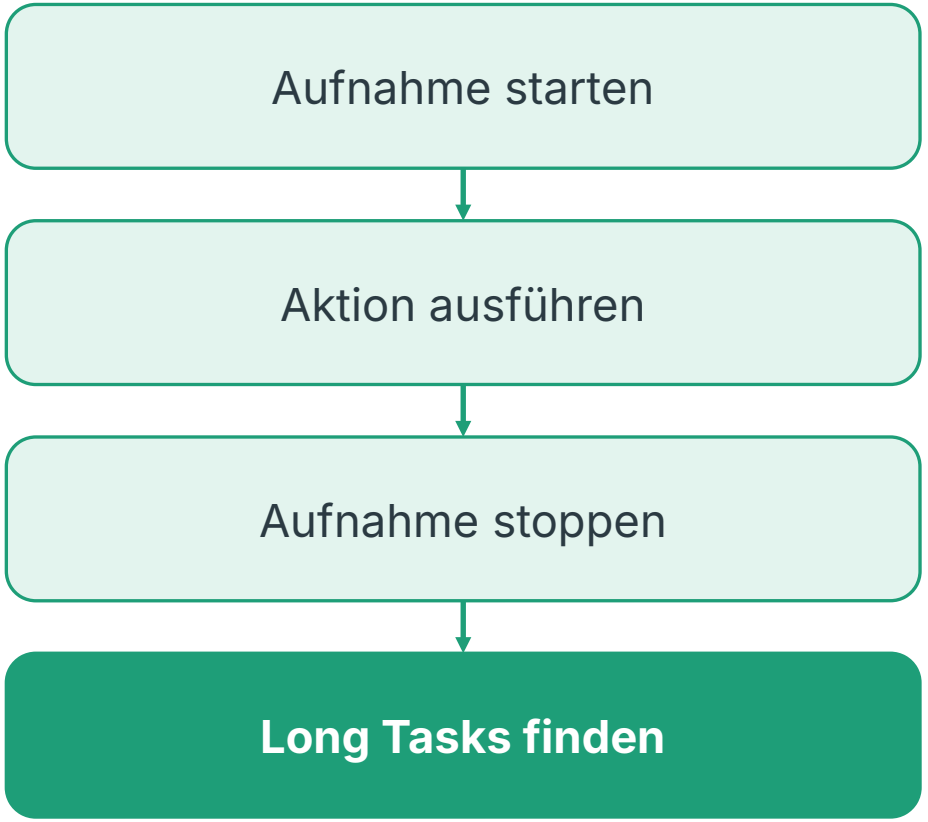
Messen: Werkzeuge im Überblick

- Chrome DevTools, Bereich Performance: was passiert wann?
- Angular DevTools, Profiler: welche Komponente kostet Zeit?
- Lighthouse: Ladeverhalten und Core Web Vitals
- ng build: Größe der Bundles, Budgets
  - --stats-json für die genaue Analyse
- Immer mit Production-Build messen



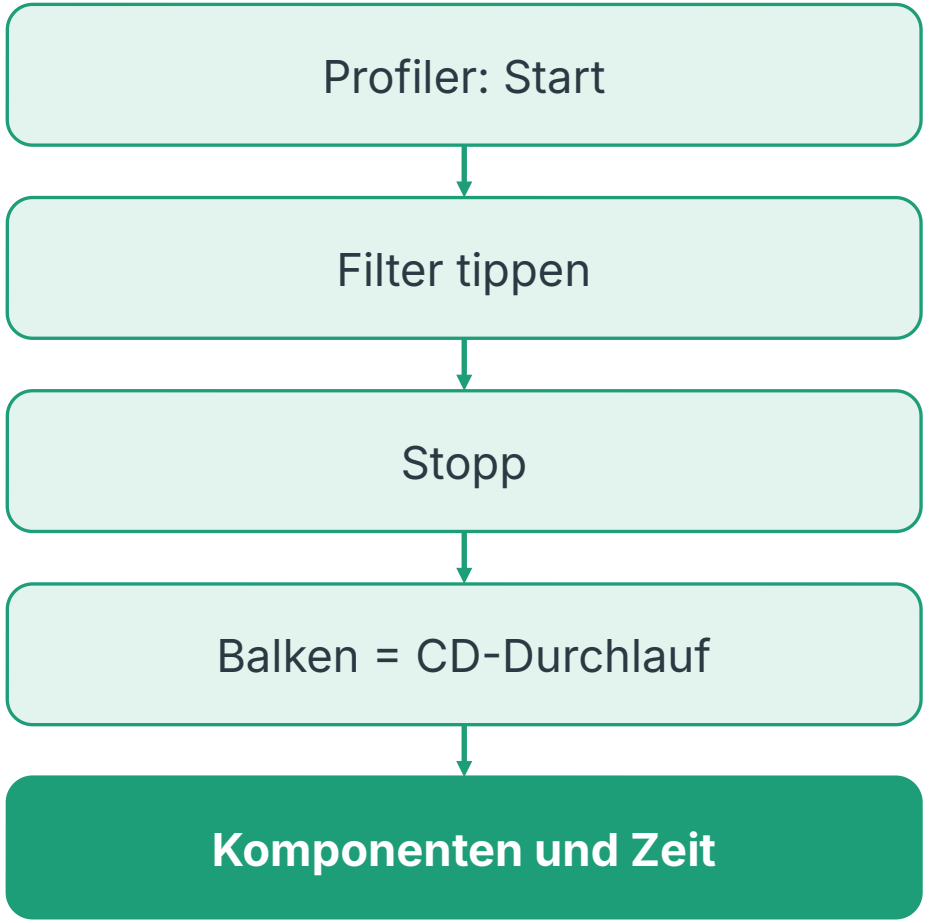
Chrome DevTools: Performance-Aufnahme

- F12, Bereich Performance, Aufnahme starten
- Aktion ausführen, z. B. im Filter tippen
- Aufnahme stoppen und Zeitachse lesen
  - Long Tasks über 50 ms sind rot markiert
  - Flamegraph: welche Funktion ruft welche auf
- CPU drosseln: langsame Geräte simulieren



Angular DevTools: der Profiler

- Browser-Erweiterung für Chrome und Firefox
- Funktioniert mit dem Entwicklungs-Build (ng serve)
- Reiter Profiler: Aufnahme starten, Aktion, stoppen
- Jeder Balken: ein Durchlauf der Change Detection
- Klick auf Balken: geprüfte Komponenten und Dauer
- Frage: Werden mehr Komponenten geprüft als nötig?



Lighthouse richtig einsetzen

- In Chrome DevTools: Bereich Lighthouse, Analyse starten
- Ergebnis: Punkte und Kennzahlen (LCP, CLS, TBT ...)
  - TBT: blockierte Zeit, Laborersatz für INP
- Mit Production-Build messen, nicht mit ng serve
  - ng build, dann dist/.../browser mit einem Webserver ausliefern
- Inkognito-Fenster: keine Erweiterungen stören
- Mehrmals messen, Schwankungen beachten
- Lighthouse CI: Messung automatisch in der Pipeline

Bundle-Analyse: was steckt im Build?

- ng build zeigt Initial chunks und Lazy chunks mit Größe
- Initial: wird beim Start geladen, soll klein bleiben
- ng build --stats-json erzeugt stats.json
  - Analyse z. B. im esbuild Bundle Analyzer
- Fragen: Welche Bibliothek ist groß? Was ist unnötig?
- Budgets in angular.json: Warnung oder Fehler

Ausgabe von ng build (Beispiel)

```
Initial chunk files | Raw size
main-XYZ.js         | 312 kB
styles-XYZ.css      |   4 kB

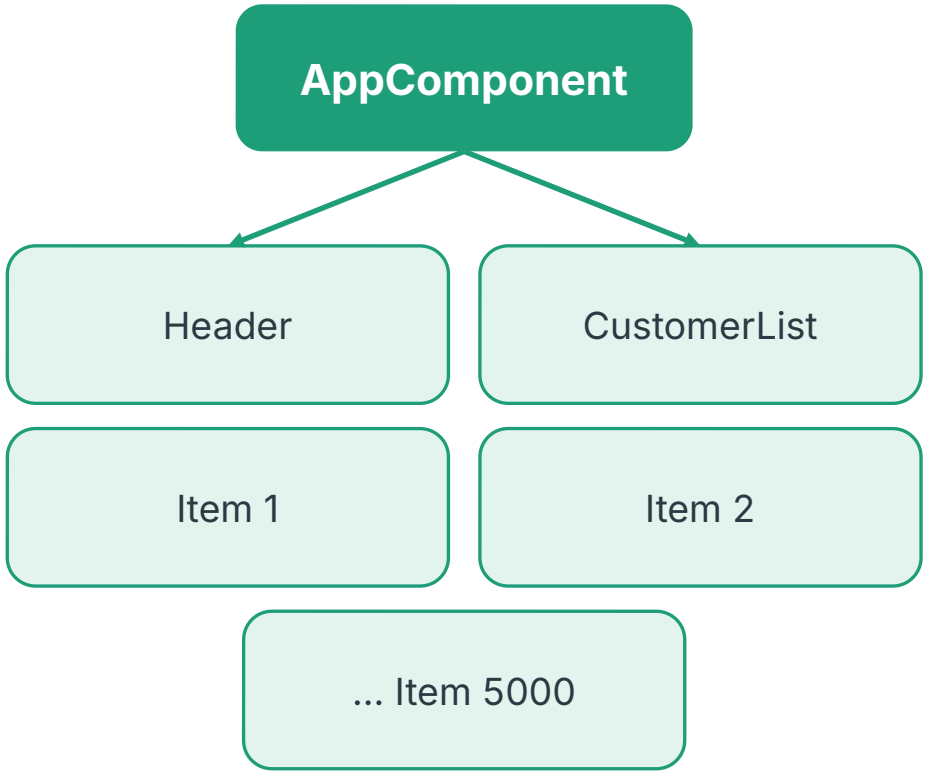
Lazy chunk files    | Raw size
chunk-ABC.js        |  48 kB
  (customer-details-
   component)
```

Budgets in angular.json

```
"configurations": {
  "production": {
    "budgets": [
      {
        "type": "initial",            // alles, was beim Start geladen wird
        "maximumWarning": "500kB",    // ab hier: Warnung
        "maximumError": "1MB"         // ab hier: Build schlägt fehl
      },
      {
        "type": "anyComponentStyle",  // CSS einer einzelnen Komponente
        "maximumWarning": "4kB",
        "maximumError": "8kB"
      }
    ]
  }
}
```

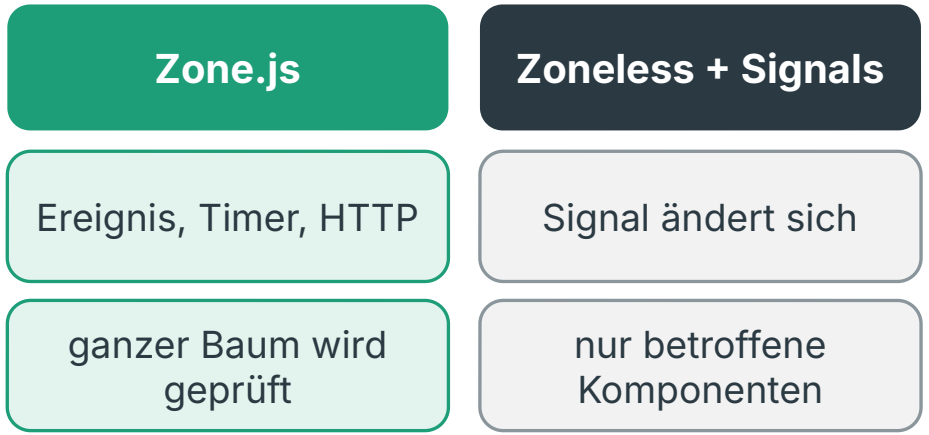
Change Detection: die Grundidee

- Komponenten bilden einen Baum, beginnend bei AppComponent
- Change Detection: Template-Bindungen prüfen, DOM anpassen
- Läuft von oben nach unten durch den Baum
- Kosten wachsen mit Zahl der geprüften Komponenten
- Ziel: nur prüfen, was sich wirklich geändert hat
- Frage: Wer sagt Angular, dass sich etwas geändert hat?



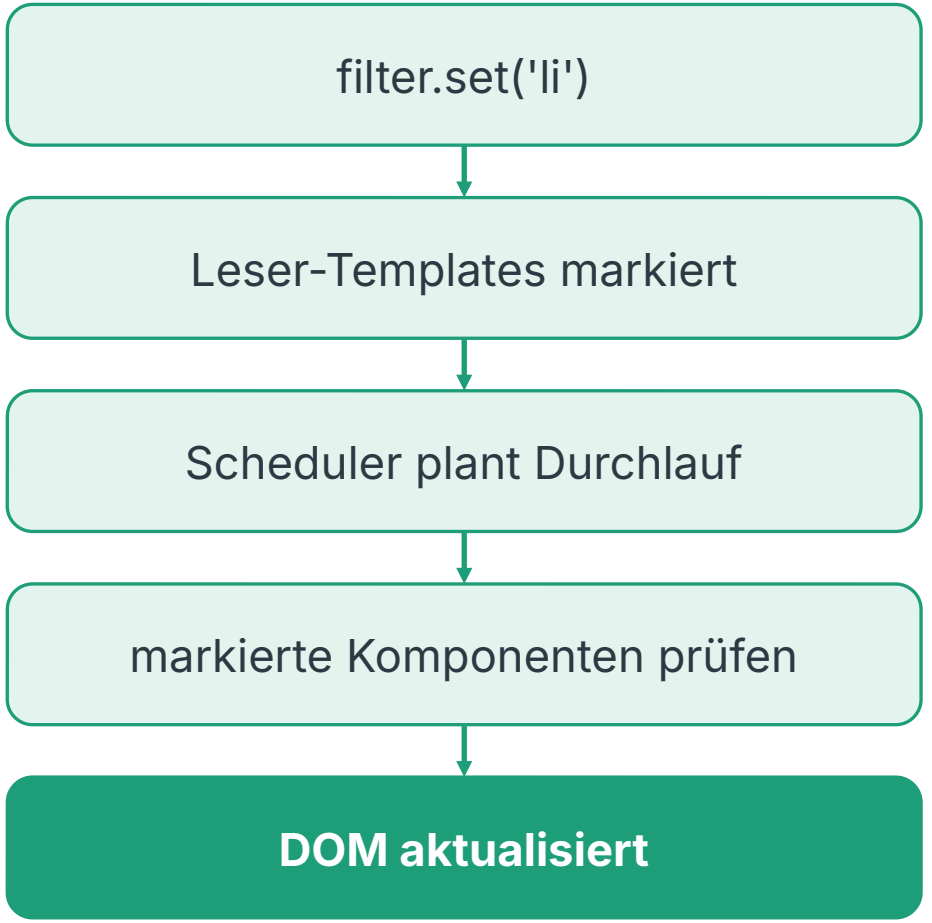
Zone.js und zoneless im Vergleich

- Zone.js (früher Standard)
  - fängt jedes Ereignis, jeden Timer, jede HTTP-Antwort ab
  - danach: ganzer Baum wird geprüft
- Zoneless (Standard in Angular 21)
  - Angular wird gezielt benachrichtigt, z. B. vom Signal
  - nur markierte Komponenten werden geprüft
- Startprojekt: provideZonelessChangeDetection()



Ablauf: Signal ändert sich (zoneless)

- Code ruft set oder update auf einem Signal auf
- Angular kennt alle Templates, die das Signal lesen
- Diese Komponenten werden als „zu prüfen“ markiert
- Der Scheduler plant einen Durchlauf ein
  - mehrere Änderungen ergeben einen Durchlauf
- Nur markierte Teile werden geprüft, DOM angepasst



Was löst eine Neu-Prüfung aus?

- Ein Signal, das im Template gelesen wird, ändert sich
- Ein Event-Handler aus dem Template läuft, z. B. (click)
- Ein Input bekommt einen neuen Wert (neue Referenz)
  - auch per componentRef.setInput im Test
- Die async-Pipe erhält einen neuen Wert
- ChangeDetectorRef.markForCheck() wird aufgerufen
- Keine Neu-Prüfung: normales Feld in setTimeout oder subscribe ändern
  - Abhilfe: den Wert in ein Signal legen

OnPush: Komponenten gezielt überspringen

- Ohne Angabe: Komponente wird bei jedem Durchlauf geprüft
- OnPush: nur prüfen, wenn sie markiert ist
  - neuer Input, Template-Event, gelesenes Signal, async-Pipe
- Sonst wird sie samt Kindern übersprungen
- Empfehlung: OnPush für alle neuen Komponenten
- Passt ideal zu Signals und input()

customer-list-item.component.ts

```
@Component({
  selector: '[customer-list-item]',
  changeDetection:
    ChangeDetectionStrategy.OnPush,
  templateUrl: '…',
})
export class
  CustomerListItemComponent {
  customer =
    input.required<Customer>();
  selected = input(false);
}
```

OnPush und Referenzen

- Inputs werden per Referenz verglichen (===)
- Objekt verändern (mutieren): gleiche Referenz, keine Prüfung
- Neues Objekt oder neue Liste: neue Referenz, Prüfung
- Bei Signals gilt dasselbe: set mit gleicher Referenz meldet nichts
- Darum: update(list ⇒ [...list, neu]) statt list.push(neu)
- Beispiel: addDemo() im CustomerStore



Struktur beeinflusst die Laufzeit

- Kleine Komponenten mit eigenen Signals ergeben kleine Aktualisierungsbereiche
- Eine große Komponente, die alles liest, wird bei jeder Änderung komplett neu bewertet
- Beispiel Invoice-App: Der Filtertext darf nur die Liste neu rendern, nicht den Kopfbereich
- Ein weiterer Grund für die Aufteilung in kleine Komponenten (Übung 31)

Zustandsstruktur und Rendering

```
// Schlecht: ein grobes Objekt-Signal, jede Änderung betrifft alle Leser
state = signal({ customers: [], filter: '', selectedId: null });

// Besser: feine Signals und computed, nur betroffene Leser werden aktualisiert
customers  = signal<Customer[]>([]);
filter     = signal('');
selectedId = signal<string | null>(null);
selected   = computed(() => this.customers().find(c => c.id === this.selectedId()));

// computed meldet nur dann eine Änderung, wenn sich sein Ergebnis wirklich ändert.
// Leser von selected werden also nicht geprüft, wenn sich nur der Filter ändert.
```

@for und track

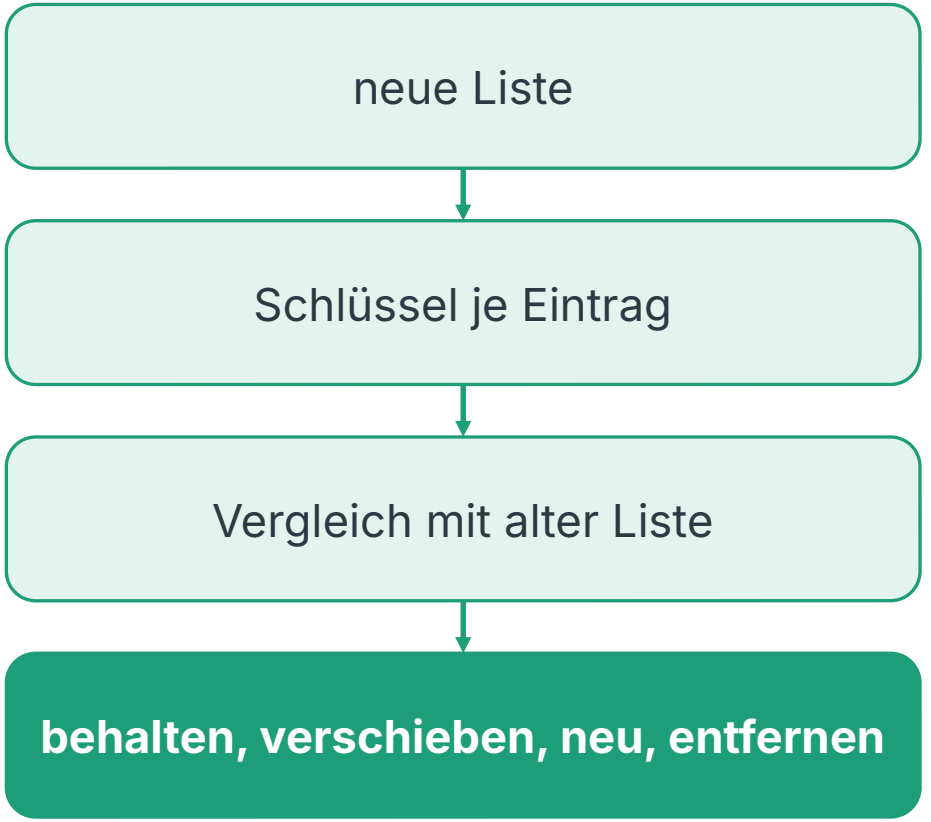
```
<!-- Schlecht: Objektidentität (so im Startprojekt) -->
@for (customer of store.filtered(); track customer) {
  <li customer-list-item [customer]="customer"></li>
}

<!-- Besser: stabile, eindeutige ID -->
@for (customer of store.filtered(); track customer.id) {
  <li customer-list-item [customer]="customer"></li>
} @empty {
  <p>Keine Kunden gefunden.</p>
}

<!-- Weitere Variablen im Block: $index, $first, $last, $even, $odd, $count -->
```

Was macht track genau?

- Angular berechnet für jeden Eintrag einen Schlüssel
- Neue Liste: Schlüssel mit der alten Liste vergleichen
  - gleicher Schlüssel: DOM wiederverwenden, ggf. verschieben
  - neuer Schlüssel: DOM neu anlegen
  - fehlender Schlüssel: DOM entfernen
- track customer: neu geladene Objekte gelten als neu



track: Regeln

- track customer: bei neu geladenen Daten entsteht alles neu
- Zustand der Zeilen (Fokus, Eingaben) geht verloren
- track $index nur bei statischen Listen
- Faustregel: stabile, eindeutige ID verwenden
- Bei Virtual Scrolling: trackBy-Funktion an cdkVirtualFor



Berechnungen im Template

```
{{ expensiveSum() }}      <!-- Methode: läuft bei jeder Prüfung der Komponente -->
{{ total() }}             <!-- computed: gecacht, rechnet nur bei geänderten Quellen -->
{{ customer.lastOrderDate | daysAgo }}   <!-- reine Pipe: gecacht je Eingabe -->
<!-- Vorsicht bei unreinen Pipes (pure: false) und vielen async-Pipes in Schleifen -->
```

Große Listen

- track mit ID: weniger Neuaufbau der DOM-Elemente
- OnPush und leichte Zeilenkomponenten: günstigeres Prüfen
- Debounce für Filtereingaben: weniger Neuberechnungen
- Virtual Scrolling (@angular/cdk/scrolling)
  - nur sichtbare Zeilen plus Puffer sind im DOM
- Paginierung oder serverseitiges Filtern: weniger Daten
- Erst messen, welche Maßnahme bei euch zählt

Virtual Scrolling mit dem CDK

```
<!-- imports: [ScrollingModule]  aus '@angular/cdk/scrolling' -->
<cdk-virtual-scroll-viewport class="viewport" itemSize="32" role="list">
  <div *cdkVirtualFor="let customer of store.filtered(); trackBy: trackById"
       customer-list-item
       [customer]="customer"
       [selected]="isSelectedCustomer(customer)"
       (chosen)="listItemClicked($event)"></div>
</cdk-virtual-scroll-viewport>

/* CSS: der Viewport braucht eine feste Höhe */
.viewport { height: 400px; }

// In der Klasse:
trackById = (index: number, customer: Customer) => customer.id;
```

Debounce für den Filter

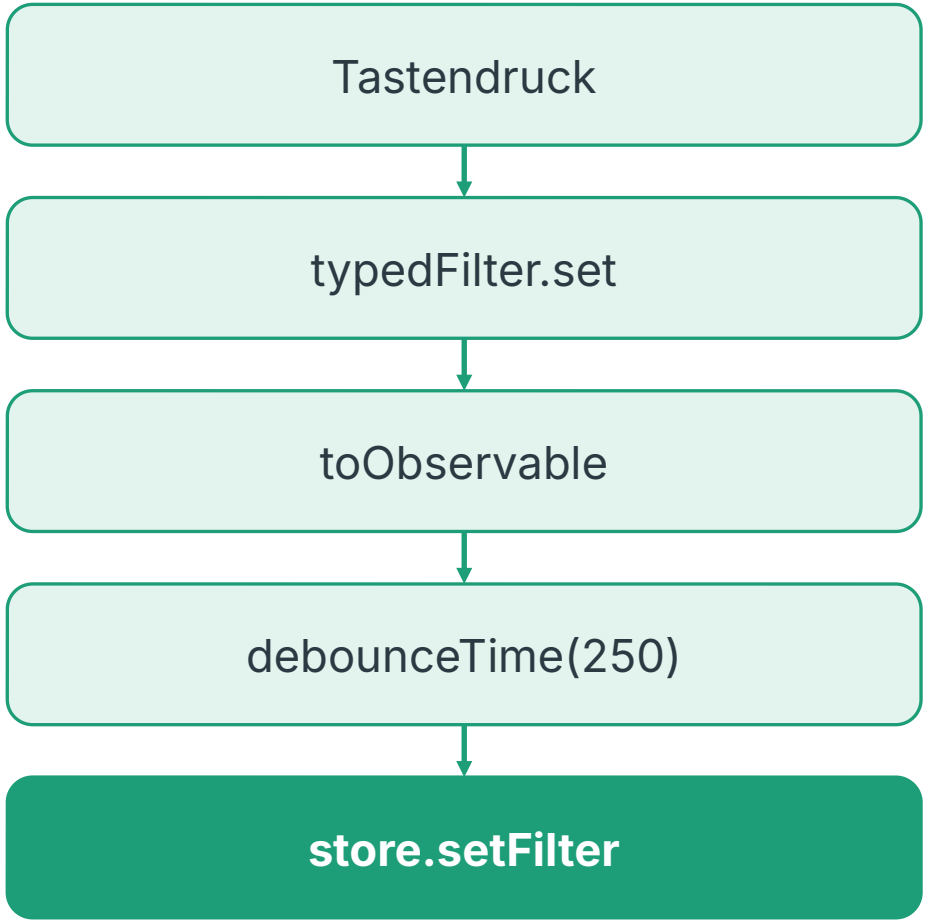
```
// Eingabefeld zeigt typedFilter sofort, der Store bekommt den Text verzögert
protected readonly typedFilter = signal(this.store.filter());

constructor() {
  toObservable(this.typedFilter).pipe(
    debounceTime(250),        // erst nach 250 ms Ruhe weitergeben
    takeUntilDestroyed()      // beim Zerstören der Komponente abmelden
  ).subscribe(text ⇒ this.store.setFilter(text));
}

onFilterChanged(text: string) {
  this.typedFilter.set(text);
}
onFilterCleared() {           // Leeren wirkt sofort, ohne Wartezeit
  this.typedFilter.set('');
  this.store.setFilter('');
}
```

Debounce: der Ablauf

- Jeder Tastendruck setzt typedFilter
- toObservable meldet jeden neuen Wert
- debounceTime(250) startet bei jedem Wert neu
- Erst nach 250 ms ohne Eingabe geht der Wert weiter
- Ergebnis: „Hans“ tippen filtert einmal statt viermal
- takeUntilDestroyed verhindert ein Speicherleck

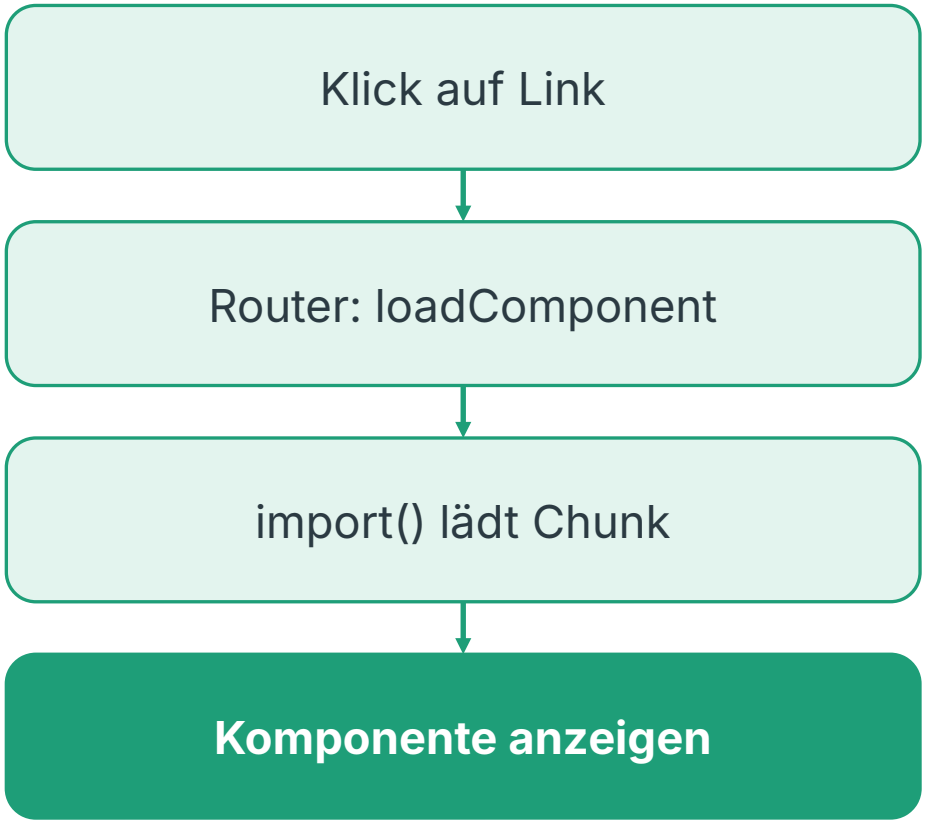


Ladezeit: nur laden, was gebraucht wird

- Lazy Routes: Seiten erst beim Aufruf laden
- @defer: Teile einer Seite erst bei Bedarf laden
- NgOptimizedImage (Übung 27): priority für das Hauptbild
  - width und height verhindern Layout-Sprünge
- Bundle-Hygiene: nur importieren, was man nutzt
  - Tree-Shaking entfernt ungenutzten Code beim Build
- Budgets halten das Start-Bundle klein

Lazy Loading von Routen

- loadComponent: eine Komponente bei Bedarf laden
- loadChildren: eine ganze Gruppe von Routen laden
- () ⇒ import('...') wird erst bei der Navigation aufgerufen
- Der Build legt dafür einen eigenen Lazy Chunk an
- Kein statischer Import der Komponente mehr
  - sonst landet sie trotzdem im Start-Bundle



nur beim ersten Aufruf geladen

loadComponent und loadChildren

```
// app.routes.ts (Übung 35)
export const routes: Routes = [
  { path: 'customer', component: CustomerListComponent, canActivate: [authGuard] },
  {
    path: 'customer/:id',
    loadComponent: () ⇒ import('./customer/customer-details/customer-details.component')
      .then(m ⇒ m.CustomerDetailsComponent),
    canActivate: [authGuard]
  },
  {
    path: 'admin',                    // Beispiel: ganzer Bereich
    loadChildren: () ⇒ import('./admin/admin.routes').then(m ⇒ m.ADMIN_ROUTES)
  },
];

// admin.routes.ts
export const ADMIN_ROUTES: Routes = [{ path: '', component: AdminHomeComponent }];
```

@defer: Teile einer Seite später laden

- Inhalt des @defer-Blocks kommt in einen eigenen Chunk
- Voraussetzung: Standalone, nicht anderswo in der Datei benutzt
- Trigger (on ...) bestimmen, wann geladen und angezeigt wird
  - on idle (Standard), on viewport, on interaction
  - on hover, on immediate, on timer(2s), when Bedingung
- prefetch: Code vorher laden, aber noch nicht anzeigen
- Blöcke: @placeholder, @loading, @error

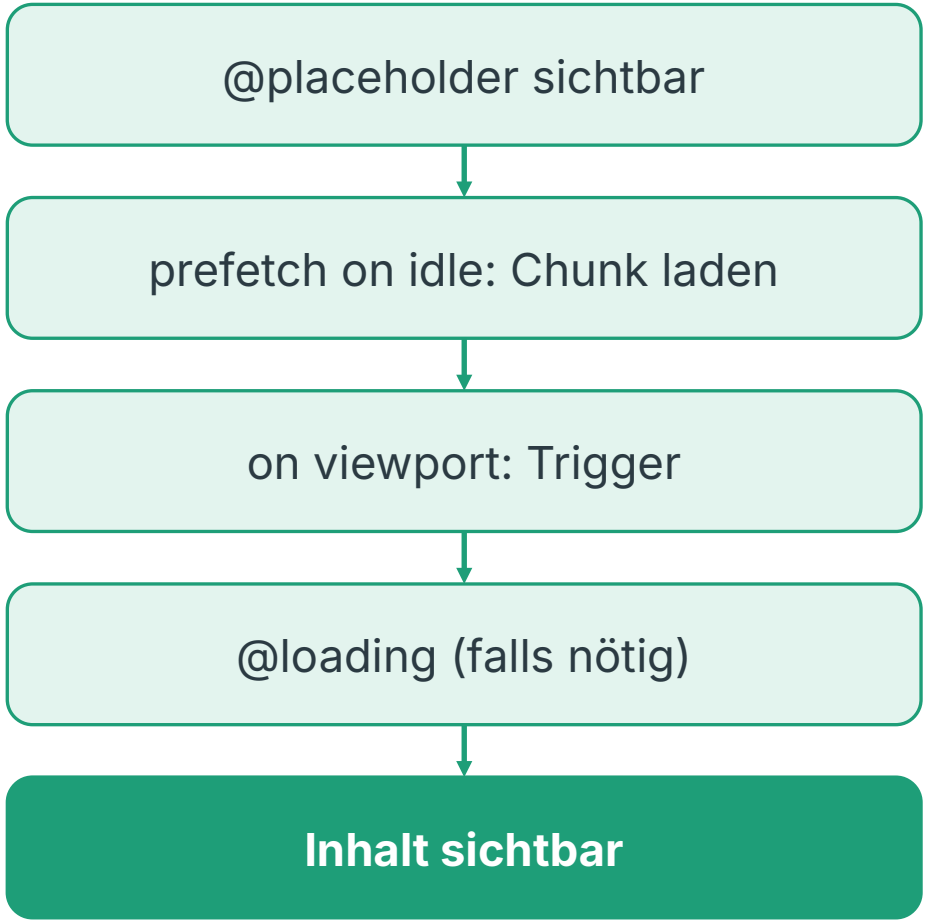
@defer mit Triggern

```
@defer (on viewport; prefetch on idle) {
  <order-history [customerId]="customer().id" />
} @placeholder (minimum 300ms) {
  <div class="box">Bestellhistorie</div>      <!-- ein Wurzelelement -->
} @loading (after 100ms; minimum 500ms) {
  <p>Lädt ...</p>
} @error {
  <p>Die Bestellhistorie konnte nicht geladen werden.</p>
}

<!-- Weitere Beispiele -->
@defer (on interaction) { <customer-chart /> } @placeholder { <button>Diagramm</button> }
@defer (when showDetails()) { <customer-details /> }
```

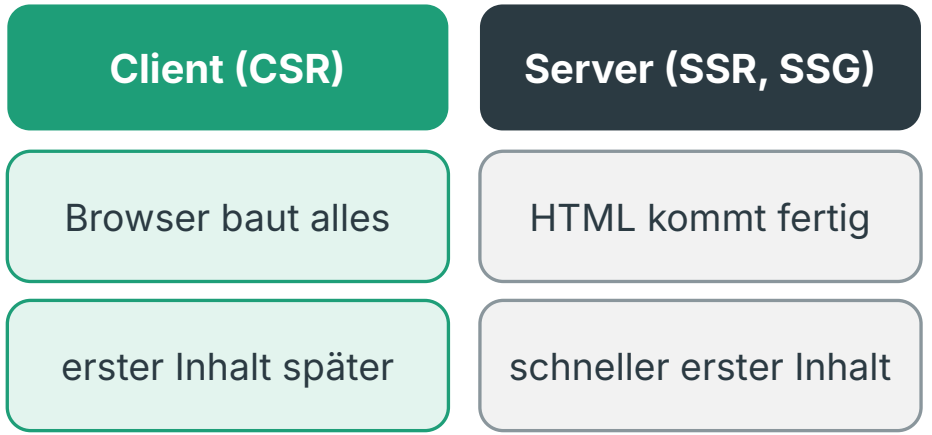
@defer: der Ablauf

- Seite lädt: @placeholder wird angezeigt
- prefetch-Trigger: Chunk wird im Hintergrund geladen
- Haupt-Trigger: Anzeige wird umgeschaltet
  - noch nicht geladen: @loading, sonst sofort Inhalt
- Fehler beim Laden: @error
- Danach bleibt der Inhalt dauerhaft geladen



Rendering-Strategien im Überblick

- CSR (Client-Side Rendering): Browser baut alles
  - einfach, aber erster Inhalt kommt später
- SSR mit Hydration: Server liefert fertiges HTML
  - Browser macht es danach bedienbar
- Prerendering (SSG): HTML entsteht beim Build
- Interne Fachanwendungen: oft reicht CSR
- Öffentliche Seiten: SSR prüfen (Folie „SSR & Hydration“)



Typische Ursachen für Laufzeitprobleme

- 1. Methodenaufrufe im Template mit teurer Berechnung
- 2. @for ohne geeigneten track-Ausdruck
- 3. Sehr große Listen ohne Virtualisierung oder Paginierung
- 4. Grobes Objekt-Signal, das alles ungültig macht
- 5. Zu viele Effects oder Effects, die Signals schreiben
- 6. Nicht abgemeldete Subscriptions, Timer und Event-Listener (Speicherlecks)
- 7. Zu große Bundles oder zu viele Bibliotheken beim Start
- 8. Ungeeignete Bildgrößen und -formate
- 9. Zu häufige HTTP-Aufrufe ohne Debounce oder Cache

Vorbereitung auf Skalierung

- Messbare Budgets (Bundle, LCP) in der Pipeline festlegen
- Performance-Tests bei Releases, z. B. mit Lighthouse CI
- Frühere Entscheidungen zahlen ein: kleine Komponenten, feine Signals, Lazy Loading
- OnPush und track mit ID von Anfang an als Standard
- Wiederholbar messen und Ergebnisse dokumentieren

Übung 35: Langsame Liste (1/2)

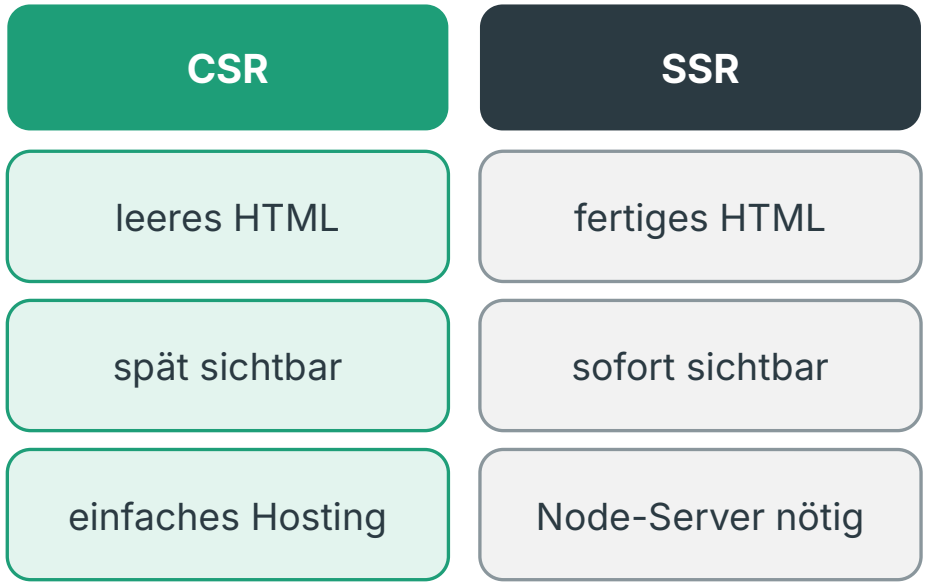
- **Öffne das Projekt 35_performance (baut auf 34_tests auf, npm install @angular/cdk)**
- **1. Last erzeugen: Lastgenerator im CustomerStore ergänzen (Code in Aufgabe.md)**
  - InjectionToken LOAD_TEST_SIZE mit factory: () ⇒ 5000 (0 = aus)
  - in load(): geladene Kunden plus createLoadTestCustomers() setzen
  - Tests mit genauen Listen: { provide: LOAD_TEST_SIZE, useValue: 0 }
- **2. Messen: Chrome DevTools (Performance) bei Eingabe im Filter**
  - und Angular DevTools Profiler: Dauer, DOM-Knoten, neu gerenderte Komponenten
- Werte als Ausgangslage in die Ergebnis-Tabelle eintragen
- **Musterlösung: 35_performance (baut auf 34_tests auf)**

Übung 35: Langsame Liste (2/2)

- **3. Optimieren, immer ein Schritt, danach neu messen:**
  - a) track customer zu track customer.id
  - b) OnPush für CustomerListItemComponent und CustomerListComponent
  - c) Debounce für den Filter (250 ms): typedFilter + toObservable + debounceTime
  - d) Virtual Scrolling mit cdk-virtual-scroll-viewport (itemSize 32)
  - e) optional: CustomerDetailsComponent als Lazy Route (loadComponent)
- **4. Tabelle füllen: Maßnahme, Messwert vorher, Messwert nachher, Fazit**
- **5. Diskussion: Was brachte am meisten? Was war unnötig?**
- **Musterlösung: 35_performance (baut auf 34_tests auf)**

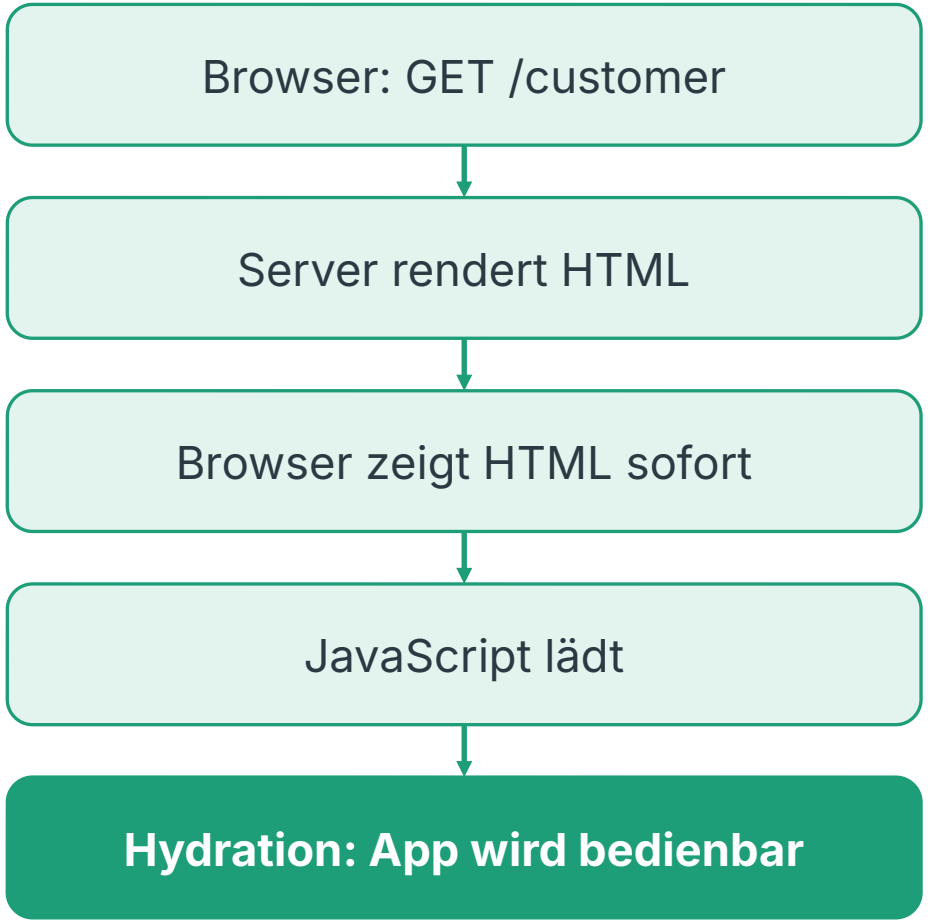
SSR: Warum Server-Side Rendering?

- Normale Single-Page-App (CSR): Server schickt fast leeres HTML
  - Inhalt erst sichtbar, wenn JavaScript geladen und gestartet ist
- SSR: Server rendert die Seite pro Anfrage zu fertigem HTML
  - Inhalt sofort sichtbar, gut für Suchmaschinen und Vorschauen
- Prerendering (SSG): HTML schon beim Build erzeugen
- Einrichten: ng new --ssr oder ng add @angular/ssr



Ablauf: Vom Server-HTML zur lebendigen App

- 1. Browser fordert eine Adresse an
- 2. Server führt Angular aus und rendert HTML
- 3. Browser zeigt das HTML sofort an – noch nicht bedienbar
- 4. JavaScript lädt im Hintergrund
- 5. Hydration: Angular übernimmt das vorhandene DOM
- Danach verhält sich die Seite wie eine normale Angular-App



Hydration und Event Replay

- provideClientHydration() in app.config.ts aktiviert Hydration
- Angular nutzt das Server-DOM weiter, statt es neu zu bauen
- withEventReplay(): Klicks vor der Hydration werden gemerkt
  - und danach automatisch nachgespielt
- HTTP-Antworten vom Server werden mitgeschickt (Transfer Cache)
- Server- und Browser-DOM müssen gleich sein (sonst NG0500)

app.config.ts

```
export const appConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(
      withFetch()),
    provideClientHydration(
      withEventReplay())
  ]
};
```

Incremental Hydration mit @defer

- Bereiche erst bei Bedarf hydrieren statt alles auf einmal
  - Server rendert den Block trotzdem sofort als HTML
  - JavaScript des Blocks lädt erst beim Auslöser
- Auslöser: on viewport, interaction, idle, hover, timer
- hydrate never: bleibt dauerhaft statisches HTML
- Aktivieren: withIncrementalHydration()

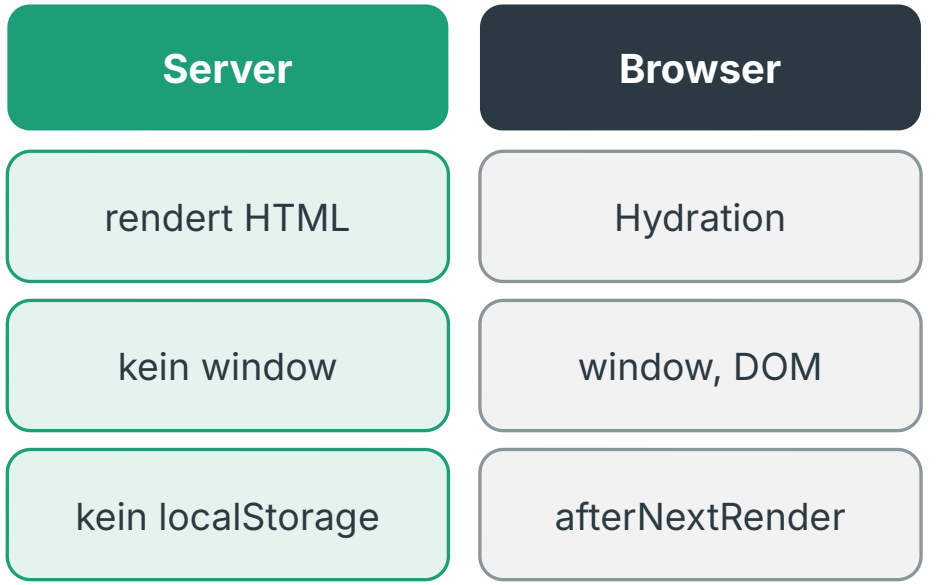
Template

```
<app-customer-header />

@defer (hydrate on viewport) {
  <app-order-history />
} @placeholder {
  <p>Bestellungen</p>
}
```

Stolperfalle: Code läuft auch auf dem Server

- Komponenten, Services, Konstruktoren laufen auf dem Server
- Dort gibt es kein window, localStorage, navigator
  - Zugriff wirft ReferenceError – Seite rendert nicht
- Kein direkter DOM-Zugriff: Bindungen, host, Renderer2
- setInterval / endlose Observables halten den Server auf
- Browser-Code in afterNextRender() oder mit isPlatformBrowser



afterNextRender: nur im Browser

```
// afterNextRender, PLATFORM_ID: @angular/core
// isPlatformBrowser: @angular/common
export class CustomerListComponent {
  private readonly platformId = inject(PLATFORM_ID);
  filterText = signal('');

  constructor() {
    // nur im Browser, einmal nach dem ersten Rendern
    afterNextRender(() => {
      this.filterText.set(localStorage.getItem('filter') ?? '');
    });
  }
  saveFilter(): void {
    if (isPlatformBrowser(this.platformId)) {
      localStorage.setItem('filter', this.filterText());
    }
  }
}
```

SSR: Render-Modi und Einordnung

- Render-Modus pro Route in app.routes.server.ts:
  - RenderMode.Server: bei jeder Anfrage auf dem Server
  - RenderMode.Prerender: einmal beim Build
  - RenderMode.Client: nur im Browser wie ohne SSR
- Lohnt sich: öffentliche Seiten, Suchmaschinen, langsame Geräte
- Oft unnötig: interne Anwendungen hinter einem Login
- Test: Seite mit abgeschaltetem JavaScript aufrufen

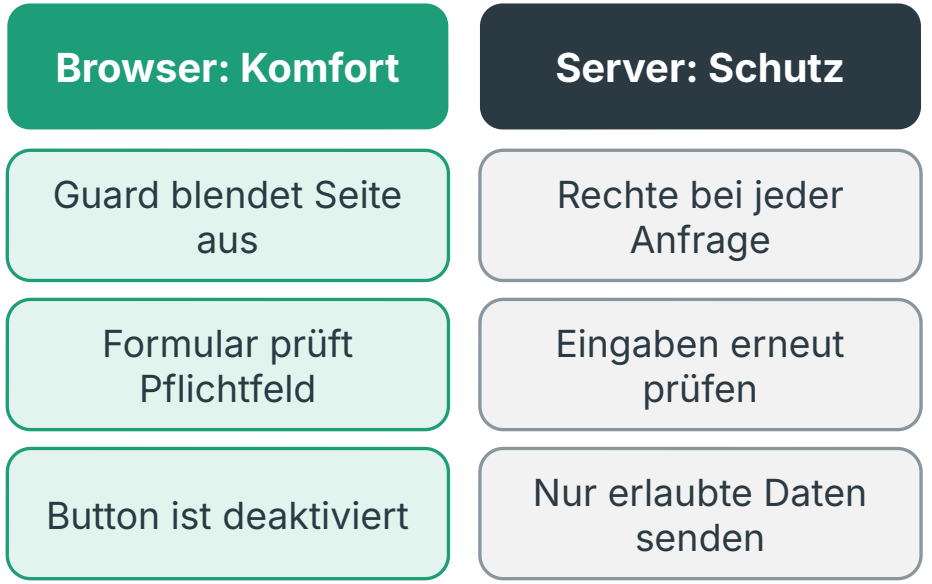
app.routes.server.ts

```
export const serverRoutes:
    ServerRoute[] = [
  { path: '',
    renderMode:
      RenderMode.Prerender },
  { path: 'customer/**',
    renderMode:
      RenderMode.Client },
  { path: '**',
    renderMode:
      RenderMode.Server }
];
```

Sicherheit im Frontend

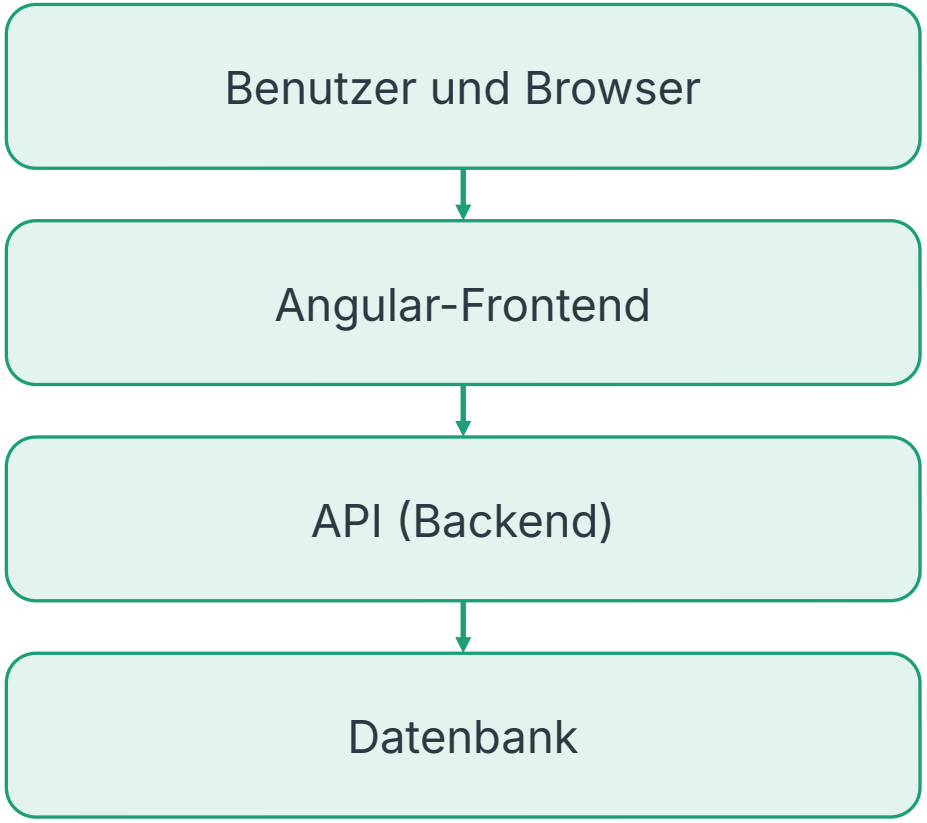
Grundsatz: Frontend ist nicht vertrauenswürdig

- Der Code läuft im Browser des Nutzers
  - Er kann ihn lesen, ändern, umgehen (Entwicklertools)
- Keine Geheimnisse im Frontend: Passwörter, Schlüssel
- Berechtigungen setzt das Backend durch
  - Guards (Übung 25) verbessern nur die Bedienung
- Validierung im Client: Komfort
- Validierung im Server: Sicherheit



Vertrauensgrenzen

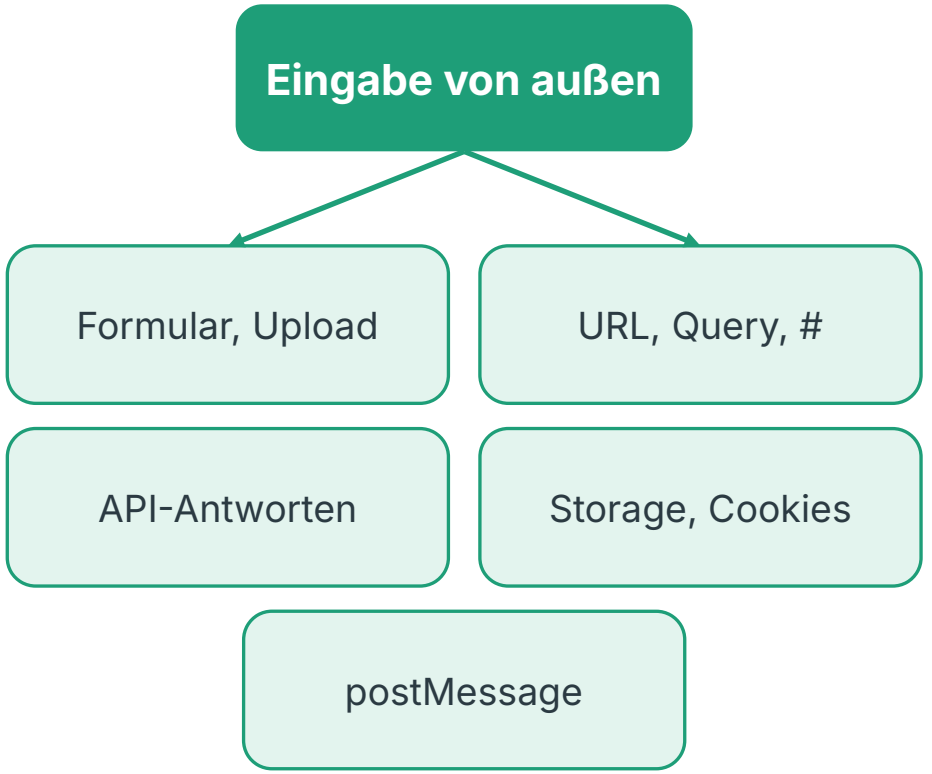
- Vertrauensgrenze: Stelle, an der Daten den Bereich wechseln
- Benutzer und Browser: alles ist manipulierbar
- Frontend: zeigt Daten an, muss sie richtig kodieren
- API: prüft Anmeldung, Rechte und Eingaben
- Datenbank: nur über die API erreichbar
- An jeder Grenze gilt: Daten von außen sind Eingabe



An jeder Grenze prüfen

Was alles „Eingabe“ ist

- Formularfelder und Dateiuploads
- URL: Pfad, Query-Parameter, Fragment nach #
- Antworten von APIs, auch vom eigenen Backend
- localStorage, sessionStorage, Cookies
- Nachrichten anderer Fenster (postMessage)
- Vertrauenswürdig: nur eigener Quelltext und Konstanten
- Angezeigte Daten je nach Kontext schützen: HTML, URL



XSS: das Hauptrisiko

- Cross-Site Scripting (XSS): fremdes Script läuft in der Seite
- Es hat die Rechte des angemeldeten Nutzers
- Gespeichert: Script steht in der Datenbank (Notiz)
- Reflektiert: Script steht im Link (Query-Parameter)
- DOM-basiert: eigener Code setzt Eingabe als HTML ein
- Ursache immer: Daten werden als Code behandelt

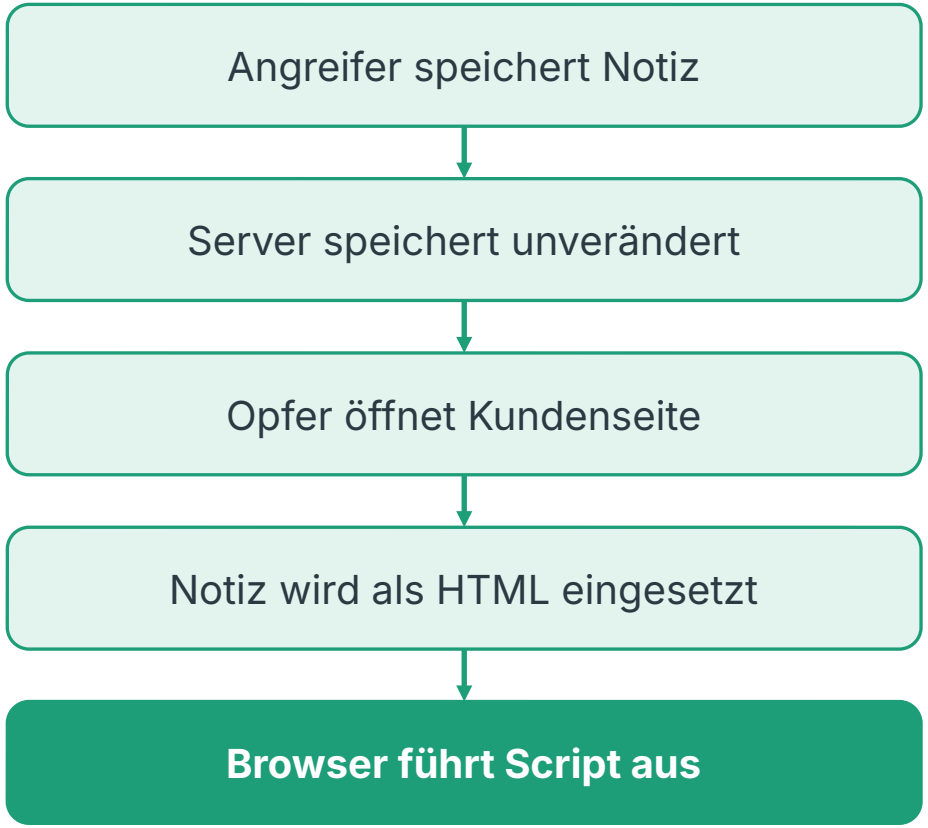
Kunden-Notiz (Eingabe)

```
<b>wichtig</b>
<img src=x
  onerror="alert('XSS')">

// src=x lädt nicht
// -> Browser ruft
//    onerror auf
// -> Script läuft
```

Ein XSS-Angriff Schritt für Schritt

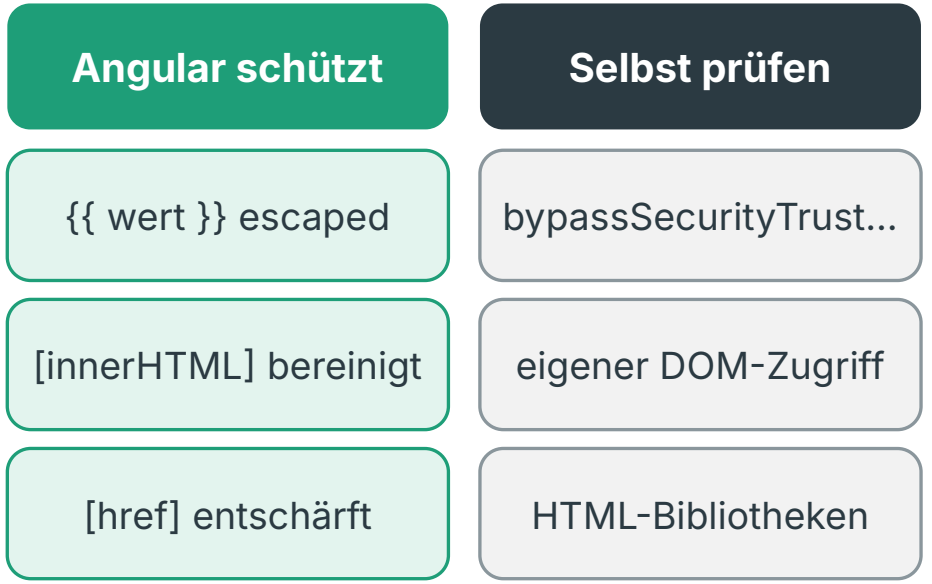
- 1. Angreifer speichert Notiz mit <img onerror=...>
- 2. Server speichert den Text unverändert
- 3. Opfer öffnet die Kundenseite
- 4. Seite setzt die Notiz ungeschützt als HTML ein
- 5. Browser führt das Script des Angreifers aus
- Folge: Token lesen, Anfragen senden, Seite fälschen



Schritt 4 verhindert Angular

Was Angular standardmäßig tut

- Interpolation {{ }}: Wert wird als Text ausgegeben
  - Zeichen wie < und > erscheinen sichtbar (escaped)
- [innerHTML]: HTML wird bereinigt (Sanitizing)
  - script, onerror, onclick werden entfernt
- [href], [src]: javascript:-URLs erhalten „unsafe:“
- Templates werden beim Build kompiliert
  - Nutzertext wird nie als Template ausgewertet



Interpolation und innerHTML im Vergleich

```
// Komponente: note = '<b>wichtig</b> <img src=x onerror="alert(\'XSS\')">'

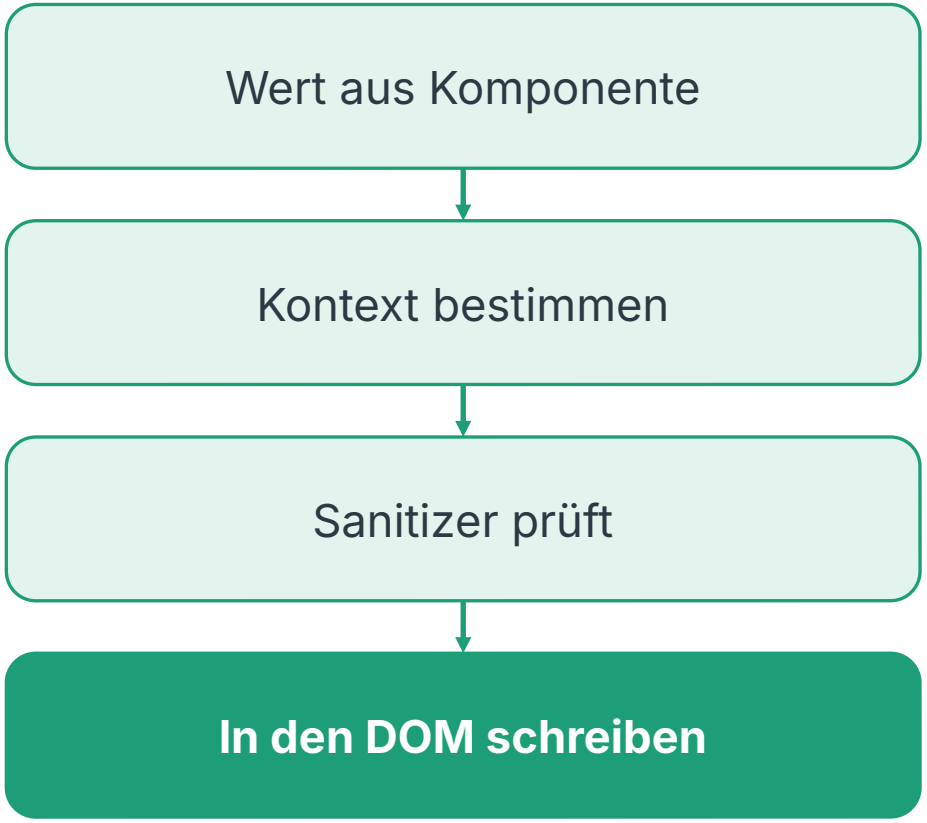
<!-- 1. Interpolation: Ausgabe als Text -->
<p>{{ customer.note }}</p>
<!-- Anzeige: alle Zeichen sichtbar, z. B. <b>wichtig</b> <img src=x onerror=... -->

<!-- 2. innerHTML: Ausgabe als HTML, vorher bereinigt -->
<div [innerHTML]="customer.note"></div>
<!-- Im DOM bleibt: <b>wichtig</b> <img src="x">  (fett, onerror entfernt, kein alert) -->

<!-- Konsole im Entwicklungsmodus:
    WARNING: sanitizing HTML stripped some content, see https://g.co/ng/security#xss -->
```

So arbeitet Angulars Sanitizer

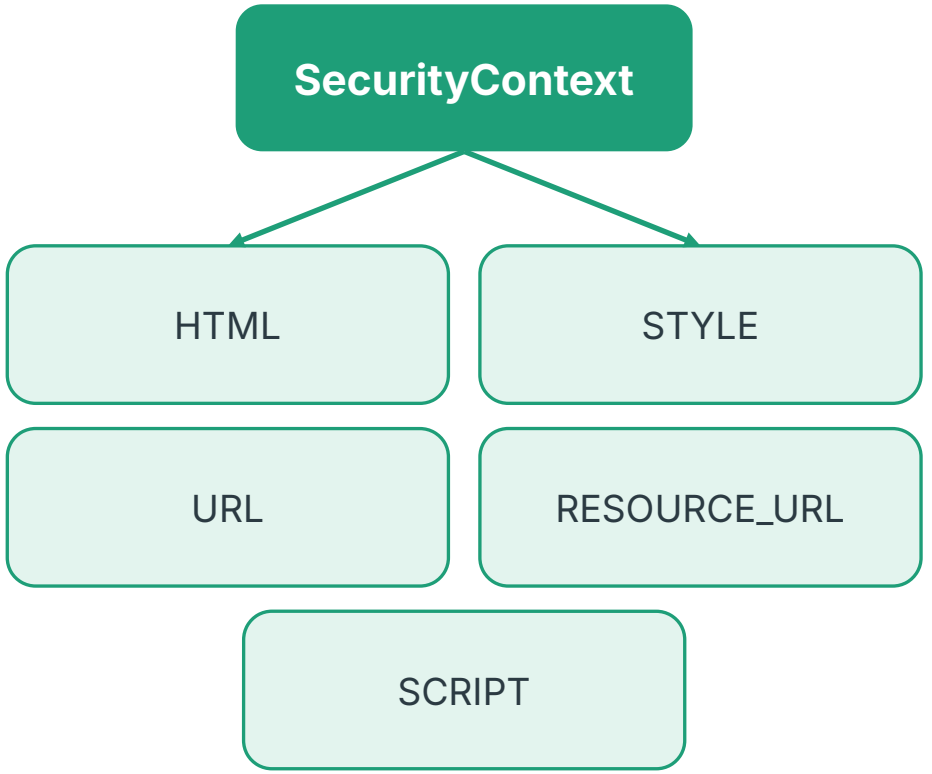
- Angular kennt für jede Bindung den Sicherheitskontext
  - Er ergibt sich aus Element und Attribut
- Der Sanitizer prüft den Wert passend zum Kontext
- Ungefährliches bleibt, Gefährliches wird entfernt
  - oder die Bindung wird mit Fehler abgelehnt
- Werte mit „SafeValue“-Markierung werden durchgelassen



Markierte SafeValues: ohne Prüfung

Die Sicherheitskontexte

- HTML: [innerHTML] – Allowlist harmloser Elemente
- Style: [style] – wird nicht verändert
  - CSS führt in heutigen Browsern kein Script aus
- URL: [href], img [src] – javascript: wird entschärft
- Resource-URL: iframe [src], script [src]
  - lädt fremden Code: Angular lehnt ab (Fehler)
- Script: Bindungen an Script-Inhalt sind nie erlaubt



URL und Resource-URL im Template

```
// Komponente
website = 'javascript:alert(1)';                // z. B. aus Kundendaten
videoUrl = 'https://www.youtube.com/embed/abc';    // Adresse für ein eingebettetes Video

<!-- URL-Kontext: Angular entschärft gefährliche Schemata -->
<a [href]="website">Webseite</a>
<!-- Ergebnis: href="unsafe:javascript:alert(1)"  Klick bewirkt nichts -->

<!-- Resource-URL-Kontext: Angular lehnt einen normalen String ab -->
<iframe [src]="videoUrl"></iframe>
<!-- Fehler: NG0904 unsafe value used in a resource URL context -->

// Lösung nur nach Prüfung gegen eine Allowlist (nächste Folien):
// sanitizer.bypassSecurityTrustResourceUrl(geprüfteUrl)
```

DomSanitizer und bypassSecurityTrust...

- DomSanitizer: Dienst von Angular, per inject() holen
- sanitize(kontext, wert): Wert selbst bereinigen
- bypassSecurityTrustHtml/Style/Url/ ResourceUrl/Script
  - Bekommt einen String, gibt einen SafeValue zurück
  - Angular prüft diesen Wert danach nicht mehr
- Der Entwickler übernimmt die volle Verantwortung

DomSanitizer

```
sanitize(ctx, value)
  : string | null

bypassSecurityTrustHtml(
  value: string): SafeHtml
bypassSecurityTrustUrl(
  value: string): SafeUrl
bypassSecurityTrustResourceUrl(
  value: string)
  : SafeResourceUrl
```

bypassSecurityTrust... im Code

```
import { Component, computed, inject, input } from '@angular/core';
import { DomSanitizer, SafeHtml } from '@angular/platform-browser';

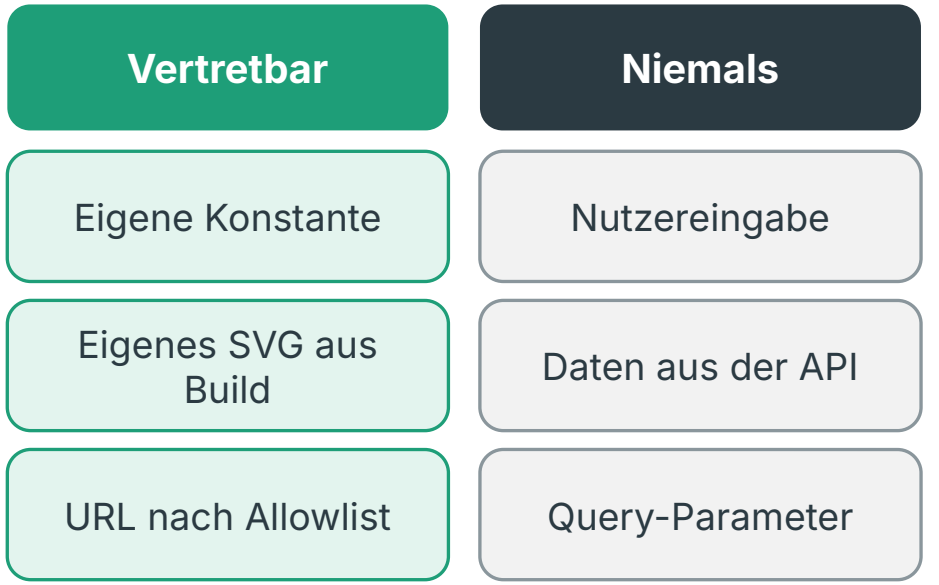
@Component({ selector: 'note-demo', template: `<div [innerHTML]="unsafeNote()"></div>` })
export class NoteDemoComponent {
  note = input<string>();                    // kommt aus dem Backend = Eingabe!
  private readonly sanitizer = inject(DomSanitizer);

  // REVIEW-ALARM: Angular prüft diesen Wert NICHT mehr. Bewusst unsicheres Gegenbeispiel.
  unsafeNote = computed<SafeHtml>(() =>
    this.sanitizer.bypassSecurityTrustHtml(this.note() ?? ''));
}

// Ergebnis mit der Notiz aus Übung 36: Das alert erscheint.
// Sichere Alternative: [innerHTML]="note()" ohne bypass, Angular bereinigt selbst.
```

bypassSecurityTrust...: Risiko einschätzen

- Vertretbar nur bei vollständig kontrollierten Inhalten
  - Konstante im Code, eigene Build-Dateien
  - URL, die vorher gegen eine Allowlist geprüft wurde
- Niemals bei Nutzereingaben, API-Daten, URL-Parametern
- Jeder Einsatz: Kommentar mit Begründung im Code
- Im Review gezielt suchen: bypassSecurityTrust

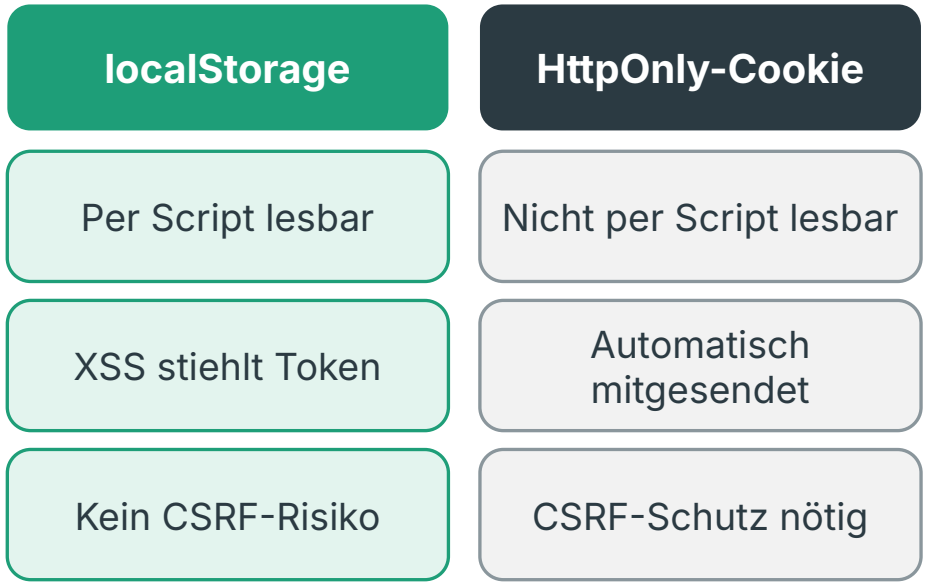


Weitere Wege am Schutz vorbei

- Direkter DOM-Zugriff ohne Sanitizer:
  - elementRef.nativeElement.innerHTML = …
  - document.write, insertAdjacentHTML
- Code aus Strings: eval, new Function, setTimeout('…')
- Templates zur Laufzeit aus Nutzertext erzeugen
- Bibliotheken, die HTML erzeugen (Markdown, Editoren)
  - Ausgabe trotzdem über [innerHTML] oder sanitize()

Wo speichere ich das Token?

- Token: Nachweis der Anmeldung bei jeder Anfrage
- localStorage: bleibt nach Reload, per Script lesbar
  - Bei XSS kann der Angreifer es stehlen
- HttpOnly-Cookie: setzt das Backend
  - Kein Script kommt heran, Browser sendet es mit
  - Dafür CSRF beachten (nächste Folien)

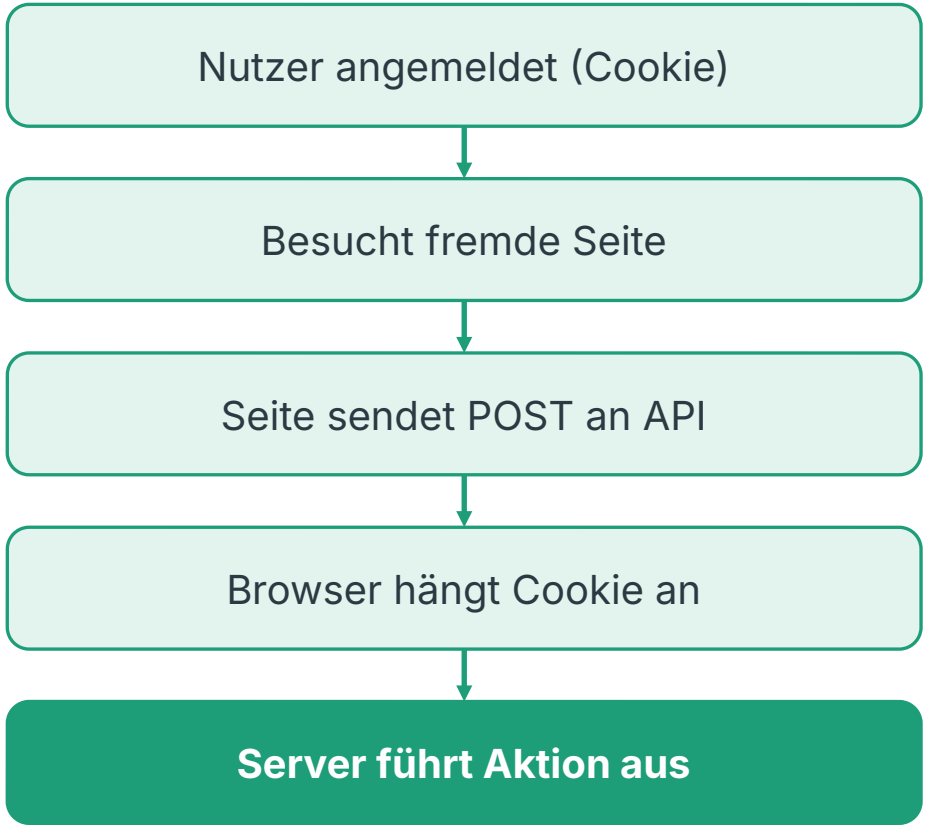


Token-Speicherung: Empfehlung

- Sitzung über Cookie mit HttpOnly, Secure, SameSite
  - Secure: nur über HTTPS; SameSite: nicht von fremden Seiten
- Cookie ohne HttpOnly: so unsicher wie localStorage
- Variable im Speicher: weg beim Reload, schwerer zu stehlen
- Tokens mit kurzer Laufzeit, Erneuerung über das Backend
- Login über erprobte Bibliothek oder Identity Provider
- AuthService aus Übung 25 (Signal isLoggedIn) ist nur Demo

CSRF: Angriff über eine fremde Seite

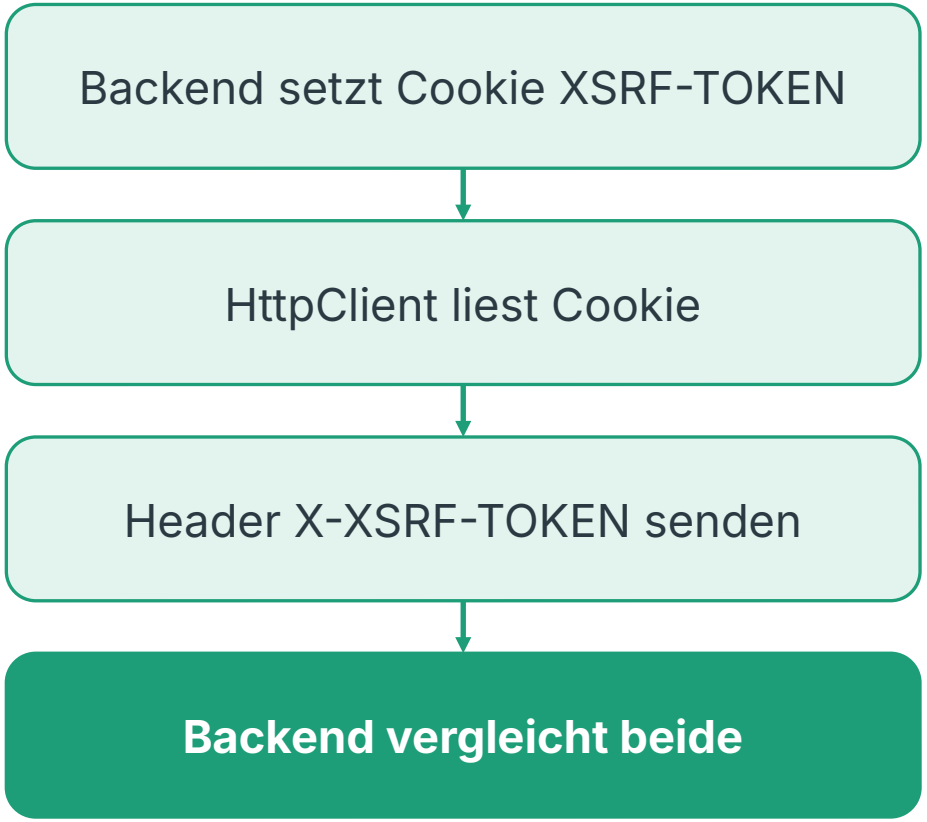
- Cross-Site Request Forgery: gefälschte Anfrage
- Nutzer ist in der App angemeldet (Cookie)
- Er besucht eine präparierte fremde Seite
- Diese schickt ein Formular an die App-API
- Browser hängt das Cookie automatisch an
- Server hält die Anfrage für echt



Cookie allein beweist nichts

Schutz: CSRF-Token (Double Submit)

- Backend setzt Cookie XSRF-TOKEN (zufälliger Wert)
- HttpClient liest das Cookie per Script
- und sendet den Wert als Header X-XSRF-TOKEN
- Backend vergleicht Cookie und Header
- Fremde Seite kann das Cookie nicht lesen
  - also kann sie den Header nicht setzen
- Zusätzlich: SameSite-Cookies



Fremde Seite kennt den Wert nicht

withXsrfConfiguration

```
// app.config.ts
import { provideHttpClient, withInterceptors, withXsrfConfiguration }
  from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([loggingInterceptor]),
      // Ohne diese Zeile gilt: Cookie "XSRF-TOKEN", Header "X-XSRF-TOKEN"
      withXsrfConfiguration({
        cookieName: 'MY-XSRF-COOKIE',   // Name des Cookies, das das Backend setzt
        headerName: 'X-MY-XSRF',        // Name des Headers, den das Backend erwartet
      }),
    ),
  ],
};

// Abschalten (nur wenn das Backend anders schützt): withNoXsrfProtection()
```

Was der HttpClient genau tut

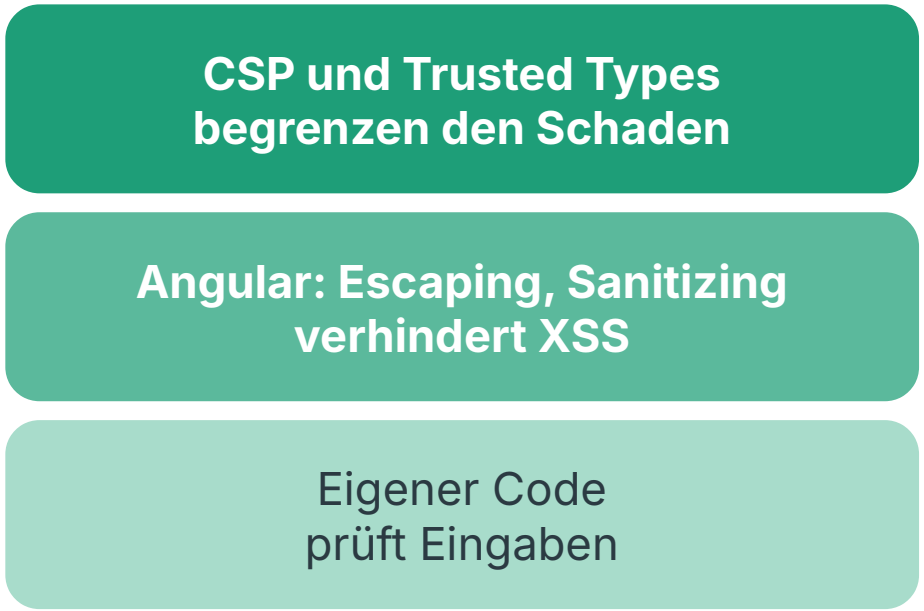
- Header nur bei verändernden Anfragen
  - POST, PUT, PATCH, DELETE – nicht bei GET und HEAD
- Nur bei relativen URLs wie /api/customer
  - Kein Token an fremde Server (absolute URL)
- Kein Cookie vorhanden: kein Header
- Prüfen muss immer das Backend
- GET-Anfragen dürfen deshalb nie etwas verändern

CORS, HTTPS und URLs

- CORS schützt nicht den Server
  - Es regelt, welche Seiten Antworten lesen dürfen
  - Anfragen kommen trotzdem beim Server an
- Immer HTTPS, auch in internen Netzen
- Keine sensiblen Daten in URLs
  - Sie landen in Logs, Verlauf und Referrer-Header
- Werte in URLs kodieren: encodeURIComponent(id)

Content Security Policy (CSP)

- HTTP-Header, den der Server mit der Seite sendet
- Legt fest, woher Skripte, Styles, Bilder kommen dürfen
- Browser blockiert alles andere, auch eingeschleuste Skripte
- Begrenzt den Schaden, wenn XSS doch gelingt
- Zusätzliche Schicht (Defense in Depth)
  - ersetzt kein sauberes Escaping



Mehrere Schutzschichten

Beispiel: ein CSP-Header

```
Content-Security-Policy:
  default-src 'self';                      # Standard: nur eigene Origin
  script-src  'self' 'nonce-r4nd0m';       # Skripte: eigene Dateien + passende Nonce
  style-src   'self' 'nonce-r4nd0m';       # Styles: Angular erzeugt <style>-Elemente
  img-src     'self' data:;                # Bilder: eigene und eingebettete (data:)
  connect-src 'self' https://api.example.com;  # Ziele für HttpClient/fetch
  object-src  'none';                      # keine Plugins (object, embed)
  base-uri    'self';                      # <base> darf nicht umgebogen werden
  frame-ancestors 'none';                  # Seite darf in keinem iframe laufen

# Die Nonce ist ein Zufallswert, den der Server bei JEDER Auslieferung neu erzeugt.
# Kommentare mit # dienen nur der Erklärung; ein echter Header steht in einer Zeile.
```

CSP mit Angular nutzen

- Angular schreibt Komponenten-Styles als <style> in die Seite
  - Diese brauchen die Nonce, sonst blockiert die CSP
- Nonce angeben: Attribut ngCspNonce am Root-Element
  - oder Provider für das Token CSP_NONCE
- Server ersetzt den Platzhalter bei jeder Auslieferung
- angular.json: security.autoCsp erzeugt Hash-CSP
- Erst im Report-Only-Modus testen, dann scharf schalten

Nonce in index.html und app.config.ts

```
<!-- index.html: der Server ersetzt RANDOM_NONCE bei jeder Auslieferung -->
<body>
  <app-root ngCspNonce="RANDOM_NONCE"></app-root>
</body>

// Alternative: app.config.ts (Wert z. B. aus einem Meta-Tag, das der Server setzt)
import { CSP_NONCE } from '@angular/core';

const nonce = document.querySelector('meta[name="csp-nonce"]')?.getAttribute('content');

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: CSP_NONCE, useValue: nonce },
  ],
};

// Server-Header dazu: script-src 'self' 'nonce-<derselbe Wert>'; style-src ...
```

Trusted Types

- Browser-Funktion, die die CSP aktiviert
- Gefährliche DOM-Stellen nehmen keine Strings mehr an
  - z. B. innerHTML, script.src, eval
- Nur Werte aus einer erlaubten Policy werden akzeptiert
- Angular bringt eigene Policies mit
  - angular, angular#unsafe-bypass (für bypass...)
- Fehlt angular#unsafe-bypass, scheitert jedes bypass

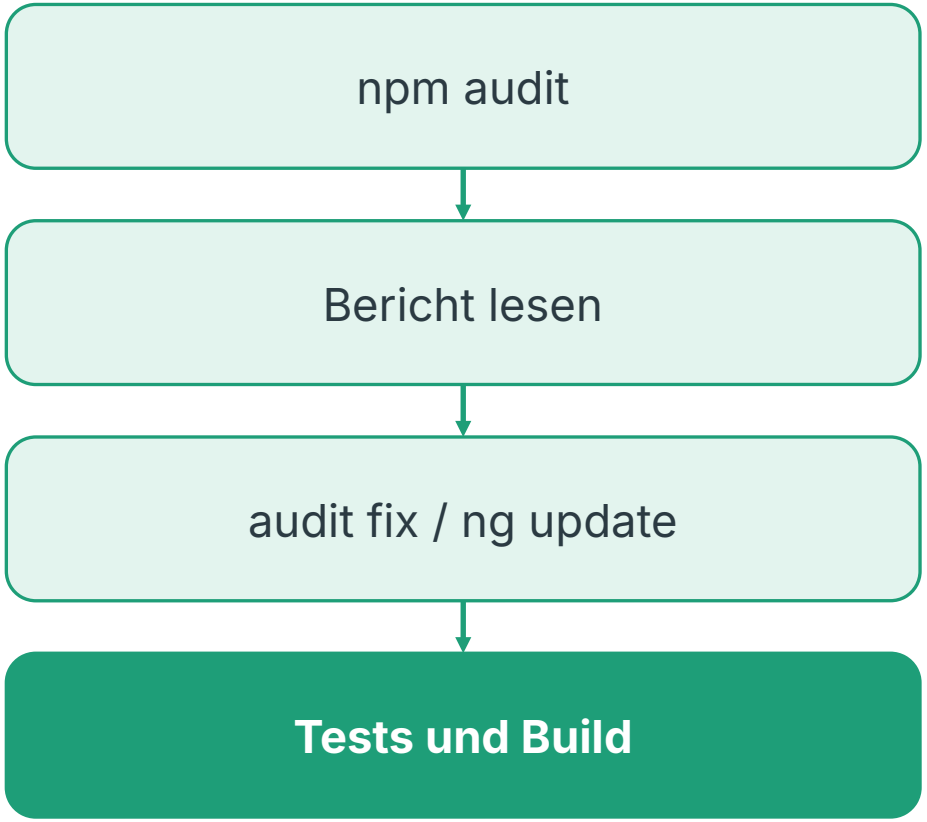
CSP-Header

```
Content-Security-Policy:
  trusted-types angular;
  require-trusted-types-for
    'script';

# mit bypass-Ausnahme:
  trusted-types angular
    angular#unsafe-bypass;
```

Abhängigkeiten: die Lieferkette

- Jedes npm-Paket läuft mit vollen Rechten in der App
- npm audit: vergleicht Lockfile mit bekannten Lücken
- npm audit fix: hebt auf sichere Versionen an
- ng update: Angular und seine Pakete aktualisieren
- package-lock.json committen, im CI mit npm ci bauen
- Neue Pakete prüfen: Pflege, Verbreitung, echter Name



Regelmäßig, z. B. im CI

npm audit in der Praxis

```
npm audit                    # alle Pakete prüfen, Bericht nach Schweregrad
npm audit --omit=dev         # nur Pakete, die in die ausgelieferte App gelangen
npm audit --audit-level=high   # Exit-Code ungleich 0 erst ab "high" (für das CI)
npm audit fix                # sichere Updates ohne neue Hauptversion

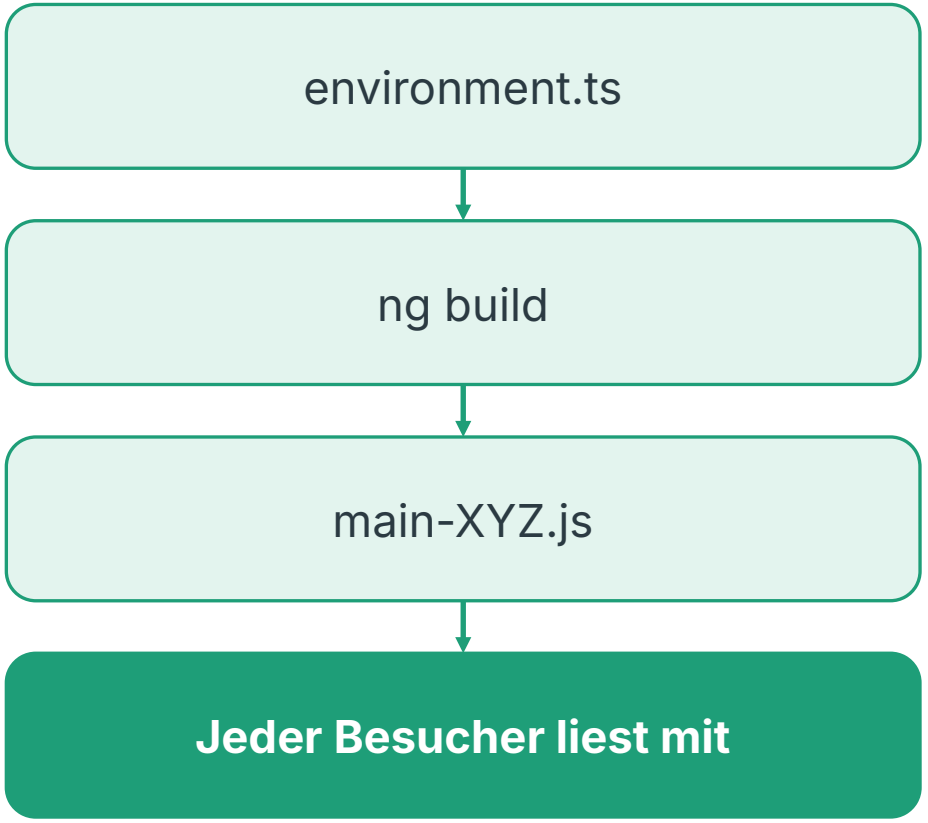
ng update                    # zeigt verfügbare Angular-Updates
ng update @angular/core @angular/cli   # aktualisiert und passt Code per Migration an

npm ci                       # installiert exakt nach package-lock.json (CI)

# Beispiel-Ausgabe (gekürzt):
# some-package  <2.4.1
# Severity: high
# Prototype Pollution - https://github.com/advisories/GHSA-...
# fix available via `npm audit fix`
```

Secrets im Frontend

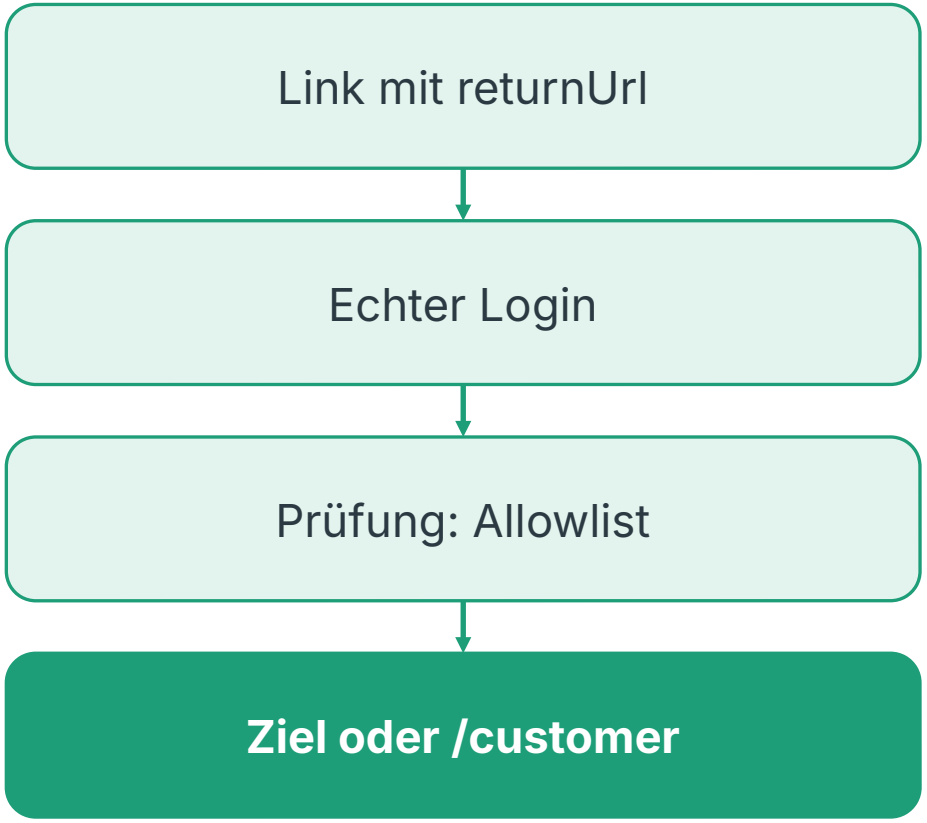
- environment.ts wird in das JavaScript-Bundle kompiliert
- Jeder kann das Bundle laden und lesen
  - Entwicklertools, Quelltext, Netzwerk-Tab
- Also: kein API-Key mit Rechten, kein Passwort im Client
- Lösung: Backend ruft den fremden Dienst auf
- Öffentliche Schlüssel (z. B. Karten): nur mit Domain-Sperre



Alles im Bundle ist öffentlich

Offene Weiterleitung: returnUrl

- Guard hängt Ziel an:
  /login?returnUrl=/customer/7
- Nach dem Login navigiert die App dorthin
- Angreifer verschickt:
  /login?returnUrl=//evil.example
  - Opfer meldet sich echt an und landet beim Angreifer
- Gegenmaßnahme: Ziel gegen Allowlist prüfen
  - nur eigene Origin, nur erlaubte Pfade



Wird in Übung 36 gebaut

safeReturnUrl: Prüfung gegen eine Allowlist

```
const ALLOWED_PREFIXES = ['/customer'];

export function safeReturnUrl(value: string | null | undefined,
    origin: string = window.location.origin): string | undefined {
  if (!value) return undefined;
  let url: URL;
  try {
    url = new URL(value, origin);    // relativ: gegen eigene Origin aufgelöst
  } catch {
    return undefined;                // kein gültiger Wert
  }
  if (url.origin !== origin) return undefined;  // //evil.example, javascript:
  const allowed = ALLOWED_PREFIXES.some(p ⇒
    url.pathname === p || url.pathname.startsWith(p + '/'));   // Pfad ist normalisiert
  return allowed ? url.pathname + url.search : undefined;
}
// Login: router.navigateByUrl(safeReturnUrl(this.returnUrl()) ?? '/customer')
```

Weitere typische Risikoquellen

- Zu viele Daten im Client: Backend liefert nur erlaubte Felder
- Sensible Daten in Logs: keine Header, Bodies, Tokens
  - loggingInterceptor (Übung 25): nur Methode, URL, Status
- Clickjacking: Seite in fremdem iframe, unsichtbar überlagert
  - Schutz: frame-ancestors oder X-Frame-Options
- Fremde Skripte (Tracking, Widgets): wirklich nötig?
  - Subresource Integrity (integrity-Attribut) und CSP

Umgang mit Daten und Benutzerinteraktionen

- Datensparsamkeit: nur anzeigen und speichern, was nötig ist (DSGVO)
- Kontextgerecht ausgeben: Text per {{ }}, HTML per Sanitizing, URL validieren
- Client-Validierung im Backend wiederholen
- Dateiuploads: Typ und Größe prüfen, Inhalte nie ausführen
- Fehlermeldungen ohne technische Details, keine Stacktraces für Endnutzer
- Nach dem Logout: Zustand im Speicher (Stores, Signals) zurücksetzen

Verantwortungsvoller Einsatz im Team

- Sicherheit ist Teamaufgabe: Review-Checkliste für jede Änderung
- Automatische Prüfungen im CI: Lint, Tests, npm audit
- Updates zeitnah mit ng update: Sicherheitsfixes in Minor- und Patch-Versionen
- Dokumentierte Ausnahmen: Wo steht bypassSecurityTrust* oder innerHTML, und warum?
- Vorfälle offen und lernorientiert besprechen, ohne Schuldzuweisung

Review-Checkliste Frontend-Sicherheit

- Wird bypassSecurityTrust* verwendet? Ist es begründet?
- Gibt es innerHTML, nativeElement.innerHTML, document.write oder eval?
- Werden URL-Parameter ungeprüft für Navigation oder Requests genutzt?
- Werden Tokens oder personenbezogene Daten geloggt oder im localStorage abgelegt?
- Stehen Geheimnisse in environment-Dateien oder im Bundle?
- Verlässt sich eine Funktion nur auf einen Guard statt auf das Backend?
- Wurden neue Abhängigkeiten geprüft (npm audit, Pflegezustand)?

Übung 36: XSS erleben und beheben (1/2)

- **Öffne das Projekt 36_sicherheit; Lastgenerator aus Übung 35 abschalten (LOAD_TEST_SIZE = 0)**
- **1. Customer erhält note?: string; der CustomerService setzt für Kunde 1 die Notiz**
  - wichtig <img src=x onerror="alert('XSS')"> (Konstante, nur für die Übung)
- **2. In CustomerDetailsComponent per Interpolation {{ c.note }} ausgeben: Was siehst du?**
- **3. Per [innerHTML] ausgeben und in den Entwicklertools (Elements) prüfen, was bleibt**
- **4. Gegenbeispiel mit bypassSecurityTrustHtml: Das alert erscheint**
  - Nur in einer getrennten Demo-Komponente hinter einem Schalter; wann wäre es akzeptabel?
- **Musterlösung: 36_sicherheit (baut auf 35_performance auf)**

Übung 36: XSS erleben und beheben (2/2)

- **5. Login: returnUrl gegen eine Allowlist prüfen**
  - Nur Pfade /customer oder /customer/…; absolute URLs und fremde Hosts verwerfen
  - Tipp: new URL(value, window.location.origin), dann origin und pathname prüfen
- **6. loggingInterceptor darf Header wie Authorization nie ausgeben (Test oder Kommentar)**
- **7. Wo fließt ein Wert aus der Adresszeile in eine Request-URL? encodeURIComponent nutzen**
- **8. Review-Checkliste (material/review-checkliste.md) für die ganze App durchgehen**
  - Funde in der Tabelle dokumentieren
- **Musterlösung: 36_sicherheit (baut auf 35_performance auf)**

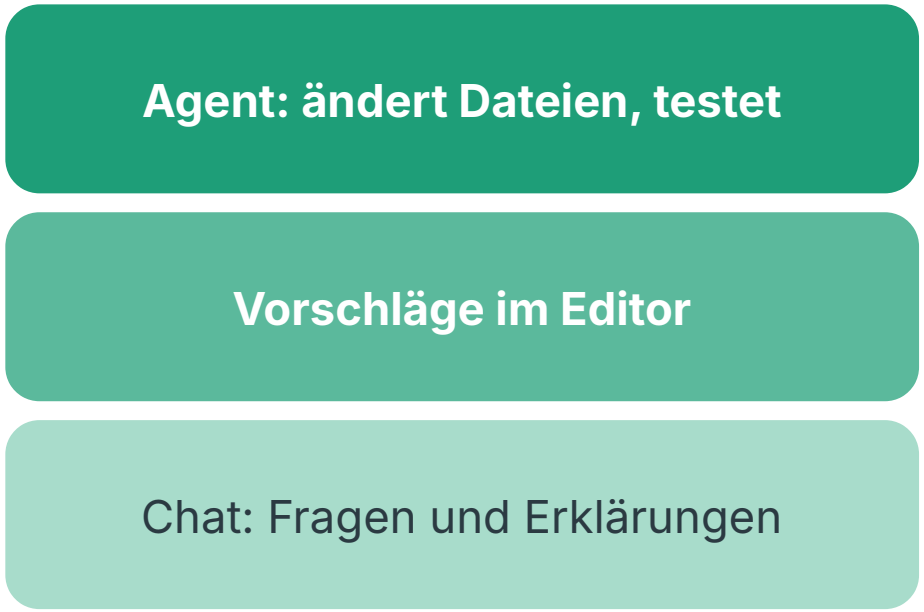
KI-gestützte Entwicklung

Wofür KI in der Entwicklung?

- Sehr gut geeignet: Gerüste, Standardcode, Umbenennungen
- Gut geeignet: fremden Code und Fehlermeldungen erklären lassen
- Gut, aber mit Review: Tests vorschlagen, Randfälle sammeln
- Gut, mit Tests: Umbauten in kleinen Schritten
- Nur als Diskussionspartner: Architektur, Fachlogik, Sicherheit
- Riskant: große Änderungen blind übernehmen
- KI ist ein schneller Assistent ohne Verantwortung: Die Verantwortung bleibt beim Entwickler

Formen der Unterstützung

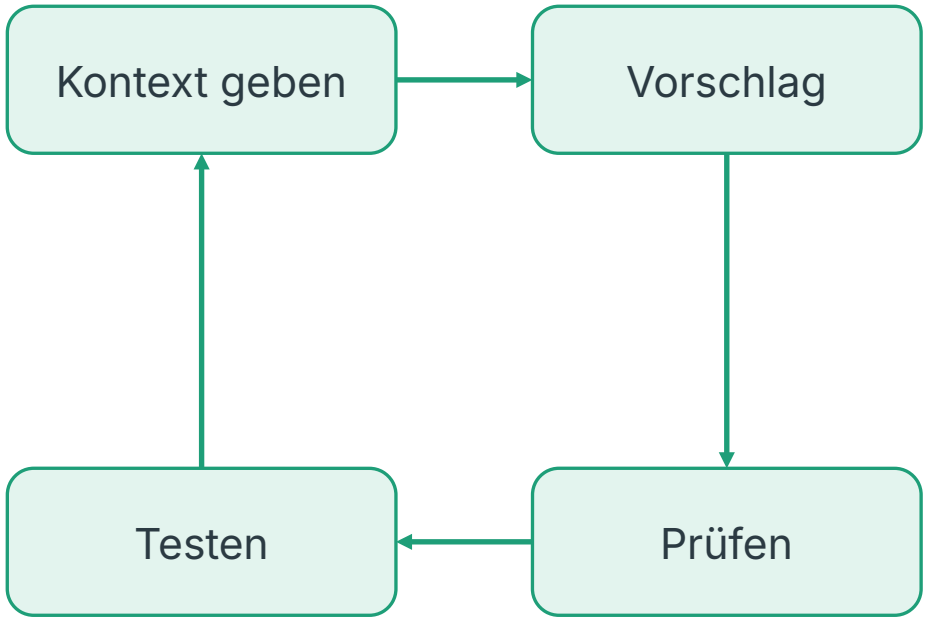
- Chat: Fragen stellen, Code erklären lassen
- Vorschläge im Editor: Zeilen oder Blöcke ergänzen
- Agenten: lesen das Projekt, ändern mehrere Dateien
  - führen Befehle aus, z. B. ng build, ng test
- Je selbstständiger das Werkzeug,
  - desto größer Prüfaufwand und möglicher Schaden



Mehr Selbstständigkeit = mehr Prüfaufwand

Arbeitsablauf mit KI: ein Kreislauf

- Kontext geben: Ziel, Dateien, Regeln, Beispiele
- Vorschlag: Plan oder Code der KI
- Prüfen: lesen, mit Checkliste vergleichen
- Testen: Build, Lint, Tests, im Browser ausprobieren
- Befunde als neuen Kontext zurückgeben
- Erst wenn alles passt: Commit

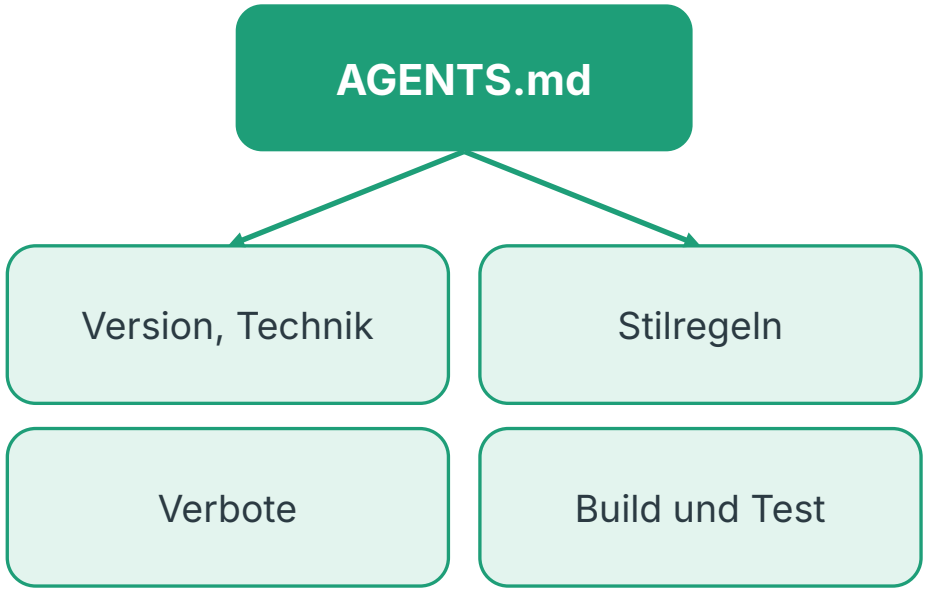


Kontext ist alles

- Aufgabe klar und klein:
  - „Erstelle CustomerFilter mit input()/output()“
  - nicht: „Baue die App um“
- Betroffene Dateien nennen oder anhängen
- Beispiele zeigen: „analog zu CustomerListComponent“
- Projektregeln mitgeben (nächste Folie)
- Version nennen: „Angular 21, zoneless, Vitest“

Projektregeln als Datei: AGENTS.md, CLAUDE.md

- Textdatei im Repository, die das Werkzeug automatisch liest
- Name je nach Werkzeug, z. B.:
  - AGENTS.md (werkzeugübergreifend), CLAUDE.md
  - .github/copilot-instructions.md
- Inhalt: Version, Stil, Verbote, Befehle für Build und Test
- Kurz und konkret, mit dem Code mitpflegen
- Wirkt wie eine Einarbeitung für jede neue Sitzung



Beispiel: Projektregeln für Angular

Projektregeln (AGENTS.md)

Technik
- Angular 21, TypeScript strict, zoneless, ChangeDetectionStrategy.OnPush.
- Standalone-Komponenten, kein NgModule. inject() statt Konstruktor-Injection.
- Control Flow @if/@for (mit track auf ID), kein *ngIf/*ngFor, kein CommonModule.
- input()/output()/model() statt @Input/@Output.
- Zustand: signal/computed; effect nur für Nebenwirkungen außerhalb von Angular.

Verbote
- Kein any. Kein innerHTML mit zusammengebautem HTML.
- Kein bypassSecurityTrust* ohne Rückfrage.
- Keine neuen npm-Pakete ohne Rückfrage.

Befehle
- Build: ng build | Tests (Vitest): ng test | Lint: ng lint
- Neue Logik immer mit Tests, die Verhalten prüfen (nicht nur toBeTruthy).

Angular-Hilfen für KI-Werkzeuge

- angular.dev/ai: offizielle Seite zu KI und Angular
  - Best-Practices-Regeln zum Kopieren in die Regeldatei
- ng new fragt nach Regeldateien für KI-Werkzeuge
  - Option --ai-config, z. B. für Claude, Copilot, Cursor
- llms.txt: Übersicht der Doku für Sprachmodelle
- Angular CLI MCP-Server: ng mcp (nächste Folie)

Der Angular CLI MCP-Server

- MCP: Model Context Protocol, Standard für Werkzeuge
- Das KI-Werkzeug ruft Funktionen eines Servers auf
- ng mcp startet den Server der Angular CLI
- Bietet z. B. aktuelle Best Practices, Doku-Suche,
  - Übersicht der Projekte im Workspace
- Konfiguration im KI-Werkzeug (Beispiel rechts)
- Ergebnisse trotzdem prüfen

MCP-Konfiguration

```
{
  "mcpServers": {
    "angular-cli": {
      "command": "npx",
      "args": ["-y",
        "@angular/cli",
        "mcp"]
    }
  }
}
```

Gute Prompts: ein Muster

Kontext: Angular 21 App, Feature customer, siehe customer-list.component.ts
Aufgabe: Extrahiere die Filter-UI in eine Darstellungskomponente
CustomerFilterComponent
Vorgaben: input()/output(), OnPush, keine Services, kein neues Paket
Abnahme: Verhalten unverändert; Test für den Output vorhanden; ng build läuft durch
Format: Zeige zuerst den Plan, dann die Änderungen als Diff

// Merkregel: Kontext, Aufgabe, Vorgaben, Abnahmekriterien, Format

Weitere Prompt-Beispiele

Erklären:
"Erkläre die Fehlermeldung NG0904 und zeige, wo sie in customer-video.component.ts entsteht. Ändere noch nichts."

Tests vorschlagen:
"Schlage für safeReturnUrl() in auth/return-url.ts Testfälle vor: erlaubte Pfade, fremde Hosts, //evil.example, javascript:, /customer/../admin. Nur Liste, kein Code."

Review anfordern:
"Prüfe den Diff mit unserer Checkliste in AGENTS.md. Nenne Befunde mit Datei, Zeile und verletzter Regel. Schreibe keinen Code."

Befunde zurückgeben (zweite Runde):
"Behebe: 1. *ngIf in Zeile 8 durch @if ersetzen. 2. any bei customers durch readonly Customer[] ersetzen. 3. effect in Zeile 20 durch computed ersetzen."

Mehrrunden-Zusammenarbeit

- Erst Plan, dann Code: Schritte besprechen
- Kleine, überprüfbare Schritte, ein Commit pro Schritt
- Ergebnisse zurückspiegeln: Fehlermeldung, Testausgabe
- Bei langen Sitzungen Regeln wiederholen
  - oder neu starten mit kurzer Zusammenfassung
- Bei Abweichung vom Plan sofort stoppen



Typische KI-Fehler bei Angular

- Veraltete Muster: *ngFor, @Input(), Konstruktor-Injection, NgModule
- Erfundene oder falsche APIs: Methoden oder Optionen, die es nicht gibt
- effect für alles: Zustand synchronisieren, statt computed zu nutzen
- any und Typ-Casts: Fehler verschwinden, statt gelöst zu werden
- Unnötige Komplexität: zusätzliche Abstraktionen, Wrapper, Hilfsklassen
- Sicherheitsblindheit: innerHTML, Tokens im localStorage, ungeprüfte Weiterleitungen
- Tests, die nichts prüfen: toBeTruthy, Spiegelung der Mocks
- Stilbruch: andere Namens- und Ordnerkonventionen

Fehlerbeispiel: veralteter KI-Vorschlag

```
@NgModule({                                    // (1) NgModule
  declarations: [CustomerBadgeComponent],
  imports: [CommonModule],
  exports: [CustomerBadgeComponent],
})
export class CustomerBadgeModule {}

@Component({
  selector: 'customer-badge',
  template: `<span *ngIf="customer">{{ label }}</span>`,  // (2) *ngIf
})
export class CustomerBadgeComponent implements OnInit {
  @Input() customer: any;                      // (3) @Input und any
  label = '';
  constructor(private store: CustomerStore) {}     // (4) Konstruktor-Injection
  ngOnInit() { this.label = this.customer.lastname + ' (' + this.store.total() + ')'; }
}                                              // (5) label wird nie aktualisiert
```

Korrektur: dasselbe in Angular 21

```
import { ChangeDetectionStrategy, Component, computed, inject, input }
  from '@angular/core';
import { Customer } from '../customer.model';
import { CustomerStore } from '../customer.store';
@Component({                                  // (1) standalone, kein NgModule
  selector: 'customer-badge',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (customer(); as c) {                  <!-- (2) Control Flow -->
      <span>{{ label() }}</span>
    }`,
})
export class CustomerBadgeComponent {
  customer = input<Customer>();               // (3) Signal-Input, echter Typ
  private readonly store = inject(CustomerStore);  // (4) inject()
  label = computed(() =>                      // (5) immer aktuell
    `${this.customer()?.lastname} (${this.store.total()})`);
}
```

Fehlerbeispiel: effect statt computed, any

```
// KI-Vorschlag (aus Übung 37, gekürzt)
@Input() customers: any;
count = signal(0);
message = signal('');
constructor() {
  effect(() ⇒ this.count.set(this.customers ? this.customers.length : 0)); // nie wieder
  effect(() ⇒ this.message.set('<b>' + this.count() + '</b> Kunden exportiert'));
}
// Template: <div *ngIf="message()" [innerHTML]="message()"></div>

// Korrektur
customers = input.required<readonly Customer[]>();
count = computed(() ⇒ this.customers().length);
// Template:
// @if (count() > 0) { <p><b>{{ count() }}</b> Kunden exportiert</p> }
```

Vorschläge bewerten: Review-Checkliste

- 1. Läuft es? Build, Lint, Tests
- 2. Passt es zur Architektur? Abhängigkeitsrichtung, Feature-Struktur, keine Parallelwelt
- 3. Ist es modern? Keine veralteten APIs (*ngIf, @Input, NgModule), keine erfundenen Methoden
- 4. Ist es sicher? innerHTML, bypassSecurityTrust*, Tokens, Logging
- 5. Ist es reaktiv korrekt? Effects missbraucht? Werte verändert statt neu gesetzt?
- 6. Ist es getestet? Prüfen die Tests Verhalten oder nur toBeTruthy?
- 7. Verstehe ich es? Was ich nicht erklären kann, committe ich nicht

KI-Tests prüfen: aussagelos oder aussagekräftig?

```
// Aussagelos (KI-Vorschlag aus Übung 37)
it('should export customers', () ⇒ {
  const csv = exportCustomers([{ id: '1', firstname: 'Hans', lastname: 'Meyer' }]);
  expect(csv).toBeTruthy();                // jeder nicht leere Text besteht
});

// Aussagekräftig: prüft konkretes Verhalten und einen Randfall
it('setzt Werte mit Komma in Anführungszeichen', () ⇒ {
  const csv = toCsv([{ id: '1', firstname: 'Hans', lastname: 'Meyer, Jr.' } as Customer]);
  expect(csv.split('\r\n')[1]).toContain('"Meyer, Jr."');
});

it('schützt vor Formeln in Excel (CSV-Injection)', () ⇒ {
  expect(escapeCsvField('=SUMME(A1)')).toBe("'=SUMME(A1)");
});
```

Grenzen und Fehlannahmen (1/2)

- Annahme: „Die KI weiß, wie unser Projekt aussieht.“
  - Realität: Nur das, was im Kontext steht
- Annahme: „Wenn sie selbstsicher klingt, stimmt es.“
  - Realität: Sie klingt immer überzeugend, auch bei Unsinn (Halluzination)
- Annahme: „Es kompiliert, also ist es richtig.“
  - Realität: Falsches Verhalten, Sicherheitslücken und Performance-Probleme bleiben möglich

Grenzen und Fehlannahmen (2/2)

- Annahme: „Tests von der KI sind ein Qualitätsnachweis.“
  - Realität: Nur wenn sie Verhalten wirklich prüfen
- Annahme: „Die KI kennt die aktuelle Angular-Version.“
  - Realität: Ihr Wissensstand kann veraltet sein, deshalb Doku oder MCP-Server nutzen
- Annahme: „Die KI erspart mir das Lernen.“
  - Realität: Ohne Wissen kann ich Ergebnisse nicht beurteilen

Datenschutz, Lizenzen, Verantwortung

- Keine Geheimnisse, Zugangsdaten oder Kundendaten in Prompts oder Regeldateien
- Firmenrichtlinien beachten: erlaubte Werkzeuge, Datenablage, Nachweispflichten
- Agenten: Rechte begrenzen, Befehle vor dem Ausführen bestätigen
- Lizenzen: Herkunft generierten Codes kann unklar sein, bei längeren Übernahmen prüfen
- Vorgeschlagene Pakete prüfen (Existenz, Pflege, Sicherheit): Namen werden manchmal erfunden
- Verantwortlich ist der Entwickler, der den Code committet

Was bleibt Entwicklerkompetenz?

- Anforderungen verstehen und präzisieren
- Architektur und Abwägungen (Aufteilung, Reaktivität, Zustandsverwaltung)
- Code lesen und beurteilen
- Testen, Messen, Absichern (Tests, Performance, Sicherheit)
- Verantwortung im Team und gegenüber den Nutzern

Übung 37: KI-Ergebnis reviewen

- **Öffne 37_ki_review; Auftrag: „Kunden als CSV exportieren“ + Komponente CustomerExportButton**
  - Eigenes KI-Werkzeug (wenn erlaubt) oder Beispielergebnis material/ki-vorschlag.md
- **1. Mit der Review-Checkliste prüfen: mindestens 5 Befunde (Stelle, Problem, Regel)**
  - Enthalten u. a.: *ngIf, any, effect schreibt Signal, innerHTML, toBeTruthy; suche weitere
- **2. Befunde in einem zweiten Prompt konkret zurückgeben („Behebe: …“), erneut prüfen**
- **3. Korrigiertes Feature einbauen: CSV-Funktion, Button in der Kundenliste, Tests**
  - Download im Browser ausprobieren
- **4. Auswertung: Was hat die KI nicht erkannt? Was war in Runde 2 neu kaputt?**
- **Musterlösung: 37_ki_review (baut auf 36_sicherheit auf)**

Übung 38: Abschlussprojekt Dashboard (1/2)

- **Öffne das Projekt 38_abschluss; Kleingruppen, KI-Unterstützung nach Wahl**
- Neues Feature: Seite /dashboard mit Kennzahlen
  - Anzahl Kunden; aktive Kunden (Bestellung in den letzten 7 Tagen); Kunden pro Stadt
- **1. Plan mit 5 Schritten schreiben und im Team abstimmen, bevor Code entsteht**
- **2. In kleinen Schritten umsetzen, je Schritt ein Commit**
- Bewertet werden: Architektur, Reaktivität, Tests, Sicherheit, Nachvollziehbarkeit
- **Musterlösung: 38_abschluss (baut auf 37_ki_review auf)**

Übung 38: Abschlussprojekt Dashboard (2/2)

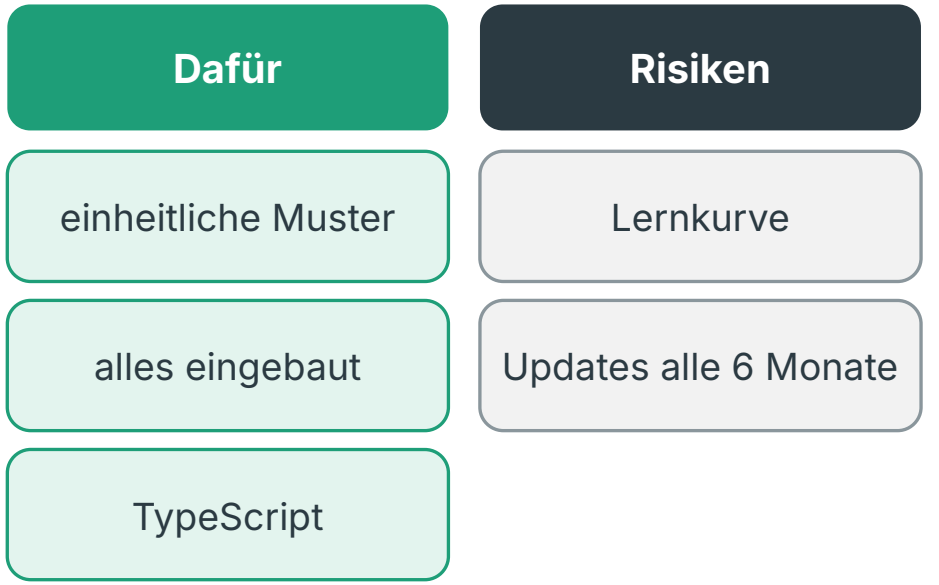
- **3. Anforderungen:**
  - Kennzahlen als computed; Daten über den CustomerStore, kein HTTP in der Komponente
  - Lazy Route /dashboard, geschützt durch authGuard; /dashboard in die returnUrl-Allowlist
  - Aktiv-Logik der CurrentActiveCustomersPipe wiederverwenden
  - Kein innerHTML, keine ungeprüften Parameter; mindestens 3 sinnvolle Tests
- **4. Selbst-Review mit den Checklisten (Sicherheit und KI-Review)**
- **5. Kurzvorstellung (5 Min. je Gruppe): Was hat die KI gut gemacht, was wurde korrigiert?**
- **Musterlösung: 38_abschluss (baut auf 37_ki_review auf)**

Rückblick-Übung: Technologieentscheidung

- Zeitpunkt: am Kursende, etwa 10 Minuten, ohne Code
- Szenario: Ein Versicherer plant ein Kundenportal
  - 8 Entwickler, 10 Jahre Laufzeit, 3 bestehende Backend-Teams
- Fragen: Passt Angular? Was aus dem Kurs spricht dafür, welche Risiken gibt es?
- Ergebnis: je Gruppe (auch zu zweit oder allein) 3 Argumente dafür und 3 Risiken
- Es gibt keine richtige Antwort
- Ziel: Entscheidungskriterien ausdrücklich benennen

Hilfen für die Technologieentscheidung

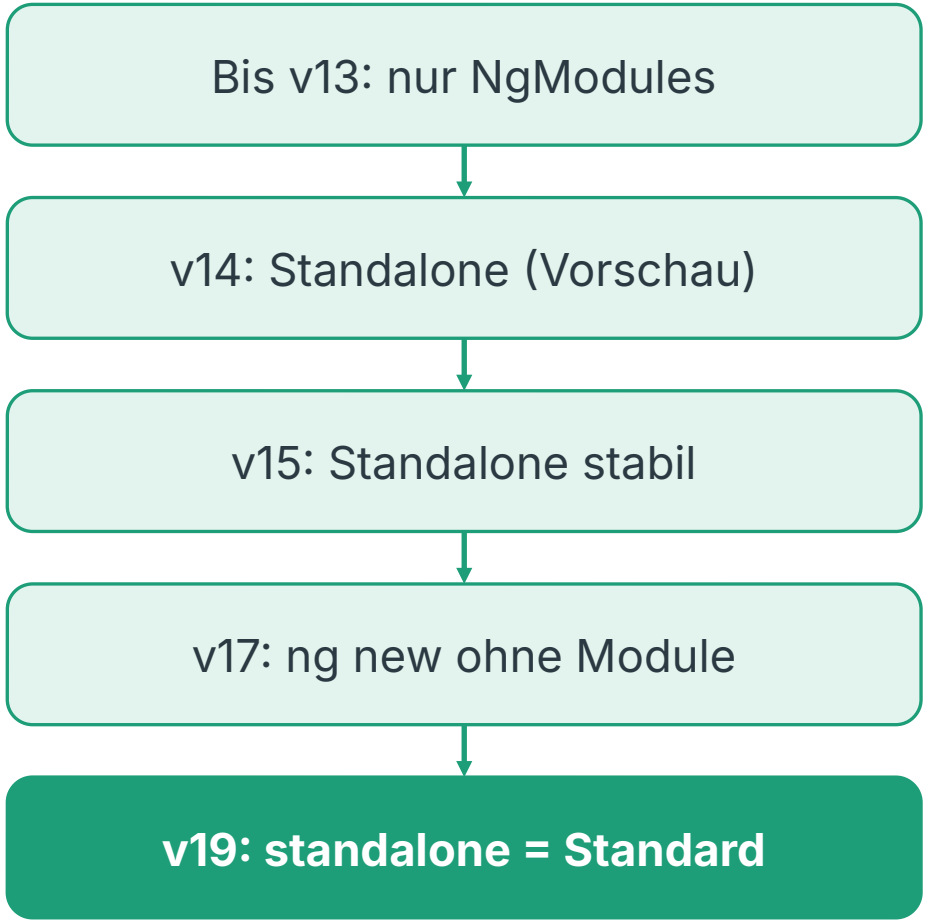
- Lebensdauer: Hauptversion etwa alle 6 Monate
  - Unterstützung je Version etwa 18 Monate
- Team: Vorwissen in TypeScript, RxJS, Signals
- Struktur: einheitliche Muster für viele Personen
  - DI, Routing, Formulare, Tests eingebaut
- Risiken: Lernkurve, regelmäßige Updates
- Kursbezug: Store, Schichten, Testbarkeit



Legacy: Angular-Module (NgModule)

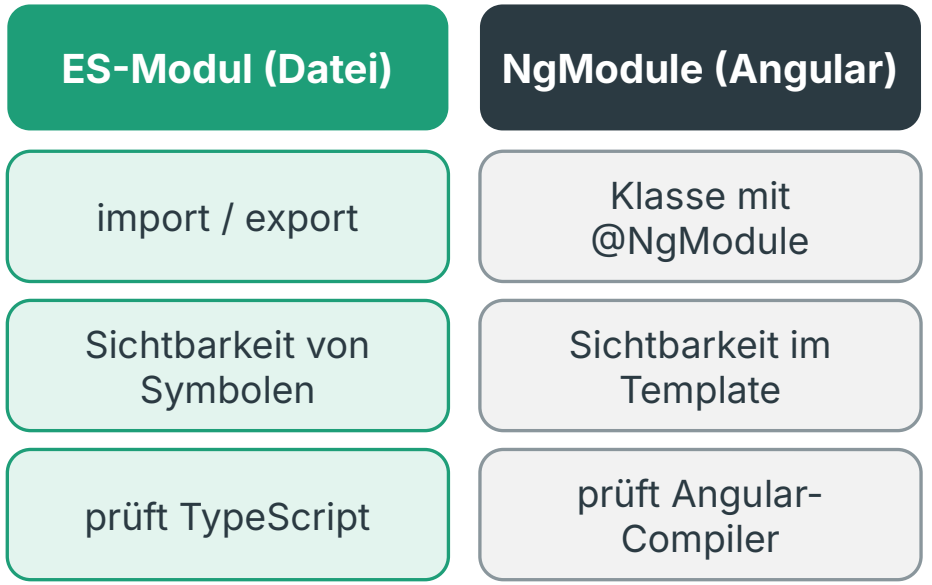
Warum dieses Kapitel?

- Viele Bestandsprojekte sind mit NgModules gebaut
- Ältere Bibliotheken, Doku und Forenbeiträge nutzen sie
- NgModules werden weiterhin unterstützt
  - Mischbetrieb mit Standalone ist möglich
- Neuer Code: immer Standalone
- Ziel: alten Code lesen, pflegen und migrieren



Was ist ein NgModule?

- Eine Klasse mit dem Dekorator @NgModule
- Fasst Komponenten, Direktiven und Pipes zusammen
- Legt fest, was im Template sichtbar ist
- Kann Services bereitstellen (providers)
- Nicht dasselbe wie ein ES-Modul (Datei mit import/export)
  - Beides wird parallel verwendet

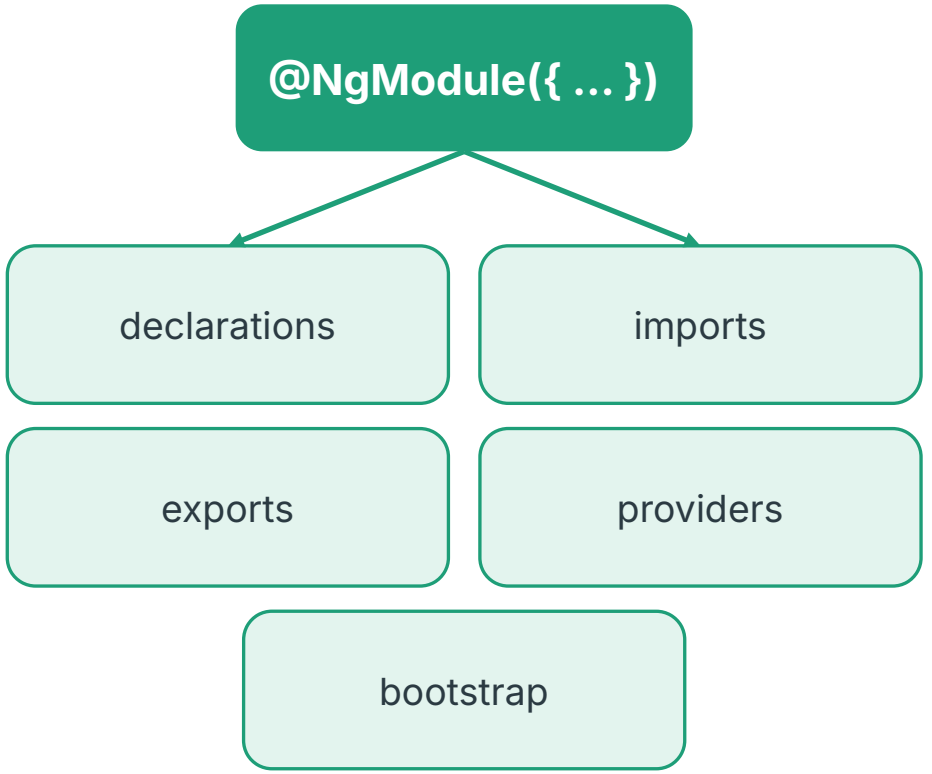


Warum gab es Module?

- Kompromiss zwischen zwei Extremen
  - AngularJS 1: alles global, „einfach reinwerfen“
  - Vorabversionen von Angular 2: jede Abhängigkeit an jeder Komponente
- Kapselung: Feature-Bereiche als Einheit
- Bibliotheken mit einem Import einbinden (z. B. FormsModule)
- Lazy Loading ganzer Bereiche über Module
- Nachteile: viel Boilerplate, indirekte Abhängigkeiten, schwer zu überblicken

Aufbau von @NgModule

- declarations: eigene Bausteine des Moduls
- imports: andere Module, deren Exporte man nutzt
- exports: was andere Module nutzen dürfen
- providers: Services für Dependency Injection
- bootstrap: Startkomponente, nur im AppModule
- Alle Felder sind Arrays und optional



AppModule im Projekt 30_modules_legacy

```
// src/app/app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';
import { AppComponent } from './app.component';
import { TrainingsModule } from './trainings/trainings.module';
import { AppRoutingModule } from './app-routing.module';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, TrainingsModule, AppRoutingModule],
  providers: [provideHttpClient(withInterceptorsFromDi())],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Die Felder im Detail (1/2)

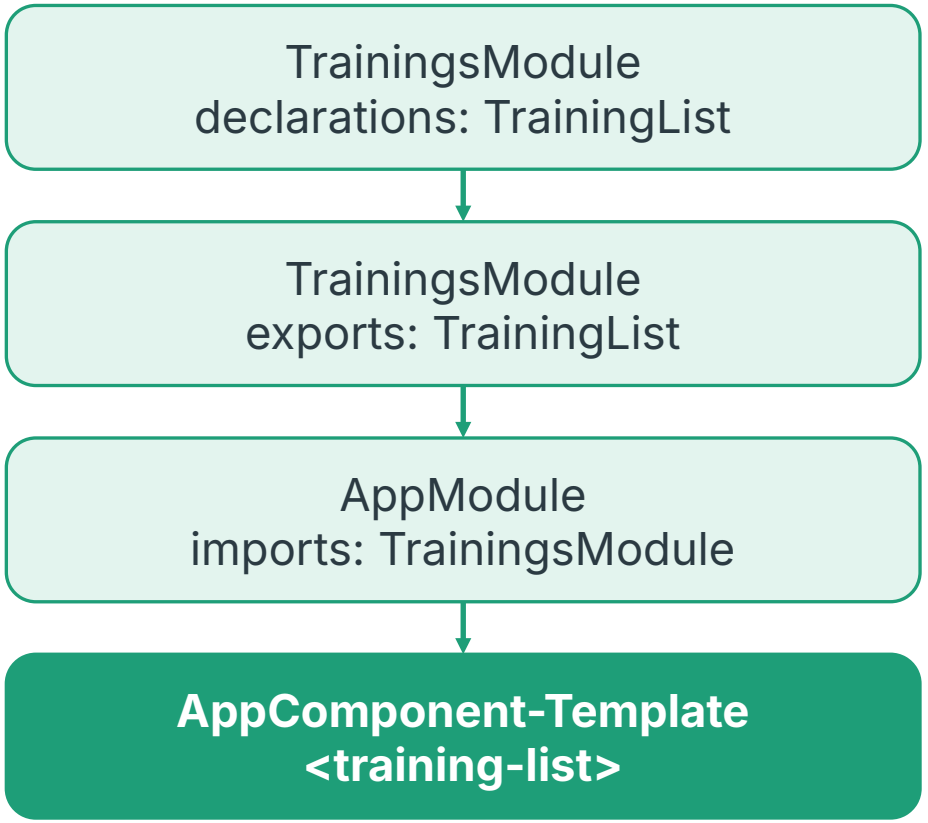
- declarations
  - Komponenten, Direktiven, Pipes mit standalone: false
  - Jeder Baustein gehört zu genau einem Modul
- imports
  - NgModules oder Standalone-Bausteine
  - Deren Exporte sind in den eigenen Templates nutzbar
- exports
  - Eigene Bausteine oder importierte Module weitergeben
  - Nicht exportiert = privat im Modul

Die Felder im Detail (2/2)

- providers
  - Services für den Injector, z. B. [TrainingService]
  - Bei normal importierten Modulen: anwendungsweit eine Instanz
- bootstrap
  - Startkomponente(n), nur im AppModule
  - Ihr Selector muss in index.html stehen
- BrowserModule nur im AppModule importieren
  - Feature-Module importieren stattdessen CommonModule

Sichtbarkeit zwischen Modulen

- Ein Template sieht nur, was sein Modul kennt
  - eigene declarations
  - Exporte der importierten Module
- Fehlt ein Glied der Kette, meldet Angular:
  - „'training-list' is not a known element“
- Prüfen: deklariert? exportiert? importiert?



Feature-Modul: TrainingsModule

```
// src/app/trainings/trainings.module.ts
@NgModule({
  declarations: [
    TrainingListComponent,
    TrainingDetailsComponent,
    TakePlaceSoonPipe
  ],
  imports: [CommonModule, FormsModule, ReactiveFormsModule, RouterModule],
  providers: [TrainingService],
  exports: [TrainingListComponent, TrainingDetailsComponent]
})
export class TrainingsModule { }
```

Dateien eines NgModule-Projekts

- index.html: Seite mit <app-root>
- main.ts: startet das AppModule
- app.module.ts: Hauptmodul
- app-routing.module.ts: Routen
- app.component.ts/.html/.css: Startkomponente
- trainings.module.ts: Feature-Modul
- Neu gegenüber Standalone: die *.module.ts-Dateien

src/

```
index.html
main.ts
app/
  app.module.ts
  app-routing.module.ts
  app.component.ts / .html
  trainings/
    trainings.module.ts
    training.service.ts
    training-list/
    training-details/
```

Komponenten im Modul: standalone: false

- @Component wie gewohnt: selector, templateUrl
- Seit v19 ist standalone: true der Standard
  - Modul-Komponenten brauchen standalone: false
- Kein imports-Array an der Komponente
  - Abhängigkeiten kommen über das Modul
- Selector entscheidet, wo sie erscheint

app.component.ts

```
@Component({
  selector: 'app-root',
  standalone: false,
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'invoice-app';
}
```

Klassischer Stil: Konstruktor-Injektion

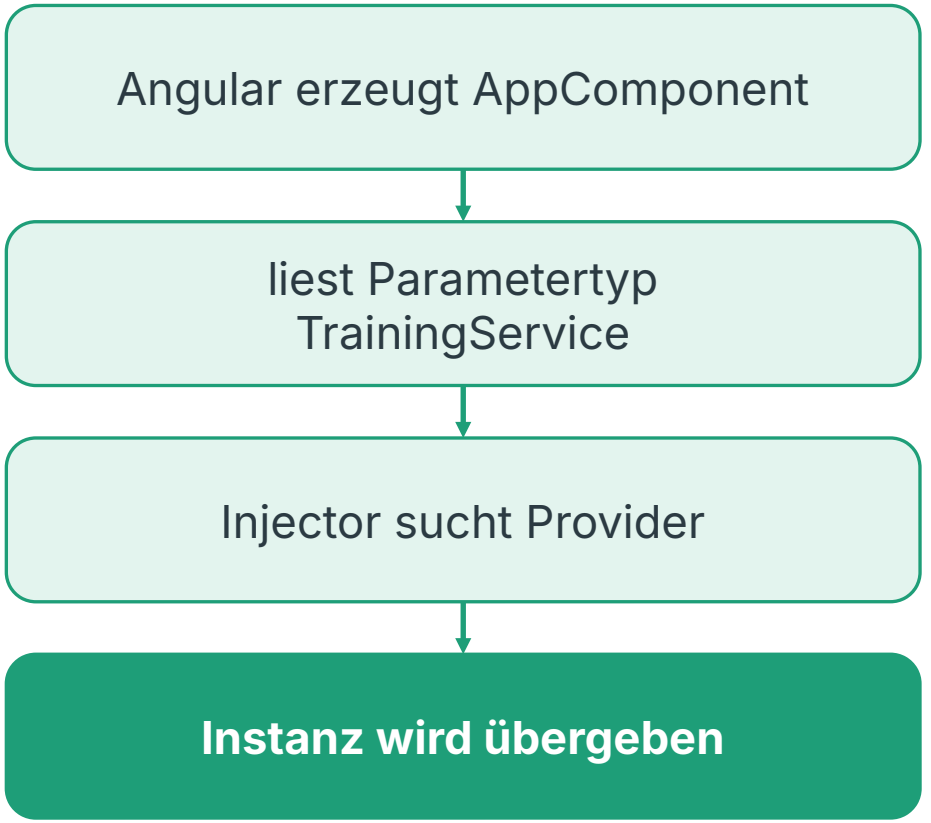
```
// src/app/app.component.ts (Projekt 30_modules_legacy)
export class AppComponent {
  title = 'invoice-app';
  public nextTraining?: Training;

  constructor(private trainingService: TrainingService) {
    trainingService.getNext().subscribe(nextTr => {
      this.nextTraining = nextTr;
    });
  }
}

// Heute üblich:
// private readonly trainingService = inject(TrainingService);
```

Konstruktor-Injektion erklärt

- „private trainingService: TrainingService“
  - TypeScript-Kurzform: Parameter wird zum Feld
- Angular liest den Typ als Token
  - Token = Schlüssel, unter dem der Injector sucht
- Injector findet Provider, liefert Instanz
- Aufrufer: Angular selbst beim Erzeugen
- Heute: inject() im Feld-Initialisierer



Bootstrapping mit Modul: main.ts

- platformBrowserDynamic()
  - erzeugt die Plattform für den Browser
- bootstrapModule(AppModule)
  - übergibt das Hauptmodul als Klasse
  - liefert ein Promise auf das gestartete Modul
- .catch(err ⇒ ...)
  - fängt Fehler beim Start ab

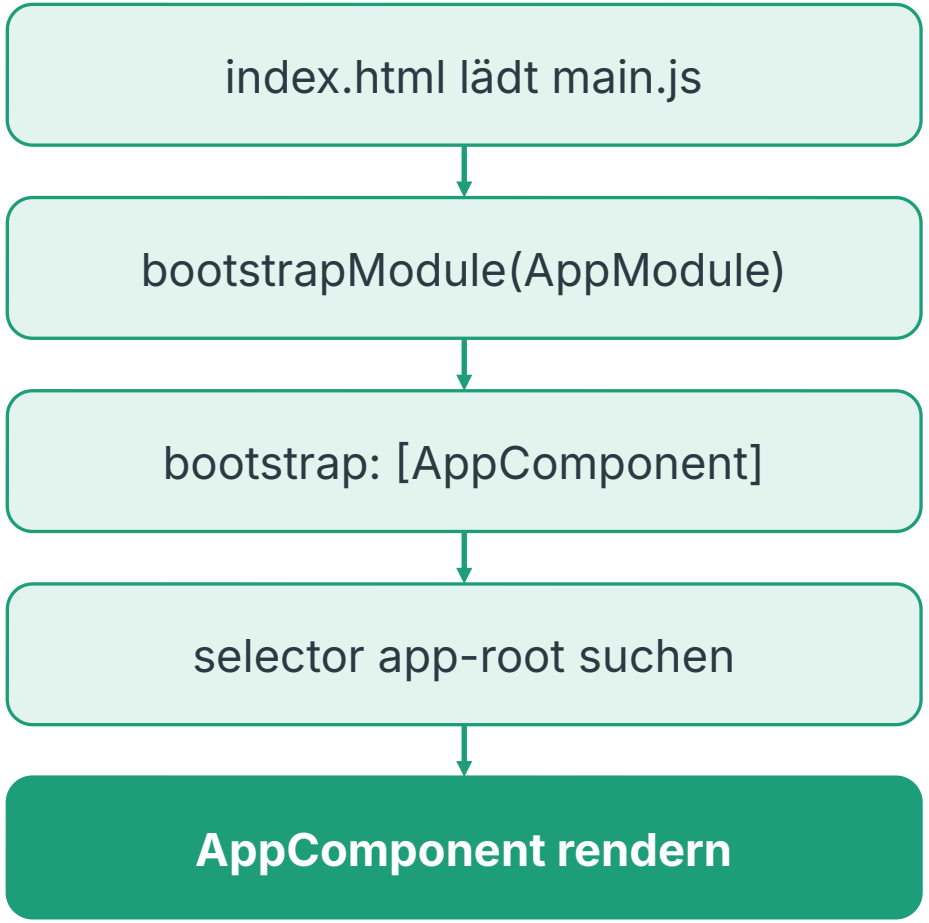
src/main.ts

```
if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err =>
    console.error(err));
```

Ablauf des Startvorgangs

- main.ts ruft bootstrapModule(AppModule) auf
- Angular liest den Dekorator von AppModule
- Feld bootstrap nennt AppComponent
- Angular liest deren selector: app-root
- Sucht <app-root> in index.html
- Erzeugt die Komponente und rendert sie dort



index.html: der Einstiegspunkt

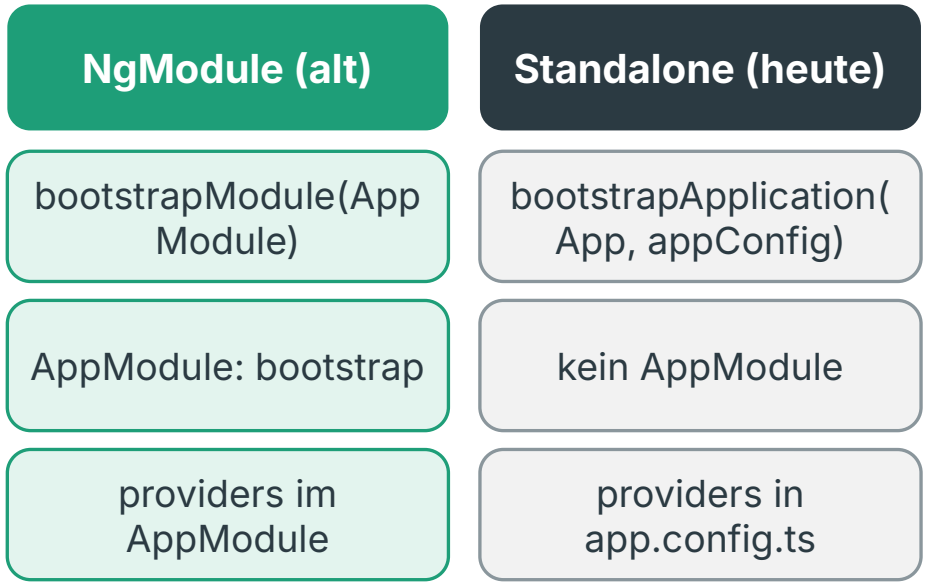
- Enthält nur den Platzhalter <app-root>
- Skripte fügt der Build automatisch ein
- <base href="/"> für das Routing
- Gleich wie im Standalone-Projekt
- Fehlt <app-root>: Fehler beim Start
  - „The selector "app-root" did not match any elements“

src/index.html

```
<!doctype html>
<html lang="de">
<head>
  <base href="/">
  <title>InvoiceApp</title>
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Start: bootstrapModule und bootstrapApplication

- Alt: Startpunkt ist ein Modul
  - globale Provider im AppModule
- Neu: Startpunkt ist eine Komponente
  - globale Provider in app.config.ts
- Ablauf danach gleich: selector suchen, rendern
- Migration stellt main.ts automatisch um



Module und Komponenten generieren

- ng generate module trainings (kurz: ng g m)
  - erzeugt trainings.module.ts mit CommonModule
  - wird NICHT automatisch importiert: selbst eintragen
  - oder Option --module app verwenden
- ng g c trainings/training-info --standalone=false
  - trägt die Komponente in declarations des nächsten Moduls ein
- Ohne --standalone=false entsteht eine Standalone-Komponente

Pipes und Direktiven im Modul

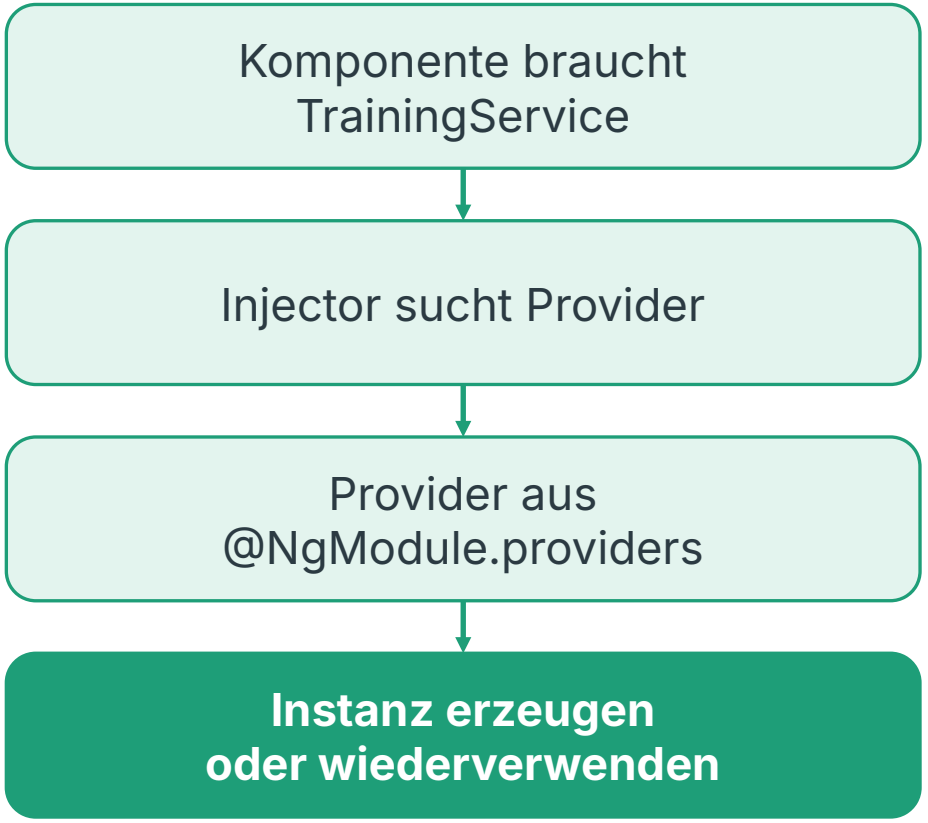
- Pipes und Direktiven werden wie Komponenten deklariert
- Nur exportierte Pipes sind in anderen Modulen nutzbar
- Eingebaute Pipes (DatePipe, ...) kommen aus CommonModule
- Heute: Pipe direkt im imports-Array der Komponente
  - imports: [HasLikesPipe, DatePipe]

shared.module.ts

```
@NgModule({
  declarations: [HasLikesPipe],
  exports: [HasLikesPipe]
})
export class SharedModule { }
```

Legacy-DI: providers am Modul

- Woher weiß Angular, was zu injizieren ist?
  - Antwort: aus einem Provider
- Alt: providers-Array im @NgModule
  - Service mit @Injectable() ohne providedIn
- Eingebundenes Modul: eine Instanz für die App
- Lazy geladenes Modul: eigener Injector, eigene Instanz



Provider-Objekte: provide und Rezept

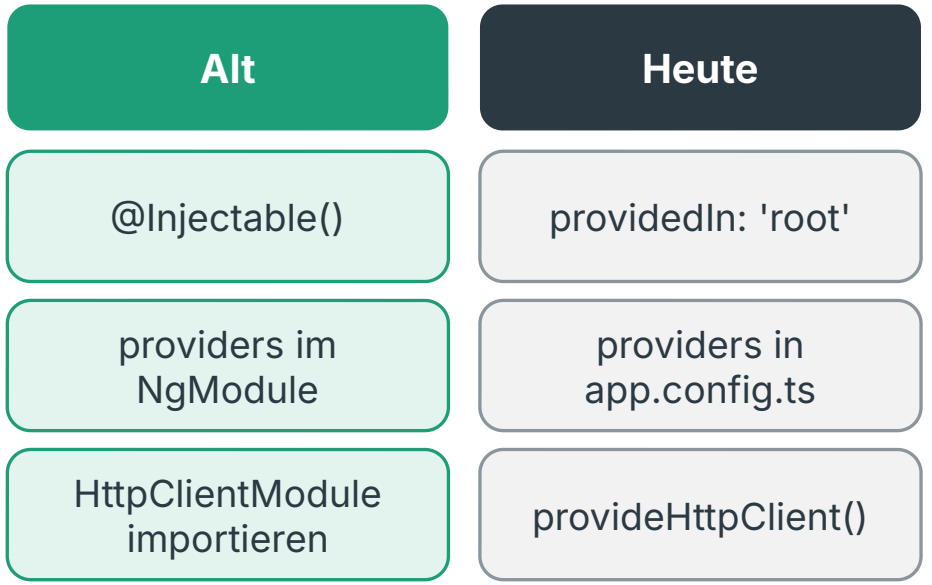
- Kurzform [ConsoleLogger] bedeutet:
  - { provide: ConsoleLogger, useClass: ConsoleLogger }
- provide: Token, unter dem gesucht wird
- Rezept, wie das Objekt entsteht:
  - useClass: Klasse mit new erzeugen
  - useValue: fertigen Wert verwenden
  - useFactory: Funktion liefert das Objekt
  - useExisting: auf anderes Token verweisen

track.module.ts

```
@NgModule({
  providers: [
    { provide: ConsoleLogger,
      useClass: ConsoleLogger },
    { provide: Logger,
      useExisting: ConsoleLogger }
  ]
})
export class TrackModule { }
```

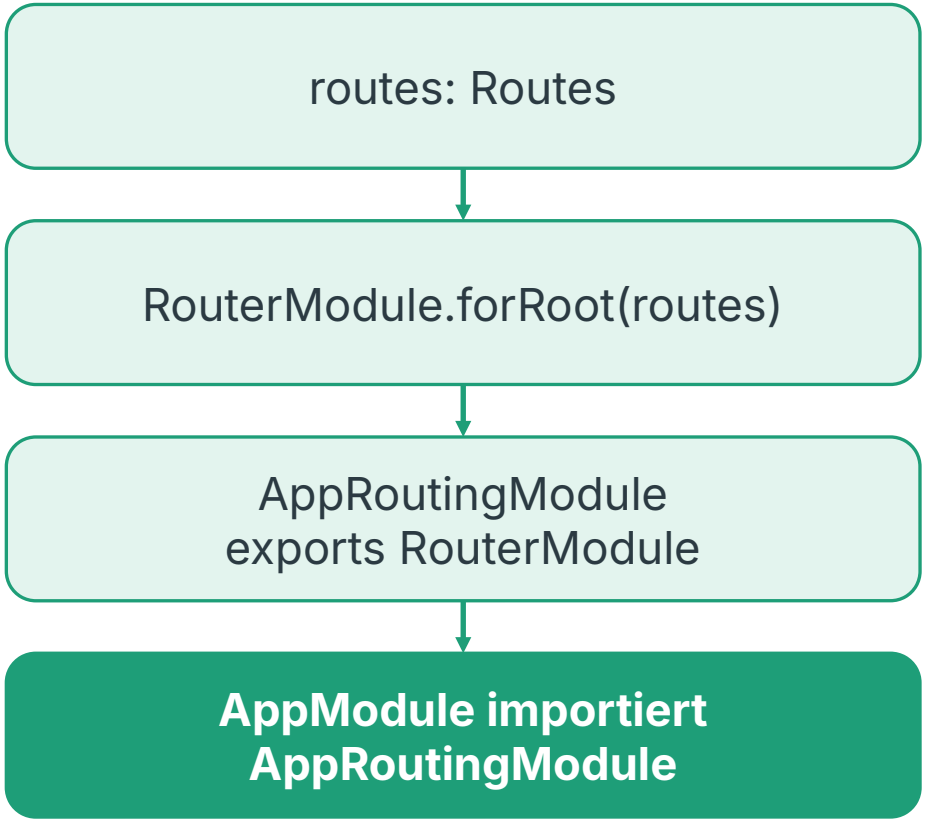
DI heute: providedIn und provide-Funktionen

- Services meist mit providedIn: 'root'
  - kein Eintrag in einem Modul nötig
  - nicht benutzte Services fallen beim Build weg
- Globale Provider in app.config.ts
- Lokale Provider im providers-Array der Komponente
- Provider-Objekte bleiben gleich



Legacy-Routing: RouterModule.forRoot

- Routen stehen in einem eigenen AppRoutingModule
- RouterModule.forRoot(routes)
  - registriert Router-Service und Routen
  - nur einmal in der ganzen Anwendung
- exports: [RouterModule]
  - router-outlet und routerLink im AppModule nutzbar
- AppModule importiert AppRoutingModule



AppRoutingModule im Projekt

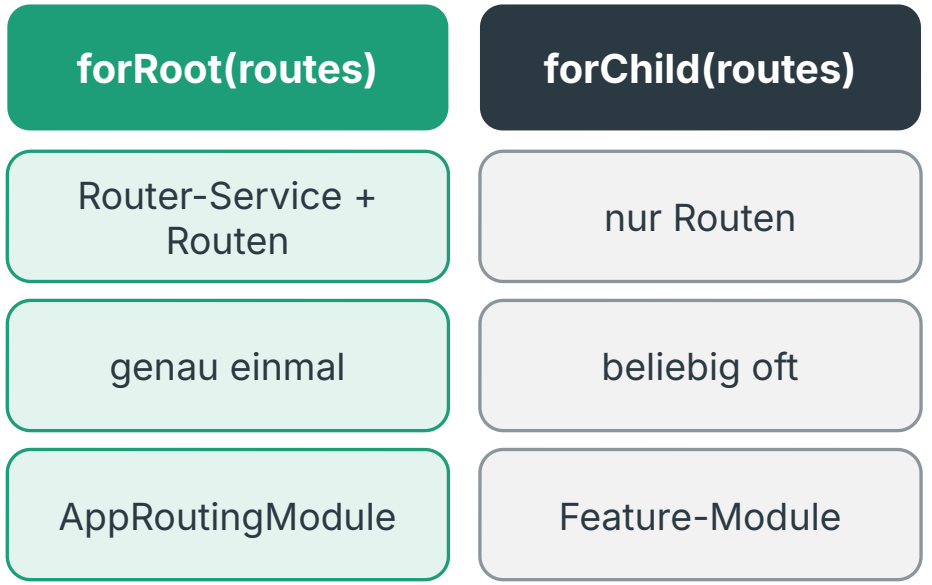
```
// src/app/app-routing.module.ts
export const routes: Routes = [
  { path: 'trainings', component: TrainingListComponent },
  { path: 'trainings/:id', component: TrainingDetailsComponent },
  { path: '**', redirectTo: '/trainings' }
];

@NgModule({
  imports: [CommonModule, RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }

// Heute: provideRouter(routes) in app.config.ts
```

forRoot und forChild

- Statische Methoden, liefern Modul plus Provider
- forRoot(routes): Router-Service und Wurzelrouten
  - genau einmal, im AppModule bzw. AppRoutingModule
- forChild(routes): nur zusätzliche Routen
  - in Feature-Modulen, v. a. bei Lazy Loading
- forRoot im Feature-Modul: zweiter Router, Fehler
- Heute: loadChildren lädt Routen-Dateien direkt



Location-Strategie im Modul

- PathLocationStrategy (Standard): /trainings/7
  - braucht <base href="/"> in index.html
  - Server muss unbekannte Pfade auf index.html leiten
- HashLocationStrategy: /#/trainings/7
  - einfacher zu hosten, Standard in AngularJS 1
- Alt: RouterModule.forRoot(routes, { useHash: true })
- Heute: provideRouter(routes, withHashLocation())

Legacy: Routenparameter mit ActivatedRoute

```
export class TrainingDetailsComponent implements OnInit, OnDestroy {
  training?: Training;
  private subscription?: Subscription;

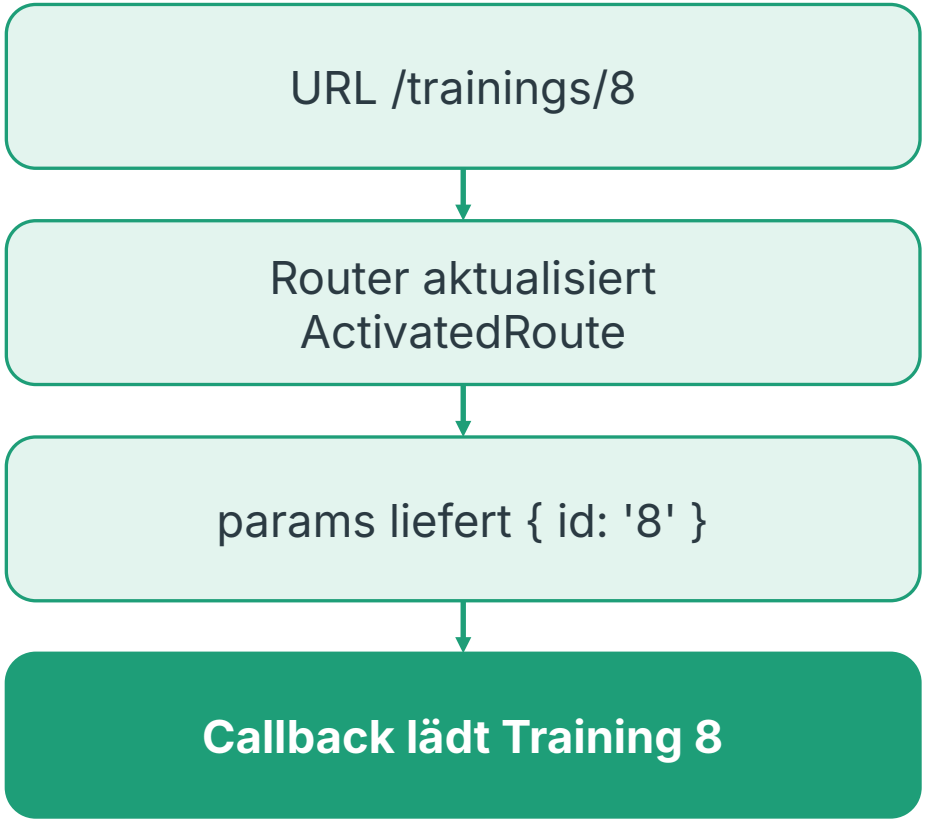
  constructor(private route: ActivatedRoute,
              private trainingService: TrainingService) {}

  ngOnInit(): void {
    this.subscription = this.route.params.subscribe(params => {
      const id = Number(params['id']);
      this.trainingService.getById(id)
        .subscribe(training => this.training = training);
    });
  }

  ngOnDestroy(): void { this.subscription?.unsubscribe(); }
}
```

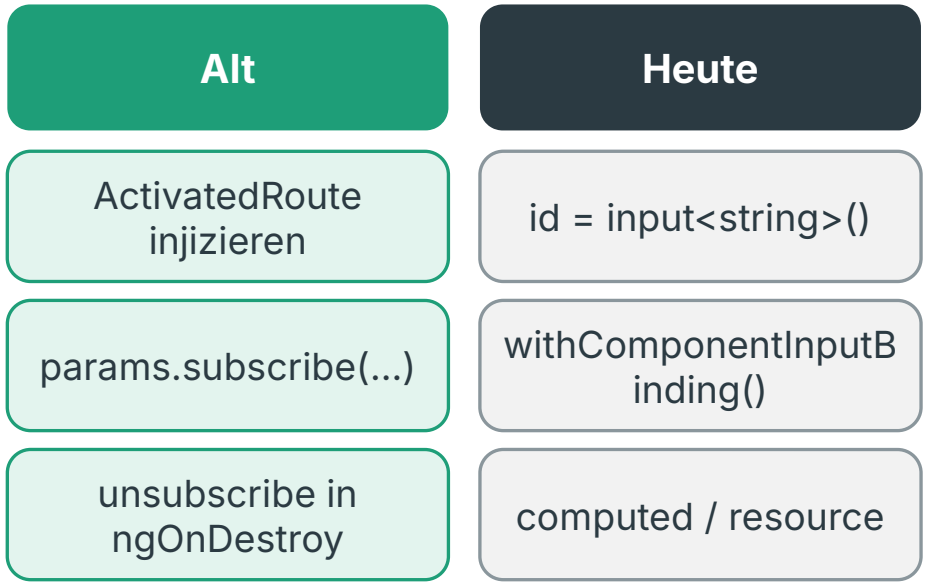
params.subscribe erklärt

- route.params ist ein Observable<Params>
- Callback-Parameter params: Objekt, z. B. { id: '7' }
  - Werte sind immer Strings: Number() nötig
- Aufruf bei jedem Parameterwechsel
  - auch /trainings/7 → /trainings/8 ohne neue Komponente
- subscribe liefert eine Subscription
  - unsubscribe in ngOnDestroy als Gewohnheit



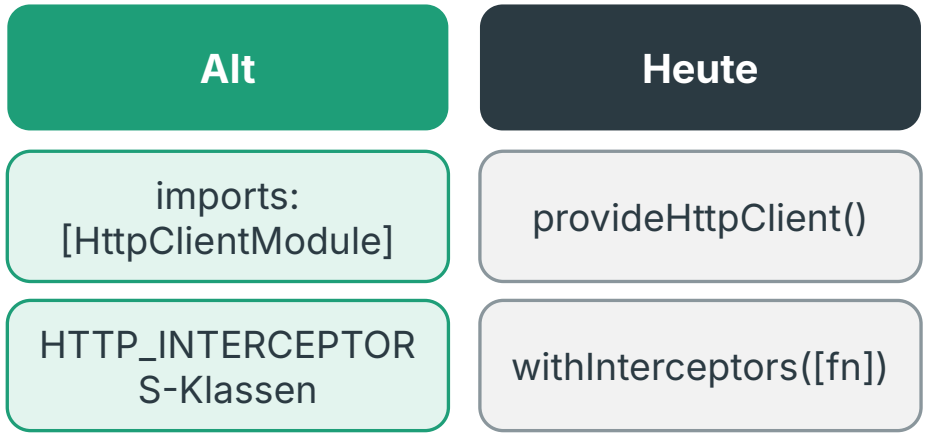
Heute: Input Binding statt subscribe

- Router übergibt Parameter als Input
  - id = input<string>()
  - einschalten: withComponentInputBinding()
- Reaktion auf Wechsel: computed, effect oder resource
- Kein subscribe, kein ngOnDestroy nötig
- Projekt 30 nutzt noch den alten Weg



Legacy: HttpClientModule

- Angular 2 bis 4: HttpModule aus @angular/http
- Ab 4.3: HttpClientModule im AppModule importieren
  - seit Angular 18 als veraltet markiert
- Heute: provideHttpClient() als Provider
  - funktioniert auch im providers-Array eines NgModule
- withInterceptorsFromDi(): klassenbasierte Interceptors
- HttpClient selbst: gleiche API wie heute



HttpClient im Service (klassisch)

- get<T>(url) liefert ein Observable<T>
  - Anfrage startet erst bei subscribe
- pipe(map(...)) formt die Antwort um
  - Parameter von map: die geparste JSON-Antwort
- Ergebnis: Observable<Training>
- Heute oft: httpResource oder rxResource

training.service.ts

```
@Injectable()
export class TrainingService {
  constructor(
    private http: HttpClient) {}

  getById(id: number) {
    const url = 'api/training/' + id;
    return this.http.get<Res>(url)
      .pipe(map(res => res.data));
  }
}
```

Mischbetrieb: Standalone und NgModule

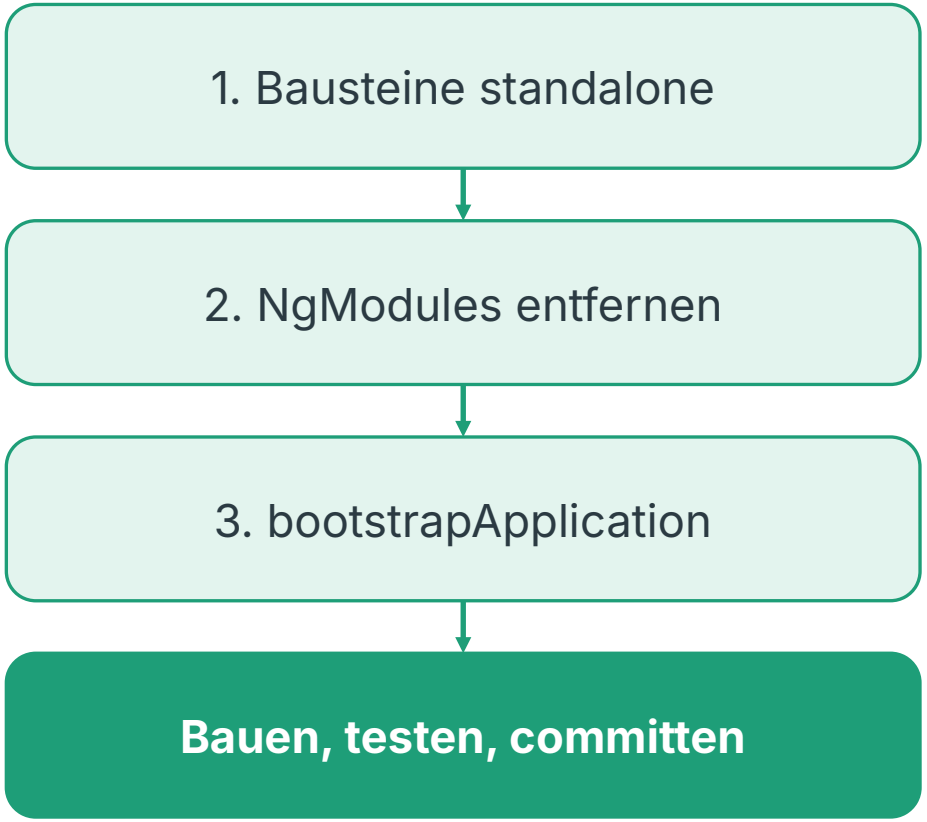
- Standalone-Komponente in einem NgModule
  - in imports eintragen, NICHT in declarations
- NgModule in einer Standalone-Komponente
  - imports: [SomeLegacyModule]
- Beides gleichzeitig im selben Projekt möglich
- Erlaubt Migration in kleinen Schritten

trainings.module.ts

```
@NgModule({
  declarations: [
    TrainingListComponent
  ],
  imports: [
    CommonModule,
    // Standalone:
    TrainingBadgeComponent
  ]
})
export class TrainingsModule { }
```

Migration: ng generate @angular/core:standalone

- Automatische Migration der Angular-Kommandozeile
- Läuft in drei Durchgängen, jeweils mit Auswahl
  - 1. Bausteine auf standalone umstellen
  - 2. Überflüssige NgModules entfernen
  - 3. Start auf bootstrapApplication umstellen
- Nach jedem Durchgang: bauen, testen, committen
- Danach von Hand nacharbeiten



Nach der Migration: weitere Modernisierung

- Control Flow: ng generate @angular/core:control-flow
  - *ngIf / *ngFor werden zu @if / @for
- inject(): ng generate @angular/core:inject
  - Konstruktor-Injektion wird zu inject()
- Von Hand: provideRouter, provideHttpClient
- Von Hand: Input Binding, Signals, resource
- Schritt für Schritt, mit Tests absichern

Gegenüberstellung: alt und neu

- Organisation: @NgModule → imports an der Komponente
- Start: bootstrapModule(AppModule) → bootstrapApplication(App, appConfig)
- Komponente: standalone: false → Standard (standalone)
- DI: providers im Modul, Konstruktor → providedIn: 'root', inject()
- Routing: RouterModule.forRoot / forChild → provideRouter, loadChildren
- Parameter: params.subscribe → input() mit withComponentInputBinding()
- HTTP: HttpClientModule → provideHttpClient()
- Template: *ngIf / *ngFor → @if / @for

Stolperfallen in Legacy-Projekten

- „is not a known element“: Deklaration, Export oder Import fehlt
- Baustein in zwei Modulen deklariert: Fehler beim Kompilieren
- standalone: false vergessen: Komponente nicht deklarierbar
- BrowserModule gehört nur ins AppModule, sonst CommonModule
- forRoot im Feature-Modul: zweiter Router
- Service in providers eines Lazy-Moduls: zweite Instanz
- Neues Modul nicht im Elternmodul importiert

Übung 30: Legacy – Angular Module (1/2)

- **Öffne das Projekt 30_modules_legacy (NgModule-basiert)**
- **1. Backend starten: Ordner server im Kursverzeichnis, npm install, node server.js**
- **2. App starten: ng serve --proxy-config dev.proxy.json**
- **3. Aufbau analysieren**
  - main.ts: platformBrowserDynamic().bootstrapModule(AppModule)
  - AppModule: declarations, imports, providers, bootstrap
  - TrainingsModule: declarations, providers, exports
  - AppRoutingModule: RouterModule.forRoot(routes), exports
- Projektordner: 30_modules_legacy (eigenständiges NgModule-Projekt)

Übung 30: Legacy – Angular Module (2/2)

- **4. Klassische Komponente erzeugen**
  - ng g c trainings/training-info --standalone=false
  - Prüfen: In welchem Modul steht sie in declarations? Exportieren?
- **5. Mischbetrieb: eine Standalone-Komponente erzeugen**
  - in imports des TrainingsModule eintragen (nicht declarations)
  - im Template einer Modul-Komponente verwenden
- **6. Optional: ng generate @angular/core:standalone**
  - Durchgänge einzeln ausführen, Ergebnis mit 23_communication vergleichen
- Projektordner: 30_modules_legacy (eigenständiges NgModule-Projekt)

Wahlthema: Reactive Forms

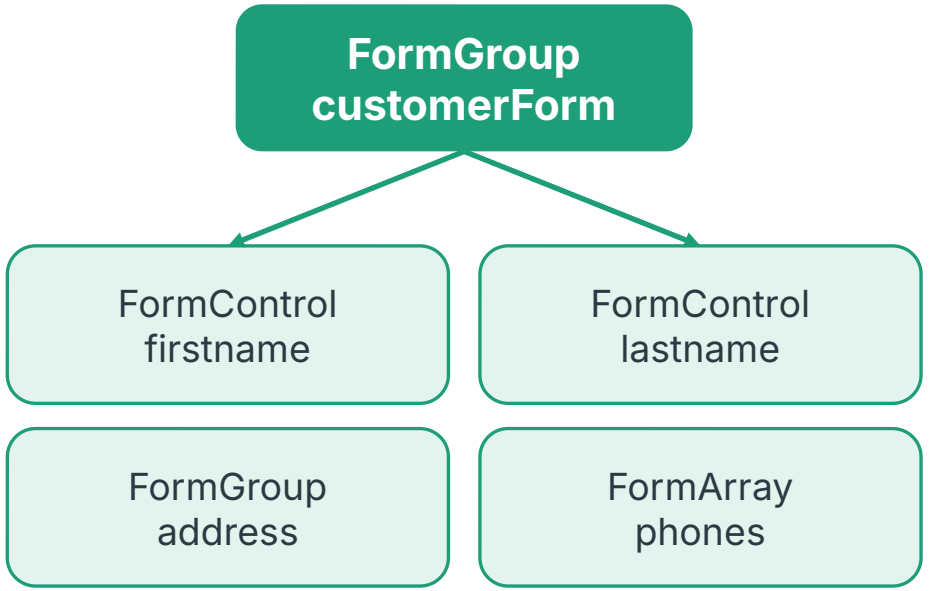
Reactive Forms: die Idee

- Formularmodell entsteht in der Klasse
- Template verbindet nur Eingabefelder mit dem Modell
- Werte und Zustand jederzeit synchron lesbar
- Prüfregeln als Funktionen in TypeScript
- Gut für dynamische Felder und komplexe Regeln
- Logik ohne Template testbar

Template-Driven	Reactive
Modell aus dem Template	Modell in der Klasse
ngModel, HTML-Attribute	FormGroup, FormControl
FormsModule	ReactiveFormsModule

Der Formular-Baum

- FormControl: ein einzelnes Feld (Blatt)
- FormGroup: Objekt aus benannten Controls
- FormArray: Liste gleichartiger Controls
- Das Formular selbst ist eine FormGroup
- Zustand wird nach oben zusammengefasst
  - ein Kind ungültig → Gruppe ungültig



Gemeinsame Basis: AbstractControl

- Alle drei erben von AbstractControl
- value: aktueller Wert
- status: VALID, INVALID, PENDING, DISABLED
- valid / invalid, errors
- touched / untouched, dirty / pristine
- Methoden: setValue, patchValue, reset, disable
- Beobachtbar: valueChanges, statusChanges

FormGroup und FormControl anlegen

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'customer-details',
  imports: [ReactiveFormsModule],
  templateUrl: './customer-details.component.html'
})
export class CustomerDetailsComponent {
  customerForm = new FormGroup({
    firstname: new FormControl('', { nonNullable: true }),
    lastname: new FormControl('', { nonNullable: true }),
    zip: new FormControl('', { nonNullable: true }),
    lastOrderDate: new FormControl<string | null>(null)
  });
}
```

Typisierte Formulare und nonNullable

- Seit Angular 14 sind Formulare streng typisiert
- Ohne Option: FormControl<string | null>
  - reset() setzt den Wert auf null
- Mit nonNullable: true: FormControl<string>
  - reset() setzt auf den Startwert zurück
- form.value: Partial, deaktivierte Felder fehlen
- form.getRawValue(): vollständiges Objekt

Typen

```
new FormControl('')
// FormControl<string | null>

new FormControl('',
  { nonNullable: true })
// FormControl<string>

form.value
// Partial<{ firstname: … }>
form.getRawValue()
// { firstname: string; … }
```

FormBuilder: kürzer schreiben

- inject(FormBuilder) liefert einen Helfer
- fb.nonNullable.group({ ... }): alle Felder nonNullable
- Feld als Array: [Startwert, Validatoren]
- Ergebnis: dieselbe typisierte FormGroup
- Alternative: inject(NonNullableFormBuilder)
- Z1-Projekt: fb.group({ firstname: [''], ... })

customer-details.component.ts

```
private fb = inject(FormBuilder);

customerForm =
  this.fb.nonNullable.group({
    firstname: [''],
    lastname: [''],
    street: [''],
    zip: [''],
    city: ['']
  });
```

Template: formGroup und formControlName

```
<form [formGroup]="customerForm" (ngSubmit)="save()">
  <label for="firstname">Vorname</label>
  <input id="firstname" type="text" formControlName="firstname">

  <label for="lastname">Nachname</label>
  <input id="lastname" type="text" formControlName="lastname">

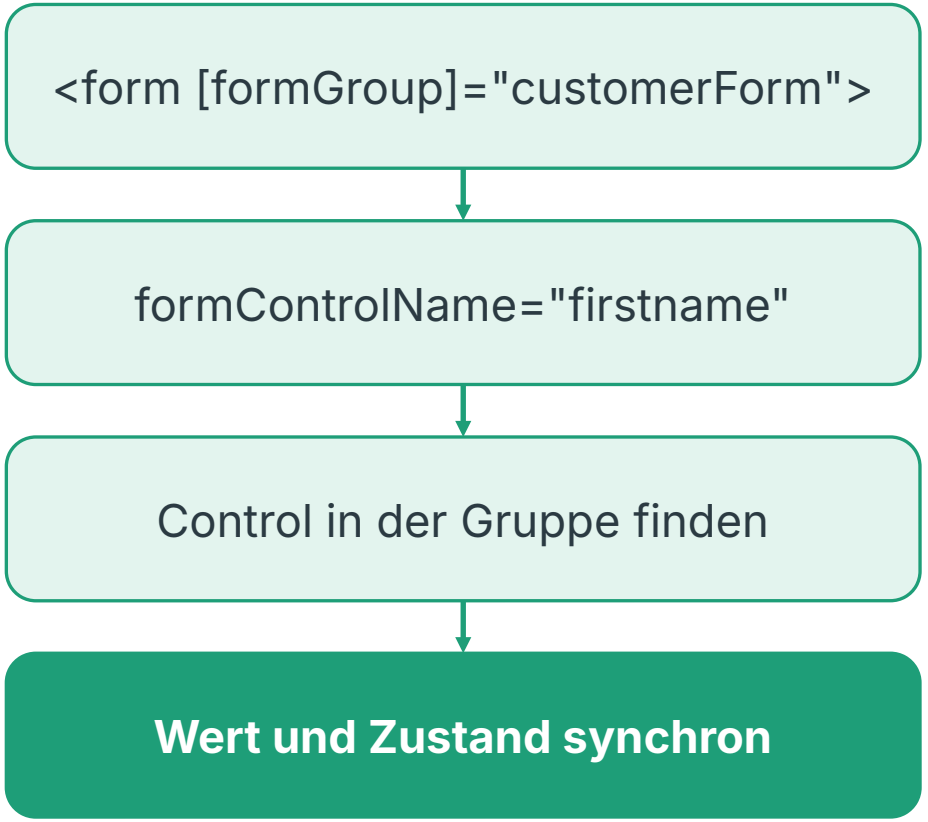
  <label for="zip">PLZ</label>
  <input id="zip" type="text" [formControl]="customerForm.controls.zip">

  <label for="date">Letzte Bestellung</label>
  <input id="date" type="date" formControlName="lastOrderDate">

  <button type="submit" [disabled]="customerForm.invalid">Speichern</button>
</form>
```

Template-Bindung erklärt

- [formGroup]: bindet das FormGroup-Objekt
- formControlName="...": sucht Control per Name
  - Tippfehler → „Cannot find control with name“
- [formControl]: bindet ein Control-Objekt direkt
- Werte fließen in beide Richtungen
- (ngSubmit): kein Neuladen der Seite
- Nicht mit [(ngModel)] mischen



Werte setzen: setValue und patchValue

- setValue(obj): alle Felder müssen dabei sein
- patchValue(obj): nur angegebene Felder
- Typisch: Daten vom Service mit patchValue
- Datum für <input type="date">: Text JJJJ-MM-TT
  - toISOString().substring(0, 10)
- reset(): Startwerte, pristine, untouched

Kunde ins Formular laden

```
const c = this.customer;
this.customerForm.patchValue({
  firstname: c.firstname,
  lastname: c.lastname,
  zip: c.zip ?? '',
  lastOrderDate: c.lastOrderDate
    ?.toISOString()
    .substring(0, 10) ?? null
});
```

Werte übernehmen beim Speichern

- (ngSubmit) ruft save() auf
- Ungültig? Felder markieren und abbrechen
  - markAllAsTouched() zeigt alle Fehler
- getRawValue(): vollständige Werte
- Datum zurück: new Date(text)
- Object.assign überträgt Werte ins Objekt
- Danach Service aufrufen

save()

```
save(): void {
  if (this.customerForm.invalid) {
    this.customerForm
      .markAllAsTouched();
    return;
  }
  const v =
    this.customerForm.getRawValue();
  Object.assign(this.customer, {
    ...v,
    lastOrderDate: v.lastOrderDate
      ? new Date(v.lastOrderDate)
      : undefined
  });
}
```

Validators: eingebaute Prüfregeln

- Validators.required: Pflichtfeld
- Validators.minLength(n) / maxLength(n)
- Validators.min(n) / max(n) für Zahlen
- Validators.email: E-Mail-Format
- Validators.pattern(regex): regulärer Ausdruck
- Mehrere Regeln als Array
  - früher: Validators.compose([...])

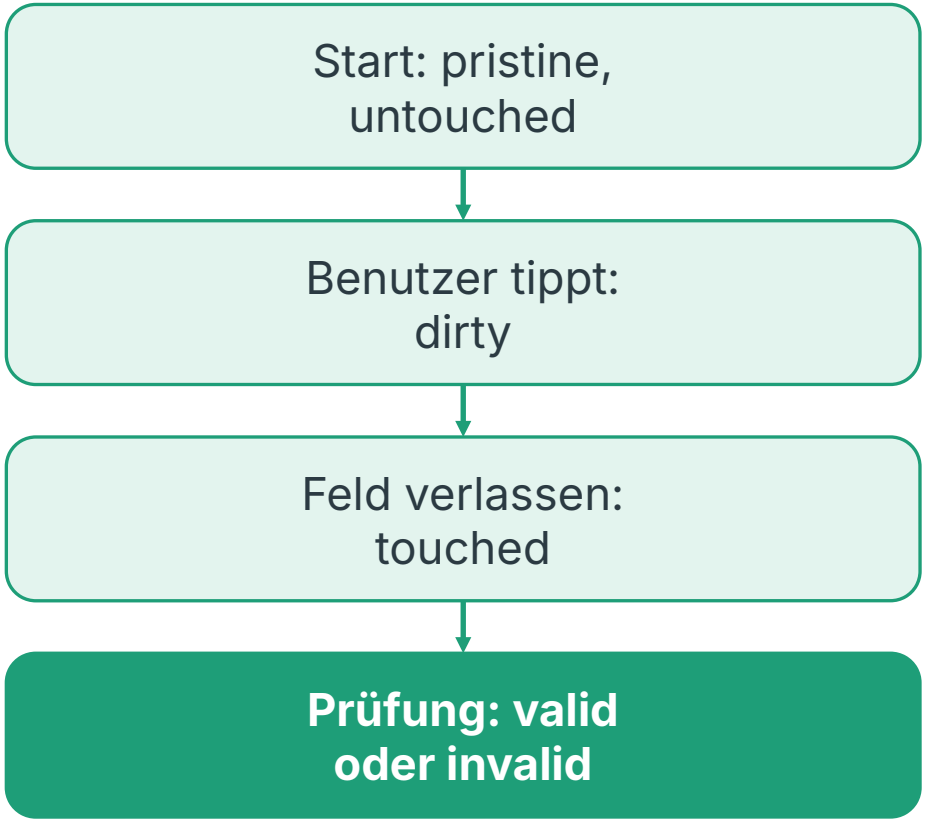
Validatoren im Formular

```
customerForm = this.fb.nonNullable.group({
  firstname: ['', Validators.required],
  lastname: ['', [Validators.required, Validators.minLength(2)]],
  city: ['', Validators.maxLength(30)],
  zip: ['', Validators.pattern('[0-9]{5}')],
  email: ['', [Validators.required, Validators.email]]
});

// Ohne FormBuilder: zweiter Parameter bzw. Optionsobjekt
firstname = new FormControl('', {
  nonNullable: true,
  validators: [Validators.required]
});
```

Feldzustände und CSS-Klassen

- pristine → dirty: Benutzer hat den Wert geändert
- untouched → touched: Feld hat den Fokus verloren
- valid / invalid / pending: Ergebnis der Prüfung
- Gilt für Feld, Gruppe und ganzes Formular
- Angular setzt passende CSS-Klassen
  - ng-valid, ng-invalid, ng-pending
  - ng-pristine, ng-dirty, ng-untouched, ng-touched



Das errors-Objekt

- errors: ValidationErrors | null
  - null = keine Fehler
- Ein Schlüssel pro verletzter Regel
  - required: true
  - minlength: { requiredLength, actualLength }
  - pattern: { requiredPattern, actualValue }
- Abfrage: control.hasError('required')

lastname.errors

```
// Feld leer:
{ required: true }

// Wert "M":
{ minlength: {
    requiredLength: 2,
    actualLength: 1 } }

// gültig:
null
```

Fehlermeldungen mit @if

```
<input id="lastname" formControlName="lastname"
      [class.has-error]="lastname.invalid && lastname.touched">

@if (lastname.touched && lastname.hasError('required')) {
  <p class="error">Nachname ist ein Pflichtfeld.</p>
}
@if (lastname.hasError('minlength')) {
  <p class="error">
    Mindestens {{ lastname.getError('minlength').requiredLength }} Zeichen.
  </p>
}
@if (zip.dirty && zip.hasError('pattern')) {
  <p class="error">Die PLZ besteht aus fünf Ziffern.</p>
}

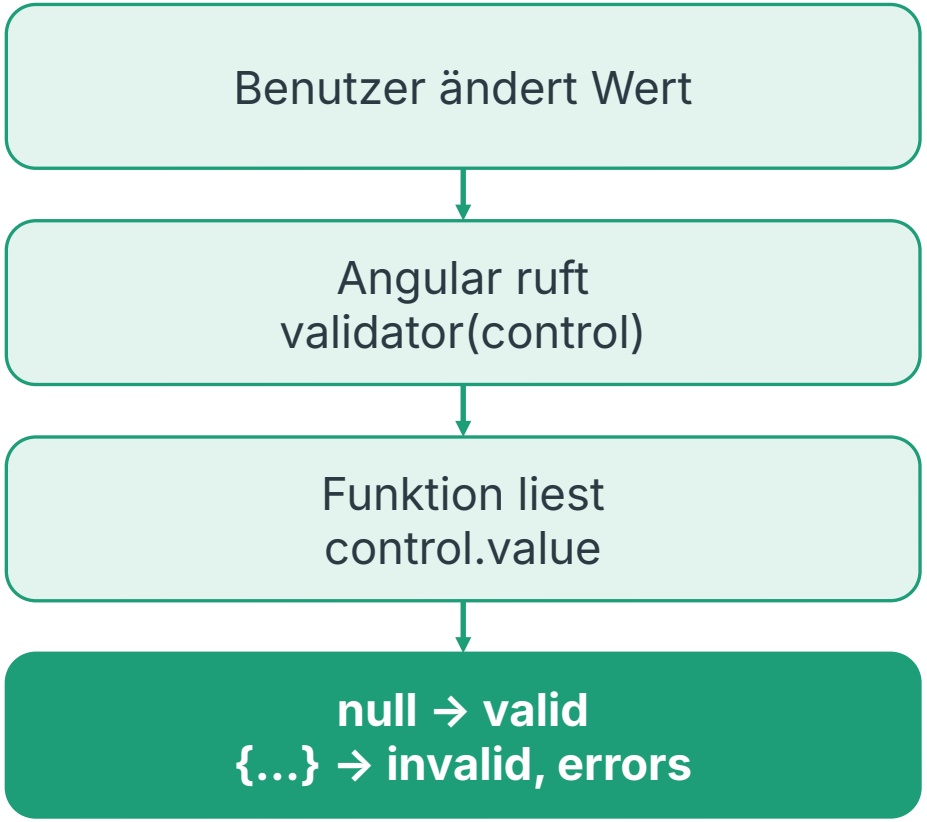
<!-- in der Klasse: lastname = this.customerForm.controls.lastname; -->
```

Fehlerzustand im Template erklärt

- Fehler nicht sofort beim leeren Formular zeigen
  - Bedingung mit touched oder dirty kombinieren
- hasError('key'): true, wenn diese Regel verletzt ist
- getError('key'): Zusatzdaten der Regel
- Submit-Button: [disabled]="customerForm.invalid"
- Oder beim Absenden markAllAsTouched()
- Früher: [hidden] und *ngIf statt @if

Eigene Validatorfunktion: Konzept

- ValidatorFn: (control: AbstractControl) ⇒
  - ValidationErrors | null
- Angular ruft sie bei jeder Wertänderung auf
- Parameter control: das geprüfte Feld
  - control.value ist der aktuelle Wert
- Rückgabe null: gültig
- Rückgabe { schluessel: ... }: ungültig



Eigener Validator: nur Ziffern

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

// 1. Direkt als Prüffunktion
export function digitsOnly(control: AbstractControl): ValidationErrors | null {
  const value = control.value as string;
  if (!value) { return null; }          // leer prüft required
  return /^[0-9]+$/.test(value) ? null : { digitsOnly: { actual: value } };
}

// 2. Mit Parameter: Fabrikfunktion liefert die Prüffunktion
export function exactLength(n: number): ValidatorFn {
  return (control) => !control.value || control.value.length === n
    ? null
    : { exactLength: { required: n, actual: control.value.length } };
}

// Verwendung: zip: ['', [Validators.required, digitsOnly, exactLength(5)]]
```

Eigener Validator erklärt

- control: AbstractControl, das geprüfte Feld
  - bei Gruppen-Validatoren: die ganze FormGroup
- Ohne Klammern übergeben: digitsOnly
- Mit Klammern aufrufen: exactLength(5) → ValidatorFn
- Schlüssel im Fehlerobjekt frei wählbar
  - Abfrage im Template: hasError('digitsOnly')
- Felder vergleichen: Validator an der FormGroup

Auf Änderungen reagieren: valueChanges

- valueChanges: Observable mit jedem neuen Wert
  - Callback-Parameter: der neue Wert des Controls
- statusChanges: VALID, INVALID, PENDING …
- takeUntilDestroyed() meldet automatisch ab
  - im Konstruktor aufrufen (Injection Context)
- Als Signal: toSignal(control.valueChanges)
- Beispiel: Stadt zur PLZ nachschlagen

constructor()

```
constructor() {
  this.customerForm.controls.zip
    .valueChanges
    .pipe(takeUntilDestroyed())
    .subscribe(zip =>
      console.log('PLZ:', zip));
}

zipValue = toSignal(
  this.customerForm.controls
    .zip.valueChanges);
```

FormArray: Listen von Feldern

- Anzahl der Felder steht erst zur Laufzeit fest
- fb.array([...]) als Feld in der FormGroup
- push(control) fügt hinzu, removeAt(i) entfernt
- controls: Array der einzelnen Controls
- Template: formArrayName + @for
  - [formControlName]="$index"

Telefonnummern

```
customerForm = this.fb.nonNullable
  .group({
    firstname: [''],
    phones: this.fb.array([''])
  });
get phones() {
  return this.customerForm
    .controls.phones;
}
<div formArrayName="phones">
 @for (p of phones.controls;
      track $index) {
  <input [formControlName]="$index">
 }
</div>
```

Best Practices für Reactive Forms

- nonNullable verwenden, Werte mit getRawValue() lesen
- Fehlermeldungen erst nach touched oder dirty zeigen
- Eigene Regeln als kleine, testbare Funktionen
- Subscriptions mit takeUntilDestroyed() beenden
- Nicht ngModel und formControlName am selben Feld
- Server prüft trotzdem: Client-Prüfung ist nur Komfort
- Ausblick: Angular 21 enthält experimentelle Signal Forms

Wahlübung Z1: Reactive Form mit FormBuilder

- **Öffne das Projekt Z1_reactive_forms, arbeite in CustomerDetailsComponent**
- **1. ReactiveFormsModule im imports-Array importieren**
- **2. Typisiertes Formular: inject(FormBuilder), fb.nonNullable.group({ … })**
  - Felder: firstname, lastname, street, zip, city, lastOrderDate
- **3. Formular mit den Kundendaten füllen: patchValue(...)**
  - Datum: lastOrderDate.toISOString().substring(0, 10)
- **4. Template: [formGroup] am <form>, formControlName an jedem <input>**
- **5. Speichern: (ngSubmit), getRawValue(), Object.assign, new Date(...)**
- **Musterlösung: Z1_reactive_forms, Schritt 1–4 (baut auf 21_dynamic_routes auf)**

Wahlübung Z1: Validierung (Reactive Forms)

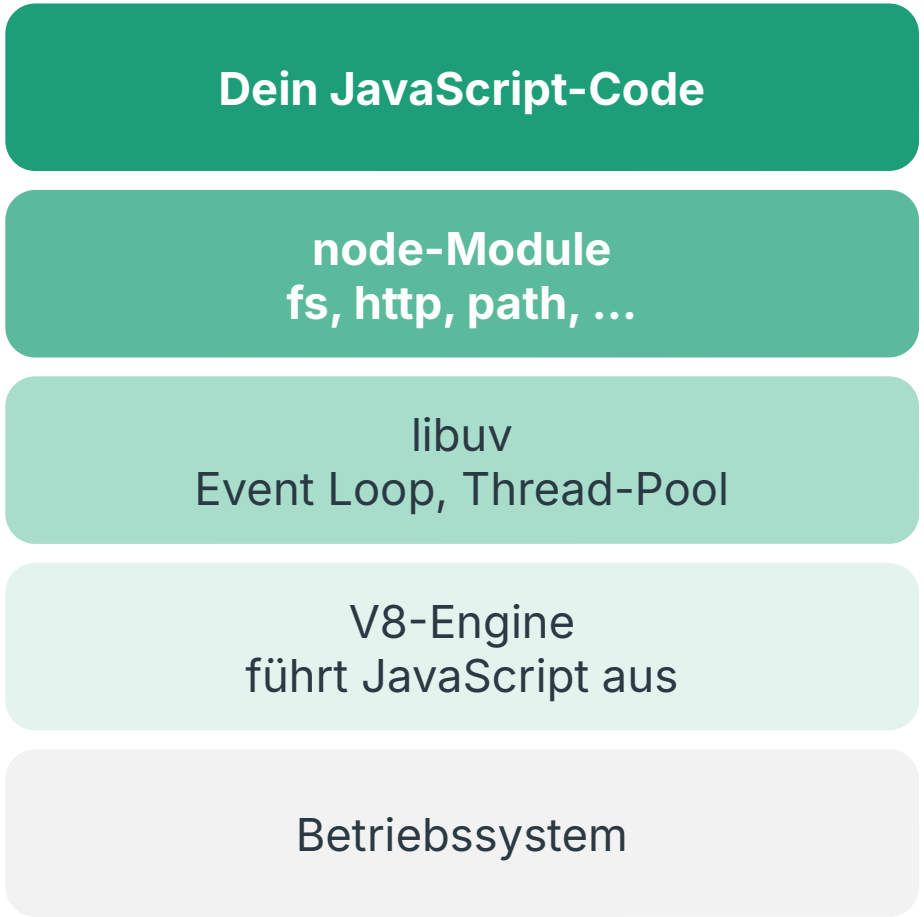
- **Weiter in CustomerDetailsComponent (Z1_reactive_forms)**
- **1. Vor- und Nachname als Pflichtfelder: Validators.required**
- **2. PLZ als Text, nur Ziffern: Validators.pattern('[0-9]*')**
  - optional: eigene Prüffunktion digitsOnly
- **3. Fehlermeldung je ungültigem Feld mit @if**
  - z. B. zip.dirty && zip.hasError('pattern')
- **4. Absenden verhindern: [disabled]="customerForm.invalid" am Submit-Button**
- **5. Im Browser testen: leere Felder, Buchstaben in der PLZ**
- **Musterlösung: keine, Validierung ist der Teil zum Selbermachen**

Anhang

node.js

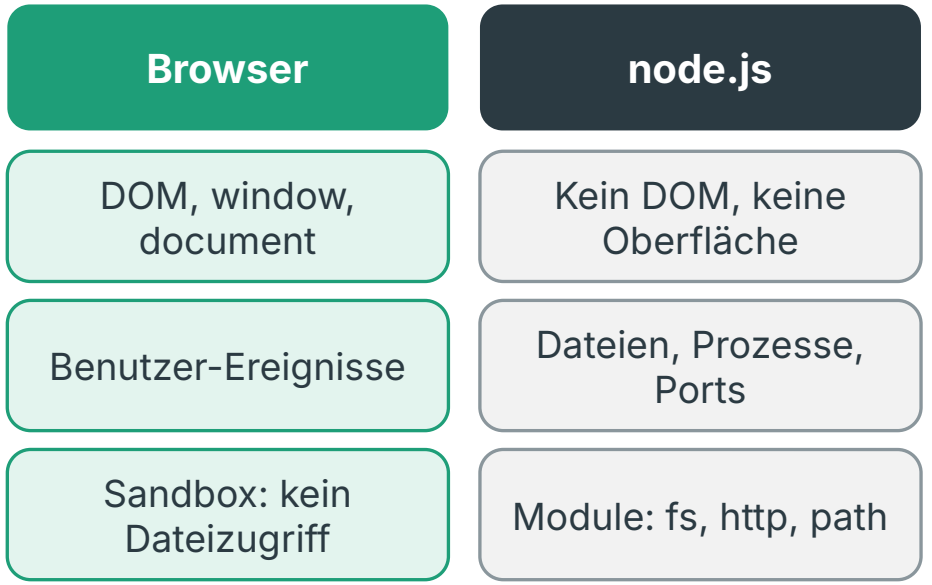
Was ist node.js?

- Laufzeitumgebung für JavaScript außerhalb des Browsers
- Kern: Google V8 – dieselbe JavaScript-Engine wie in Chrome
- libuv: Event Loop und Thread-Pool für Dateien, Netzwerk, Timer
- Eigene Bibliotheken (Module): fs, http, path, crypto, …
- Einsatz: Server, Kommandozeilen-Werkzeuge, Build-Tools
- Doku: nodejs.org/docs



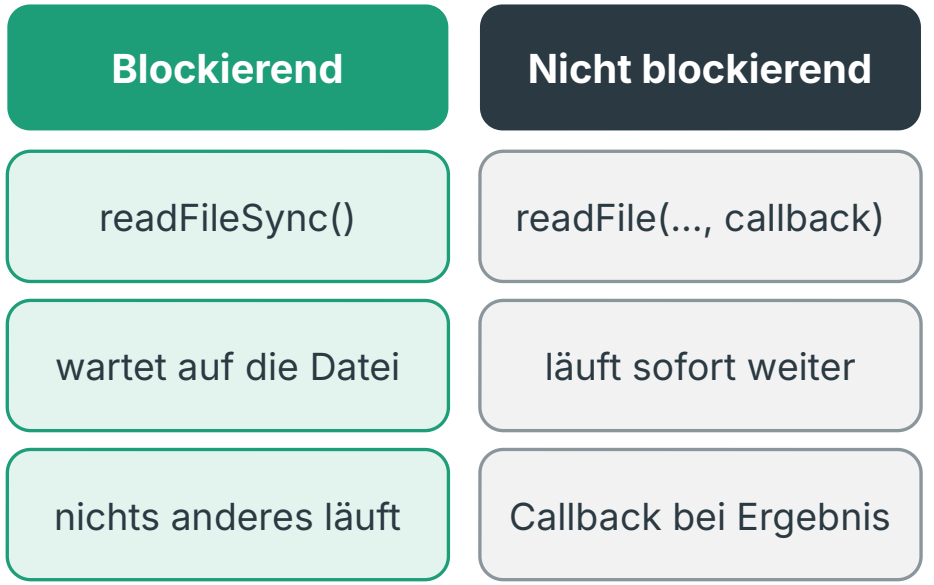
node.js und Browser im Vergleich

- Gleiche Sprache, andere Umgebung
- node.js hat kein Fenster, kein HTML-Dokument, kein DOM
- Keine Browser-APIs wie window, document, localStorage
- Dafür Zugriff auf Dateien, Prozesse, Netzwerk-Ports
- Gemeinsam: console, setTimeout, fetch, Promise, JSON



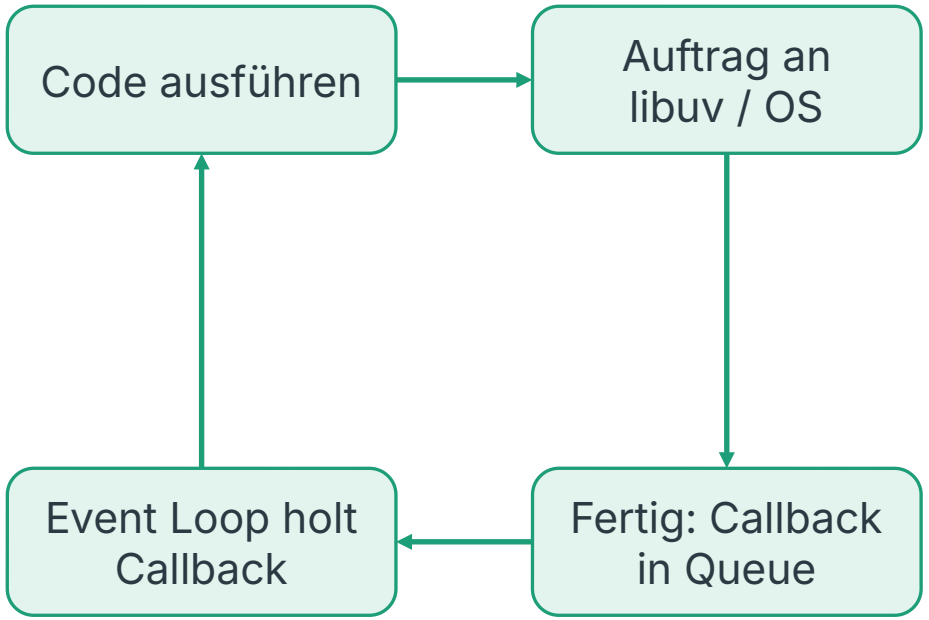
Nicht blockierendes I/O

- I/O = Ein- und Ausgabe: Datei lesen, Netzwerk, Datenbank
- Blockierend: Programm wartet, bis die Daten da sind
- Nicht blockierend: Auftrag abgeben, sofort weiterarbeiten
- Ergebnis kommt später über einen Callback oder ein Promise
- Ein JavaScript-Thread bedient so viele Anfragen gleichzeitig
- Rechenintensive Aufgaben blockieren dagegen alles



Die Event Loop: Ablauf

- 1. Code läuft von oben nach unten durch
- 2. Asynchrone Aufträge gehen an libuv bzw. das Betriebssystem
- 3. Fertige Aufträge landen in einer Warteschlange
- 4. Ist der Stapel leer, holt die Event Loop den nächsten Callback
- 5. Der Callback läuft vollständig, dann der nächste
- Programm endet, wenn nichts mehr aussteht



Beispiel: Datei asynchron lesen

```
// lesen.js – starten mit: node lesen.js
const fs = require('node:fs');

console.log('1: Auftrag wird abgegeben');

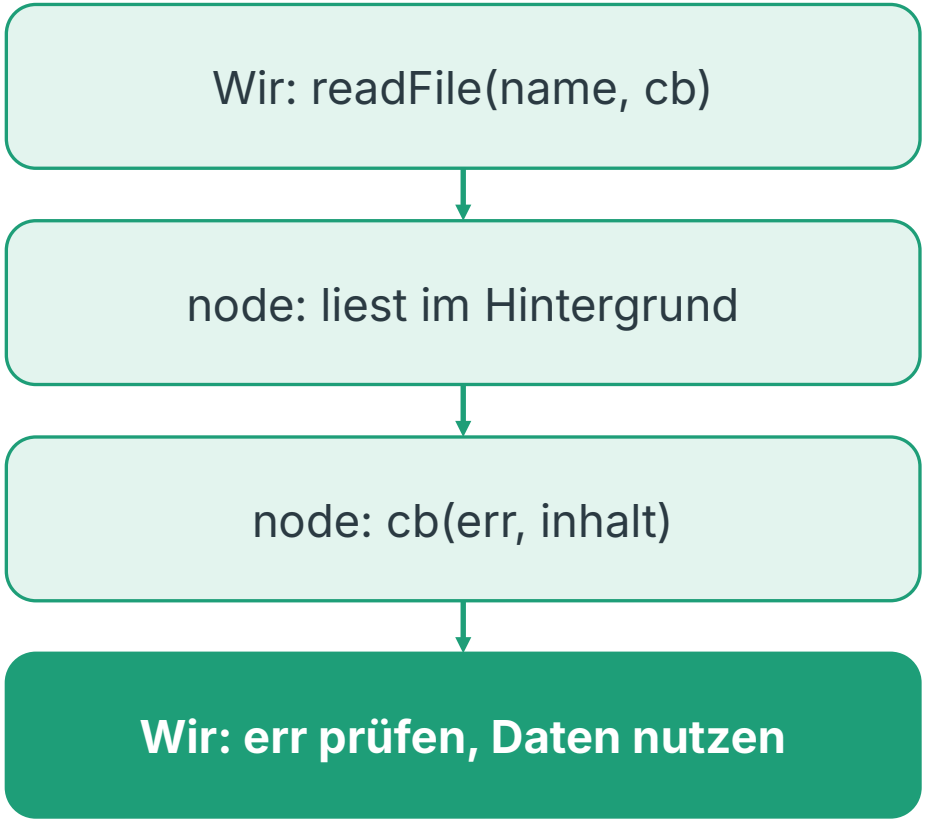
fs.readFile('daten.txt', 'utf8', (err, inhalt) ⇒ {
  if (err) {
    console.error('Fehler beim Lesen:', err.message);
    return;
  }
  console.log('3: Datei gelesen, Länge:', inhalt.length);
});

console.log('2: Programm läuft weiter');

// Ausgabe: 1, 2, 3 – der Callback kommt erst später dran
```

Der Callback: wer ruft wann was auf?

- Wir übergeben die Funktion, node.js ruft sie auf
- Zeitpunkt: wenn die Datei gelesen ist – nicht sofort
- 1. Parameter err: Fehlerobjekt oder null
- 2. Parameter inhalt: die Daten (hier ein String)
- Regel „error first“: immer zuerst err prüfen
- Rückgabewert des Callbacks wird ignoriert



Dasselbe mit Promise und await

```
// lesen.mjs – ES-Modul, starten mit: node lesen.mjs
import { readFile } from 'node:fs/promises';

try {
  const inhalt = await readFile('daten.txt', 'utf8');
  console.log('Datei gelesen, Länge:', inhalt.length);
} catch (err) {
  console.error('Fehler beim Lesen:', err.message);
}

// readFile liefert hier ein Promise statt einen Callback zu erwarten.
// await wartet auf das Ergebnis, ohne node.js zu blockieren.
// Fehler kommen als Exception und landen im catch-Block.
```

Beispiel: Ein kompletter HTTP-Server

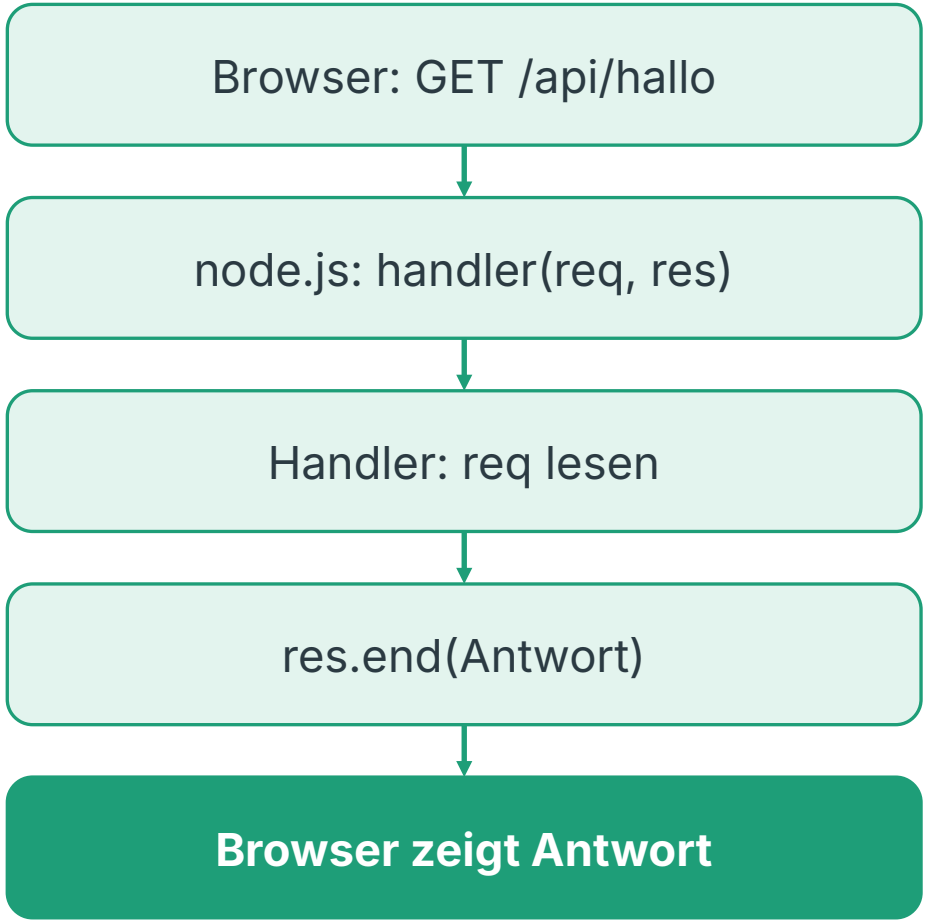
```
// server.js – starten mit: node server.js
const http = require('node:http');

const server = http.createServer((req, res) ⇒ {
  if (req.method === 'GET' && req.url === '/api/hallo') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ text: 'Hallo von node.js' }));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('Nicht gefunden');
  }
});

server.listen(6061, '127.0.0.1', () ⇒ {
  console.log('Server läuft auf http://127.0.0.1:6061');
});
```

HTTP-Server: was bei einer Anfrage passiert

- createServer(handler) registriert den Handler, ruft ihn noch nicht auf
- listen() öffnet den Port, das Programm läuft dauerhaft
- Pro Anfrage ruft node.js handler(req, res) auf
- Der Handler wertet req aus und schreibt in res
- res.end() schickt die Antwort ab – Pflicht!
- Callback von listen(): einmal, wenn der Server bereit ist



Die Parameter req und res

- req (IncomingMessage) – die Anfrage, nur lesen
  - req.method: GET, POST, … req.url: Pfad mit Query
  - req.headers: Kopfzeilen; Body kommt als Stream
- res (ServerResponse) – die Antwort, wird befüllt
  - res.writeHead(status, headers) oder res.statusCode / setHeader()
  - res.write(teil): Daten senden, beliebig oft
  - res.end(letzterTeil?): Antwort abschließen, genau einmal
- Für jede Anfrage gibt es ein neues Paar req / res

Beispiel: Statische Dateien ausliefern

```
const http = require('node:http');
const fs = require('node:fs');
const path = require('node:path');

const ROOT = path.join(__dirname, 'static-content');
const TYPES = { '.html': 'text/html', '.css': 'text/css',
                '.js': 'text/javascript' };
http.createServer((req, res) => {
  const name = path.basename(req.url) || 'index.html'; // kein ../ möglich
  const datei = path.join(ROOT, name);
  const typ = TYPES[path.extname(datei)] ?? 'text/plain';

  const stream = fs.createReadStream(datei);
  stream.on('open', () => res.writeHead(200, { 'Content-Type': typ }));
  stream.on('error', () => { res.writeHead(404); res.end(); });
  stream.pipe(res);
}).listen(6061, '127.0.0.1');
```

Streams, pipe und Frameworks

- Stream: Daten kommen in Stücken, nicht alles auf einmal
- pipe() leitet Lese-Stream in Schreib-Stream (hier: res)
- Vorteil: wenig Speicher, auch bei großen Dateien
- Sicherheit: Pfade aus der URL nie ungeprüft verwenden
- In der Praxis: Framework wie Express statt reinem http
- Unser Kurs-Backend server/server.js nutzt Express

Express (Kurs-Backend)

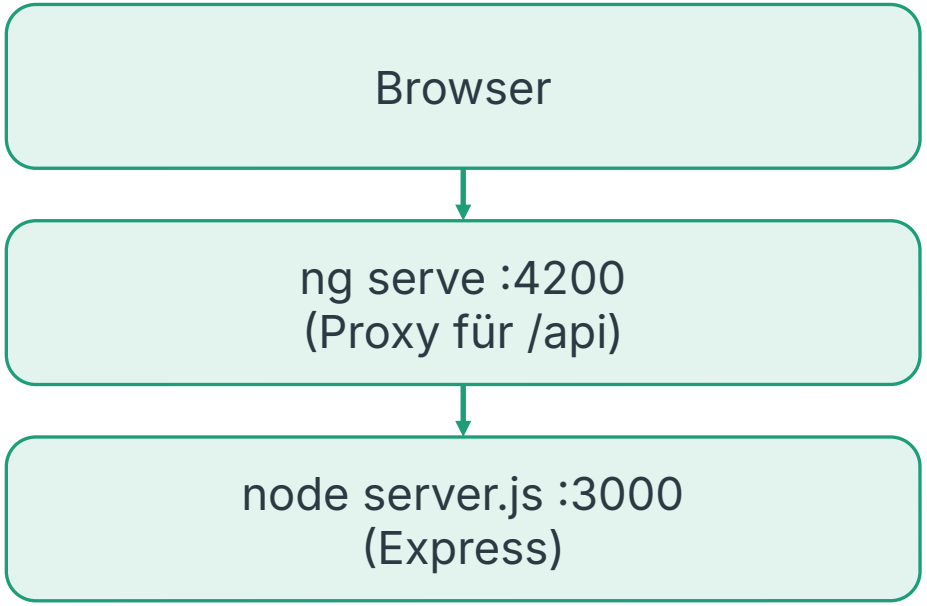
```
const express = require('express');
const app = express();

app.get('/api/pigs', (req, res) => {
  res.send({ data: pigArray });
});

app.listen(3000);
```

Das Kurs-Backend im Zusammenspiel

- Terminal 1: cd server, npm install, node server.js
  - Express-Server auf Port 3000, Daten unter /api/...
- Terminal 2: ng serve --proxy-config dev.proxy.json
  - Dev-Server auf Port 4200, liefert die Angular-App
- Browser fragt nur Port 4200 an
- Proxy leitet /api/... an Port 3000 weiter
- Zwei node-Prozesse laufen gleichzeitig



dev.proxy.json: /api → localhost:3000

Installation von node.js

- Download: nodejs.org – Version mit „LTS“ wählen
- LTS = Long Term Support: gerade Nummern, ca. 30 Monate gepflegt
- Ungerade Nummern (Current) nur kurz unterstützt
- Angular 21 braucht node ^20.19, ^22.12 oder ^24
- node 20 wird seit April 2026 nicht mehr gepflegt
- Empfehlung: aktuelle LTS, die Angular unterstützt (z. B. 24)
- npm wird automatisch mitinstalliert

Mehrere Versionen mit nvm

- Problem: Projekte brauchen verschiedene node-Versionen
- nvm = Node Version Manager (macOS, Linux)
- Windows: nvm-windows oder fnm (eigene Projekte)
- Datei .nvmrc im Projekt hält die Version fest
- nvm use liest .nvmrc und schaltet um
- Globale Pakete gelten pro node-Version

Terminal

```
nvm install 24
nvm use 24
nvm alias default 24
nvm ls

# .nvmrc im Projekt:
24

nvm use   # liest .nvmrc
```

Testen der Installation

```
$ node -v
v24.11.0                # Beispiel – die genaue Nummer kann abweichen

$ npm -v
11.6.1

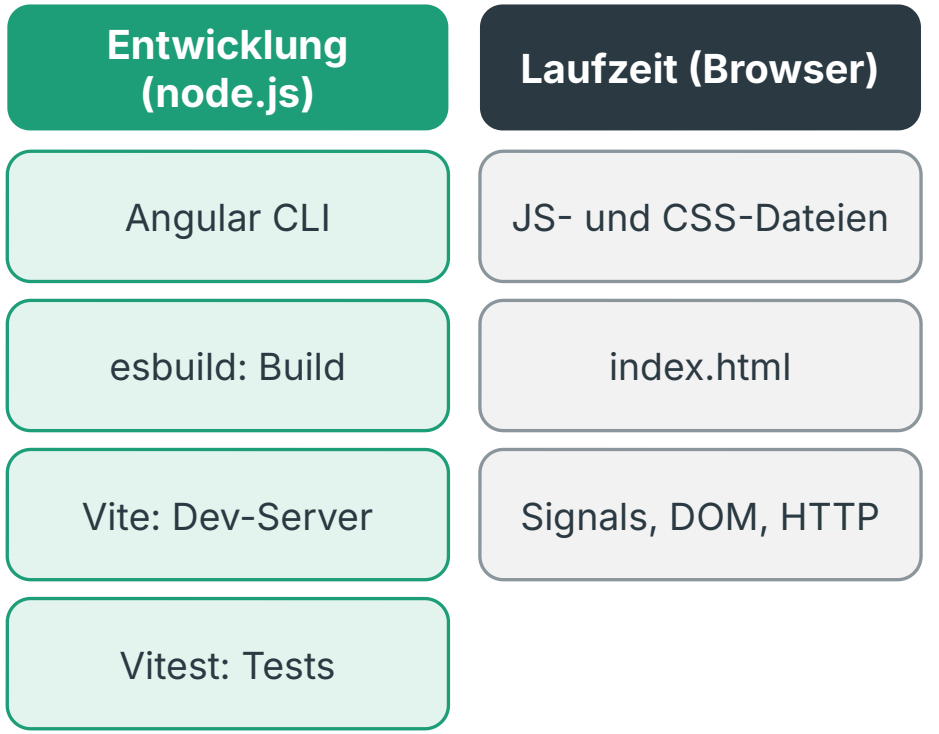
$ node -e "console.log('Hallo von node', process.version)"
Hallo von node v24.11.0

$ node                  # interaktive Konsole (REPL)
> 1 + 2
3
> .exit

$ ng version            # zeigt Angular CLI, node- und Paketversionen
```

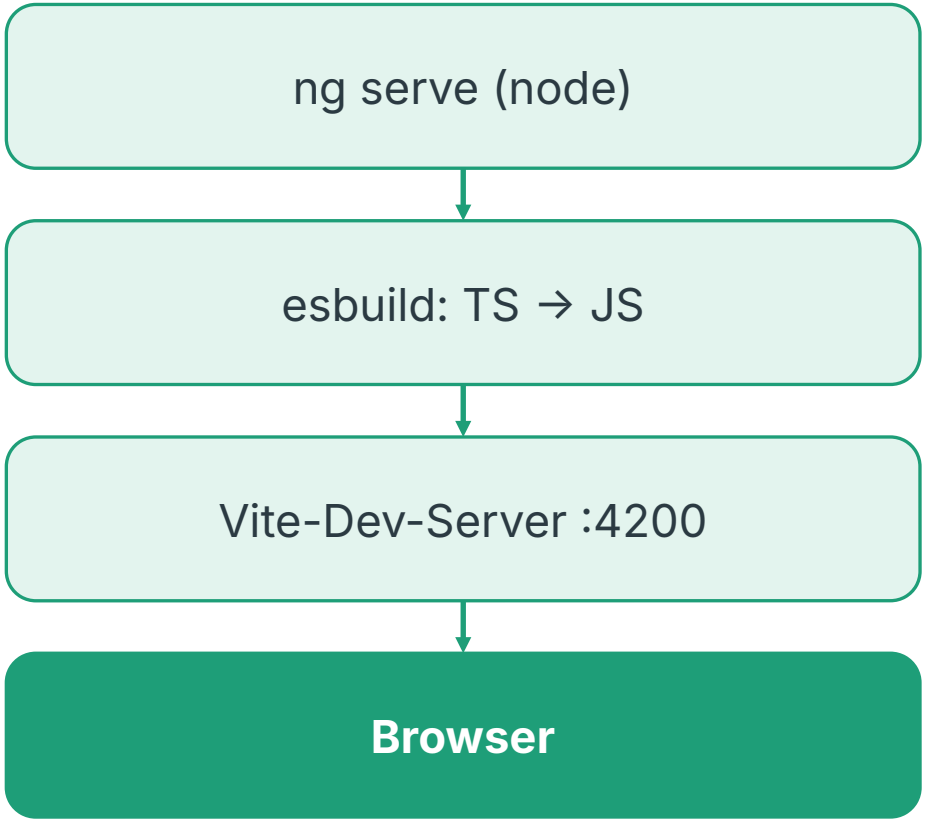
Wofür Angular node.js braucht

- node.js läuft nur auf dem Entwickler-Rechner bzw. im Build-Server
- Angular CLI (ng new, ng generate, ng serve, ng build)
- Build: TypeScript und Templates übersetzen mit esbuild
- Dev-Server: Vite liefert die App aus, lädt bei Änderungen neu
- Tests: ng test startet Vitest
- Im Browser läuft nur das Ergebnis: JavaScript, HTML, CSS



Ablauf von ng serve

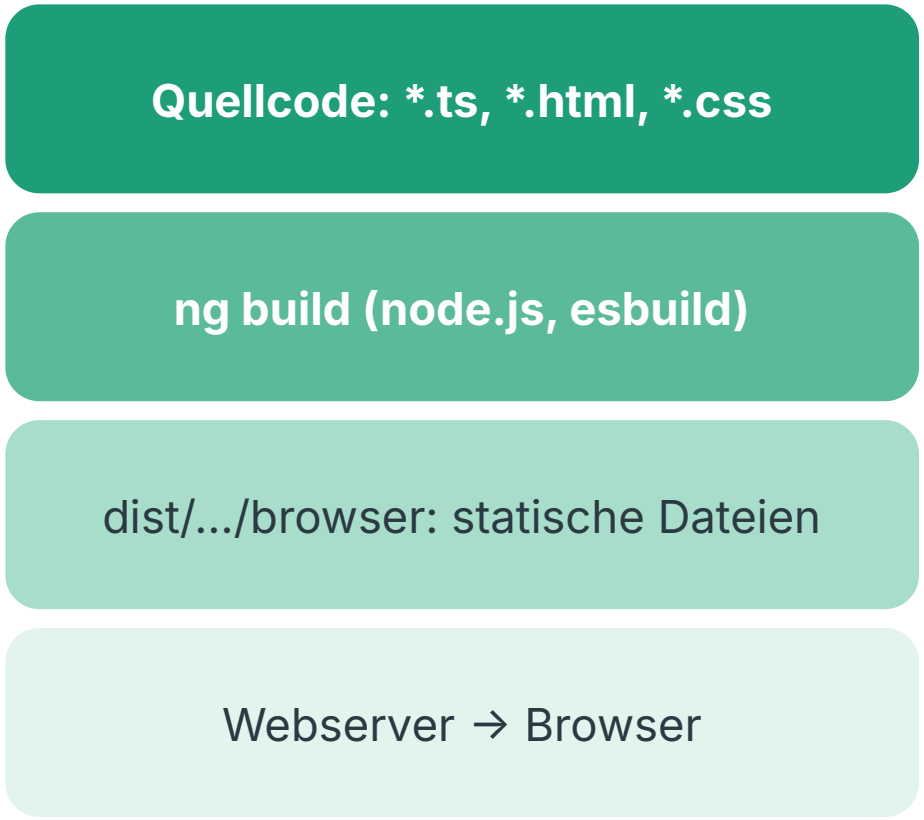
- ng serve startet die CLI als node-Prozess
- esbuild übersetzt *.ts und Templates nach JavaScript
- Vite-Dev-Server liefert die Dateien aus dem Speicher
- Browser lädt localhost:4200
- Datei geändert: nur betroffene Teile neu bauen
- Browser lädt neu bzw. tauscht Teile aus (HMR)



Änderung → neu bauen → neu laden

ng build und der Betrieb

- ng build erzeugt dist/<projekt>/browser/
- Inhalt: index.html, JavaScript- und CSS-Dateien, Assets
- Dateien sind statisch: jeder Webserver kann sie ausliefern
  - z. B. nginx, Apache, IIS, CDN
- Im Betrieb ist dafür kein node.js nötig
- Ausnahme: Server-Side Rendering (SSR) läuft auf node.js

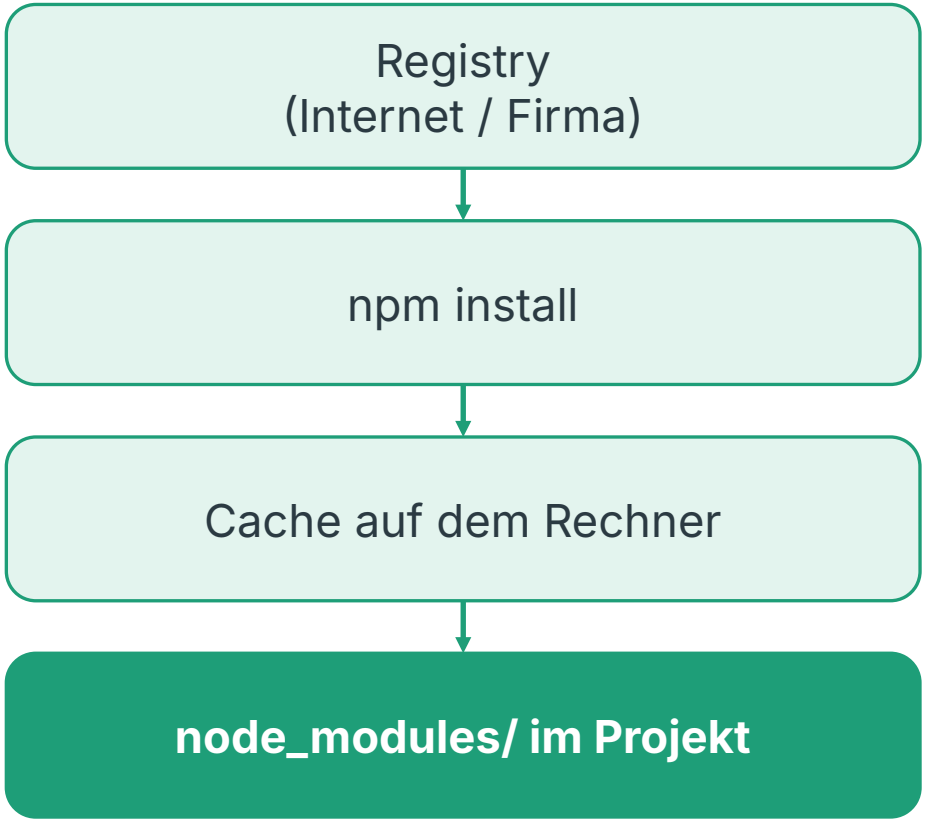


node.js nur beim Bauen (außer SSR)

npm – Der Node Package Manager

Was ist npm?

- Paketmanager: lädt Bibliotheken und Werkzeuge (Pakete)
- Wird mit node.js installiert, prüfen mit npm -v
- Öffentliche Registry: registry.npmjs.org
  - Größte Sammlung von Software-Paketen
- Firmen: eigene Registry, z. B. Nexus, Artifactory
  - Einstellung in der Datei .npmrc (registry=...)
- Alternativen mit gleicher Idee: pnpm, Yarn



npm-Befehle im Überblick

- npm install – Abhängigkeiten installieren oder Paket hinzufügen
- npm ci – exakt nach package-lock.json installieren (CI, Build)
- npm run <name> – Script aus package.json ausführen
- npm outdated – zeigt veraltete Pakete
- npm update – aktualisiert im erlaubten Versionsbereich
- npm audit – prüft auf bekannte Sicherheitslücken
- npx <paket> – Werkzeug ausführen, ohne global zu installieren
- Hilfe: npm help <befehl>, npm <befehl> -h, docs.npmjs.com

npm install: Varianten

```
npm install                 # alle Abhängigkeiten aus package.json installieren
npm i                       # Kurzform

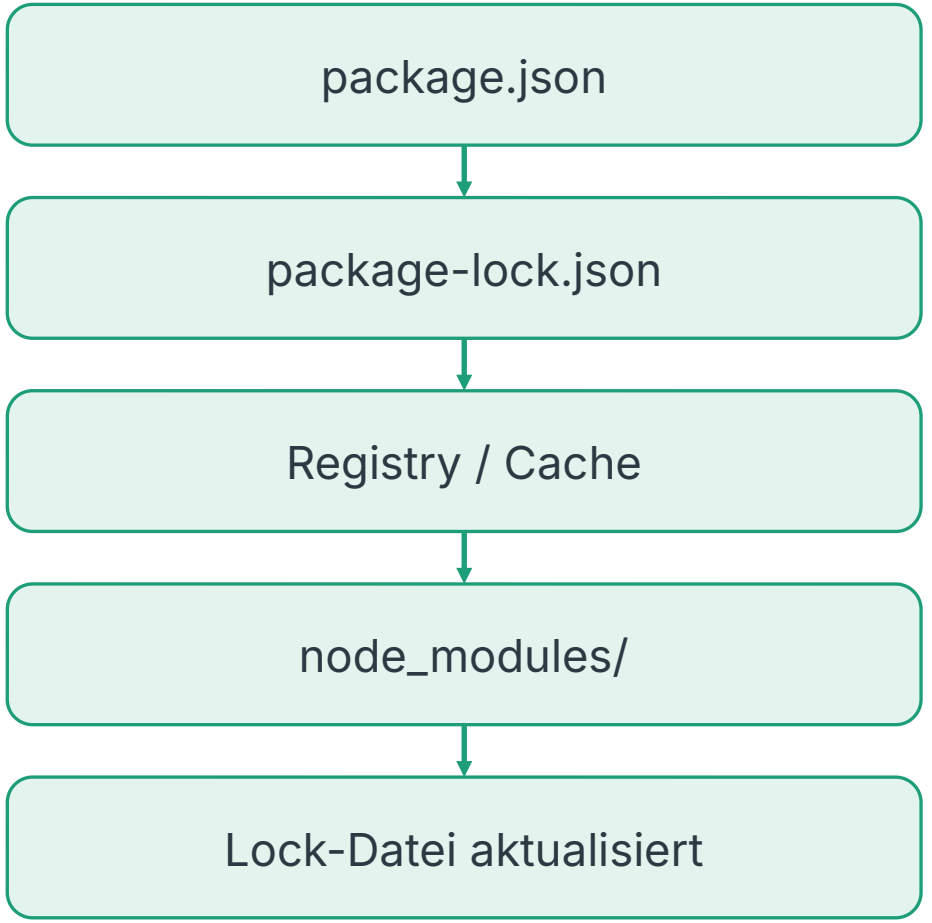
npm install rxjs            # Paket hinzufügen → "dependencies"
npm install -D vitest       # nur zum Entwickeln → "devDependencies"
npm install lodash@4.17.21  # bestimmte Version
npm install @angular/core@21  # neueste 21.x.x

npm uninstall lodash        # Paket entfernen

npm install -g @angular/cli   # global: Befehl "ng" überall verfügbar
```

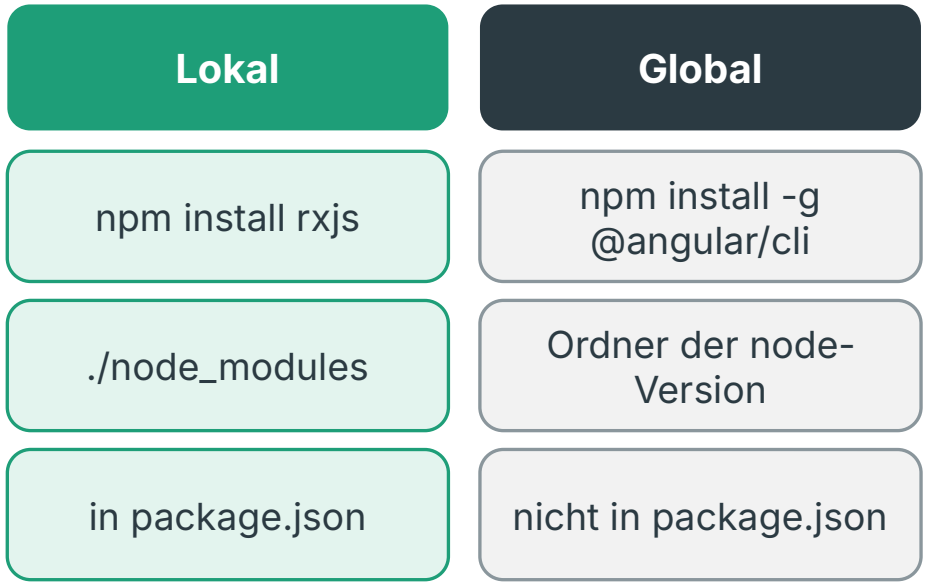
Was npm install im Detail macht

- 1. package.json lesen: welche Pakete, welche Bereiche?
- 2. Mit package-lock.json abgleichen bzw. Versionen auflösen
- 3. Pakete laden (oder aus dem Cache nehmen)
- 4. Nach node_modules/ entpacken, Abhängigkeiten der Pakete mit
- 5. package-lock.json schreiben oder aktualisieren
- Danach: offline arbeiten möglich



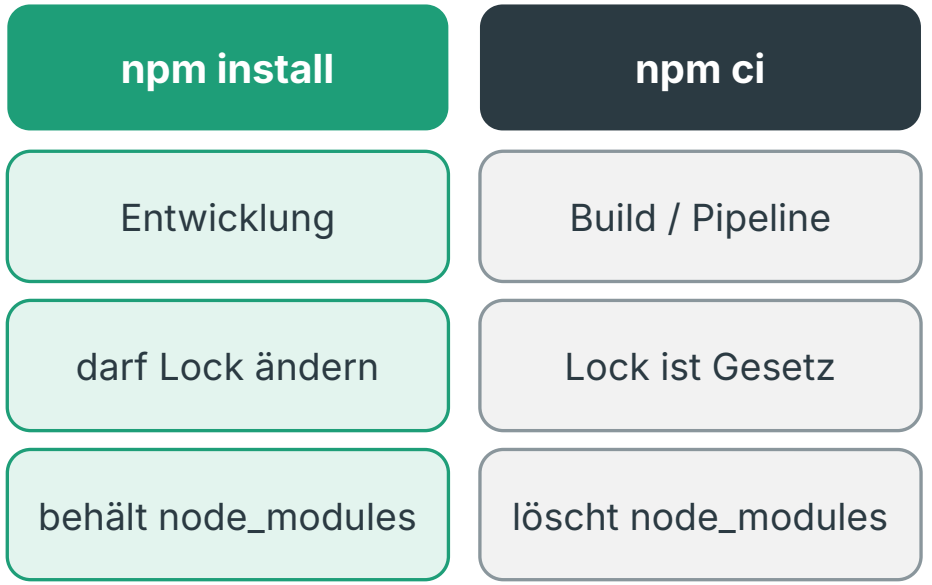
Lokal oder global installieren?

- Lokal (Standard): ./node_modules im Projekt
  - Für Bibliotheken und Werkzeuge des Projekts
  - Version steht in package.json, alle nutzen dieselbe
- Global (-g): einmal pro Rechner bzw. node-Version
  - Für allgemeine Werkzeuge, z. B. @angular/cli
- ng im Projekt nutzt trotzdem die lokale CLI-Version
- Alternative zu global: npx



npm ci oder npm install?

- npm install: darf Lock-Datei anpassen, behält node_modules
- npm ci (clean install): löscht node_modules zuerst
  - Installiert exakt package-lock.json
  - Bricht ab, wenn package.json und Lock nicht passen
- Einsatz von npm ci: Build-Server, Pipeline, frischer Checkout
- Beim Entwickeln: npm install



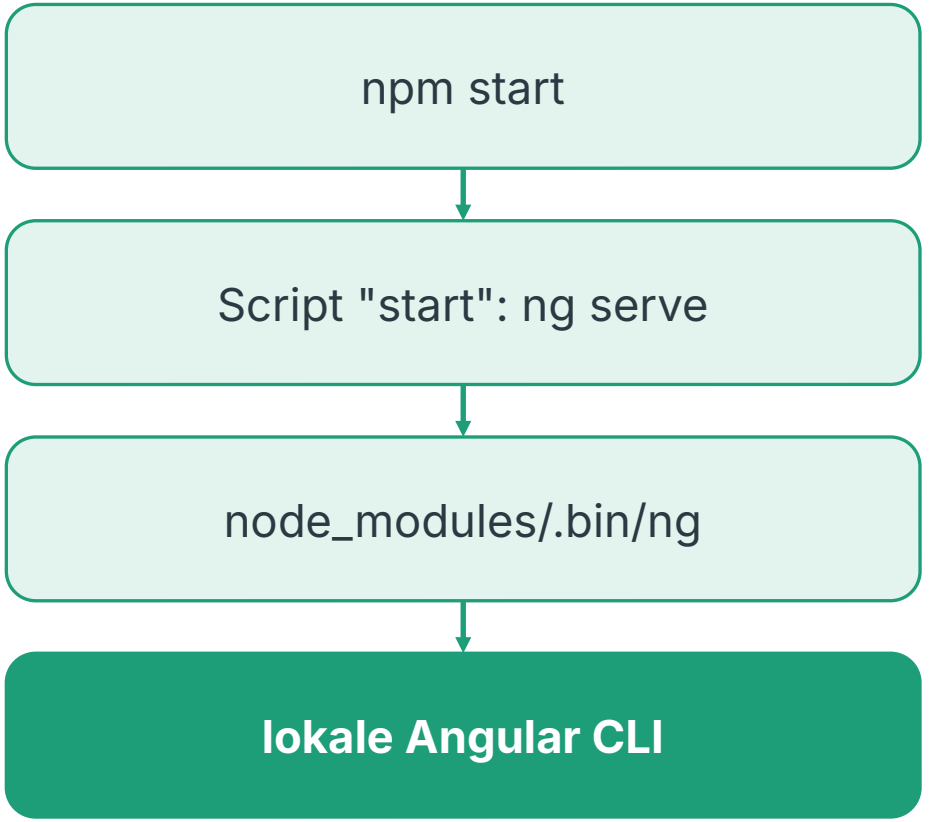
npm run: Scripts ausführen

```
// package.json von 26_resourceApi/invoice-app
"scripts": {
  "ng": "ng",
  "start": "ng serve",
  "build": "ng build",
  "watch": "ng build --watch --configuration development",
  "test": "ng test"
}

npm start                    # Kurzform von: npm run start   → ng serve
npm test                     # Kurzform von: npm run test    → ng test
npm run build                  # → ng build
npm run watch
npm run build -- --configuration development   # nach "--": Argumente an ng
npm run                      # listet alle Scripts
```

Warum Scripts den lokalen ng finden

- npm run ergänzt den Suchpfad um node_modules/.bin
- Dort liegen Befehle der installierten Pakete: ng, vitest, tsc
- Also läuft immer die Projekt-Version – ohne globale Installation
- Einheitliche Befehle im Team: npm start, npm test
- Pipeline ruft dieselben Scripts auf
- Eigene Scripts möglich, z. B. "lint": "ng lint"



npm outdated und npm update

```
$ npm outdated
Package      Current   Wanted   Latest   Location
@angular/core  21.0.1   21.0.6   21.0.6   node_modules/@angular/core
rxjs           7.8.1    7.8.2    7.8.2   node_modules/rxjs
jsdom          28.0.0   28.1.0   29.0.0   node_modules/jsdom

# Current: installiert   Wanted: höchste im erlaubten Bereich   Latest: neueste
$ npm update                  # aktualisiert bis "Wanted", passt Lock-Datei an

$ npm install -D jsdom@29       # Sprung auf neue Hauptversion: bewusst und einzeln

# Für Angular-Pakete besser das Angular-Werkzeug (passt auch Code an):
$ ng update @angular/core @angular/cli
```

npm audit: Sicherheitslücken finden

- Gleicht installierte Versionen mit einer Datenbank bekannter Lücken ab
- Ausgabe: Paket, Schweregrad (low … critical), Abhilfe
- npm audit fix: aktualisiert im erlaubten Versionsbereich
- npm audit fix --force: auch neue Hauptversionen – Vorsicht!
- Lücken in devDependencies betreffen meist nur die Entwicklung
- Regelmäßig prüfen, z. B. automatisch in der Pipeline

npx: Werkzeuge ohne Installation

- npx führt den Befehl eines Pakets aus
- Sucht zuerst in node_modules/.bin, sonst lädt es temporär
- Neues Projekt ohne globale CLI
- Bestimmte Version gezielt verwenden: paket@version
- Bei unbekannten Paketen fragt npx vor dem Download

Terminal

```
# neues Projekt mit CLI 21
npx @angular/cli@21 new shop

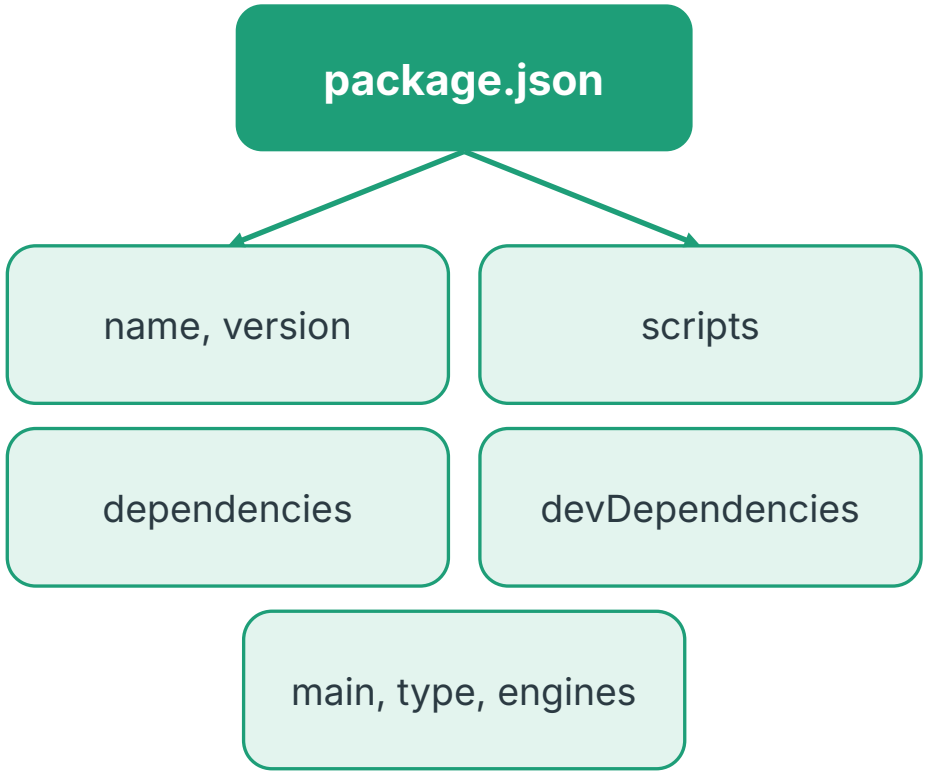
# lokale CLI des Projekts
npx ng version

# lokales Vitest direkt
npx vitest --version
```

Node-Modules und package.json

Die package.json

- Beschreibt Projekt bzw. Paket im JSON-Format
- name, version: Name und Version (SemVer)
- private: true – verhindert versehentliches Veröffentlichen
- scripts: Kurzbefehle für npm run
- dependencies / devDependencies: benötigte Pakete
- Weitere: description, author, license, main, type, engines

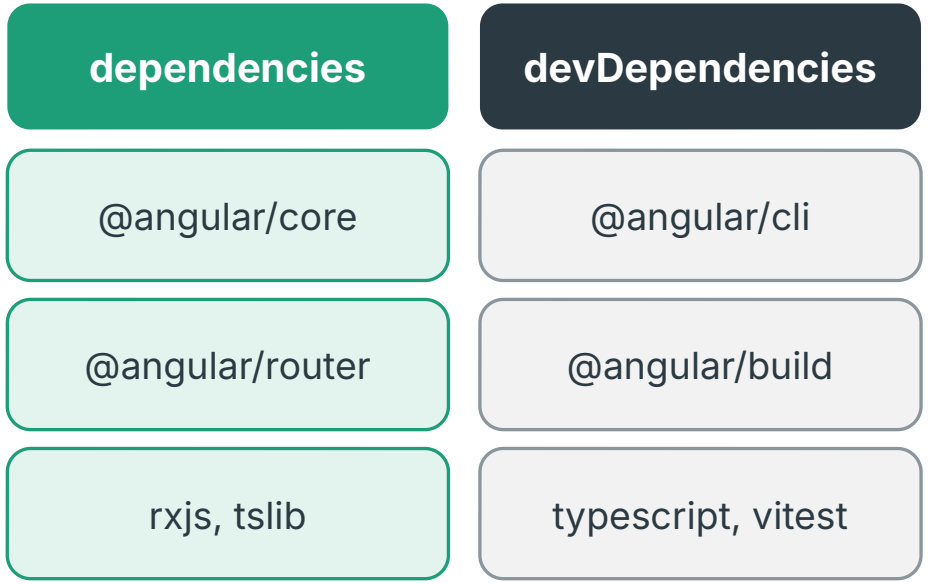


package.json eines Übungsprojekts

```
{
  "name": "invoice-app",
  "version": "0.0.0",
  "private": true,
  "scripts": { "start": "ng serve", "build": "ng build", "test": "ng test" },
  "dependencies": {
    "@angular/core": "^21.0.0", "@angular/router": "^21.0.0",
    "rxjs": "^7.8.0", "tslib": "^2.1.0"
  },
  "devDependencies": {
    "@angular/build": "^21.0.0", "@angular/cli": "^21.0.0",
    "typescript": "~5.9.0", "vitest": "~4.0.18"
  }
}
```

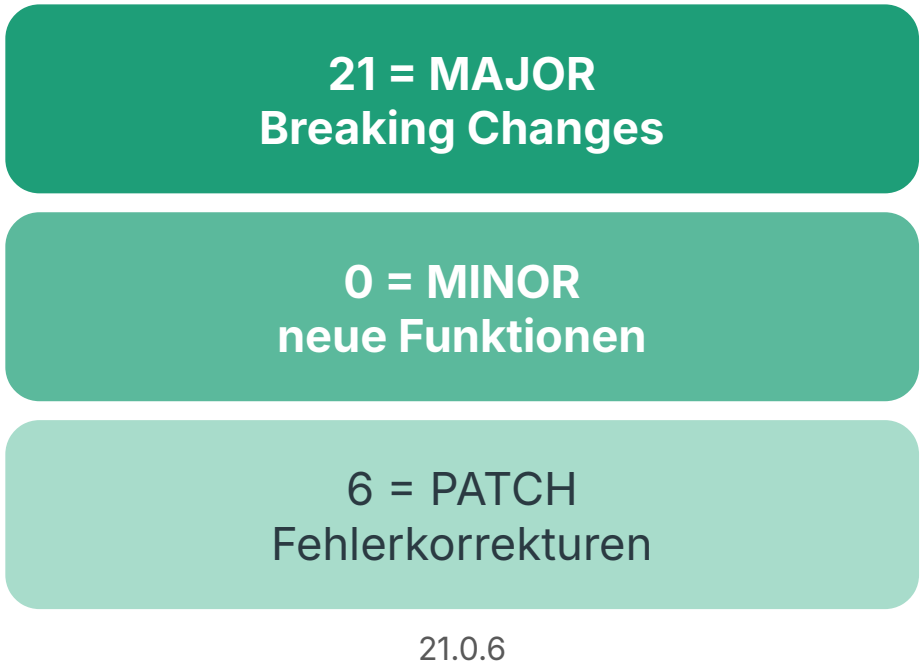
dependencies oder devDependencies?

- dependencies: Code, der später im Browser läuft
  - @angular/core, @angular/router, rxjs, tslib
- devDependencies: nur zum Bauen, Testen, Prüfen
  - @angular/cli, @angular/build, typescript, vitest
- npm install installiert beide
- npm install --omit=dev lässt devDependencies weg
- Bei Angular entscheidet der Build, was ins Bundle kommt



Versionsnummern: Semantic Versioning

- Aufbau: MAJOR.MINOR.PATCH, z. B. 21.0.6
- MAJOR: inkompatible Änderungen (Breaking Changes)
- MINOR: neue Funktionen, abwärtskompatibel
- PATCH: nur Fehlerkorrekturen
- Vorabversionen: 22.0.0-next.1, 22.0.0-rc.0
- Angular: alle 6 Monate eine neue Hauptversion



Versionsbereiche: ^ und ~

- ^21.0.0 (Dach): gleiche Hauptversion
  - >= 21.0.0 und < 22.0.0 – erlaubt Minor- und Patch-Updates
- ~5.9.0 (Tilde): gleiche Minor-Version
  - >= 5.9.0 und < 5.10.0 – nur Patch-Updates
- 21.0.6 ohne Zeichen: genau diese Version
- Sonderfall ^0.x: ^0.2.3 erlaubt nur < 0.3.0
- TypeScript mit ~: Angular unterstützt nur bestimmte Minor-Versionen

Was ist erlaubt?

```
^21.0.0
  21.0.6  ja
  21.2.0  ja
  22.0.0  nein

~5.9.0
  5.9.3   ja
  5.10.0  nein
```

package-lock.json

- Hält den exakten Stand aller installierten Pakete fest
  - Version, Download-Adresse (resolved), Prüfsumme (integrity)
  - Auch alle indirekten Abhängigkeiten
- Wird von npm geschrieben – nicht von Hand bearbeiten
- Immer mit einchecken (Git)
- Grund: alle bauen mit identischen Versionen, npm ci braucht sie
- Merge-Konflikt: Datei aus Ziel-Branch nehmen, npm install



Ein eigenes Projekt anlegen: npm init

- Jedes npm-Projekt ist selbst ein Paket (Node-Module)
- npm init fragt Name, Version, Beschreibung, … ab
- npm init -y übernimmt alle Vorgaben ohne Fragen
- Ergebnis: eine neue package.json
- Bei Angular übernimmt ng new diesen Schritt
  - inklusive Abhängigkeiten, Scripts und npm install

Terminal

```
mkdir npm-sample
cd npm-sample
npm init -y
npm install express

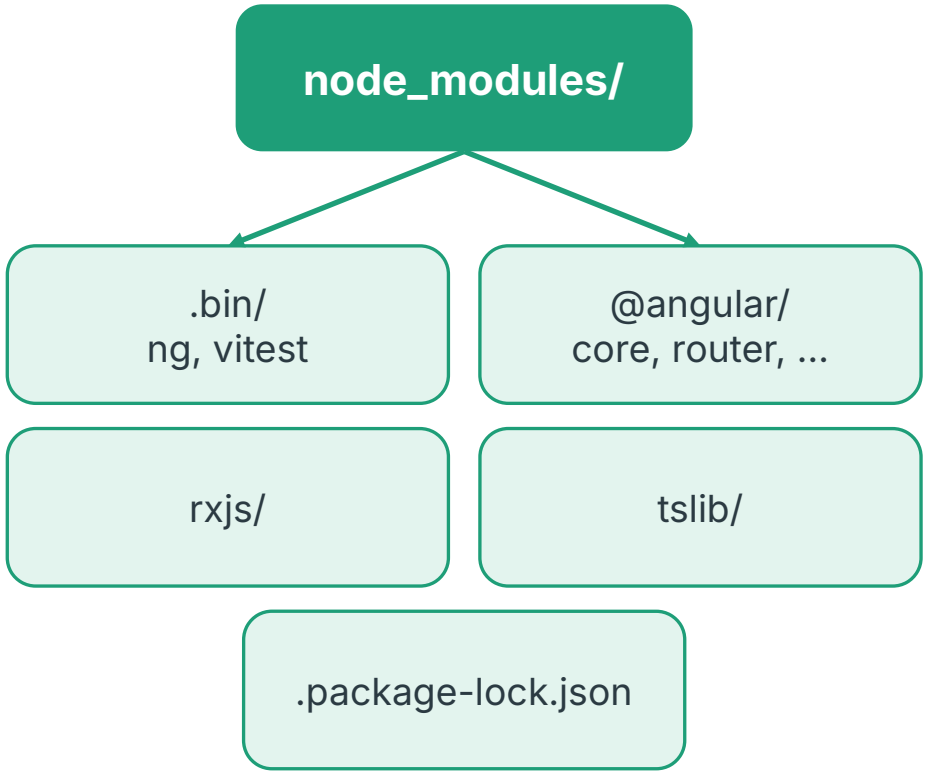
# Angular:
ng new shop
```

Beispiel: package.json nach npm init

```
{
  "name": "npm-sample",
  "version": "1.0.0",
  "description": "a simple training project",
  "main": "index.js",
  "type": "commonjs",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["training"],
  "author": "theAuthor",
  "license": "ISC",
  "engines": { "node": ">=22.12" }
}
```

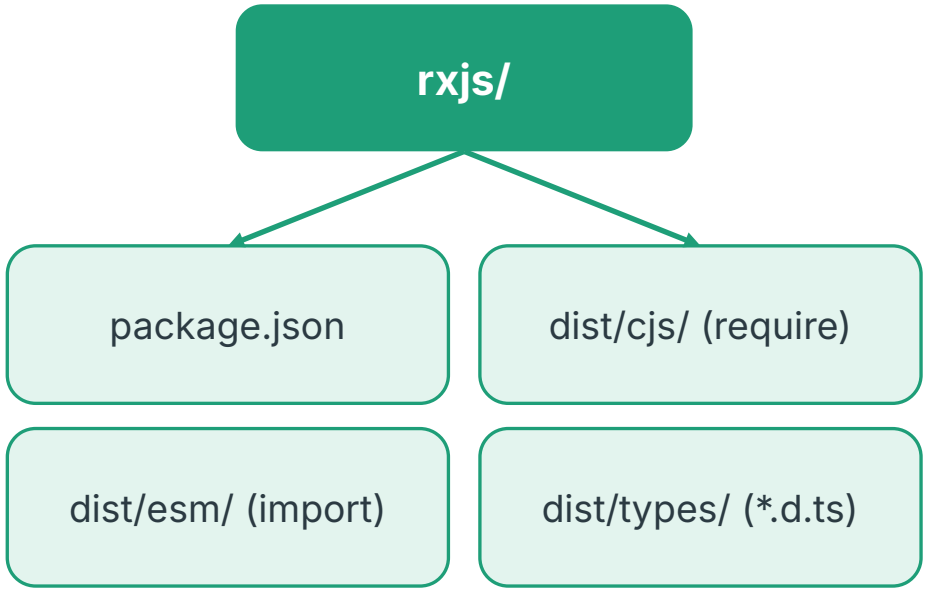
Der Ordner node_modules

- Enthält alle installierten Pakete, je mit eigener package.json
- Flach: Abhängigkeiten von Abhängigkeiten liegen meist oben
- Verschachtelt nur bei Versionskonflikten
- .bin/ enthält die Befehle (ng, vitest, tsc)
- Groß (oft Hunderte MB) – nie einchecken (.gitignore)
- Kaputt? Ordner löschen und npm install bzw. npm ci



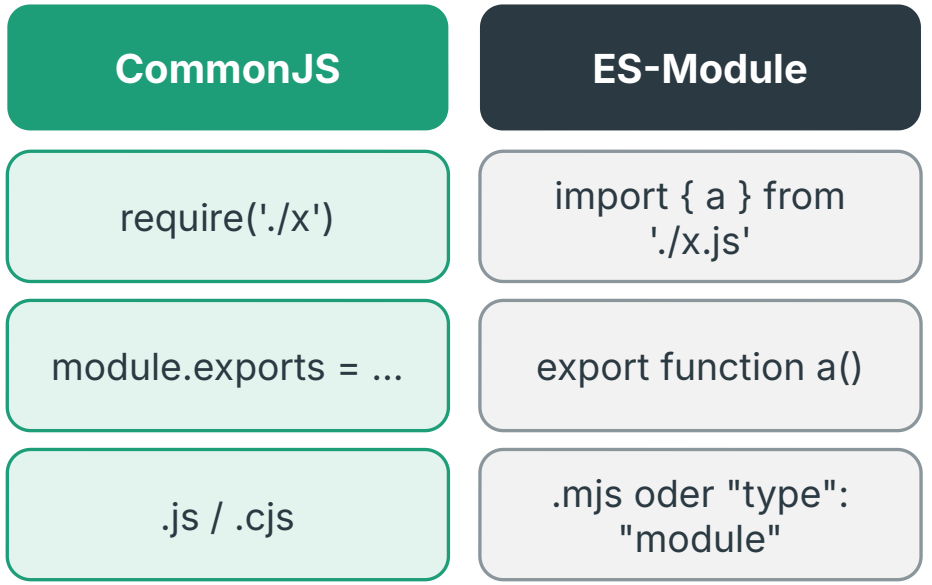
Aufbau eines Pakets

- package.json: Name, Version, eigene Abhängigkeiten
- Einstiegspunkt: main oder exports
  - Datei, die beim Import des Paketnamens geladen wird
- types: TypeScript-Typen (*.d.ts) für den Editor
- Abhängigkeiten werden mit installiert
- Beispiel: node_modules/rxjs/package.json



CommonJS und ES-Module

- CommonJS (CJS): das klassische Modulsystem von node.js
  - require() lädt, module.exports gibt frei – synchron
- ES-Module (ESM): Standard von JavaScript seit ES2015
  - import / export – Browser, TypeScript, Angular
- Neue node-Projekte nutzen zunehmend ESM
- Im Angular-Code schreibt man immer import / export



Woran node.js das Modulsystem erkennt

- Endung .mjs: immer ES-Modul
- Endung .cjs: immer CommonJS
- Endung .js: entscheidet "type" in der nächsten package.json
  - "type": "module" → ESM, sonst CommonJS
- Bei ESM in node: Dateiendung im Pfad angeben (./x.js)
- Neuere node-Versionen laden ESM auch per require()

Eigenes Modul mit CommonJS

```
// gruss.js – das Modul
const anrede = 'Hallo';                    // privat: nicht exportiert

function gruesse(name) {
  return `${anrede}, ${name}!`;
}

module.exports = { gruesse, version: '1.0.0' };   // Freigabe nach außen

// app.js – das Programm, starten mit: node app.js
const gruss = require('./gruss.js');          // Rückgabe = module.exports
console.log(gruss.gruesse('Angular'));        // Hallo, Angular!
console.log(gruss.version);                   // 1.0.0

// Kurzform mit Destrukturierung:
const { gruesse } = require('./gruss.js');
```

Dasselbe Modul als ES-Modul

```
// gruss.mjs – das Modul
const anrede = 'Hallo';                    // privat

export function gruesse(name) {            // benannter Export
  return `${anrede}, ${name}!`;
}
export const version = '1.0.0';

// app.mjs – das Programm, starten mit: node app.mjs
import { gruesse, version } from './gruss.mjs';   // Endung angeben!
console.log(gruesse('Angular'));           // Hallo, Angular!
console.log(version);

// Eingebaute Module und Pakete:
import { readFile } from 'node:fs/promises';
import { map } from 'rxjs';
```

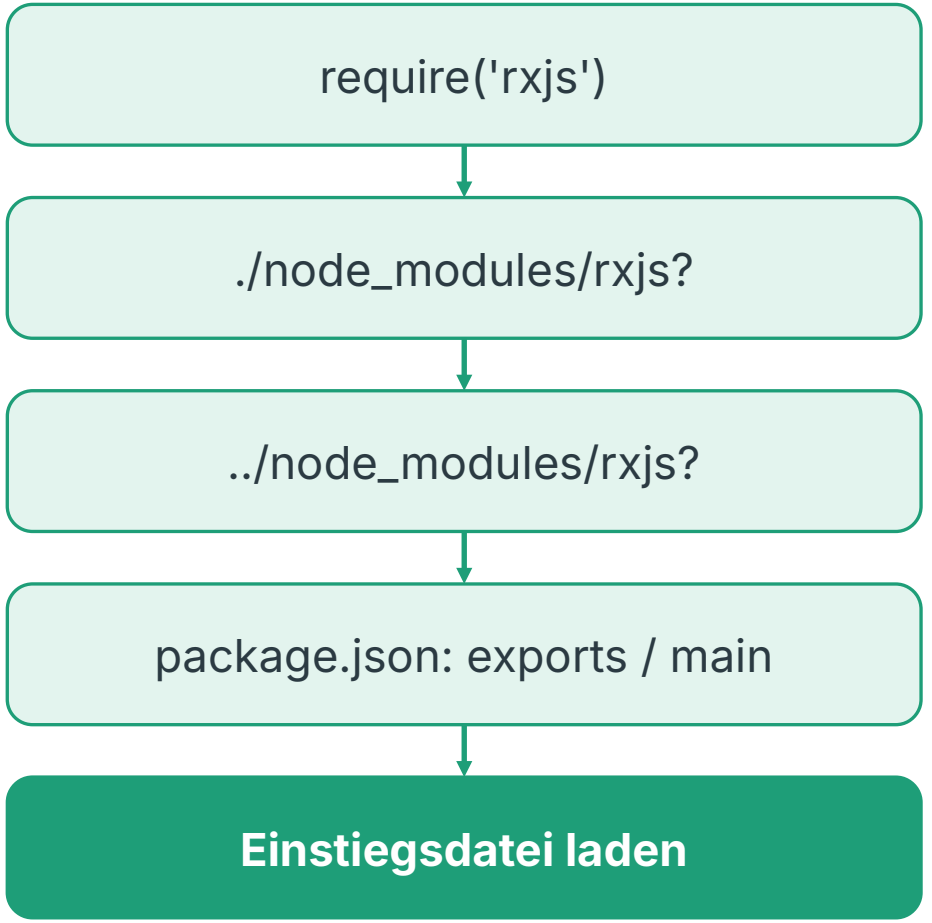
Was beim Laden eines Moduls passiert

- Der Code des Moduls läuft beim ersten Laden genau einmal
- Das Ergebnis wird zwischengespeichert
  - Jedes weitere require/import bekommt dasselbe Objekt
- Jedes Modul hat einen eigenen Gültigkeitsbereich
  - Variablen sind privat, solange sie nicht exportiert werden
- Ähnlich in Angular: eine Datei = ein Modul mit Exporten



Wie node.js ein Modul findet

- Pfad mit ./ oder ../ – eigene Datei relativ zur aktuellen Datei
- Präfix „node:“ – eingebautes Modul (node:fs, node:http)
- Nur ein Name (rxjs, express) – Paket in node_modules
  - Sucht im eigenen Ordner, dann in übergeordneten Ordnern
  - Im Paket bestimmen exports bzw. main die Datei
- Nicht gefunden: Fehler „Cannot find module“



Zusammenfassung Anhang

- node.js: JavaScript außerhalb des Browsers, V8 + Event Loop
- Für Angular: CLI, Build (esbuild), Dev-Server (Vite), Tests (Vitest)
- Im Browser läuft nur das gebaute Ergebnis
- npm: Pakete aus der Registry, Befehle install, ci, run, update, audit, npx
- package.json: Bereiche (^, ~), Scripts, dependencies / devDependencies
- package-lock.json: exakter Stand, immer einchecken
- node_modules: nie einchecken, jederzeit neu erzeugbar
- Module: CommonJS (require) und ES-Module (import / export)